ACCOUNTING ANALYTICS WITH PYTHON

Data-Driven Insights for Financial Analysis, Audit, and Decision Making



FINANCIAL ANALYSIS | AUDIT ANALYTICS | DATA WRANGLING | MACHINE LEARNING

SHARIF ISLAM, DBA, CPA, CMA

Accounting Analytics with Python

Analytics Powered by Large Language Models (LLM)

Sharif Islam, DBA, CPA, CMA

2026-10-11

Brief Contents

	Preface	1
	About the Author	3
	Acknowledgement	5
1	Introduction to Accounting Analytics	9
2	Python and Quarto Environment Setup	15
3	Data Fundamentals for Accountants	31
4	Data Wrangling with `Pandas` and `Polars`	43
5	Exploratory Data Analysis for Financial Data	79
6	Data Visualization for Accounting	95
7	Financial Statement Analytics	119
8	Management Accounting Analytics	151
9	Audit Analytics	179
10	Fraud and Anomaly Detection	221
11	Revenue Recognition and Transaction-Level Analysis	253
12	Tax Analytics	269
13	Introduction to Predictive Modeling	295
14	Text Analytics in Accounting	321
15	Generative AI in Accounting	349
16	Automated Reporting and Dashboards with Quarto	363
17	Data Ethics, Governance, and Reproducibility	381
18	Capstone Project	391
	References	397
A	A Primer on Python	399
B	The Accounting Cycle: From Journal to Financial Statements	409
C	An Introduction to EDGAR Database	423

Table of Contents

Preface . . . 1
About the Author . . . 3
Acknowledgement . . . 5

I. Part I: Foundations 7

1 Introduction to Accounting Analytics . . . 9
- 1.1. What Is Accounting Analytics? . . . 9
- 1.2. Why Now? The Shift Toward Data-Driven Accounting . . . 10
- 1.3. Accounting Analytics vs. Traditional Accounting and Auditing . . . 10
- 1.4. The Four Types of Analytics . . . 11
- 1.5. The Accounting Analytics Workflow . . . 11
- 1.6. Where Accounting Analytics Shows Up in Practice . . . 12
- 1.7. Careers and Certifications . . . 12
- 1.8. A Note on Tools: Why Python and Quarto? . . . 13
- 1.9. Chapter Summary . . . 13
- 1.10. Discussion Questions . . . 14
- 1.11. Exercises . . . 14

2 Python and Quarto Environment Setup . . . 15
- 2.1. Why This Chapter Matters . . . 15
- 2.2. The Three Tools You Need . . . 15
- 2.3. Step 1: Install Python . . . 16
- 2.4. Step 2: Install Quarto . . . 17
- 2.5. Step 3: Install a Code Editor . . . 17
- 2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document . . . 18
  - 2.6.1. 1. A Plain Python Script (`.py`) . . . 18
  - 2.6.2. 2. A Jupyter Notebook (`.ipynb`) . . . 19
  - 2.6.3. 3. A Quarto Document (`.qmd`) . . . 20
- 2.7. Your First Quarto Document . . . 23
- 2.8. Rendering This Book's Project (HTML and PDF) . . . 25
- 2.9. Installing the Python Packages You'll Need . . . 26
- 2.10. Organizing Your Project Folder . . . 27
- 2.11. Troubleshooting Common Setup Issues . . . 28

2.12. Chapter Summary . . . 28
2.13. Exercises . . . 29
3 Data Fundamentals for Accountants . . . 31
3.1. Why Data Structure Matters Before You Start Coding . . . 31
3.2. Tidy Data Principles . . . 31
3.3. Levels of Measurement . . . 33
3.4. Data Types You'll Encounter . . . 34
3.4.1. A Special Note on Dates . . . 34
3.5. Core Accounting Data Structures . . . 34
3.5.1. Chart of Accounts (COA) . . . 35
3.5.2. General Ledger (GL) . . . 35
3.5.3. Subsidiary Ledgers . . . 35
3.5.4. Trial Balance . . . 35
3.6. Transaction-Level vs. Summarized Data . . . 36
3.7. Common Data Quality Issues in Accounting Data . . . 36
3.8. A Preview Look at a Real Dataset . . . 37
3.9. Chapter Summary . . . 38
3.10. Discussion Questions . . . 38
3.11. Exercises . . . 39

II. Part II: Core Data Skills 41
4 Data Wrangling with `Pandas` and `Polars` . . . 43
4.1. What is Data Wrangling? . . . 43
4.1.1. Data Wrangling Steps and Techniques . . . 44
4.1.1.1. Discovery . . . 45
4.1.1.2. Structuring . . . 45
4.1.1.3. Cleaning . . . 45
4.1.1.4. Enriching . . . 45
4.1.1.5. Validating . . . 45
4.1.1.6. Publishing . . . 46
4.1.2. Data Wrangling vs. Data Cleaning . . . 46
4.1.3. Importance of Data Wrangling . . . 47
4.2. Two Tools, One Set of Concepts . . . 47
4.3. Reading Data In . . . 48
4.4. pandas . . . 48
4.5. polars . . . 49
4.6. pandas . . . 49
4.7. polars . . . 50
4.7.1. Reading with an Explicit Schema . . . 50
4.8. pandas . . . 50

4.9. polars . . . 51
4.10. Selecting, Filtering, and Sorting . . . 51
4.10.1. Filtering on a Single Condition . . . 51
4.11. pandas . . . 51
4.12. polars . . . 52
4.12.1. Combining Multiple Conditions . . . 52
4.13. pandas . . . 52
4.14. polars . . . 53
4.14.1. Sorting . . . 53
4.15. pandas . . . 54
4.16. polars . . . 54
4.17. Cleaning Data . . . 54
4.17.1. Step 1: Fix Currency Formatting . . . 55
4.18. pandas . . . 55
4.19. polars . . . 55
4.20. pandas . . . 56
4.21. polars . . . 57
4.21.1. Step 2: Standardize Text Fields . . . 57
4.22. pandas . . . 57
4.23. polars . . . 58
4.23.1. Step 3: Fix Inconsistent Dates . . . 59
4.24. pandas . . . 59
4.25. polars . . . 60
4.25.1. Step 4: Handle Missing Values . . . 61
4.26. pandas . . . 61
4.27. polars . . . 62
4.27.1. Step 5: Check for Duplicates . . . 62
4.28. pandas . . . 62
4.29. polars . . . 63
4.30. Merging and Joining Tables . . . 63
4.31. pandas . . . 63
4.32. polars . . . 64
4.33. Types of Merging or Joining . . . 64
4.33.1. Inner Join . . . 65
4.34. pandas . . . 65
4.35. polars . . . 66
4.35.1. Left Join . . . 66
4.36. pandas . . . 66
4.37. polars . . . 67
4.37.1. Right Join . . . 67
4.37.2. Full Join . . . 68

4.37.3. Anti Join . . . 68
4.38. Reshaping Data . . . 68
4.38.1. Wide: Total Debits by Account and Department . . . 68
4.39. pandas . . . 68
4.40. polars . . . 69
4.40.1. Back to Long (Tidy) Format . . . 70
4.41. pandas . . . 70
4.42. polars . . . 70
4.43. Grouping and Aggregating . . . 71
4.44. pandas . . . 71
4.45. polars . . . 71
4.45.1. Multi-Level Grouping . . . 72
4.46. pandas . . . 72
4.47. polars . . . 73
4.48. Polars Lazy Frames: Wrangling Large Accounting Datasets . . . 74
4.48.1. Eager vs. Lazy: The Same Pipeline, Two Ways . . . 74
4.49. Chapter Summary . . . 77
4.50. Discussion Questions . . . 77
4.51. Exercises . . . 77
5 Exploratory Data Analysis for Financial Data . . . 79
5.1. What Is EDA, and Why Do It First? . . . 79
5.2. pandas . . . 80
5.3. polars . . . 80
5.4. Choosing the Right Summary Statistic . . . 80
5.5. pandas . . . 81
5.6. polars . . . 81
5.7. Measures of Central Tendency . . . 82
5.8. pandas . . . 82
5.9. polars . . . 82
5.10. Measures of Dispersion . . . 83
5.11. pandas . . . 83
5.12. polars . . . 83
5.13. Understanding Distributions . . . 84
5.14. Detecting Outliers . . . 86
5.14.1. The Z-Score Method . . . 87
5.15. pandas . . . 87
5.16. polars . . . 87
5.16.1. The IQR Method . . . 88
5.17. pandas . . . 88
5.18. polars . . . 88
5.18.1. Outlier or Error? A Judgment Call . . . 89

5.19. Grouped EDA: Comparing Across Categories . . . 90
5.20. pandas . . . 90
5.21. polars . . . 91
5.22. Putting It Together: A First-Pass EDA Checklist . . . 92
5.23. Chapter Summary . . . 92
5.24. Discussion Questions . . . 93
5.25. Exercises . . . 93
6 Data Visualization for Accounting . . . 95
6.1. Why Visualize? From Numbers to Insight . . . 95
6.2. Two Visualization Philosophies: Seaborn vs. Plotnine . . . 95
6.3. Bar Charts: Comparing Categories . . . 97
6.4. seaborn . . . 98
6.5. plotnine . . . 98
6.6. Line Charts: Trends Over Time . . . 99
6.7. seaborn . . . 100
6.8. plotnine . . . 101
6.9. Distribution Plots: Revisiting Chapter 5, Properly Styled . . . 101
6.10. seaborn . . . 102
6.11. plotnine . . . 102
6.12. seaborn . . . 103
6.13. plotnine . . . 104
6.14. Scatter Plots: Spotting Outliers Visually . . . 105
6.15. seaborn . . . 105
6.16. plotnine . . . 106
6.17. Heatmaps: Cross-Tabulated Data . . . 107
6.18. seaborn . . . 108
6.19. plotnine . . . 108
6.20. Faceting: Small Multiples . . . 109
6.21. seaborn . . . 110
6.22. plotnine . . . 111
6.23. When to Reach for Matplotlib . . . 111
6.24. Design Principles for Accounting Visualizations . . . 113
6.24.1. Truncated Axes Exaggerate Change . . . 113
6.24.2. Other Design Principles . . . 114
6.25. Chapter Summary . . . 115
6.26. Discussion Questions . . . 115
6.27. Exercises . . . 116

III. Part III: Applied Accounting Analytics **117**
7 Financial Statement Analytics . . . 119

7.1. From General Ledger to Financial Statements . . . 119
7.2. pandas . . . 120
7.3. polars . . . 120
7.4. pandas . . . 120
7.5. polars . . . 121
7.6. pandas . . . 122
7.7. polars . . . 122
7.8. Horizontal (Trend) Analysis . . . 122
7.9. pandas . . . 123
7.10. polars . . . 123
7.10.1. Indexed Trend Analysis . . . 125
7.11. pandas . . . 125
7.12. polars . . . 125
7.13. Vertical (Common-Size) Analysis . . . 126
7.14. pandas . . . 126
7.15. polars . . . 127
7.16. pandas . . . 127
7.17. polars . . . 128
7.18. Ratio Analysis . . . 129
7.18.1. Liquidity Ratios . . . 129
7.19. pandas . . . 129
7.20. polars . . . 130
7.20.1. Profitability Ratios . . . 131
7.21. pandas . . . 131
7.22. polars . . . 132
7.22.1. Leverage Ratios . . . 133
7.23. pandas . . . 133
7.24. polars . . . 134
7.24.1. Efficiency Ratios . . . 135
7.25. pandas . . . 135
7.26. polars . . . 136
7.27. Automating Ratio Calculations . . . 137
7.28. pandas . . . 137
7.29. polars . . . 138
7.30. DuPont Decomposition: Explaining *Why* ROE Changed . . . 141
7.31. pandas . . . 141
7.32. polars . . . 142
7.33. Visualizing the Trends . . . 143
7.34. seaborn . . . 143
7.35. plotnine . . . 144
7.36. Putting It Together: A Financial Health Memo . . . 146

7.37. Why These Numbers Matter to Capital Markets . . . 147
7.37.1. Financial Statements as an Information Bridge . . . 147
7.37.2. Market Reactions to Financial Information . . . 147
7.37.3. How Ratios Connect to Valuation and the Cost of Capital . . . 148
7.37.4. A Word of Caution: Markets Use More Than Ratios . . . 148
7.38. Chapter Summary . . . 149
7.39. Discussion Questions . . . 149
7.40. Exercises . . . 150
8 Management Accounting Analytics . . . 151
8.1. Financial vs. Management Accounting Analytics . . . 151
8.2. pandas . . . 152
8.3. polars . . . 152
8.4. Standard Costing Variance Analysis . . . 153
8.4.1. Materials Variances . . . 153
8.5. pandas . . . 153
8.6. polars . . . 154
8.6.1. Labor Variances . . . 156
8.7. pandas . . . 156
8.8. polars . . . 157
8.8.1. Overhead Variances . . . 158
8.8.2. Visualizing the Full Variance Picture . . . 160
8.9. seaborn . . . 160
8.10. plotnine . . . 161
8.11. Cost-Volume-Profit (CVP) Analysis . . . 162
8.12. pandas . . . 162
8.13. polars . . . 163
8.13.1. Contribution Margin and Break-Even . . . 163
8.14. pandas . . . 165
8.15. polars . . . 165
8.15.1. The Break-Even Chart . . . 166
8.16. seaborn . . . 167
8.17. plotnine . . . 168
8.17.1. Target Profit Analysis . . . 169
8.18. Activity-Based Costing (ABC) . . . 169
8.19. pandas . . . 170
8.20. polars . . . 170
8.20.1. Traditional Allocation (Single Base: Machine Hours) . . . 171
8.21. pandas . . . 171
8.22. polars . . . 171
8.22.1. Activity-Based Allocation (Three Cost Pools) . . . 172
8.23. pandas . . . 172

8.24. polars . . . 173
8.25. seaborn . . . 174
8.26. plotnine . . . 176
8.27. Chapter Summary . . . 177
8.28. Discussion Questions . . . 177
8.29. Exercises . . . 178
9 Audit Analytics . . . 179
9.1. From EDA to Audit Testing . . . 179
9.2. pandas . . . 180
9.3. polars . . . 180
9.4. Benford's Law . . . 180
9.5. pandas . . . 181
9.6. polars . . . 182
9.7. matplotlib . . . 184
9.8. seaborn . . . 185
9.9. plotnine . . . 186
9.9.1. Testing Statistical Significance . . . 188
9.10. pandas . . . 188
9.11. polars . . . 189
9.12. Journal Entry Testing . . . 190
9.13. pandas . . . 190
9.14. polars . . . 191
9.14.1. Test 1: Weekend Postings . . . 191
9.15. pandas . . . 192
9.16. polars . . . 192
9.16.1. Test 2: Round-Dollar Amounts . . . 193
9.17. pandas . . . 193
9.18. polars . . . 193
9.18.1. Test 3: Period-End Cutoff Timing . . . 194
9.19. pandas . . . 195
9.20. polars . . . 195
9.20.1. Test 4: Duplicate Entries . . . 196
9.21. pandas . . . 196
9.22. polars . . . 196
9.22.1. Test 5: Unusual Posting Activity . . . 197
9.23. pandas . . . 197
9.24. polars . . . 198
9.25. pandas . . . 198
9.26. polars . . . 199
9.27. Combining Red Flags into a Composite Risk Score . . . 200
9.28. pandas . . . 200

9.29. polars . . . 202
9.30. Sampling Techniques . . . 206
9.31. pandas . . . 206
9.32. polars . . . 206
9.32.1. Simple Random Sampling . . . 206
9.33. pandas . . . 207
9.34. polars . . . 207
9.34.1. Systematic Sampling . . . 207
9.35. pandas . . . 208
9.36. polars . . . 208
9.36.1. Stratified Sampling . . . 208
9.37. pandas . . . 208
9.38. polars . . . 209
9.39. pandas . . . 210
9.40. polars . . . 211
9.40.1. Monetary Unit Sampling (MUS) . . . 212
9.41. pandas . . . 212
9.42. polars . . . 213
9.43. pandas . . . 214
9.44. polars . . . 215
9.44.1. Comparing Coverage Across Methods . . . 217
9.45. pandas . . . 217
9.46. polars . . . 218
9.47. Chapter Summary . . . 219
9.48. Discussion Questions . . . 219
9.49. Exercises . . . 220
10 Fraud and Anomaly Detection . . . 221
10.1. From Red Flags to Statistical Anomaly Detection . . . 221
10.2. Z Score - A Statistical Method for Anomaly Detection . . . 222
10.3. pandas . . . 223
10.4. polars . . . 223
10.5. Feature Engineering for Anomaly Detection . . . 224
10.6. pandas . . . 227
10.7. polars . . . 227
10.8. K-Means Clustering for Anomaly Detection . . . 228
10.8.1. Attempt 1: Clustering on Amount and Weekend Indicator . . . 228
10.9. pandas . . . 228
10.10. polars . . . 229
10.11. pandas . . . 230
10.12. polars . . . 231
10.12.1. Attempt 2: Clustering on Amount-Focused Features Only . . . 232

10.13. pandas . . . 232
10.14. polars . . . 232
10.15. pandas . . . 233
10.16. polars . . . 234
10.17. seaborn . . . 235
10.18. plotnine . . . 236
10.19. Isolation Forest . . . 238
10.20. pandas . . . 239
10.21. polars . . . 240
10.21.1. The `contamination` Parameter Controls the Precision-Recall Tradeoff 241
10.22. pandas . . . 242
10.23. polars . . . 242
10.24. Combining Model-Based and Rule-Based Detection . . . 244
10.25. pandas . . . 244
10.26. polars . . . 245
10.27. Evaluating Unsupervised Methods Without Labels . . . 249
10.28. Chapter Summary . . . 250
10.29. Discussion Questions . . . 250
10.30. Exercises . . . 251
11 Revenue Recognition and Transaction-Level Analysis . . . 253
11.1. Why Revenue Recognition Is a Common Risk Area . . . 253
11.2. pandas . . . 254
11.3. polars . . . 254
11.4. Detecting Quarter-End Revenue Concentration . . . 255
11.5. pandas . . . 255
11.6. polars . . . 256
11.7. seaborn . . . 257
11.8. plotnine . . . 258
11.8.1. Which Customers Drive the Concentration? . . . 259
11.9. pandas . . . 259
11.10. polars . . . 260
11.11. Detecting Bill-and-Hold Patterns . . . 261
11.12. pandas . . . 261
11.13. polars . . . 262
11.14. pandas . . . 262
11.15. polars . . . 263
11.16. Accounts Receivable Aging . . . 264
11.17. pandas . . . 264
11.18. polars . . . 264
11.19. Days Sales Outstanding (DSO) by Quarter . . . 266
11.20. pandas . . . 266

11.21. polars . . . 266
11.22. Chapter Summary . . . 267
11.23. Discussion Questions . . . 268
11.24. Exercises . . . 268
12 Tax Analytics . . . 269
12.1. Two Kinds of Tax Analytics . . . 269
12.2. pandas . . . 269
12.3. polars . . . 270
12.4. Effective Tax Rate by Jurisdiction . . . 270
12.5. seaborn . . . 271
12.6. plotnine . . . 272
12.7. Consolidated Effective Tax Rate . . . 273
12.8. pandas . . . 273
12.9. polars . . . 274
12.10. Spotting Disproportionate Profit Allocation . . . 274
12.11. pandas . . . 274
12.12. polars . . . 275
12.12.1. Profit per Employee: A Sharper View of the Same Pattern . . . 277
12.13. pandas . . . 277
12.14. polars . . . 277
12.15. seaborn . . . 278
12.16. plotnine . . . 279
12.17. Rate Reconciliation . . . 280
12.18. pandas . . . 280
12.19. polars . . . 281
12.20. pandas . . . 282
12.21. polars . . . 283
12.22. Sales Tax Compliance Testing . . . 284
12.23. pandas . . . 284
12.24. polars . . . 284
12.25. pandas . . . 285
12.26. polars . . . 286
12.26.1. Quantifying the Exposure . . . 287
12.27. pandas . . . 287
12.28. polars . . . 287
12.28.1. The Materiality Threshold Matters . . . 288
12.29. pandas . . . 288
12.30. polars . . . 288
12.31. Chapter Summary . . . 289
12.32. Discussion Questions . . . 290
12.33. Exercises . . . 290

IV. Part IV: Advanced Analytics and Artificial Intelligence **293**
13 Introduction to Predictive Modeling . . . 295
13.1. From Descriptive to Predictive Analytics . . . 295
13.2. pandas . . . 296
13.3. polars . . . 296
13.4. Regression: Predicting Days to Collect . . . 296
13.4.1. Feature Engineering . . . 296
13.5. pandas . . . 296
13.6. polars . . . 297
13.6.1. Train/Test Split . . . 298
13.6.2. Fitting and Interpreting the Model . . . 299
13.7. scikit-learn . . . 299
13.8. pyfixest . . . 299
13.8.1. Evaluating the Model . . . 303
13.9. seaborn . . . 303
13.10. plotnine . . . 304
13.11. Classification: Predicting Financial Distress . . . 306
13.11.1. Comparing Distressed vs. Healthy Companies . . . 306
13.12. pandas . . . 306
13.13. polars . . . 307
13.13.1. Train/Test Split and Standardization . . . 307
13.13.2. Fitting the Model . . . 308
13.14. scikit-learn . . . 308
13.15. pyfixest . . . 309
13.16. seaborn . . . 312
13.17. plotnine . . . 313
13.17.1. Evaluating the Model . . . 314
13.17.2. Confusion Matrix and ROC Curve . . . 315
13.18. seaborn . . . 316
13.19. plotnine . . . 317
13.20. Chapter Summary . . . 318
13.21. Discussion Questions . . . 319
13.22. Exercises . . . 319
14 Text Analytics in Accounting . . . 321
14.1. From Structured to Unstructured Data . . . 321
14.2. Where Text Shows Up in Accounting Practice . . . 322
14.3. Text Preprocessing Fundamentals . . . 322
14.4. pandas . . . 322
14.5. polars . . . 323
14.5.1. Tokenization and Cleaning . . . 323
14.6. Not pandas . . . 323

14.7. pandas . . . 324
14.8. polars . . . 324
14.8.1. Removing Stopwords . . . 326
14.9. Not pandas . . . 326
14.10. pandas . . . 326
14.11. polars . . . 327
14.11.1. Stemming . . . 327
14.12. Not pandas . . . 328
14.13. pandas . . . 328
14.14. polars . . . 329
14.14.1. Lemmatization . . . 329
14.15. pandas . . . 330
14.16. polars . . . 331
14.17. From Text to Numbers: Bag-of-Words and TF-IDF . . . 331
14.18. Visualizing Word Frequency . . . 333
14.19. seaborn . . . 333
14.20. plotnine . . . 334
14.21. Financial Sentiment Analysis . . . 339
14.22. seaborn . . . 341
14.23. plotnine . . . 342
14.24. Text Classification . . . 343
14.25. Limits and Risks of Text Analytics in Accounting . . . 345
14.26. Chapter Summary . . . 346
14.27. Discussion Questions . . . 346
14.28. Exercises . . . 347
15 Generative AI in Accounting . . . 349
15.1. Generative AI vs. the Predictive Models in Chapter 12 . . . 349
15.2. Current Use Cases in Accounting and Audit Practice . . . 350
15.3. Prompt Engineering Basics for Accounting Tasks . . . 350
15.4. Hands-On: Drafting a Financial Narrative with an LLM API . . . 351
15.5. pandas . . . 351
15.6. polars . . . 352
15.7. The Hallucination Check: Verifying AI Output Against Source Data . . . 354
15.8. Using GenAI for Synthetic Text Generation . . . 355
15.9. Risks and Limitations . . . 356
15.10. The Current Regulatory and Professional Landscape . . . 356
15.11. Designing an "AI vs. Analyst" Exercise . . . 357
15.12. Chapter Summary . . . 357
15.13. Discussion Questions . . . 358
15.14. Exercises . . . 358

V. Part V: Communication and Capstone **361**

16 Automated Reporting and Dashboards with Quarto . . . 363
16.1. From One-Off Analysis to Repeatable Reporting . . . 363
16.2. Parameterized Reports . . . 363
16.2.1. Declaring a Parameter . . . 364
16.3. pandas . . . 364
16.4. polars . . . 364
16.4.1. Building the Rest of the Report Around the Parameter . . . 365
16.5. pandas . . . 365
16.6. polars . . . 366
16.6.1. Rendering the Same Template for a Different Department . . . 369
16.7. Batch Rendering Multiple Reports . . . 370
16.8. Quarto Dashboards . . . 371
16.8.1. Building the Dashboard's Content . . . 371
16.8.2. An Interactive Chart with Plotly . . . 372
16.8.3. Assembling the Full Dashboard . . . 373
16.9. Putting It Together: A Recurring Reporting Workflow . . . 378
16.10. Chapter Summary . . . 378
16.11. Discussion Questions . . . 379
16.12. Exercises . . . 379
17 Data Ethics, Governance, and Reproducibility . . . 381
17.1. Why This Chapter Matters . . . 381
17.2. Data Privacy and Confidentiality . . . 382
17.2.1. Pseudonymization . . . 382
17.2.2. Generalization . . . 383
17.3. Data Governance and Audit Trails . . . 384
17.3.1. Version Control as an Audit Trail . . . 385
17.3.2. Logging Data Transformations . . . 385
17.3.3. Access Control . . . 386
17.4. Reproducibility . . . 386
17.4.1. Fixed Random Seeds . . . 387
17.4.2. Capturing the Software Environment . . . 387
17.4.3. Quarto's Own Reproducibility Features . . . 388
17.5. A Practical Ethics, Governance, and Reproducibility Checklist . . . 388
17.6. Chapter Summary . . . 389
17.7. Discussion Questions . . . 389
17.8. Exercises . . . 390
18 Capstone Project . . . 391
18.1. The Scenario . . . 391
18.2. Choosing a Track . . . 392
18.2.1. Track A: Audit & Fraud Analytics . . . 392

18.2.2. Track B: Financial Reporting & Revenue Analytics . . . 392
18.2.3. Track C: Management Accounting & Predictive Analytics . . . 393
18.3. Deliverables . . . 393
18.4. Suggested Milestones . . . 394
18.5. Example Starter Questions . . . 394
18.6. Grading Rubric . . . 395
18.7. Final Thoughts . . . 395
References . . . 397

Appendices **399**
A A Primer on Python . . . 399
A.1. What Is Python, and Why This Book Uses It . . . 399
A.2. Data Types . . . 399
A.3. Data Structures . . . 400
A.4. Functions . . . 402
A.5. NumPy and Arrays . . . 403
A.5.1. Why Not Just Use a List? . . . 403
A.5.2. Creating Arrays . . . 403
A.5.3. Vectorized Operations and Filtering . . . 404
A.5.4. Common Aggregate Functions . . . 404
A.5.5. Reshaping Arrays . . . 404
A.5.6. Conditional Logic with `np.where()` . . . 405
A.5.7. Random Number Generation . . . 405
A.6. Classes, Objects, Attributes, and Methods . . . 405
A.6.1. Defining a Class . . . 406
A.6.2. Creating and Using an Object . . . 406
A.7. Summary . . . 407
B The Accounting Cycle: From Journal to Financial Statements . . . 409
B.1. The Accounting Cycle - at a Glance . . . 409
B.2. Our Worked Example: Bright Consulting LLC . . . 410
B.3. Step 1: Journalize Transactions . . . 410
B.4. Step 2: Post to the Ledger . . . 412
B.5. Step 3: Unadjusted Trial Balance . . . 412
B.6. Step 4: Adjusting Entries . . . 413
B.6.1. (a) Deferred Expense: Supplies Used . . . 413
B.6.2. (b) Deferred Expense: Insurance Expired . . . 414
B.6.3. (c) Depreciation . . . 414
B.6.4. (d) Accrued Expense: Unrecorded Salaries . . . 414
B.6.5. (e) Deferred Revenue: Unearned Revenue Now Earned . . . 415
B.7. Step 5: Adjusted Trial Balance . . . 415

B.8. Step 6: Prepare the Financial Statements . . . 416
B.8.1. Income Statement . . . 416
B.8.2. Statement of Retained Earnings . . . 416
B.8.3. Balance Sheet . . . 416
B.9. Step 7: Closing Entries . . . 417
B.10. Step 8: Post-Closing Trial Balance . . . 418
B.11. Seeing the Whole Cycle in Python . . . 419
B.12. Summary . . . 420
C An Introduction to EDGAR Database . . . 423
C.1. EDGAR Database and Its Implications . . . 423
C.2. XBRL . . . 424
C.3. edgartools - Python Package for Parsing EDGAR Database . . . 425
C.4. Unstructured Data in Company Filings . . . 428
C.5. tidyfinance - Python package for Share Price Data . . . 433
C.6. Summary . . . 437

Preface

Welcome to *Accounting Analytics with Python*. This book is intended for accounting undergraduate students. However, students from other disciplines such as Finance are very likely to be benefited from using the book.

About the Author

Sharif Islam, DBA, CPA, CMA is an Associate professor in School of Accountancy in Southern Illinois University Carbondale (SIUC). He is a licensed CPA in Illinois and a Certified Management Accountant (CMA). He teaches Advanced Cost Accounting, Auditing, Accounting Information Systems, Machine Learning, and Analytics for Accounting Data. He earned his doctorate from Louisiana Tech University. His mansucripts are selected for "Best Research Paper Award" in several conferences of American Accounting Association (AAA). His research also got 2024 "Notable Contribution to the Literature Award" by AIS section of AAA. He published research in *Accounting Horizons*, *Journal of Accounting and Public Policy*, *Journal of Information Systems*, *Advances in Accounting*, *Journal of Emerging Technologies in Accounting*, *Issues in Accounting Education*, and *Managerial Auditing Journal*. His research interests lie at the intersection of Accounting and Data Science. More about his teaching and research can be found in his personal website.

Acknowledgement

To prepare the book, I took help from many sources on the internet and published materials. Many of them are cited in the book. I acknowledge the contribution of all of those resources that help me to prepare the book for the students.

Part I.

Part I: Foundations

1. Introduction to Accounting Analytics

Learning Objectives

By the end of this chapter, you should be able to:

- Define accounting analytics and distinguish it from traditional accounting and auditing
- Explain why data analytics has become central to accounting practice
- Describe the four major types of analytics (descriptive, diagnostic, predictive, prescriptive) and give an accounting example of each
- Outline the typical accounting analytics workflow, from raw data to a decision
- Identify career paths and professional certifications related to accounting analytics

1.1. What Is Accounting Analytics?

Accounting has always been about data. Every transaction a business records — a sale, a payment, a depreciation entry — becomes a data point that eventually rolls up into financial statements. What has changed in the last two decades is not *that* accountants work with data, but the *volume*, *speed*, and *tools* available for working with it.

Accounting analytics is the application of data analysis techniques — statistical, visual, and computational — to accounting and financial data in order to support decisions in areas such as financial reporting, auditing, tax, and management accounting. It sits at the intersection of three fields:

- **Accounting knowledge** — understanding what the numbers mean (revenue recognition, internal controls, GAAP/IFRS rules)
- **Data skills** — the ability to collect, clean, and analyze data (which is where Python comes in)
- **Business judgment** — knowing what question is worth asking and what a result actually implies for a decision

A useful way to think about it: traditional accounting produces the numbers; accounting analytics asks *what the numbers are telling us* and *what we should do about it*.

1.2. Why Now? The Shift Toward Data-Driven Accounting

Several forces have pushed analytics from a "nice to have" to a core accounting skill:

1. **Data volume.** Modern ERP systems (SAP, Oracle, NetSuite) log every transaction electronically. A mid-sized company can generate millions of journal entries a year — far more than any auditor or accountant could review manually.
2. **Regulatory and audit expectations.** Standard-setters and regulators (including the PCAOB) increasingly expect auditors to test *entire populations* of transactions rather than small manual samples, which requires analytics tools.
3. **Automation of routine work.** Many manual bookkeeping and reconciliation tasks are increasingly automated, shifting accountants' time toward analysis, interpretation, and communication of results.
4. **The rise of accessible tools.** Python, R, and business intelligence tools (Power BI, Tableau) have made analytics accessible to accountants without a computer science background — you don't need to be a software engineer to benefit from these tools.
5. **CPA Exam evolution.** The AICPA has incorporated data analytics content into the CPA Exam Blueprint, particularly in the Business Analysis and Reporting (BAR) discipline, signaling that the profession considers this a core competency, not an elective skill.

> Connect to Practice
>
> Throughout this book, look for "Connect to Practice" boxes like this one, which tie a concept back to how it's used in real accounting and audit work.

1.3. Accounting Analytics vs. Traditional Accounting and Auditing

It's worth being precise about how this course differs from other accounting courses you've taken:

	Traditional Accounting/Auditing	Accounting Analytics
Primary tool	Spreadsheets, manual ledgers	Python, SQL, visualization tools
Sample size	Often a sample of transactions	Frequently the *entire population*
Focus	Was this recorded correctly?	What patterns, risks, or insights exist in the data?

	Traditional Accounting/Auditing	Accounting Analytics
Output	Journal entries, financial statements, audit opinions	Dashboards, models, flagged anomalies, predictions
Skillset emphasis	Rules and standards (GAAP/IFRS)	Rules and standards *plus* coding, statistics, visualization

This isn't a replacement for the accounting knowledge you've built in other courses — it's a set of tools that make that knowledge more powerful and scalable.

1.4. The Four Types of Analytics

A common framework in the analytics field breaks analysis into four types, moving from "what happened" to "what should we do." Every technique in this book fits into one of these categories.

1. **Descriptive analytics** — *What happened?* Summarizing past data. Example: calculating total revenue by month, or the average days-sales-outstanding over the last year.
2. **Diagnostic analytics** — *Why did it happen?* Investigating causes and relationships. Example: determining why gross margin declined in Q3 by breaking revenue and cost data down by product line.
3. **Predictive analytics** — *What is likely to happen?* Using historical data to forecast. Example: predicting which customers are likely to become uncollectible accounts receivable.
4. **Prescriptive analytics** — *What should we do?* Recommending an action based on analysis. Example: recommending which vendors to flag for additional audit procedures based on a fraud-risk score.

Most introductory accounting analytics work — including much of this book — lives in the descriptive and diagnostic space, with an introduction to predictive methods later in the book (Chapter 13). Prescriptive analytics tends to require more advanced modeling and is largely beyond the scope of an undergraduate course, but it's useful to know where the field is heading.

1.5. The Accounting Analytics Workflow

Most accounting analytics projects — whether a homework assignment in this course or a real audit engagement — follow a similar sequence:

1. **Define the question.** What decision or risk are we trying to inform? (e.g., "Are there unusual journal entries near period-end that warrant investigation?")

2. **Obtain the data.** Extract data from a source system — a general ledger export, a CSV file, a database query.
3. **Clean and prepare the data.** Real data is messy: missing values, inconsistent formats, duplicate entries. This step often takes the most time in practice.
4. **Explore and analyze.** Apply descriptive statistics, visualizations, or models to look for patterns.
5. **Interpret results in context.** A statistical pattern is not automatically meaningful — this is where accounting judgment matters most.
6. **Communicate findings.** Produce a report, dashboard, or memo that a non-technical stakeholder (a partner, a controller, a client) can act on.

This book is structured around this workflow. Chapters 2–3 build foundational tools and data literacy; Chapters 4–6 cover steps 2–4 in depth; Chapters 7–11 apply the workflow to specific accounting domains (financial statements, audit, fraud, revenue, tax); and later chapters extend into predictive methods, GenAI, and communicating results.

1.6. Where Accounting Analytics Shows Up in Practice

A few examples of accounting analytics in action:

- **External audit** — Using full-population testing and anomaly detection to identify unusual journal entries for further audit procedures, rather than testing a small manual sample.
- **Internal audit** — Continuous monitoring dashboards that flag control violations or policy exceptions as they occur, not just at year-end.
- **Financial planning & analysis (FP&A)** — Building forecasting models and variance-analysis dashboards to support budgeting decisions.
- **Forensic accounting** — Applying techniques like Benford's Law (covered in Chapter 9) to detect potential financial statement fraud.
- **Tax** — Analyzing large transactional datasets to identify tax planning opportunities or compliance risks across jurisdictions.

1.7. Careers and Certifications

Accounting analytics skills are increasingly valued across public accounting, industry, and government. A few relevant credentials and pathways worth knowing about as you plan your career:

- **CPA (Certified Public Accountant)** — increasingly incorporates data analytics content, especially through the BAR discipline
- **CISA (Certified Information Systems Auditor)** — relevant for IT audit and control-focused roles

- **Data-analytics-specific certificates** offered by the AICPA and various universities

You don't need to become a data scientist to benefit from this course — the goal is *analytics literacy*: being comfortable enough with data tools that you can perform basic analyses yourself, and knowledgeable enough to critically evaluate analyses done by others (including, increasingly, by AI tools).

1.8. A Note on Tools: Why Python and Quarto?

This book uses **Python** as its programming language and **Quarto** as its authoring and reporting tool, for a few reasons that matter to your future career:

- Python is one of the most widely used languages in data analytics and is free and open-source, meaning you can continue using it after graduation without needing a paid license.
- The pandas library (introduced in Chapter 4) is purpose-built for exactly the kind of tabular, spreadsheet-like data accountants work with every day.
- Quarto allows you to combine narrative writing, code, and output (tables, charts) into a single reproducible document — which mirrors how analytics work is increasingly delivered in practice, as a reproducible report rather than a static spreadsheet.

You'll get hands-on with both tools starting in Chapter 2.

1.9. Chapter Summary

- Accounting analytics combines accounting knowledge, data skills, and business judgment to support decisions.
- The shift toward analytics is driven by data volume, regulatory expectations, automation, and accessible tools.
- Analytics can be descriptive, diagnostic, predictive, or prescriptive — most of this course focuses on the first two, with an introduction to the third.
- A consistent workflow (define, obtain, clean, analyze, interpret, communicate) underlies most accounting analytics projects, and structures the rest of this book.
- These skills map directly onto current and evolving areas of accounting practice, including the CPA Exam.

1.10. Discussion Questions

1. Choose one accounting role (e.g., external auditor, tax accountant, FP&A analyst). How might accounting analytics change the day-to-day work of someone in that role over the next five years?
2. Think of a business decision (not necessarily accounting-related) you or your family have made recently. Which of the four analytics types (descriptive, diagnostic, predictive, prescriptive) best describes the thinking that went into it?
3. Why do you think regulators like the PCAOB have pushed auditors toward analyzing full populations of transactions rather than samples? What are the potential benefits and risks of this shift?

1.11. Exercises

1. Find one recent (last 12 months) news article or company disclosure describing the use of data analytics or AI in an accounting, audit, or finance context. Write a short (150-word) summary of what the technology does and what problem it solves.
2. Look up the current AICPA CPA Exam Blueprint for the Business Analysis and Reporting (BAR) discipline. Identify two specific data-analytics-related skills listed and briefly explain, in your own words, what they mean.

2. Python and Quarto Environment Setup

Learning Objectives

By the end of this chapter, you should be able to:

- Install Python, Quarto, and a code editor on your own computer
- Explain the difference between a Jupyter notebook (`.ipynb`), a plain Python script (`.py`), and a Quarto document (`.qmd`)
- Create, edit, and render a Quarto document containing Python code
- Render a Quarto book to both HTML and PDF
- Install and import the Python packages used throughout this book (pandas, numpy, matplotlib)

2.1. Why This Chapter Matters

Every chapter after this one assumes you can do three things: write Python code, run it, and see the results in a nicely formatted document. This chapter gets that pipeline working on your own laptop, once, so that the rest of the course is about *accounting analytics* rather than *fighting with software installation*.

If you get stuck at any point in this chapter, don't worry — environment setup is almost always the hardest part of a data analytics course, and it only gets easier from here.

2.2. The Three Tools You Need

Tool	What it does	Analogy
Python	The programming language that does the actual data work	The engine

Tool	What it does	Analogy
Quarto	Combines your writing, Python code, and output into one document (HTML, PDF, Word)	The publishing system
A Code Editor (Also called Integrated Development Environment(IDE))	Where you write and run your code (this book uses Positron, but VS Code works identically)	The workshop

You need all three installed before you can complete the exercises in this book.

2.3. Step 1: Install Python

You can install the python from https://www.python.org/downloads/. We recommend you use this link to install a specific version of python. Your professor might recommend a specific version that you should use. If you use macOS, please click `macOS` to install python suitable for mac.

In addition, you can also install Python via the **Anaconda** or **Miniconda** distribution, because it comes bundled with the data packages we'll use (pandas, numpy, matplotlib) and makes managing versions easier than a bare Python install.

1. Download Miniconda from https://docs.conda.io/en/latest/miniconda.html (choose the installer for your operating system).
2. Run the installer with default settings.
3. Confirm the install worked by opening a terminal and typing:

```
python --version
```

You should see something like `Python 3.11.x` (the exact version doesn't need to match this).

> **Connect to Practice**
>
> If your future employer uses a specific data platform (e.g., a cloud data warehouse, or a company-managed Python environment), the *concepts* you learn here transfer directly — only the installation step changes.

2.4. Step 2: Install Quarto

1. Go to https://quarto.org/docs/get-started/ and download the installer for your operating system.
2. Run the installer with default settings.
3. Confirm the install worked:

```
quarto --version
```

Since we'll render to PDF as well as HTML, also install a LaTeX distribution — Quarto includes a lightweight one you can install with a single command:

```
quarto install tinytex
```

This only needs to be done once per computer.

2.5. Step 3: Install a Code Editor

This book's examples use **Positron**, which can be downloaded from https://positron.posit.co/download.html. Positron is a code editor built specifically for data science that will feel very familiar if you've used VS Code before — the two share the same underlying editing engine, so keyboard shortcuts, extensions, and layout all work the same way. (If your instructor prefers plain VS Code, everything in this chapter applies identically.)

Figure 2.1 shows the general layout you'll be working in for the rest of this book: a file explorer on the left, an editor pane in the middle where you write your `.qmd` files, a terminal at the bottom where you run commands like `quarto render`, and a Source Control panel for Git (covered separately if your instructor uses version control in this course).

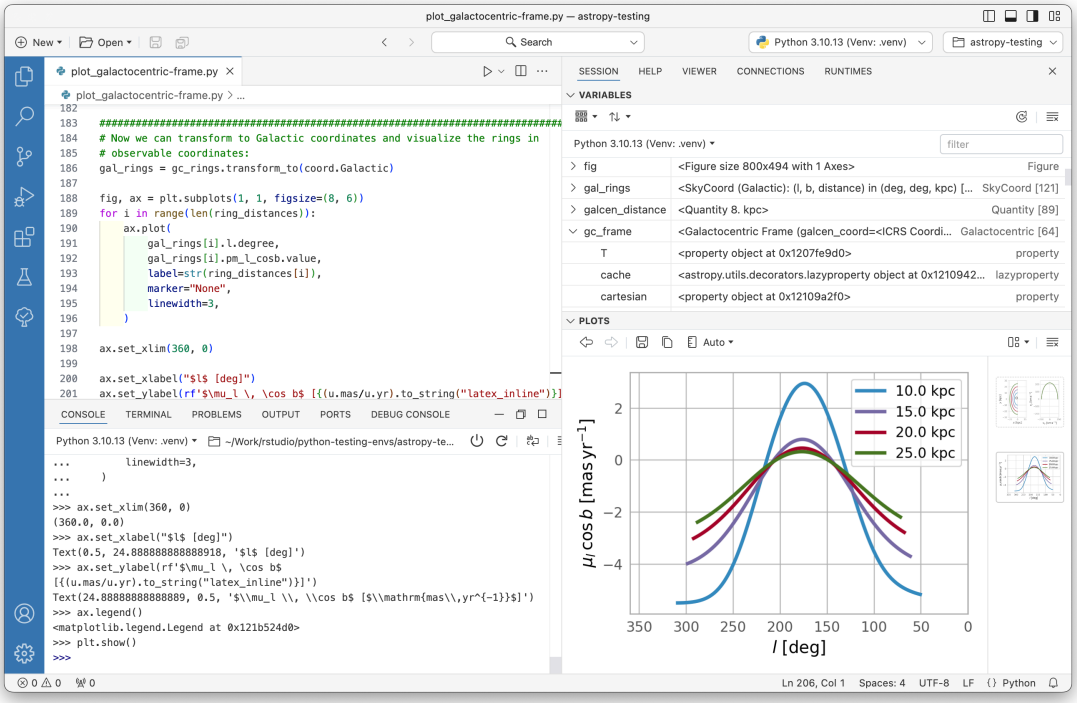


Figure 2.1.: A typical code-editor layout: file explorer, editor pane, session, and plots

Once installed, open your project folder in Positron/VS Code using **File □ Open Folder**, and select the folder containing your `_quarto.yml` file.

2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document

As you start doing accounting analytics work, you'll encounter three common file types for writing Python. It's worth understanding the difference early, since this book uses all three at different points.

2.6.1. 1. A Plain Python Script (`.py`)

A `.py` file contains only code — no narrative text, no formatted output. You run the whole file at once (or line by line) and see output printed to a terminal or console. This is the format used for production code, reusable functions, or automated jobs that don't need explanation baked in.

```python
# analysis.py
import pandas as pd

df = pd.read_csv("data/trial_balance.csv")
print(df.head())
```

2.6.2. 2. A Jupyter Notebook (`.ipynb`)

A Jupyter notebook mixes code and narrative text in a sequence of **cells**, and shows output (tables, charts) directly underneath the code that produced it. This makes notebooks excellent for exploratory analysis — trying things out, looking at results immediately, and iterating.

Figure 2.2 shows a simplified notebook: a markdown cell explaining what's about to happen, a code cell that loads data, and the output (a data table) displayed directly below it.

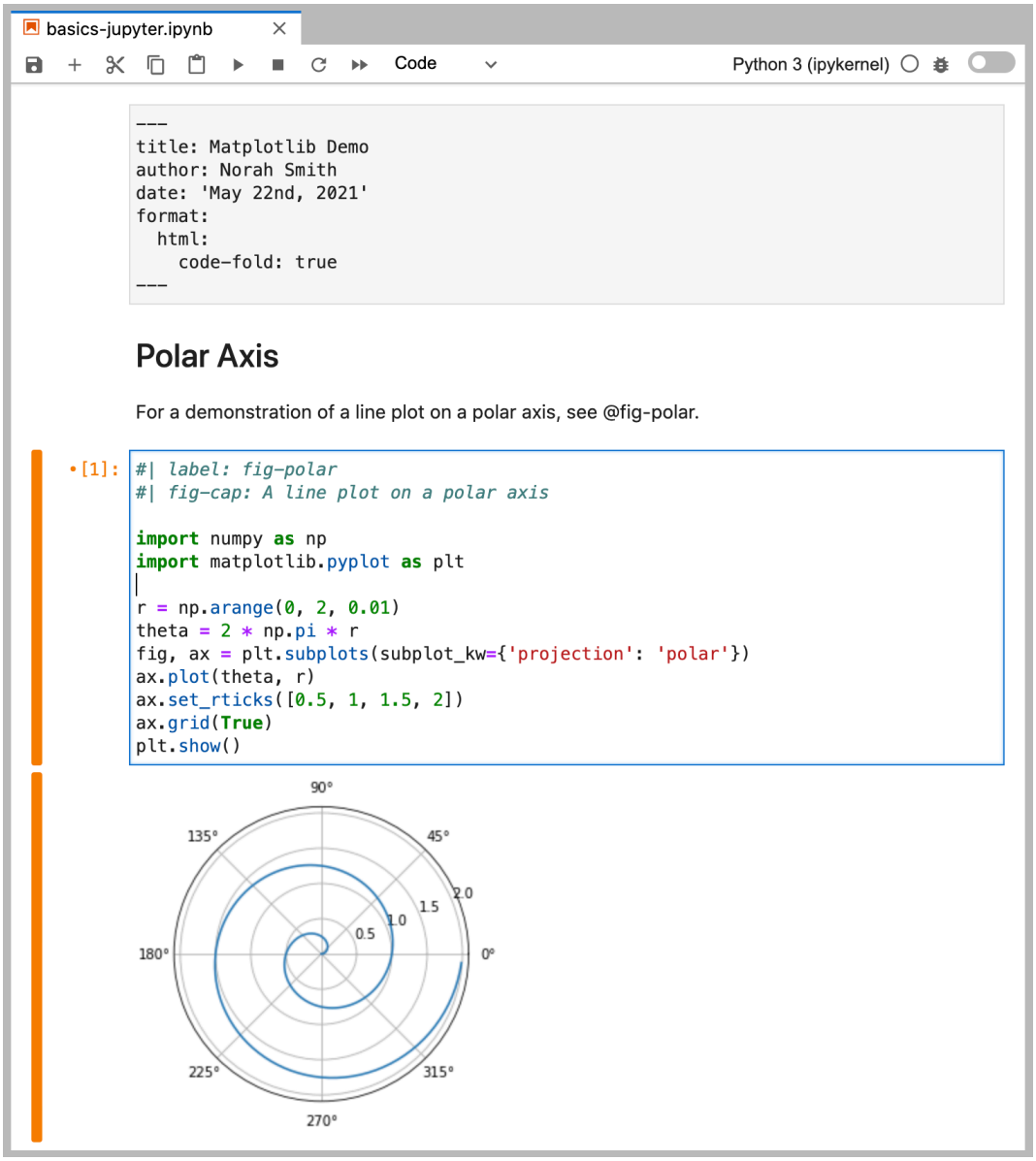


Figure 2.2.: A Jupyter notebook combines markdown cells, code cells, and their output in one scrollable document

You can open and run `.ipynb` files directly in Positron/VS Code, the same as in classic Jupyter.

2.6.3. 3. A Quarto Document (`.qmd`)

A Quarto document looks a lot like a notebook conceptually — narrative text plus embedded, runnable code — but it's a **plain text file** (easier to track with Git) that can render to multiple polished output formats: HTML, PDF, Word, and more. This is the format used for every chapter in this book.

A Quarto document has three parts:

1. A **YAML header** at the top (between `---` lines) with metadata like the title
2. **Narrative text** written in Markdown
3. **Code chunks**, marked with three backticks and curly braces

````
---
title: "Example Chapter"
---

## Analyzing the Trial Balance

Here we load the trial balance and preview the first few rows.

::: {.cell execution_count=1}
``` {.python .cell-code}
import pandas as pd

df = pd.read_csv("data/trial_balance.csv")
df.head()
```

::: {.cell-output .cell-output-display execution_count=1}
```{=html}
<div>
<style scoped>
    .dataframe tbody tr th:only-of-type {
        vertical-align: middle;
    }

    .dataframe tbody tr th {
        vertical-align: top;
    }

    .dataframe thead th {
        text-align: right;
    }
</style>
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
````
````

```
      <th>account</th>
      <th>account_type</th>
      <th>debit</th>
      <th>credit</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>Cash</td>
      <td>Asset</td>
      <td>12500</td>
      <td>0</td>
    </tr>
    <tr>
      <th>1</th>
      <td>Accounts Receivable</td>
      <td>Asset</td>
      <td>8200</td>
      <td>0</td>
    </tr>
    <tr>
      <th>2</th>
      <td>Inventory</td>
      <td>Asset</td>
      <td>15300</td>
      <td>0</td>
    </tr>
    <tr>
      <th>3</th>
      <td>Prepaid Insurance</td>
      <td>Asset</td>
      <td>1200</td>
      <td>0</td>
    </tr>
    <tr>
      <th>4</th>
      <td>Equipment</td>
      <td>Asset</td>
      <td>42000</td>
      <td>0</td>
```

```
    </tr>
  </tbody>
</table>
</div>
```
:::
:::

The output above shows account balances before adjusting entries.

> **!** Which format does this book use?
>
> Every chapter in *this* book is written as a `.qmd` file, since we want a single, polished, professional document (in both HTML and PDF) at the end of the semester — not a folder of separate notebooks. However, your instructor may ask you to *practice and experiment* in a Jupyter notebook before copying finished code into your chapter or assignment `.qmd` file. Think of the notebook as your scratch paper and the `.qmd` file as your final submission.

2.7. Your First Quarto Document

Let's create a small Quarto document from scratch to make sure everything works end to end.

1. **Create a new file** called `test.qmd` in your project folder with the following content:

```
---
title: "My First Quarto Document"
format: html
---

## Testing My Setup

::: {.cell execution_count=2}
```{.python .cell-code}
import pandas as pd

data = {"account": ["Cash", "Accounts Receivable", "Inventory"],
        "balance": [12500, 8200, 15300]}
```
```

````
df = pd.DataFrame(data)
df
```

::: {.cell-output .cell-output-display execution_count=2}
```{=html}
<div>
<style scoped>
    .dataframe tbody tr th:only-of-type {
        vertical-align: middle;
    }

    .dataframe tbody tr th {
        vertical-align: top;
    }

    .dataframe thead th {
        text-align: right;
    }
</style>
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>account</th>
      <th>balance</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>Cash</td>
      <td>12500</td>
    </tr>
    <tr>
      <th>1</th>
      <td>Accounts Receivable</td>
      <td>8200</td>
    </tr>
    <tr>
      <th>2</th>
````

```
      <td>Inventory</td>
      <td>15300</td>
    </tr>
  </tbody>
</table>
</div>
```
:::
:::


```
If you can see a table above with three accounts, your Python and
 ↪  Quarto setup is working correctly.
```

2. Render it. In the terminal, run:

```
quarto render test.qmd
```

3. Open the result. This creates `test.html` in the same folder — open it in your web browser and confirm you can see the table.

If this worked, your environment is fully set up. If you got an error, check the Troubleshooting section at the end of this chapter before moving on.

2.8. Rendering This Book□s Project (HTML and PDF)

Your course textbook project is set up as a **Quarto book** (multiple chapters combined into one publication), which is slightly different from rendering a single file. From the terminal, inside your project folder (the one containing `_quarto.yml`):

```
quarto render
```

This single command renders *every* chapter listed in `_quarto.yml` and produces both formats we configured:

- `_book/index.html` — an HTML website you can open in a browser or host online
- a single PDF file inside `_book/` — a downloadable, printable version of the entire book

Figure 2.3 summarizes this process: your `.qmd` source files go in, `quarto render` executes the Python code and assembles the document, and both an HTML and a PDF version come out the other side from the *same source file* — you never have to write your content twice.

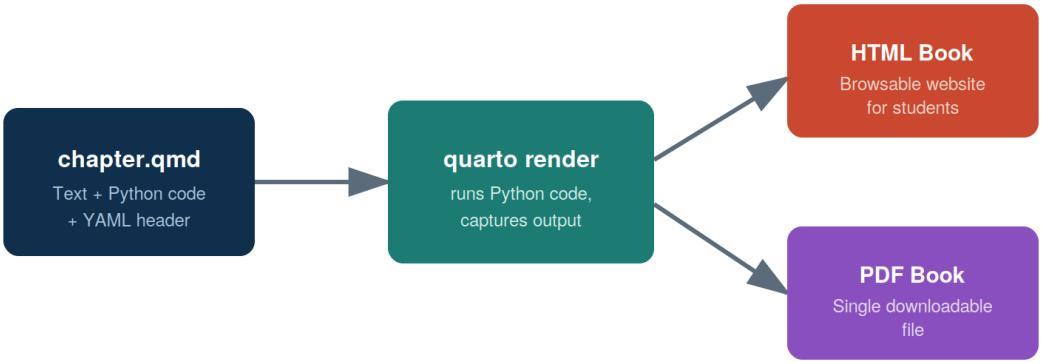


Figure 2.3.: The Quarto rendering workflow: one source document, two polished output formats

To preview your book live while editing (auto-refreshing as you save changes), use:

```
quarto preview
```

2.9. Installing the Python Packages You□ll Need

This book relies on a small set of Python packages. If you installed Python via Anaconda/Miniconda, most of these are likely already available. To install (or confirm) them, run:

```
pip install pandas numpy matplotlib jupyter
```

Confirm they're installed correctly by running this in a `.qmd` code chunk or a notebook cell:

````
```{python}
import pandas as pd
import numpy as np
import matplotlib
```
````

```
print(pd.__version__)
print(np.__version__)
print(matplotlib.__version__)
```

If this runs without errors and prints three version numbers, you're ready for Chapter 3.

2.10. Organizing Your Project Folder

As your book (or your homework project) grows, keeping files organized will save you real time. Figure 2.4 shows the recommended structure, which matches how this book's own project folder is organized: one `.qmd` file per chapter, a shared `images/` folder for figures, and the configuration file (`_quarto.yml`) at the top level.

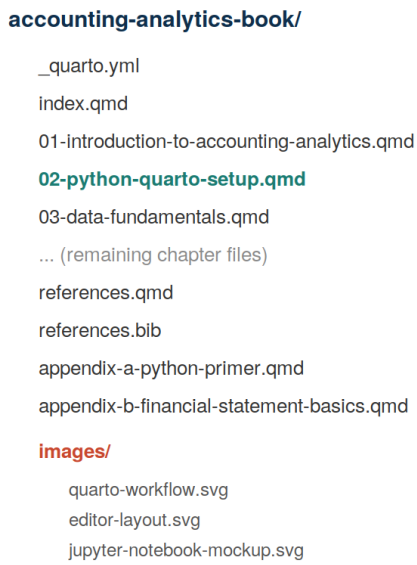


Figure 2.4.: Recommended project folder structure: one file per chapter, plus a shared images folder

A couple of habits worth building now:

- Keep all figures in a single `images/` folder rather than scattering them across the project — it makes the project easier to navigate and easier to back up.
- Give files and images descriptive, lowercase, hyphenated names (e.g., `revenue-by-quarter.png`, not `Figure1_FINAL_v2.png`) — this avoids broken links when you rename things later and matches conventions used in real data projects.

2.11. Troubleshooting Common Setup Issues

⚠ "quarto: command not found"

This usually means Quarto wasn't added to your system's PATH during installation. Try closing and reopening your terminal (and your code editor) after installing Quarto — this alone fixes the issue most of the time. If it persists, reinstall Quarto and confirm you allowed it to modify your PATH during setup.

⚠ "ModuleNotFoundError: No module named 'pandas' "

Python can't find the package because it isn't installed in the environment your editor is using. Run `pip install pandas` in the same terminal your editor uses, and confirm you're using the Python interpreter you expect (in Positron/VS Code, check the interpreter shown in the bottom status bar).

⚠ PDF rendering fails but HTML works fine

This almost always means the LaTeX installation is missing or incomplete. Run `quarto install tinytex` and try again. If it still fails, re-render with `quarto render --to pdf --log render.log` and check the log file for the specific missing package.

2.12. Chapter Summary

- Python, Quarto, and a code editor (Positron or VS Code) are the three tools needed for this course, and each is installed once at the start of the semester.
- A `.py` script, a `.ipynb` notebook, and a `.qmd` Quarto document all run Python code, but serve different purposes: scripts for production code, notebooks for exploration, and Quarto documents for polished, reproducible reports — which is why this book is written entirely in `.qmd` files.
- `quarto render` turns your `.qmd` source into both HTML and PDF from a single source of truth.
- Keeping a consistent project folder structure (chapters at the top level, a shared `images/` folder) will save you time throughout the semester.

2.13. Exercises

1. Complete the "Your First Quarto Document" walkthrough in this chapter. Take a screenshot of your rendered HTML output showing the table, and submit it as instructed by your course syllabus.
2. In your own words (2–3 sentences), explain when you would choose to use a Jupyter notebook versus a Quarto document for a piece of analytical work.
3. Intentionally introduce one small error into your `test.qmd` file (e.g., misspell `pandas` as `panadas`), render it, and copy the error message you get. Then fix it and re-render successfully. This exercise is designed to make error messages feel less intimidating early in the course.

3. Data Fundamentals for Accountants

Learning Objectives

By the end of this chapter, you should be able to:

- Explain tidy data principles and identify whether a table is tidy or messy
- Identify the four levels of measurement and match them to common accounting variables
- Recognize the main data types encountered in accounting data (numeric, categorical, date/time, text, boolean)
- Describe the structure and relationship between the chart of accounts, general ledger, trial balance, and subledgers
- Explain the difference between transaction-level and summarized data, and why analytics often needs the former
- Identify common data quality issues in real-world accounting datasets

3.1. Why Data Structure Matters Before You Start Coding

It's tempting to jump straight into pandas and start writing code — and we will, starting next chapter. But most real-world accounting analytics problems are **data problems**, not coding problems. A well-written line of Python code applied to poorly structured or misunderstood data will still give you a wrong (or misleading) answer.

This chapter builds the conceptual foundation you need before touching code: what "well-structured" data looks like, what kinds of data you'll encounter, how core accounting datasets are organized, and what tends to go wrong with real data. Think of this as learning to read a map before you start driving.

3.2. Tidy Data Principles

Tidy data is a simple, widely-used standard for organizing tabular data so that it's easy to analyze. A dataset is tidy when:

1. **Each variable forms a column.**
2. **Each observation forms a row.**
3. **Each type of observational unit forms a table.**

This sounds abstract, so let's look at an example. Suppose you have quarterly revenue by product line.

Messy (wide) version — each quarter is its own column:

product_line	Q1	Q2	Q3	Q4
Consulting	42,000	45,500	47,200	50,100
Software	61,000	58,000	63,500	70,200

This looks fine to read as a human, and it's a completely normal way to present numbers in a report. But it's **not tidy**: "quarter" is a variable that's been spread across four columns instead of living in a single column.

Tidy (long) version — the same data, restructured:

product_line	quarter	revenue
Consulting	Q1	42,000
Consulting	Q2	45,500
Consulting	Q3	47,200
Consulting	Q4	50,100
Software	Q1	61,000
Software	Q2	58,000
Software	Q3	63,500
Software	Q4	70,200

> **Connect to Practice**
>
> Tidy data isn't just an academic preference. In Chapter 4, you'll see that pandas operations — filtering, grouping, plotting — are dramatically easier when your data is tidy. A wide table like the "messy" example above requires extra reshaping steps before you can, say, plot revenue trends across all product lines on one chart.

You don't need to memorize a rulebook here — the practical test is: *if I wanted to filter, group, or plot by any single variable, is that variable sitting in one clearly labeled column?* If yes, you're likely looking at tidy data.

3.3. Levels of Measurement

Before you calculate anything — an average, a sum, a comparison — it's worth asking: *what kind of variable is this, and what operations actually make sense on it?* This is the idea behind **levels of measurement**, a classification used throughout statistics.

There are four levels, each allowing more mathematical operations than the last:

Level	Description	Accounting Example	Meaningful Operations
Nominal	Categories with no inherent order	Account type (Asset, Liability, Equity), Department, Vendor name	Count, mode, group by
Ordinal	Ordered categories, but differences between levels aren't meaningful	Credit risk rating (Low/Medium/High), Audit opinion severity	Count, mode, median, ranking
Interval	Meaningful differences, but no true zero	Fiscal year, Credit score (in some scales)	Addition/subtraction, mean, but not meaningful ratios
Ratio	True zero point; all arithmetic is meaningful	Account balance, Transaction amount, Units sold	Mean, sum, ratios, all standard arithmetic

! Why This Matters in Practice

It's easy to make a technically-valid-looking calculation that is conceptually meaningless. For example: averaging a customer's credit risk rating of "Low, Medium, High" by assigning them 1, 2, 3 and computing a mean of 1.8 *looks* like a real number, but "1.8" doesn't correspond to any real risk category — ordinal data doesn't support that kind of averaging. Similarly, summing account numbers from your chart of accounts (nominal data, even though it's stored as a number) produces a number with no accounting meaning at all.
We'll return to this idea in Chapter 5, where choosing the right summary statistic depends directly on a variable's level of measurement.

3.4. Data Types You□ll Encounter

Related to levels of measurement, but more about how data is *stored and represented* in a dataset, are data types. In accounting data, you'll regularly see:

- **Numeric □ continuous**: dollar amounts (account balances, transaction amounts) — ratio-level, all arithmetic makes sense
- **Numeric □ discrete**: counts (units sold, number of invoices) — ratio-level, but only whole numbers make sense
- **Categorical**: account type, department, cost center, vendor — typically nominal, sometimes ordinal (e.g., risk tier)
- **Date/time**: transaction date, fiscal period, posting date — deserves special attention, covered next
- **Text/string**: transaction descriptions, memo fields, vendor names — often messy and inconsistently formatted
- **Boolean/flag**: `is_reversed`, `is_intercompany`, `is_reconciled` — true/false indicators, nominal with only two categories

3.4.1. A Special Note on Dates

Dates cause more headaches in accounting data than almost any other data type, for a few reasons:

- They can be stored as **text** (`"01/15/2025"`) instead of an actual date type, which prevents sorting and date arithmetic from working correctly
- **Format ambiguity**: is `03/04/2025` March 4th or April 3rd? This depends on regional conventions (US vs. most of the rest of the world) and is a common source of silent errors
- **Fiscal year vs. calendar year**: a company with a fiscal year ending June 30 will label transactions in "FY2025" that occurred in *calendar* 2024

We'll handle dates properly with pandas in Chapter 4 — for now, just know that a date column that "looks fine" when you eyeball a spreadsheet is one of the most common sources of hidden bugs in accounting analytics.

3.5. Core Accounting Data Structures

Most accounting analytics work touches one or more of four related data structures. Figure 3.1 shows how they connect.

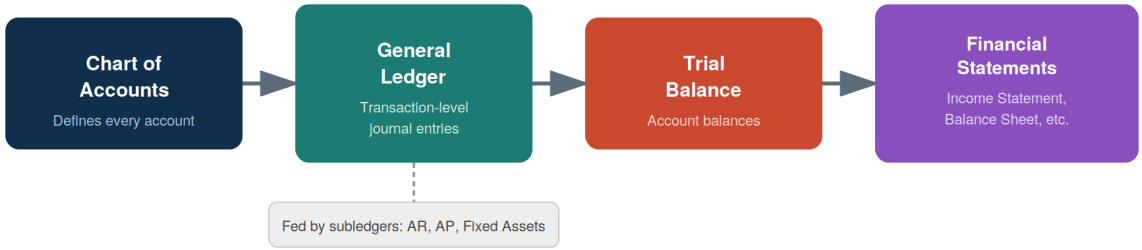


Figure 3.1.: How core accounting data structures relate: the chart of accounts defines every account, transactions flow through the general ledger (fed by subledgers), and roll up into a trial balance and ultimately the financial statements

3.5.1. Chart of Accounts (COA)

The chart of accounts is the master list of every account a company uses — essentially a data dictionary for the rest of the accounting system. A typical COA entry includes an account number, account name, and account type (Asset, Liability, Equity, Revenue, Expense).

3.5.2. General Ledger (GL)

The general ledger contains **transaction-level detail** — every individual journal entry, each with a debit and a credit side, a date, a description, and (typically) the account(s) affected. This is the most granular, information-rich accounting dataset, and the one most audit and fraud analytics work is performed on.

3.5.3. Subsidiary Ledgers

Subsidiary ledgers (Accounts Receivable, Accounts Payable, Fixed Assets, Payroll, etc.) hold detailed records for a specific area that then roll up into summary entries in the general ledger. For example, the AR subledger tracks every individual customer invoice and payment, but the GL might only see a periodic summary entry.

3.5.4. Trial Balance

The trial balance is a **summarized snapshot**: the ending balance of every account in the chart of accounts, as of a point in time. This is the level of detail you worked with in Chapter 2 — one

row per account, not one row per transaction. A trial balance is a checkpoint used to confirm that total debits equal total credits before financial statements are prepared.

3.6. Transaction-Level vs. Summarized Data

This distinction matters more than it might first appear:

- A **trial balance** can look completely normal — accounts balanced, nothing unusual — while individual **transactions** underneath it hide errors, unusual patterns, or fraud.
- For example, imagine a "Travel Expense" account with a perfectly reasonable ending balance of $45,000 for the year. That summary number gives no indication that $38,000 of it came from a single suspicious transaction posted on a Sunday by an employee who doesn't normally have travel expenses.

This is exactly why audit analytics (Chapter 8) and fraud detection (Chapter 9) typically require **transaction-level general ledger data**, not just trial balances — the interesting patterns often only show up at the most granular level.

3.7. Common Data Quality Issues in Accounting Data

Real accounting data is rarely clean. Some of the most common issues you'll encounter:

- **Missing values**: blank department codes, missing dates, unpopulated memo fields
- **Duplicate entries**: the same transaction entered twice, sometimes with slightly different formatting
- **Inconsistent formatting**: dates stored as text in different formats, currency values stored with `$` symbols or commas (which prevents numeric calculations until cleaned), inconsistent capitalization in account or vendor names (`"Cash"` vs. `"cash"` vs. `"CASH "` with trailing whitespace)
- **Inconsistent categorical labels**: the same department entered as `"HR"`, `"H.R."`, and `"Human Resources"` across different rows
- **Outliers vs. errors**: an unusually large transaction might be a legitimate one-time event (a building purchase) or a data entry error (an extra zero) — these require judgment, not just a statistical rule, to tell apart

> ⚠ A Realistic Messy Example
>
> Imagine you received this general ledger excerpt exactly as extracted from a client's system:

entry_date	account	debit	department
01/15/2025	Cash	$4,500.00	Sales
1-15-2025	cash	4500	sales
01/15/2025	Cash	4,500	
01/32/2025	Cash	45,000	Sales

This tiny sample already contains: inconsistent date formats, inconsistent account name capitalization, inconsistent currency formatting, a missing department value, a likely duplicate entry, an impossible date (`01/32/2025`), and a value that may be a data-entry error (an extra zero turning $4,500 into $45,000). Spotting these issues *before* you calculate anything is a core skill — we'll clean data like this hands-on in Chapter 4.

3.8. A Preview Look at a Real Dataset

To connect these ideas to something concrete, look at the sample transaction-level general ledger below (`general_ledger_sample.csv`), covering two weeks of journal entries for a small company. Each journal entry (`entry_id`) has at least two rows — one debit, one credit — matching how double-entry accounting actually works.

entry_id	entry_date	account	account_type	description	debit	credit	department	posted_by
JE1001	2025-01-03	Cash	Asset	Customer payment received	4500	0	Sales	jsmith
JE1001	2025-01-03	Accounts Receivable	Asset	Customer payment received	0	4500	Sales	jsmith
JE1002	2025-01-05	Inventory	Asset	Purchase of merchandise	7200	0	Operations	mchen
JE1002	2025-01-05	Accounts Payable	Liability	Purchase of merchandise	0	7200	Operations	mchen

A few things worth noticing just from visual inspection, before we write a single line of pandas

code in Chapter 4:

- This table is **tidy**: each column is a single variable (date, account, debit, credit, etc.), and each row is one line of a journal entry.
- The **level of measurement** varies by column: `debit/credit` are ratio-level (arithmetic is meaningful), `account_type` and `department` are nominal (categories only), `entry_date` is a date type requiring careful handling.
- This is **transaction-level** data — far more granular than the trial balance from Chapter 2 — which is exactly the kind of dataset audit and fraud analytics techniques (Chapters 9-10) are designed to work with.

We'll load and work with this exact file in Chapter 4.

3.9. Chapter Summary

- Tidy data — one variable per column, one observation per row, one observational unit per table — makes analysis dramatically easier and is the standard this book (and pandas) assumes.
- Levels of measurement (nominal, ordinal, interval, ratio) determine which calculations are actually meaningful for a given variable — averaging an ordinal rating or summing a nominal code produces numbers without real meaning.
- Accounting data typically falls into a few types: numeric, categorical, date/time, text, and boolean — each with its own quirks, dates especially.
- The chart of accounts, general ledger, subledgers, and trial balance are related but distinct structures, moving from transaction-level detail to summarized snapshots.
- Analytics work — especially audit and fraud analytics — usually requires transaction-level data, since summarized data can hide the patterns that matter most.
- Real accounting data commonly has quality issues: missing values, duplicates, inconsistent formatting, and ambiguous outliers, all of which need to be identified before analysis begins.

3.10. Discussion Questions

1. Look back at the "wide" quarterly revenue example early in this chapter. Why might a company's finance team *prefer* the wide format for a printed report, even though it's not tidy for analysis purposes?
2. Give one example (not from this chapter) of an accounting variable at each level of measurement: nominal, ordinal, interval, and ratio.
3. Why might a company's trial balance appear completely normal in a given month, even if fraudulent transactions occurred during that period?

3.11. Exercises

1. Take the "messy" wide revenue table from this chapter and manually rewrite it in tidy (long) format on paper or in a spreadsheet.
2. Using the messy general ledger excerpt in the callout box above, list every data quality issue you can find, and for each one, briefly describe how you would investigate or resolve it if this were real client data.
3. Download `general_ledger_sample.csv` (provided alongside this chapter) and open it in a spreadsheet program. Identify: (a) how many unique journal entries it contains, (b) how many unique departments appear, and (c) whether total debits equal total credits.

Part II.

Part II: Core Data Skills

4. Data Wrangling with `Pandas` and `Polars`

Learning Objectives

By the end of this chapter, you should be able to:

- Understand what data wrangling is and its importance
- Import tabular accounting data (CSV) into both pandas and polars DataFrames
- Select, filter, and sort accounting data using both libraries
- Clean messy accounting data: fix currency formatting, inconsistent text, bad dates, and missing values
- Merge/join accounting tables and explain the difference between inner and left joins
- Reshape data between wide and long formats
- Group and aggregate transaction-level data into account-level summaries
- Explain lazy vs. eager evaluation in polars, and why it matters for large accounting datasets

4.1. What is Data Wrangling?

Data wrangling—also called *data cleaning*, *data remediation*, or *data munging*—refers to a variety of processes designed to transform raw data into more readily used formats. The exact methods differ from project to project depending on the data you're leveraging and the goal you're trying to achieve (Stobierski, 2021).

Some examples of data wrangling include:

- Merging multiple data sources into a single dataset for analysis
- Identifying gaps in data (for example, empty cells in a spreadsheet) and either filling or deleting them
- Deleting data that's either unnecessary or irrelevant to the project you're working on

- Identifying extreme outliers in data and either explaining the discrepancies or removing them so that analysis can take place

Data wrangling can be a manual or automated process. In scenarios where datasets are exceptionally large, automated data cleaning becomes a necessity. In organizations that employ a full data team, a data scientist or other team member is typically responsible for data wrangling. In smaller organizations, non-data professionals are often responsible for cleaning their data before leveraging it (Stobierski, 2021).

4.1.1. Data Wrangling Steps and Techniques

Each data project requires a unique approach to ensure its final dataset is reliable and accessible. That being said, several processes typically inform the approach. These are commonly referred to as data wrangling steps or activities. Figure 4.1 shows the steps in Data Warngling (Stobierski, 2021).



Figure 4.1.: Steps in Data Wrangling

4.1.1.1. Discovery

Discovery refers to the process of familiarizing yourself with data so you can conceptualize how you might use it. You can liken it to looking in your refrigerator before cooking a meal to see what ingredients you have at your disposal.

During discovery, you may identify trends or patterns in the data, along with obvious issues, such as missing or incomplete values that need to be addressed. This is an important step, as it will inform every activity that comes afterward (Stobierski, 2021).

4.1.1.2. Structuring

Raw data is typically unusable in its raw state because it's either incomplete or misformatted for its intended application. Data structuring is the process of taking raw data and transforming it to be more readily leveraged. The form your data takes will depend on the analytical model you use to interpret it (Stobierski, 2021).

4.1.1.3. Cleaning

Data cleaning is the process of removing inherent errors in data that might distort your analysis or render it less valuable. Cleaning can come in different forms, including deleting empty cells or rows, removing outliers, and standardizing inputs. The goal of data cleaning is to ensure there are no errors (or as few as possible) that could influence your final analysis. Identifying and removing any bad data greatly impacts the rest of the wrangling processes (Stobierski, 2021).

4.1.1.4. Enriching

Once you understand your existing data and have transformed it into a more usable state, you must determine whether you have all of the data necessary for the project at hand. If not, you may choose to enrich or augment your data by incorporating values from other datasets. For this reason, it's important to understand what other data is available for use (Stobierski, 2021).

If you decide that enrichment is necessary, you need to repeat the steps above for any new data.

4.1.1.5. Validating

Data validation refers to the process of verifying that your data is both consistent and of a high enough quality. During validation, you may discover issues you need to resolve or conclude that your data is ready to be analyzed. Validation is typically achieved through various automated processes and requires programming (Stobierski, 2021).

4.1.1.6. Publishing

Once your data has been validated, you can publish it. This involves making it available to others within your organization for analysis. The format you use to share the information—such as a written report or electronic file—will depend on your data and the organization's goals (Stobierski, 2021).

4.1.2. Data Wrangling vs. Data Cleaning

Despite the terms being used interchangeably, data wrangling and data cleaning are two different processes. It's important to make the distinction that data cleaning is a critical step in the data wrangling process to remove inaccurate and inconsistent data. Meanwhile, data-wrangling is the overall process of transforming raw data into a more usable form (Stobierski, 2021). Figure 4.2 compares both.

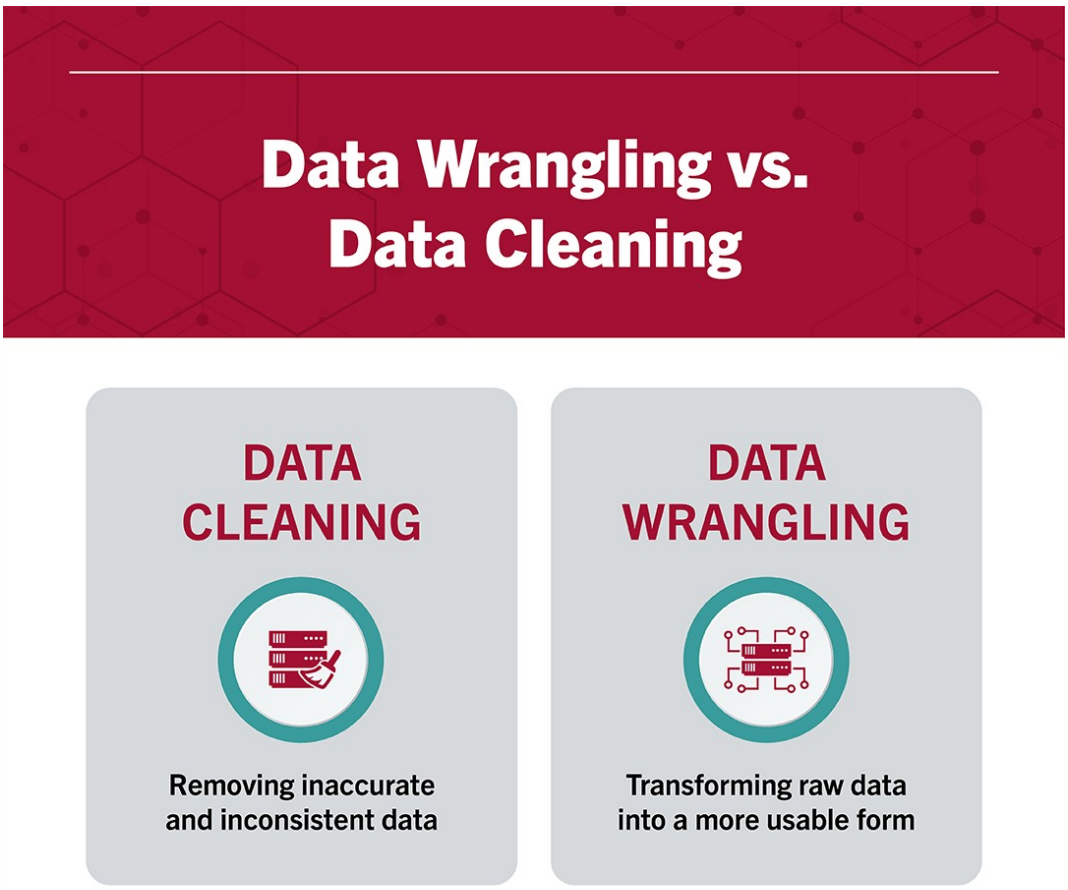


Figure 4.2.: Data Wrangling vs Data Cleaning

4.1.3. Importance of Data Wrangling

Any analyses an accountant or CPA performs will ultimately be constrained by the data that informs them. If data is incomplete, unreliable, or faulty, then analyses will be too—diminishing the value of any critical insights gleaned.

Data wrangling seeks to remove that risk by ensuring data is in a reliable state before it's analyzed and leveraged. This makes it a critical part of the analytical process.

It's important to note that data wrangling can be time-consuming and taxing on resources, particularly when done manually. This is why many organizations institute policies and best practices that help employees streamline the data cleanup process—for example, requiring that data include certain information or be in a specific format before it's uploaded to a database.

For this reason, it's vital to understand the steps of the data wrangling process and the negative outcomes associated with incorrect or faulty data (Stobierski, 2021).

4.2. Two Tools, One Set of Concepts

This chapter teaches data wrangling using **two** Python libraries side by side: **pandas**, the long-standing standard for tabular data analysis in Python, and **polars**, a newer library built for speed on large datasets.

The good news: the underlying *concepts* — filtering, joining, grouping, reshaping — are identical no matter which library you use. Only the syntax changes. Learning both at once actually reinforces the concepts, because you'll see the same logic expressed two different ways.

	pandas	polars
Maturity	Released 2008; the long-standing standard	Released 2020; newer, rapidly growing
Performance	Good for small-to-medium data; single-threaded by default	Built for speed; multi-threaded, memory-efficient
Syntax style	Label-based indexing (`df[df["x"] > 0]`)	Expression-based API (`pl.col("x") > 0`)
Industry adoption (accounting/audit)	Still the dominant tool in most firms and tutorials	Growing adoption, especially for large transaction datasets
Lazy evaluation	Not built in (see Dask/Modin for scaling, not covered here)	Built in — covered in this chapter

> **Connect to Practice**
>
> You are very likely to encounter pandas in an internship or entry-level role today. Polars is worth learning now because its adoption is accelerating quickly, especially anywhere teams work with large general ledger exports (multi-million-row datasets), where its performance advantage becomes very noticeable.

We'll use two datasets you already have from previous chapters — `general_ledger_sample.csv` and `trial_balance.csv` — plus two new ones for this chapter: `chart_of_accounts.csv` (for joins) and `messy_general_ledger.csv` (for cleaning practice).

4.3. Reading Data In

Let's start by loading the general ledger sample into both libraries and inspecting its structure.

4.4. pandas

```
import pandas as pd
import polars as pl

# pandas
gl_pd = pd.read_csv("data/general_ledger_sample.csv")
print(gl_pd.shape)
print(gl_pd.dtypes)
gl_pd.head(3)
```

```
(20, 9)
entry_id          str
entry_date        str
account           str
account_type      str
description       str
debit           int64
credit          int64
department        str
posted_by         str
dtype: object
```

	entry_id	entry_date	account	account_type	description	debit	cred
0	JE1001	2025-01-03	Cash	Asset	Customer payment received	4500	0
1	JE1001	2025-01-03	Accounts Receivable	Asset	Customer payment received	0	450
2	JE1002	2025-01-05	Inventory	Asset	Purchase of merchandise	7200	0

4.5. polars

```
# polars
gl_pl = pl.read_csv("data/general_ledger_sample.csv")
print(gl_pl.shape)
print(gl_pl.dtypes)
gl_pl.head(3)
```

```
(20, 9)
[String, String, String, String, String, Int64, Int64, String, String]
```

entry_id str	entry_date str	account str	account_type str	description str	debit i64	cr i6
"JE1001"	"2025-01-03"	"Cash"	"Asset"	"Customer payment received"	4500	0
"JE1001"	"2025-01-03"	"Accounts Receivable"	"Asset"	"Customer payment received"	0	45
"JE1002"	"2025-01-05"	"Inventory"	"Asset"	"Purchase of merchandise"	7200	0

Both libraries correctly infer that `debit` and `credit` are numeric and everything else is text. You can get quick summary statistics with `.describe()` in both libraries:

4.6. pandas

```
gl_pd.describe()
```

	debit	credit
count	20.000000	20.000000
mean	3012.500000	3012.500000
std	4296.873864	4296.873864
min	0.000000	0.000000

	debit	credit
25%	0.000000	0.000000
50%	425.000000	425.000000
75%	5375.000000	5375.000000
max	15400.000000	15400.000000

4.7. polars

```
gl_pl.describe()
```

statistic str	entry_id str	entry_date str	account str	account_type str	description str	debit f64
”count”	”20”	”20”	”20”	”20”	”20”	20.0
”null_count”	”0”	”0”	”0”	”0”	”0”	0.0
”mean”	null	null	null	null	null	3012.5
”std”	null	null	null	null	null	4296.873
”min”	”JE1001”	”2025-01-03”	”Accounts Payable”	”Asset”	”Biweekly payroll”	0.0
”25%”	null	null	null	null	null	0.0
”50%”	null	null	null	null	null	850.0
”75%”	null	null	null	null	null	5100.0
”max”	”JE1010”	”2025-01-31”	”Utilities Expense”	”Revenue”	”Vendor payment”	15400.0

4.7.1. Reading with an Explicit Schema

By default, both libraries *guess* each column’s data type. This usually works, but for accounting data it’s good practice to be explicit — especially for ID columns that look numeric but should stay text (an `entry_id` of `1001` should never be treated as a number you could accidentally average).

4.8. pandas

```
# pandas: force entry_id to stay text
gl_pd2 = pd.read_csv("data/general_ledger_sample.csv",
 ↪  dtype={"entry_id": str})
```

```
print(gl_pd2["entry_id"].dtype)
```

```
str
```

4.9. polars

```
# polars: same idea, using schema_overrides
gl_pl2 = pl.read_csv(
    "data/general_ledger_sample.csv",
    schema_overrides={"entry_id": pl.Utf8}
)
print(gl_pl2["entry_id"].dtype)
```

```
String
```

> **Connect to Practice**
>
> Excel files are just as common as CSVs in accounting practice. Both libraries can read them directly: `pd.read_excel("file.xlsx")` and `pl.read_excel("file.xlsx")` (polars requires the `fastexcel` package, installable with `pip install fastexcel`).

4.10. Selecting, Filtering, and Sorting

4.10.1. Filtering on a Single Condition

Let's find every transaction with a debit over $5,000 — a common first step in transaction testing.

4.11. pandas

```
# pandas
large_pd = gl_pd[gl_pd["debit"] > 5000]
large_pd[["entry_id", "account", "debit"]]
```

	entry_id	account	debit
2	JE1002	Inventory	7200
4	JE1003	Salaries Expense	15400
6	JE1004	Accounts Receivable	9800
12	JE1007	Cash	6200
14	JE1008	Cost of Goods Sold	5100
16	JE1009	Accounts Payable	7200

4.12. polars

```
# polars
large_pl = gl_pl.filter(pl.col("debit") > 5000)
large_pl.select(["entry_id", "account", "debit"])
```

entry_id str	account str	debit i64
"JE1002"	"Inventory"	7200
"JE1003"	"Salaries Expense"	15400
"JE1004"	"Accounts Receivable"	9800
"JE1007"	"Cash"	6200
"JE1008"	"Cost of Goods Sold"	5100
"JE1009"	"Accounts Payable"	7200

4.12.1. Combining Multiple Conditions

Now let's narrow this to transactions in the Operations department with a nonzero debit — combining two conditions with `&` (AND).

4.13. pandas

```
# pandas
combo_pd = gl_pd[(gl_pd["department"] == "Operations") &
 ↪  (gl_pd["debit"] > 0)]
combo_pd[["entry_id", "account", "debit", "department"]]
```

	entry_id	account	debit	department
2	JE1002	Inventory	7200	Operations
8	JE1005	Rent Expense	3000	Operations
10	JE1006	Utilities Expense	850	Operations
14	JE1008	Cost of Goods Sold	5100	Operations
16	JE1009	Accounts Payable	7200	Operations
18	JE1010	Depreciation Expense	1000	Operations

4.14. polars

```
# polars
combo_pl = gl_pl.filter(
    (pl.col("department") == "Operations") & (pl.col("debit") > 0)
)
combo_pl.select(["entry_id", "account", "debit", "department"])
```

entry_id str	account str	debit i64	department str
”JE1002”	”Inventory”	7200	”Operations”
”JE1005”	”Rent Expense”	3000	”Operations”
”JE1006”	”Utilities Expense”	850	”Operations”
”JE1008”	”Cost of Goods Sold”	5100	”Operations”
”JE1009”	”Accounts Payable”	7200	”Operations”
”JE1010”	”Depreciation Expense”	1000	”Operations”

! A Common Mistake

In pandas, you must use `&` and `|` (not Python's `and`/`or`) when combining conditions, and each condition needs its own parentheses: `(gl_pd["department"] == "Operations") & (gl_pd["debit"] > 0)`. Forgetting the parentheses is one of the most common pandas errors for beginners.

4.14.1. Sorting

Let's sort by debit amount (largest first), breaking ties by date (earliest first):

4.15. pandas

```
# pandas
gl_pd.sort_values(["debit", "entry_date"], ascending=[False,
 ↪  True]).head()
```

	entry_id	entry_date	account	account_type	description	debit	cre
4	JE1003	2025-01-08	Salaries Expense	Expense	Biweekly payroll	15400	0
6	JE1004	2025-01-12	Accounts Receivable	Asset	Sale on account	9800	0
2	JE1002	2025-01-05	Inventory	Asset	Purchase of merchandise	7200	0
16	JE1009	2025-01-29	Accounts Payable	Liability	Vendor payment	7200	0
12	JE1007	2025-01-22	Cash	Asset	Customer payment received	6200	0

4.16. polars

```
# polars
gl_pl.sort(["debit", "entry_date"], descending=[True, False]).head()
```

entry_id str	entry_date str	account str	account_type str	description str	debit i64	c i
”JE1003”	”2025-01-08”	”Salaries Expense”	”Expense”	”Biweekly payroll”	15400	0
”JE1004”	”2025-01-12”	”Accounts Receivable”	”Asset”	”Sale on account”	9800	0
”JE1002”	”2025-01-05”	”Inventory”	”Asset”	”Purchase of merchandise”	7200	0
”JE1009”	”2025-01-29”	”Accounts Payable”	”Liability”	”Vendor payment”	7200	0
”JE1007”	”2025-01-22”	”Cash”	”Asset”	”Customer payment received”	6200	0

4.17. Cleaning Data

Real accounting data is rarely as clean as `general_ledger_sample.csv`. Let's work through `messy_general_ledger.csv`, which has the same kinds of issues introduced in Chapter 3: currency symbols, inconsistent formatting, an impossible date, a missing value, and a likely duplicate entry.

4.17.1. Step 1: Fix Currency Formatting

4.18. pandas

```
messy_pd = pd.read_csv("data/messy_general_ledger.csv")
messy_pd
```

	entry_id	entry_date	account	debit	credit	department
0	JE2001	01/15/2025	Cash	$4,500.00	0	Sales
1	JE2001	1-15-2025	cash	4500	0	sales
2	JE2002	01/16/2025	Accounts Receivable	0	$4,500.00	Sales
3	JE2003	01/18/2025	Inventory	7200	0	Operations
4	JE2003	01/18/2025	Accounts Payable	0	7200	Operations
5	JE2004	02/30/2025	Rent Expense	3000	0	Operations
6	JE2004	02/30/2025	Cash	0	3000	Operations
7	JE2005	01/22/2025	SALARIES EXPENSE	15400	0	NaN
8	JE2005	01/22/2025	Cash	0	15400	HR
9	JE2006	01/25/2025	Utilities Expense	850	0	Operations
10	JE2006	01/25/2025	Accounts Payable	0	8500	Operations

4.19. polars

```
# polars
messy_pl = pl.read_csv("data/messy_general_ledger.csv")
messy_pl
```

entry_id str	entry_date str	account str	debit str	credit str	department str
"JE2001"	"01/15/2025"	"Cash"	"$4,500.00"	"0"	"Sales"
"JE2001"	"1-15-2025"	"cash "	"4500"	"0"	" sales"
"JE2002"	"01/16/2025"	"Accounts Receivable"	"0"	"$4,500.00"	"Sales"
"JE2003"	"01/18/2025"	"Inventory"	"7200"	"0"	"Operations"
"JE2003"	"01/18/2025"	"Accounts Payable"	"0"	"7200"	"Operations"
…	…	…	…	…	…
"JE2004"	"02/30/2025"	"Cash"	"0"	"3000"	"Operations"

entry_id str	entry_date str	account str	debit str	credit str	department str
”JE2005”	”01/22/2025”	”SALARIES EXPENSE”	”15400”	”0”	null
”JE2005”	”01/22/2025”	”Cash”	”0”	”15400”	”HR”
”JE2006”	”01/25/2025”	”Utilities Expense”	”850”	”0”	”Operations”
”JE2006”	”01/25/2025”	”Accounts Payable”	”0”	”8500”	”Operations”

4.20. pandas

```
df_pd = messy_pd.copy()
for col in ["debit", "credit"]:
    df_pd[col] = (
        df_pd[col].astype(str)
        .str.replace("$", "", regex=False)
        .str.replace(",", "", regex=False)
    )
    df_pd[col] = pd.to_numeric(df_pd[col])

df_pd[["entry_id", "debit", "credit"]]
```

	entry_id	debit	credit
0	JE2001	4500.0	0.0
1	JE2001	4500.0	0.0
2	JE2002	0.0	4500.0
3	JE2003	7200.0	0.0
4	JE2003	0.0	7200.0
5	JE2004	3000.0	0.0
6	JE2004	0.0	3000.0
7	JE2005	15400.0	0.0
8	JE2005	0.0	15400.0
9	JE2006	850.0	0.0
10	JE2006	0.0	8500.0

4.21. polars

```
df_pl = messy_pl.with_columns([
    pl.col("debit").str.replace_all(r"\$", "").str.replace_all(",",
 ↪  "").cast(pl.Float64),
    pl.col("credit").str.replace_all(r"\$", "").str.replace_all(",",
 ↪  "").cast(pl.Float64),
])
df_pl.select(["entry_id", "debit", "credit"])
```

entry_id str	debit f64	credit f64
"JE2001"	4500.0	0.0
"JE2001"	4500.0	0.0
"JE2002"	0.0	4500.0
"JE2003"	7200.0	0.0
"JE2003"	0.0	7200.0
…	…	…
"JE2004"	0.0	3000.0
"JE2005"	15400.0	0.0
"JE2005"	0.0	15400.0
"JE2006"	850.0	0.0
"JE2006"	0.0	8500.0

The `debit` and `credit` columns contain `$` and `,` characters, which force them to be read as text instead of numbers.

4.21.1. Step 2: Standardize Text Fields

The `account` and `department` columns have inconsistent capitalization and stray whitespace.

4.22. pandas

```
# pandas
df_pd["account"] = df_pd["account"].str.strip().str.title()
```

```
df_pd["department"] = df_pd["department"].str.strip().str.title()
df_pd[["account", "department"]]
```

	account	department
0	Cash	Sales
1	Cash	Sales
2	Accounts Receivable	Sales
3	Inventory	Operations
4	Accounts Payable	Operations
5	Rent Expense	Operations
6	Cash	Operations
7	Salaries Expense	NaN
8	Cash	Hr
9	Utilities Expense	Operations
10	Accounts Payable	Operations

4.23. polars

```
# polars
df_pl = df_pl.with_columns([
    pl.col("account").str.strip_chars().str.to_titlecase(),
    pl.col("department").str.strip_chars().str.to_titlecase(),
])
df_pl.select(["account", "department"])
```

account str	department str
"Cash"	"Sales"
"Cash"	"Sales"
"Accounts Receivable"	"Sales"
"Inventory"	"Operations"
"Accounts Payable"	"Operations"
…	…
"Cash"	"Operations"
"Salaries Expense"	null
"Cash"	"Hr"

account str	department str
”Utilities Expense”	”Operations”
”Accounts Payable”	”Operations”

⚠ Automated Cleaning Still Needs a Human Check

Look closely at the department column: `"HR"` became `"Hr"` after title-casing, since `.title()` (and its polars equivalent) simply capitalizes the first letter of each word — it doesn't know `HR` is an acronym that should stay fully capitalized. This is a good example of why you should always spot-check the results of automated cleaning steps rather than assuming they worked perfectly. A more robust fix here would be an explicit mapping (e.g., `{"hr": "HR", "Hr": "HR"}`) for known acronyms in your data.

4.23.1. Step 3: Fix Inconsistent Dates

The `entry_date` column mixes `01/15/2025` and `1-15-2025` formats, and contains one impossible date (`02/30/2025` — February never has 30 days).

4.24. pandas

```
# pandas: format="mixed" handles multiple formats; errors="coerce"
 ↪  turns bad dates into NaT
df_pd["entry_date"] = pd.to_datetime(df_pd["entry_date"],
 ↪  format="mixed", errors="coerce")
df_pd[["entry_id", "entry_date"]]
```

	entry_id	entry_date
0	JE2001	2025-01-15
1	JE2001	2025-01-15
2	JE2002	2025-01-16
3	JE2003	2025-01-18
4	JE2003	2025-01-18
5	JE2004	NaT
6	JE2004	NaT
7	JE2005	2025-01-22

	entry_id	entry_date
8	JE2005	2025-01-22
9	JE2006	2025-01-25
10	JE2006	2025-01-25

```
# pandas: find the rows that failed to parse
df_pd[df_pd["entry_date"].isna()]
```

	entry_id	entry_date	account	debit	credit	department
5	JE2004	NaT	Rent Expense	3000.0	0.0	Operations
6	JE2004	NaT	Cash	0.0	3000.0	Operations

4.25. polars

Polars doesn't have an automatic "mixed format" parser, so we normalize the separators first, then parse with `strict=False` so invalid dates become `null` instead of raising an error.

```
# polars
df_pl = df_pl.with_columns(
    pl.col("entry_date")
      .str.replace_all("-", "/")
      .str.to_date(format="%m/%d/%Y", strict=False)
      .alias("entry_date")
)
df_pl.select(["entry_id", "entry_date"])
```

entry_id str	entry_date date
"JE2001"	2025-01-15
"JE2001"	2025-01-15
"JE2002"	2025-01-16
"JE2003"	2025-01-18
"JE2003"	2025-01-18
…	…
"JE2004"	null
"JE2005"	2025-01-22

entry_id str	entry_date date
"JE2005"	2025-01-22
"JE2006"	2025-01-25
"JE2006"	2025-01-25

! What Should You Do With an Invalid Date?

Notice that `02/30/2025` became a missing value (`NaT` in pandas, `null` in polars) rather than an error that stops your code. This is intentional — but it means *you* now have to decide what to do about it: go back to the source system to find the correct date, exclude the transaction, or flag it for follow-up. Silently ignoring nulls at this point is how real errors slip through into financial reports.

4.25.1. Step 4: Handle Missing Values

The `department` column has one missing value.

4.26. pandas

```
# pandas
df_pd["department"] = df_pd["department"].fillna("Unassigned")
df_pd[["entry_id", "department"]]
```

	entry_id	department
0	JE2001	Sales
1	JE2001	Sales
2	JE2002	Sales
3	JE2003	Operations
4	JE2003	Operations
5	JE2004	Operations
6	JE2004	Operations
7	JE2005	Unassigned
8	JE2005	Hr
9	JE2006	Operations
10	JE2006	Operations

4.27. polars

```
# polars
df_pl = df_pl.with_columns(
    pl.col("department").fill_null("Unassigned")
)
df_pl.select(["entry_id", "department"])
```

entry_id str	department str
”JE2001”	”Sales”
”JE2001”	”Sales”
”JE2002”	”Sales”
”JE2003”	”Operations”
”JE2003”	”Operations”
…	…
”JE2004”	”Operations”
”JE2005”	”Unassigned”
”JE2005”	”Hr”
”JE2006”	”Operations”
”JE2006”	”Operations”

4.27.1. Step 5: Check for Duplicates

After cleaning, compare rows on the fields that should uniquely identify a real transaction line.

4.28. pandas

```
# pandas
dupes_pd = df_pd[df_pd.duplicated(subset=["account", "debit",
 ↪  "credit"], keep=False)]
dupes_pd
```

	entry_id	entry_date	account	debit	credit	department
0	JE2001	2025-01-15	Cash	4500.0	0.0	Sales

	entry_id	entry_date	account	debit	credit	department
1	JE2001	2025-01-15	Cash	4500.0	0.0	Sales

4.29. polars

```
dupes_pl = df_pl.filter(df_pl.is_duplicated())
dupes_pl
```

entry_id str	entry_date date	account str	debit f64	credit f64	department str
"JE2001"	2025-01-15	"Cash"	4500.0	0.0	"Sales"
"JE2001"	2025-01-15	"Cash"	4500.0	0.0	"Sales"

The two JE2001 rows for Cash are now revealed as duplicates once formatting differences are removed — before cleaning, "cash " (lowercase, trailing space) looked like a different value from "Cash", hiding the duplicate.

4.30. Merging and Joining Tables

Accounting data rarely lives in a single table. Data may reside in multiple tables. In such cases, we might need to merge or join them together. In order to join two datasets, both datasets must have at least one common column such as CustomerID, or OrderID, or ProductSKU that acts as the bridge between rows. Let's enrich the general ledger with account type information from chart_of_accounts.csv.

4.31. pandas

```
coa_pd = pd.read_csv("data/chart_of_accounts.csv")
coa_pd.head()
```

	account	account_number	account_type
0	Cash	1000	Asset

	account	account_number	account_type
1	Accounts Receivable	1100	Asset
2	Inventory	1200	Asset
3	Prepaid Insurance	1300	Asset
4	Equipment	1500	Asset

4.32. polars

```
# polars
coa_pl = pl.read_csv("data/chart_of_accounts.csv")
```

4.33. Types of Merging or Joining

Data joins are fundamentally defined by **how they match rows** across separate datasets using a shared, identical key column. The primary objective is to align corresponding records horizontally to create a unified view of your data. Figure 4.3 illustrates the four most common types of joins: inner, left, right, and full. Each type serves a distinct purpose and is chosen based on the specific requirements of your analysis.

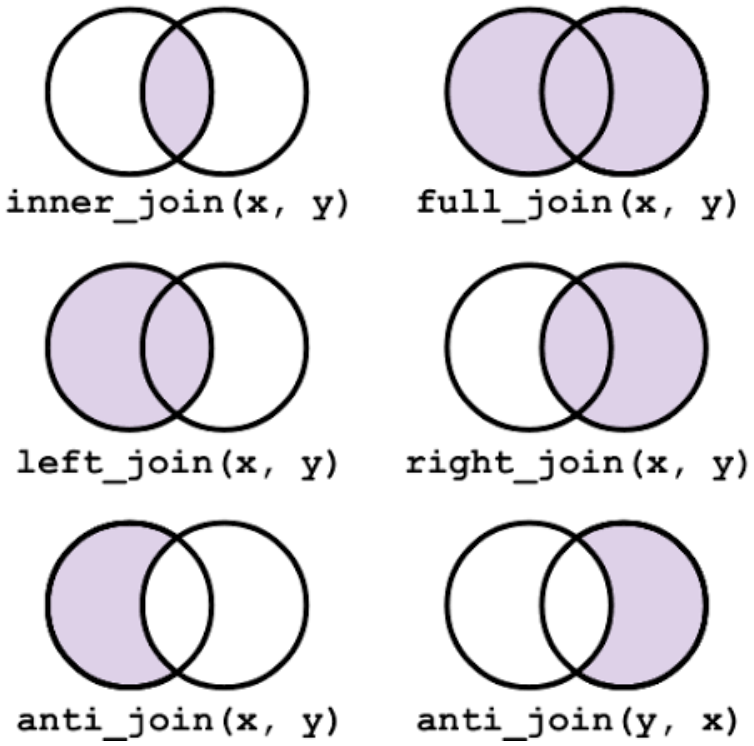


Figure 4.3.: Types of Data Merging or Joining

4.33.1. Inner Join

An **inner join** keeps only rows where the join key (`account`) matches in *both* tables. An inner join serves as a strict filter that combines rows from two tables only when their designated keys match exactly. If a row's key exists in one table but is missing from the other, that row is entirely excluded from the final output dataset. This type of join is ideal when your analysis demands complete information from both sources, leaving you with a clean, fully intersected table free of missing values on the join keys.

4.34. pandas

```
# pandas
inner_pd = gl_pd.merge(coa_pd, on="account", how="inner")
print(inner_pd.shape)
sorted(inner_pd["account"].unique())
```

```
(19, 11)
```

```
['Accounts Payable',
 'Accounts Receivable',
 'Accumulated Depreciation - Equipment',
 'Cash',
 'Cost of Goods Sold',
 'Depreciation Expense',
 'Inventory',
 'Rent Expense',
 'Salaries Expense',
 'Sales Revenue']
```

4.35. polars

```
# polars
inner_pl = gl_pl.join(coa_pl, on="account", how="inner")
print(inner_pl.shape)
```

```
(19, 11)
```

Notice the inner join result has fewer rows than the original general ledger (19 vs. 20). One account is missing.

4.35.1. Left Join

A **left join** keeps *every* row from the left table (the general ledger), filling in `null`/`NaN` where there's no match in the chart of accounts. A left join prioritizes the primary dataset by preserving every single row from the left table, regardless of whether a matching partner exists in the secondary table. When a key in the left table finds a match on the right, the corresponding attributes are pulled in; when no match is found, the missing slots are simply filled with null values. Analysts frequently default to this approach because it prevents data loss from the master dataset while safely appending supplementary context where available.

4.36. pandas

```
# pandas
left_pd = gl_pd.merge(coa_pd, on="account", how="left")
missing_pd = left_pd[left_pd["account_number"].isna()]
```

```
missing_pd[["entry_id", "account"]].drop_duplicates()
```

	entry_id	account
10	JE1006	Utilities Expense

4.37. polars

```
# polars
left_pl = gl_pl.join(coa_pl, on="account", how="left")
missing_pl = left_pl.filter(pl.col("account_number").is_null())
missing_pl.select(["entry_id", "account"]).unique()
```

entry_id	account
str	str
"JE1006"	"Utilities Expense"

Both approaches reveal the same finding: □**Utilities Expense**□ appears in the general ledger but not in the chart of accounts — likely a new account that hasn't been added to the COA yet, or a naming mismatch (e.g., "Utilities" vs. "Utilities Expense") that needs investigation.

> **!** Why This Matters
>
> An inner join would have silently *dropped* the Utilities Expense transactions from any analysis without telling you — the row count would just be smaller, with no error or warning. A left join preserves every general ledger transaction and makes the mismatch visible. When enriching transaction data with reference tables, prefer a left join and explicitly check for unmatched rows, rather than defaulting to an inner join.

4.37.1. Right Join

A **right join** acts as the exact mirror image of a left join, ensuring that every record from the right-hand table is maintained in the final result. If a row in this secondary table cannot find a corresponding key in the left table, the missing left columns are returned as null values. While it can be functionally replaced by simply swapping the table order in a left join, it remains highly useful when you are incrementally feeding your core tables into a new, dominant reference dataset.

4.37.2. Full Join

A **full join** is an all-inclusive combination that retains every single record from both the left and right tables, leaving no data behind. When keys align, the rows are cleanly merged into a single entry; when they do not align, the missing halves are populated with null values for whichever side lacks the data. This join is indispensable for comprehensive audits or data synchronization tasks where you need a holistic view of all available records across both systems.

4.37.3. Anti Join

An **anti join** acts as a powerful exclusion filter, returning rows from the left table only when their keys are completely absent from the right table. It isolates the anomalies and non-matching records, discarding any entries that find a counterpart in the secondary dataset. Data engineers frequently leverage anti joins for debugging, identifying orphan records, or isolating missing entries between two database systems.

4.38. Reshaping Data

Recall the tidy data discussion from Chapter 3. Many times, we might need to reshape our data to better suit our analysis needs. **Reshaping a dataset** involves transforming its structure to make it clean, readable, and ready for analysis or visualization. This process typically means converting data between a wide format (where variables occupy individual columns) and a long format (where variables are stacked sequentially down rows). Reshaping does not alter the underlying information, but rather rearranges the layout so that software tools, statistical models, and charting libraries can interpret the data correctly. Let's build a **wide** summary table — one common way accountants like to view data — and then reshape it back to **long** (tidy) format.

4.38.1. Wide: Total Debits by Account and Department

4.39. pandas

```
# pandas
wide_pd = gl_pd.pivot_table(
    index="account", columns="department", values="debit",
    aggfunc="sum", fill_value=0
)
wide_pd
```

department account	HR	Operations	Sales
Accounts Payable	0	7200	0
Accounts Receivable	0	0	9800
Accumulated Depreciation - Equipment	0	0	0
Cash	0	0	10700
Cost of Goods Sold	0	5100	0
Depreciation Expense	0	1000	0
Inventory	0	7200	0
Rent Expense	0	3000	0
Salaries Expense	15400	0	0
Sales Revenue	0	0	0
Utilities Expense	0	850	0

4.40. polars

```
# polars
wide_pl = gl_pl.pivot(
    index="account", on="department", values="debit",
    aggregate_function="sum"
).fill_null(0)
wide_pl
```

account str	Sales i64	Operations i64	HR i64
”Cash”	10700	0	0
”Accounts Receivable”	9800	0	0
”Inventory”	0	7200	0
”Accounts Payable”	0	7200	0
”Salaries Expense”	0	0	15400
…	…	…	…
”Rent Expense”	0	3000	0
”Utilities Expense”	0	850	0
”Cost of Goods Sold”	0	5100	0
”Depreciation Expense”	0	1000	0
”Accumulated Depreciation - Equ…	0	0	0

4.40.1. Back to Long (Tidy) Format

4.41. pandas

```
# pandas
long_pd = wide_pd.reset_index().melt(
    id_vars="account", var_name="department", value_name="debit"
)
long_pd.head()
```

	account	department	debit
0	Accounts Payable	HR	0
1	Accounts Receivable	HR	0
2	Accumulated Depreciation - Equipment	HR	0
3	Cash	HR	0
4	Cost of Goods Sold	HR	0

4.42. polars

```
# polars
long_pl = wide_pl.unpivot(
    index="account", variable_name="department", value_name="debit"
)
long_pl.head()
```

account str	department str	debit i64
"Cash"	"Sales"	10700
"Accounts Receivable"	"Sales"	9800
"Inventory"	"Sales"	0
"Accounts Payable"	"Sales"	0
"Salaries Expense"	"Sales"	0

Both directions matter in practice: the wide format is easy for a human to scan in a report, while the long/tidy format is what you want as an input to further analysis, grouping, or plotting (Chapter 6).

4.43. Grouping and Aggregating

One of the most common accounting analytics tasks is summarizing transaction-level data into account-level totals — in other words, **rebuilding a trial balance from the general ledger.**

4.44. pandas

```
# pandas
tb_pd = gl_pd.groupby("account", as_index=False)[["debit",
 ↪  "credit"]].sum()
tb_pd
```

	account	debit	credit
0	Accounts Payable	7200	8050
1	Accounts Receivable	9800	10700
2	Accumulated Depreciation - Equipment	0	1000
3	Cash	10700	25600
4	Cost of Goods Sold	5100	0
5	Depreciation Expense	1000	0
6	Inventory	7200	5100
7	Rent Expense	3000	0
8	Salaries Expense	15400	0
9	Sales Revenue	0	9800
10	Utilities Expense	850	0

```
print("Total debits:", tb_pd["debit"].sum())
print("Total credits:", tb_pd["credit"].sum())
```

```
Total debits: 60250
Total credits: 60250
```

4.45. polars

```
# polars
tb_pl = (
```

```
    gl_pl.group_by("account")
    .agg([pl.col("debit").sum(), pl.col("credit").sum()])
    .sort("account")
)
tb_pl
```

account str	debit i64	credit i64
"Accounts Payable"	7200	8050
"Accounts Receivable"	9800	10700
"Accumulated Depreciation - Equ…	0	1000
"Cash"	10700	25600
"Cost of Goods Sold"	5100	0
…	…	…
"Inventory"	7200	5100
"Rent Expense"	3000	0
"Salaries Expense"	15400	0
"Sales Revenue"	0	9800
"Utilities Expense"	850	0

Notice that total debits equal total credits ($60,250 each) — exactly what we'd expect, since this confirms the sample general ledger is internally balanced, the same check you'd run on any real trial balance.

4.45.1. Multi-Level Grouping

You can group by more than one column at once — for example, account *within* department:

4.46. pandas

```
# pandas
multi_pd = gl_pd.groupby(["department", "account"],
 ↪  as_index=False)[["debit", "credit"]].sum()
multi_pd
```

	department	account	debit	credit
0	HR	Cash	0	15400
1	HR	Salaries Expense	15400	0
2	Operations	Accounts Payable	7200	8050
3	Operations	Accumulated Depreciation - Equipment	0	1000
4	Operations	Cash	0	10200
5	Operations	Cost of Goods Sold	5100	0
6	Operations	Depreciation Expense	1000	0
7	Operations	Inventory	7200	5100
8	Operations	Rent Expense	3000	0
9	Operations	Utilities Expense	850	0
10	Sales	Accounts Receivable	9800	10700
11	Sales	Cash	10700	0
12	Sales	Sales Revenue	0	9800

4.47. polars

```
# polars
multi_pl = (
    gl_pl.group_by(["department", "account"])
    .agg([pl.col("debit").sum(), pl.col("credit").sum()])
    .sort(["department", "account"])
)
multi_pl
```

department str	account str	debit i64	credit i64
”HR”	”Cash”	0	15400
”HR”	”Salaries Expense”	15400	0
”Operations”	”Accounts Payable”	7200	8050
”Operations”	”Accumulated Depreciation - Equ…	0	1000
”Operations”	”Cash”	0	10200
…	…	…	…
”Operations”	”Rent Expense”	3000	0
”Operations”	”Utilities Expense”	850	0
”Sales”	”Accounts Receivable”	9800	10700
”Sales”	”Cash”	10700	0

department str	account str	debit i64	credit i64
”Sales”	”Sales Revenue”	0	9800

4.48. Polars Lazy Frames: Wrangling Large Accounting Datasets

Everything so far has used **eager evaluation** — each line of code runs immediately, and the result is fully computed and stored in memory before you move to the next step. This is how pandas always works, and how polars works by default with `pl.read_csv()`.

Polars also offers **lazy evaluation**: instead of running each step immediately, you build up a *query plan* describing everything you want to do, and only execute it once, at the end, with `.collect()`. This matters enormously for large datasets, because polars can look at your *entire* pipeline before running anything and optimize it — for example, reading only the columns you actually use, or applying filters before an expensive join instead of after.

4.48.1. Eager vs. Lazy: The Same Pipeline, Two Ways

Let’s take a realistic pipeline — filter to nonzero debits, join to the chart of accounts, then total debits by account type — and run it both ways.

```
# EAGER: every step executes immediately and materializes a full
 ↪  result in memory
result_eager = (
    gl_pl
    .filter(pl.col("debit") > 0)
    .join(coa_pl, on="account", how="inner")
    .group_by("account_type")
    .agg(pl.col("debit").sum().alias("total_debit"))
    .sort("account_type")
)
result_eager
```

account_type str	total_debit i64
”Asset”	27700

account_type str	total_debit i64
"Expense"	24500
"Liability"	7200

```
# LAZY: scan_csv builds a query plan; nothing runs until .collect()
lazy_query = (
    pl.scan_csv("data/general_ledger_sample.csv")
    .filter(pl.col("debit") > 0)
    .join(pl.scan_csv("data/chart_of_accounts.csv"), on="account",
 ↪  how="inner")
    .group_by("account_type")
    .agg(pl.col("debit").sum().alias("total_debit"))
    .sort("account_type")
)

result_lazy = lazy_query.collect()
result_lazy
```

account_type str	total_debit i64
"Asset"	27700
"Expense"	24500
"Liability"	7200

Both approaches give an identical result — but they got there differently. You can inspect *how* polars plans to execute the lazy version before running it, using `.explain()`:

```
print(lazy_query.explain())
```

```
SORT BY [col("account_type")]
  AGGREGATE[maintain_order: false]
    [col("debit").sum().alias("total_debit")] BY [col("account_type")]
    FROM
    simple  2/2 ["account_type", "debit"]
      INNER JOIN:
      LEFT PLAN ON: [col("account")]
        simple  3/3 ["account_type", "debit", ... 1 other column]
```

```
    Csv SCAN [data/general_ledger_sample.csv]
    PROJECT 3/9 COLUMNS
    SELECTION: col("debit") > 0
    ESTIMATED ROWS: 23
RIGHT PLAN ON: [col("account")]
  Csv SCAN [data/chart_of_accounts.csv]
  PROJECT 1/3 COLUMNS
  ESTIMATED ROWS: 37
END INNER JOIN
```

Look closely at the plan: notice it shows `PROJECT 3/9 COLUMNS` for the general ledger scan. Polars has figured out — entirely on its own — that only 3 of the 9 available columns are actually needed anywhere in this pipeline, and it will only read those columns from disk. It has also pushed the `debit > 0` filter down to happen *during* the file scan, rather than loading the full dataset first and filtering afterward.

! Why This Matters for Real Accounting Data

A university dataset like `general_ledger_sample.csv` has 20 rows — the difference between eager and lazy evaluation is invisible here. But a real company's general ledger for a single fiscal year can easily contain **millions of transaction lines** across dozens of columns. In that setting:

- Reading only the 3 needed columns instead of all 9 can cut memory use dramatically
- Filtering before a join (rather than joining everything, then filtering) avoids doing expensive join work on rows you're about to discard anyway
- These optimizations happen **automatically** once you write your pipeline in lazy form — you don't have to manually reorder your steps for performance

This is precisely the kind of workload common in audit analytics (Chapter 8) and fraud detection (Chapter 9), where you may need to scan an entire year of transactions rather than a sample. Lazy evaluation is one reason polars is gaining adoption for exactly this kind of large-scale accounting analytics work.

A Note on Pandas and Scale

Pandas itself doesn't have a lazy evaluation mode. For very large datasets in a pandas-based workflow, teams sometimes turn to libraries like **Dask** or **Modin**, which mimic the pandas API while distributing computation — these are beyond the scope of this book, but worth knowing the names of if you encounter a pandas dataset too large to fit comfortably in memory.

4.49. Chapter Summary

- pandas and polars share the same core wrangling concepts — filtering, sorting, cleaning, joining, reshaping, grouping — but express them with different syntax; pandas remains the current industry standard, while polars is a fast-growing alternative especially suited to large datasets.
- Cleaning real accounting data typically involves fixing currency formatting, standardizing inconsistent text, parsing unreliable dates, filling missing values, and checking for duplicates — and automated cleaning steps (like `.title()`) still need a human sanity check.
- Inner joins silently drop unmatched rows; left joins preserve every row from your primary table and reveal mismatches — prefer left joins when enriching transaction data with a reference table like a chart of accounts.
- Reshaping between wide (human-readable) and long/tidy (analysis-ready) formats is a routine step, supported in both libraries via pivot/melt (pandas) and pivot/unpivot (polars).
- Grouping and aggregating transaction-level data lets you rebuild summary views — like a trial balance — directly from the general ledger, and is a natural way to sanity-check that debits equal credits.
- Polars' lazy evaluation (`pl.scan_csv() + .collect()`) lets the library optimize an entire pipeline before running it — reading only needed columns and pushing filters earlier — which becomes increasingly valuable as accounting datasets grow into the millions of rows.

4.50. Discussion Questions

1. Why might an inner join be the *wrong* default choice when enriching general ledger transactions with chart-of-accounts data, even though it produces a "cleaner-looking" result with fewer rows to review?
2. The `.title()` cleaning step turned `"HR"` into `"Hr"`. Can you think of another accounting example (an account name, a department, an abbreviation) where automatic text cleaning could introduce a *new* error while fixing another?
3. In your own words, explain why lazy evaluation matters more for a company with millions of general ledger transactions than for the 20-row sample used in this chapter.

4.51. Exercises

1. Load `general_ledger_sample.csv` in both pandas and polars, and confirm both libraries produce the same total debits and total credits when grouped by account.

2. Using `messy_general_ledger.csv`, complete the full cleaning workflow (currency formatting, text standardization, dates, missing values, duplicates) in whichever library you're more comfortable with, then repeat it in the other library.
3. Using `chart_of_accounts.csv` and `general_ledger_sample.csv`, perform a left join and identify every general ledger account with no chart-of-accounts match. Write one sentence describing what you'd do next if this were a real client engagement.
4. Rewrite the eager pipeline in the Lazy Frames section as your own lazy pipeline, but group by `department` instead of `account_type`. Run `.explain()` on your version and describe, in your own words, what optimization(s) you can identify in the output.

5. Exploratory Data Analysis for Financial Data

Learning Objectives

By the end of this chapter, you should be able to:

- Explain the goals of exploratory data analysis (EDA) and where it fits in the analytics workflow
- Choose appropriate summary statistics based on a variable's level of measurement
- Compute and interpret measures of central tendency and dispersion for financial data
- Visually inspect the distribution of a financial variable and recognize skewness
- Distinguish between an outlier and an error, and apply the z-score and IQR methods to flag unusual transactions
- Perform a first-pass EDA on transaction-level general ledger data, including comparisons across categories

5.1. What Is EDA, and Why Do It First?

Exploratory data analysis (EDA) is the process of getting to know a dataset — its shape, its typical values, its oddities — before you calculate ratios (Chapter 7), run audit tests (Chapter 8), or build a predictive model (Chapter 12). Skipping this step is one of the most common ways analytics work goes wrong: it's easy to compute a technically correct number from data you don't actually understand yet, and end up with a confidently wrong conclusion.

The EDA mindset is simple: **look before you calculate.** Let the data surprise you before you start asking it questions.

This chapter introduces EDA using a new dataset: `general_ledger_large.csv`, a simulated general ledger with **1,504 transaction lines** (752 journal entries) spanning a full fiscal year. It's large enough to show real distribution shapes and genuine outliers — something our earlier 10–20 row samples were too small to demonstrate convincingly.

5.2. pandas

```
import pandas as pd
gl_pd = pd.read_csv("data/general_ledger_large.csv")
print(gl_pd.shape)
gl_pd.head()
```

```
(1504, 7)
```

	entry_id	entry_date	account	debit	credit	department	posted_by
0	JE3253	2025-01-01	Cost of Goods Sold	1231.76	0.00	Operations	mchen
1	JE3253	2025-01-01	Inventory	0.00	1231.76	Operations	mchen
2	JE3413	2025-01-01	Accounts Receivable	3024.76	0.00	Sales	agarcia
3	JE3413	2025-01-01	Sales Revenue	0.00	3024.76	Sales	agarcia
4	JE3476	2025-01-01	Cash	4476.03	0.00	Sales	rpatel

5.3. polars

```
import polars as pl
gl_pl = pl.read_csv("data/general_ledger_large.csv")
gl_pl.head()
```

entry_id str	entry_date str	account str	debit f64	credit f64	department str	posted_by str
"JE3253"	"2025-01-01"	"Cost of Goods Sold"	1231.76	0.0	"Operations"	"mchen"
"JE3253"	"2025-01-01"	"Inventory"	0.0	1231.76	"Operations"	"mchen"
"JE3413"	"2025-01-01"	"Accounts Receivable"	3024.76	0.0	"Sales"	"agarcia"
"JE3413"	"2025-01-01"	"Sales Revenue"	0.0	3024.76	"Sales"	"agarcia"
"JE3476"	"2025-01-01"	"Cash"	4476.03	0.0	"Sales"	"rpatel"

5.4. Choosing the Right Summary Statistic

Recall from Chapter 3 that a variable's **level of measurement** determines which calculations are actually meaningful:

Level	Example in this dataset	Meaningful summary
Nominal	`account`, `department`, `posted_by`	Count, mode
Ratio	`debit`, `credit`	Mean, median, std, all arithmetic

It's worth pausing on this before diving into code. `department` is nominal — Sales, Operations, HR, Finance are categories with no inherent order. Computing an "average department" is meaningless. `debit`, on the other hand, is ratio-level — dollar amounts have a true zero and every arithmetic operation applies. The rest of this chapter focuses on summarizing `debit` and `credit`, and *counting* (not averaging) categorical fields like `department`.

5.5. pandas

```
gl_pd["department"].value_counts()
```

```
department
Operations    700
Sales         572
HR            132
Finance       100
Name: count, dtype: int64
```

5.6. polars

```
gl_pl['department'].value_counts()
# or
gl_pl.select(pl.col("department").value_counts())
```

department
struct[2]
{"HR",132}
{"Sales",572}
{"Finance",100}
{"Operations",700}

5.7. Measures of Central Tendency

Let's compute the mean, median, and mode of transaction debit amounts (restricting to rows where a debit actually occurred, since half the rows in a double-entry dataset have a debit of 0).

5.8. pandas

```
debit_pd = gl_pd[gl_pd["debit"] > 0]["debit"]

print("Mean:  ", debit_pd.mean())
print("Median:", debit_pd.median())
print("Mode:  ", debit_pd.mode().iloc[0])
```

```
Mean:   4044.065132978723
Median: 2457.11
Mode:   1739.52
```

5.9. polars

```
debit_pl = gl_pl.filter(pl.col("debit") >
 ↪  0).select("debit").to_series()

print("Mean:  ", debit_pl.mean())
print("Median:", debit_pl.median())
print("Mode:  ", debit_pl.mode()[0])
```

```
Mean:   4044.0651329787233
Median: 2457.11
Mode:   1739.52
```

Notice the **mean ($4,044) is noticeably higher than the median ($2,457)**. This is a classic sign of a right-skewed distribution: most transactions are modest in size, but a handful of very large ones pull the mean upward. The median is a more representative "typical transaction size" here — a pattern you'll see constantly in financial data, and a good reason to always check both rather than reporting the mean alone.

> **Connect to Practice**
>
> When a client or manager asks for "the average transaction size," it's worth asking whether they want the mean or the median — for skewed financial data, these can tell noticeably different stories, and reporting only the mean can be misleading.

5.10. Measures of Dispersion

Central tendency tells you where the data is centered; dispersion tells you how spread out it is.

5.11. pandas

```
print("Standard deviation:", debit_pd.std())
print("Variance:          ", debit_pd.var())

q1, q3 = debit_pd.quantile([0.25, 0.75])
iqr = q3 - q1
print("IQR:               ", iqr)

cv = debit_pd.std() / debit_pd.mean()
print("Coefficient of variation:", cv)
```

```
Standard deviation: 11497.579308527369
Variance:           132194329.95587668
IQR:                2576.855
Coefficient of variation: 2.8430747108315333
```

5.12. polars

```
print("Standard deviation:", debit_pl.std())
print("Variance:          ", debit_pl.var())

q1_pl = debit_pl.quantile(0.25, interpolation="linear")
q3_pl = debit_pl.quantile(0.75, interpolation="linear")
iqr_pl = q3_pl - q1_pl
print("IQR:               ", iqr_pl)
```

```
cv_pl = debit_pl.std() / debit_pl.mean()
print("Coefficient of variation:", cv_pl)
```

```
Standard deviation: 11497.579308527369
Variance:           132194329.95587671
IQR:                2576.855
Coefficient of variation: 2.843074710831533
```

> **!** Why the Coefficient of Variation?
>
> Standard deviation alone can't tell you whether an account is "volatile" in any meaningful sense, because it's on the same scale as the data itself — a standard deviation of $10,000 is huge for a Rent Expense account but unremarkable for total Sales Revenue. The **coefficient of variation** (standard deviation ÷ mean) is a unitless ratio, which makes it possible to fairly compare variability *across* accounts of very different sizes — something we'll do in the grouped EDA section below.

5.13. Understanding Distributions

Numbers alone don't always reveal a distribution's shape — visualizing it does. Full charting techniques are the focus of Chapter 6, but a histogram and boxplot are useful enough that we'll use them here too.

```
import matplotlib.pyplot as plt

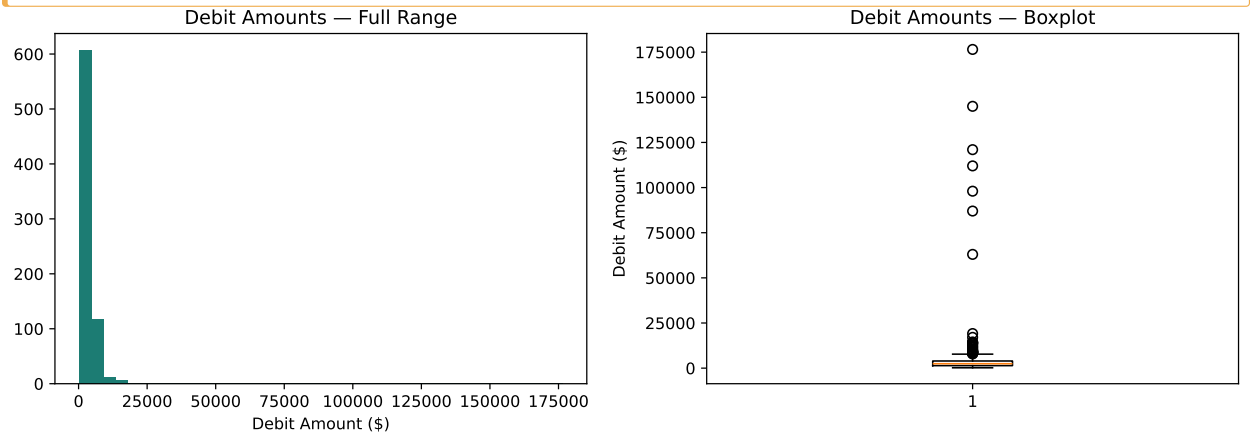
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].hist(debit_pd, bins=40, color="#1c7c73")
axes[0].set_title("Debit Amounts - Full Range")
axes[0].set_xlabel("Debit Amount ($)")

axes[1].boxplot(debit_pd, vert=True)
axes[1].set_title("Debit Amounts - Boxplot")
axes[1].set_ylabel("Debit Amount ($)")

plt.tight_layout()
plt.show()
```

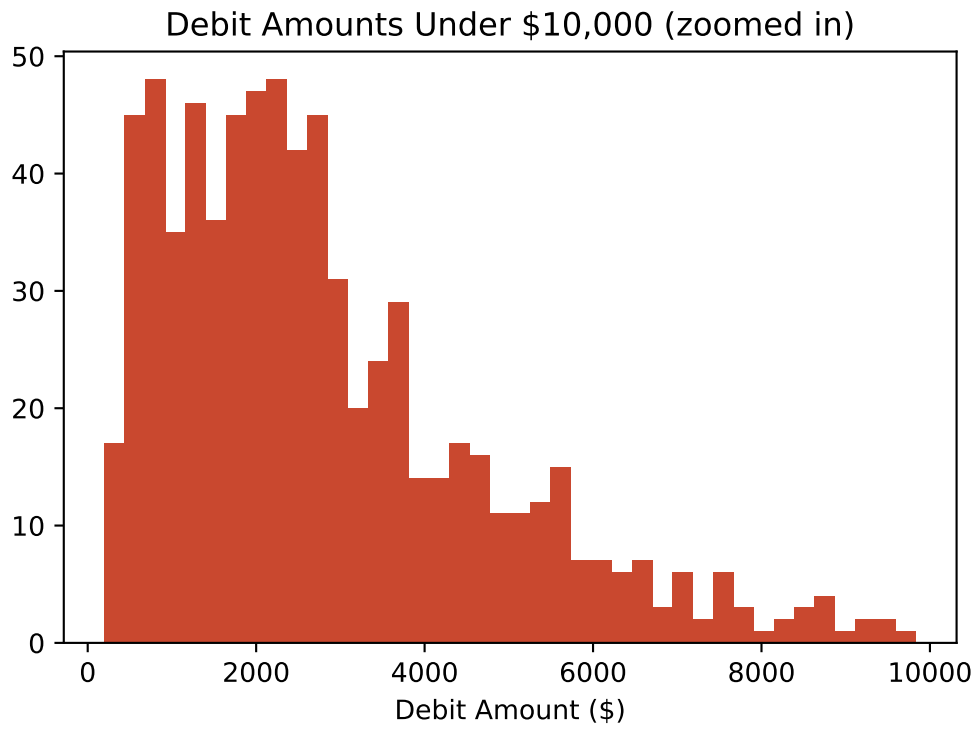
chart, because a handful of very large transactions stretch the x-axis all the way out to $176,500. This is a common problem with skewed financial data — the outliers you'll learn to detect in the next section can also distort your ability to see the underlying distribution. A quick fix: zoom in by temporarily excluding the extreme values.




```
zoomed = debit_pd[debit_pd < 10000]

plt.figure(figsize=(6, 4))
plt.hist(zoomed, bins=40, color="#c9482f")
plt.title("Debit Amounts Under $10,000 (zoomed in)")
plt.xlabel("Debit Amount ($)")
plt.show()

print(f"{len(zoomed)} of {len(debit_pd)} transactions shown
 ↪  ({len(zoomed)/len(debit_pd):.1%})")
```




```
731 of 752 transactions shown (97.2%)
```

Zooming in reveals what was hidden before: a right-skewed but otherwise unimodal (single-peaked) distribution, with most transactions clustered in the low thousands of dollars. This confirms numerically what we already suspected from the mean-vs-median gap: **skewness**, not a data error, largely explains the shape.

> Connect to Practice
>
> A **bimodal** distribution (two distinct peaks) in an account balance often signals that two different populations have been lumped into one account — for example, a "Travel Expense" account mixing routine local travel with occasional expensive international trips. Spotting this kind of pattern is often the first clue that a variable needs to be split or segmented before further analysis.

5.14. Detecting Outliers

An **outlier** is a value that differs substantially from the rest of the data. Whether it's a *mistake* or a *legitimate rare event* is a judgment call — statistics can only flag candidates for you to investigate, not make that determination on its own.

5.14.1. The Z-Score Method

A z-score measures how many standard deviations a value is from the mean. A common rule of thumb flags anything with `|z| > 3` as an outlier.

5.15. pandas

```
z_scores = (debit_pd - debit_pd.mean()) / debit_pd.std()
outliers_z_pd = gl_pd.loc[debit_pd.index][z_scores.abs() > 3]
outliers_z_pd[["entry_id", "entry_date", "account", "debit",
 ↪  "department"]]
```

	entry_id	entry_date	account	debit	department
322	JE3900	2025-03-14	Accounts Receivable	145000.0	Sales
458	JE3901	2025-04-22	Cash	98000.0	Sales
668	JE3902	2025-06-09	Cost of Goods Sold	112000.0	Operations
1002	JE3903	2025-08-30	Accounts Receivable	176500.0	Sales
1068	JE3905	2025-09-19	Accounts Payable	63000.0	Operations
1222	JE3904	2025-11-02	Salaries Expense	87000.0	HR
1466	JE3906	2025-12-24	Cash	121000.0	Sales

5.16. polars

```
mean_val = debit_pl.mean()
std_val = debit_pl.std()

gl_with_z = (
    gl_pl.filter(pl.col("debit") > 0)
    .with_columns(
        ((pl.col("debit") - mean_val) / std_val).alias("z_score")
    )
)

outliers_z_pl = gl_with_z.filter(pl.col("z_score").abs() > 3)
outliers_z_pl.select(["entry_id", "entry_date", "account", "debit",
 ↪  "department"])
```

entry_id str	entry_date str	account str	debit f64	department str
"JE3900"	"2025-03-14"	"Accounts Receivable"	145000.0	"Sales"
"JE3901"	"2025-04-22"	"Cash"	98000.0	"Sales"
"JE3902"	"2025-06-09"	"Cost of Goods Sold"	112000.0	"Operations"
"JE3903"	"2025-08-30"	"Accounts Receivable"	176500.0	"Sales"
"JE3905"	"2025-09-19"	"Accounts Payable"	63000.0	"Operations"
"JE3904"	"2025-11-02"	"Salaries Expense"	87000.0	"HR"
"JE3906"	"2025-12-24"	"Cash"	121000.0	"Sales"

The z-score method flags exactly 7 **transactions** — all well over $60,000, an order of magnitude above the typical transaction size.

5.16.1. The IQR Method

An alternative, less sensitive to extreme values than the mean/std-based z-score, uses the interquartile range: flag anything above `Q3 + 1.5 × IQR`.

5.17. pandas

```
upper_bound = q3 + 1.5 * iqr
outliers_iqr_pd = gl_pd.loc[debit_pd.index][debit_pd > upper_bound]

print("IQR upper bound:", upper_bound)
print("Transactions flagged:", len(outliers_iqr_pd))
```

```
IQR upper bound: 7827.0975
Transactions flagged: 38
```

5.18. polars

```
upper_bound_pl = q3_pl + 1.5 * iqr_pl
outliers_iqr_pl = gl_with_z.filter(pl.col("debit") > upper_bound_pl)

print("IQR upper bound:", upper_bound_pl)
print("Transactions flagged:", outliers_iqr_pl.height)
```

```
IQR upper bound: 7827.0975
Transactions flagged: 38
```

Notice the IQR method flags **38 transactions** — far more than the 7 flagged by the z-score method. This isn't a bug; it's a real methodological difference:

> **!** Z-Score vs. IQR: Why Do They Disagree?
>
> The z-score method uses the **mean and standard deviation**, both of which are themselves heavily influenced by extreme values — a few huge transactions inflate the standard deviation, which makes the z-score threshold more forgiving. The IQR method uses the **median and quartiles**, which are far more resistant to extreme values, so it stays "strict" even when a few huge outliers are present. In practice, this means:
>
> - The z-**score method** here caught only the most extreme injected anomalies (all above $60,000)
> - The **IQR method** also caught moderately large, but still plausible, legitimate transactions
>
> Neither method is "correct" on its own — the IQR method casts a wider net and will produce more false positives requiring manual review, while the z-score method may miss moderately unusual transactions when the data itself is highly skewed. In practice, many audit and fraud analytics workflows use IQR-style thresholds as a first-pass filter specifically *because* it's more sensitive, and treat the resulting list as candidates for follow-up, not confirmed problems.

5.18.1. Outlier or Error? A Judgment Call

Statistical flagging is only the first step. Consider the seven z-score outliers above — some questions a real investigation would ask:

- Is a $176,500 sale plausible for this business, or does it look like a data entry error (an extra digit)?
- Two of the flagged transactions were posted on a weekend — is that normal for this company, or unusual enough to warrant follow-up?
- Do the posting employees for these entries normally handle transactions of this size?

```
gl_pd["entry_date"] = pd.to_datetime(gl_pd["entry_date"])
outliers_z_pd_with_day = outliers_z_pd.copy()
outliers_z_pd_with_day["entry_date"] =
 ↪  pd.to_datetime(outliers_z_pd_with_day["entry_date"])
outliers_z_pd_with_day["day_of_week"] =
 ↪  outliers_z_pd_with_day["entry_date"].dt.day_name()
outliers_z_pd_with_day[["entry_id", "entry_date", "day_of_week",
 ↪  "account", "debit", "posted_by"]]
```

	entry_id	entry_date	day_of_week	account	debit	posted_by
322	JE3900	2025-03-14	Friday	Accounts Receivable	145000.0	dwilson
458	JE3901	2025-04-22	Tuesday	Cash	98000.0	ktaylor
668	JE3902	2025-06-09	Monday	Cost of Goods Sold	112000.0	dwilson
1002	JE3903	2025-08-30	Saturday	Accounts Receivable	176500.0	dwilson
1068	JE3905	2025-09-19	Friday	Accounts Payable	63000.0	ktaylor
1222	JE3904	2025-11-02	Sunday	Salaries Expense	87000.0	dwilson
1466	JE3906	2025-12-24	Wednesday	Cash	121000.0	mchen

Two of the seven outliers were posted on a **Saturday or Sunday** — worth flagging for follow-up, since unusual timing combined with an unusually large amount is a stronger signal than either one alone. We'll build on exactly this kind of reasoning in Chapter 9 (Fraud Detection and Anomaly Detection).

5.19. Grouped EDA: Comparing Across Categories

Summary statistics computed on the whole dataset can hide differences between groups. Let's compare debit amounts by department.

5.20. pandas

```
grouped_pd = (
    gl_pd[gl_pd["debit"] > 0]
    .groupby("department")["debit"]
    .agg(["count", "mean", "median", "std"])
)
grouped_pd["cv"] = grouped_pd["std"] / grouped_pd["mean"]
```

```
grouped_pd
```

department	count	mean	median	std	cv
Finance	50	668.231200	657.435	253.988623	0.380091
HR	66	6953.794091	5534.105	10128.610229	1.456559
Operations	350	2608.582057	1987.360	6825.529984	2.616567
Sales	286	5719.480035	3184.825	16080.360228	2.811507

5.21. polars

```
grouped_pl = (
    gl_pl.filter(pl.col("debit") > 0)
    .group_by("department")
    .agg([
        pl.len().alias("count"),
        pl.col("debit").mean().alias("mean"),
        pl.col("debit").median().alias("median"),
        pl.col("debit").std().alias("std"),
    ])
    .with_columns((pl.col("std") / pl.col("mean")).alias("cv"))
    .sort("department")
)
grouped_pl
```

department str	count u32	mean f64	median f64	std f64	cv f64
”Finance”	50	668.2312	657.435	253.988623	0.380091
”HR”	66	6953.794091	5534.105	10128.610229	1.456559
”Operations”	350	2608.582057	1987.36	6825.529984	2.616567
”Sales”	286	5719.480035	3184.825	16080.360228	2.811507

A few things worth noticing:

- **Finance** has by far the *lowest* variability (small coefficient of variation) — consistent with routine, predictable transactions like depreciation and insurance amortization.

- **Sales** and **HR** show much higher variability, partly because several of our injected large outliers landed in those departments.
- **Operations** has the highest transaction *count* by far, reflecting frequent inventory and vendor-payment activity.

This kind of grouped comparison is often how an unusual pattern first gets noticed — not from a single transaction, but from *one department's numbers looking different from the others* in a way that invites a closer look.

5.22. Putting It Together: A First-Pass EDA Checklist

The steps in this chapter form a reusable sequence you can apply to *any* new dataset before moving on to deeper analysis:

1. **Check shape and structure** — row/column count, data types (Chapter 3–4 territory)
2. **Choose appropriate summary statistics** based on each variable's level of measurement
3. **Compute central tendency and dispersion** — and compare mean vs. median for signs of skew
4. **Visualize the distribution** — histogram and boxplot, zooming in if outliers dominate the scale
5. **Flag outliers** — using both z-score and IQR methods, since they disagree in informative ways
6. **Compare across categories** — group by a relevant categorical variable (department, account, employee) to see if any group stands out

We applied all six steps to `general_ledger_large.csv` in this chapter. In Chapter 6, we'll build on this foundation with more advanced and polished visualizations; in Chapters 8–9, we'll apply this same EDA mindset directly to audit and fraud analytics tasks.

5.23. Chapter Summary

- EDA means getting to know your data — its center, its spread, its shape, its oddities — before running any formal analysis, audit test, or model.
- A variable's level of measurement determines which summary statistics are meaningful; this dataset's ratio-level `debit/credit` columns support full arithmetic, while nominal columns like `department` only support counting.
- Mean and median can diverge substantially in skewed financial data — always check both, since the mean alone can be misleading.
- The coefficient of variation allows fair comparison of variability across accounts or groups on very different scales.

- Histograms can be distorted by extreme outliers; zooming in (temporarily excluding extreme values) often reveals the true underlying shape.
- The z-score and IQR methods for outlier detection can disagree substantially — IQR is more sensitive (and produces more false positives) since it's based on the more outlier-resistant median and quartiles, while z-score is based on the more outlier-sensitive mean and standard deviation.
- A flagged outlier is not automatically an error — determining that requires investigation and accounting judgment, not just a statistical threshold.
- Grouped EDA (comparing summary statistics across categories) is often how an unusual pattern first becomes visible.

5.24. Discussion Questions

1. Why does the mean of the debit amounts in this chapter's dataset exceed the median? What would it mean if the opposite were true — median greater than mean?
2. The IQR method flagged 38 transactions, the z-score method only 7. If you were leading an audit team with limited time, which method would you use for an initial screen, and why?
3. Two of the seven z-score outliers were posted on a weekend. On its own, is a weekend posting evidence of a problem? What additional information would make you more or less concerned?

5.25. Exercises

1. Using `general_ledger_large.csv`, compute the mean, median, and standard deviation of `debit` for just the **Sales** department, and compare it to the whole-dataset statistics computed in this chapter.
2. Re-run the z-score and IQR outlier detection methods using the `credit` column instead of `debit`. Are the same journal entries flagged? Explain why or why not, given how the dataset is structured.
3. Produce a histogram of debit amounts grouped by department (four separate histograms, or one overlaid chart) and describe, in a few sentences, how the distributions differ across departments.
4. Pick one of the seven z-score-flagged outliers. Write a short (100-word) memo describing what additional information you would request before concluding whether it's a legitimate transaction or an error.

6. Data Visualization for Accounting

Learning Objectives

By the end of this chapter, you should be able to:

- Explain why visualization complements (rather than replaces) the numeric EDA from Chapter 5
- Describe the difference between seaborn's statistical-plotting approach and plotnine's grammar-of-graphics approach
- Create bar charts, line charts, distribution plots, scatter plots, heatmaps, and faceted charts for accounting data using both libraries
- Recognize misleading chart design choices, especially truncated axes, and apply basic design principles for financial visualizations
- Choose an appropriate chart type for a given accounting analytics question

6.1. Why Visualize? From Numbers to Insight

Chapter 5 gave us *numbers*: a mean of $4,044, a median of $2,457, seven z-score outliers. Those numbers are precise, but they take effort to absorb, and they don't always reveal a pattern's shape at a glance. A well-designed chart can communicate the same finding to a controller or audit committee in seconds, and can reveal patterns — a skew, a cluster, a trend — that are much harder to spot from a table of statistics alone.

This chapter builds on the same dataset from Chapter 5, `general_ledger_large.csv`, and turns several of that chapter's numeric findings into visuals.

6.2. Two Visualization Philosophies: Seaborn vs. Plotnine

This chapter teaches two Python visualization libraries - `seaborn` and `plotnine` - so you can compare the same chart built two different ways:

- **Seaborn**, built on top of matplotlib, provides ready-made statistical plotting functions (`barplot`, `histplot`, `scatterplot`) — you call a function that matches your chart type, and pass in your data and variables.
- **Plotnine**, a Python implementation of R's `ggplot2`, uses a **grammar of graphics**: you build a plot by layering independent components — data, aesthetic mappings, geometric shapes — with `+`.

	Seaborn	Plotnine
Approach	Pick a function matching your chart type	Layer components: data + aesthetics + geometry
Built on	matplotlib	Its own rendering, inspired by R's `ggplot2`
Learning curve	Fast to get a standard chart working	Slightly steeper at first, very consistent once learned
Customization	Matplotlib escape hatch available for fine control	Everything is a layer you can add/replace
Common in accounting/finance practice	Very common, well-documented	Growing, especially among analysts with R backgrounds

We'll also use plain **matplotlib** in one section later in this chapter, for a case where low-level control over a custom layout is genuinely useful — but for the vast majority of charts, `seaborn` or `plotnine` will get you there faster.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from plotnine import *

gl = pd.read_csv("data/general_ledger_large.csv")
gl["entry_date"] = pd.to_datetime(gl["entry_date"])

# Add account_type, matching each account to its category
# (In Chapter 4, we got this via a join to chart_of_accounts.csv - a
 ↪  simple
# mapping works just as well here since this chapter's focus is
 ↪  visualization)
account_type_map = {
    "Cash": "Asset", "Accounts Receivable": "Asset", "Inventory":
     ↪  "Asset",
    "Prepaid Insurance": "Asset", "Accumulated Depreciation -
     ↪  Equipment": "Contra-Asset",
```

```
    "Accounts Payable": "Liability", "Notes Payable": "Liability",
    "Sales Revenue": "Revenue",
    "Cost of Goods Sold": "Expense", "Salaries Expense": "Expense",
    "Rent Expense": "Expense", "Utilities Expense": "Expense",
    "Depreciation Expense": "Expense", "Insurance Expense": "Expense",
    "Interest Expense": "Expense",
}
gl["account_type"] = gl["account"].map(account_type_map)

debit = gl[gl["debit"] > 0].copy()
debit.head()
```

	entry_id	entry_date	account	debit	credit	department	posted_by	account_typ
0	JE3253	2025-01-01	Cost of Goods Sold	1231.76	0.0	Operations	mchen	Expense
2	JE3413	2025-01-01	Accounts Receivable	3024.76	0.0	Sales	agarcia	Asset
4	JE3476	2025-01-01	Cash	4476.03	0.0	Sales	rpatel	Asset
6	JE3030	2025-01-02	Accounts Payable	1488.48	0.0	Operations	agarcia	Liability
8	JE3066	2025-01-02	Cost of Goods Sold	2705.81	0.0	Operations	rpatel	Expense

6.3. Bar Charts: Comparing Categories

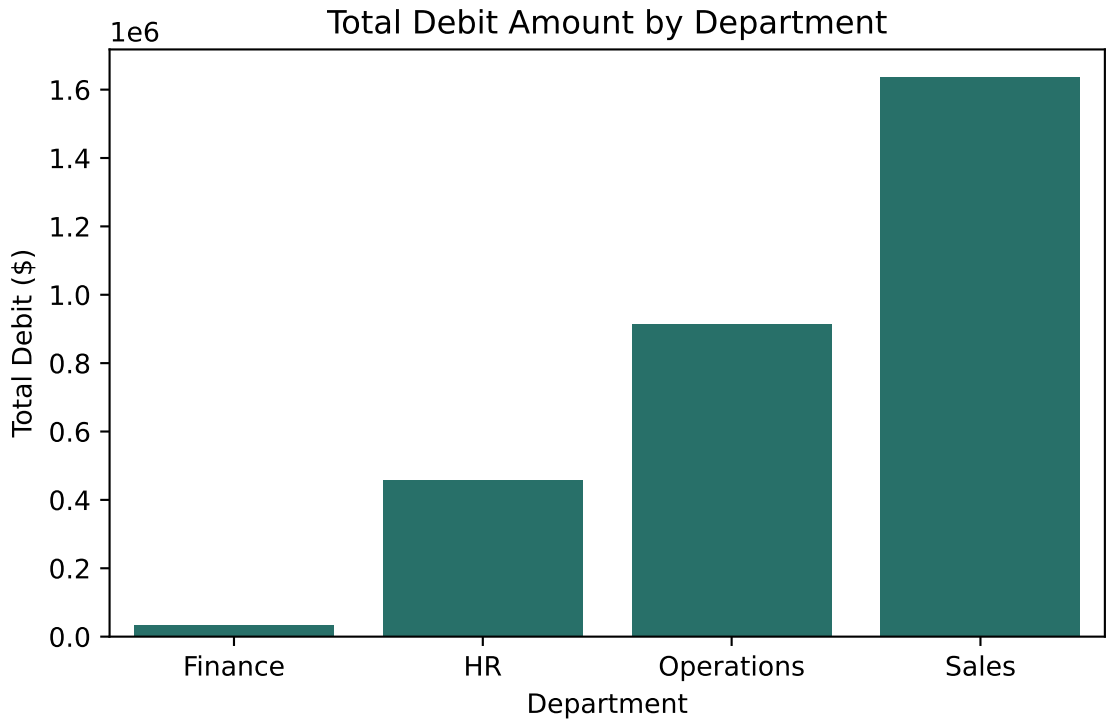
A bar chart is the natural choice for comparing a total across categories — here, total debit amount by department.

```
dept_totals = debit.groupby("department",
 ↪  as_index=False)["debit"].sum()
dept_totals
```

	department	debit
0	Finance	33411.56
1	HR	458950.41
2	Operations	913003.72
3	Sales	1635771.29

6.4. seaborn

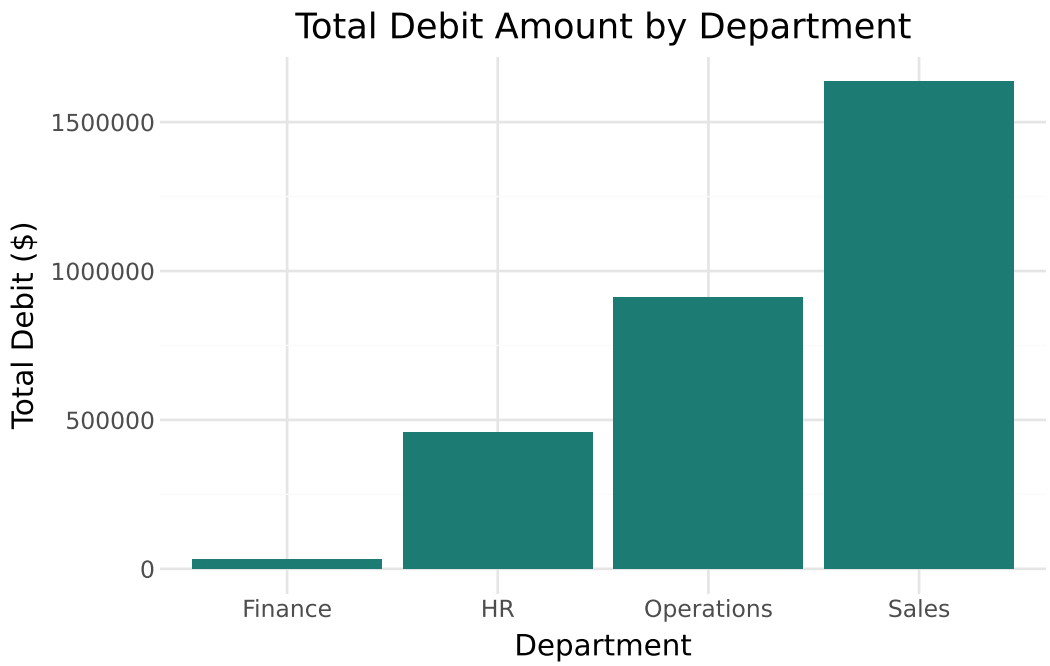
```
plt.figure(figsize=(6, 4))
sns.barplot(data=dept_totals, x="department", y="debit",
 ↪  color="#1c7c73")
plt.title("Total Debit Amount by Department")
plt.ylabel("Total Debit ($)")
plt.xlabel("Department")
plt.tight_layout()
plt.show()
```



6.5. plotnine

```
(
    ggplot(dept_totals, aes(x="department", y="debit"))
    + geom_col(fill="#1c7c73")
    + labs(title="Total Debit Amount by Department", x="Department",
     ↪  y="Total Debit ($)")
    + theme_minimal()
```

```
)
```



Sales accounts for by far the largest share of debit volume, followed by Operations, HR, and Finance — consistent with the grouped statistics we computed in Chapter 5.

6.6. Line Charts: Trends Over Time

Line charts are the natural choice for showing change over time — here, total debit volume by month across the fiscal year.

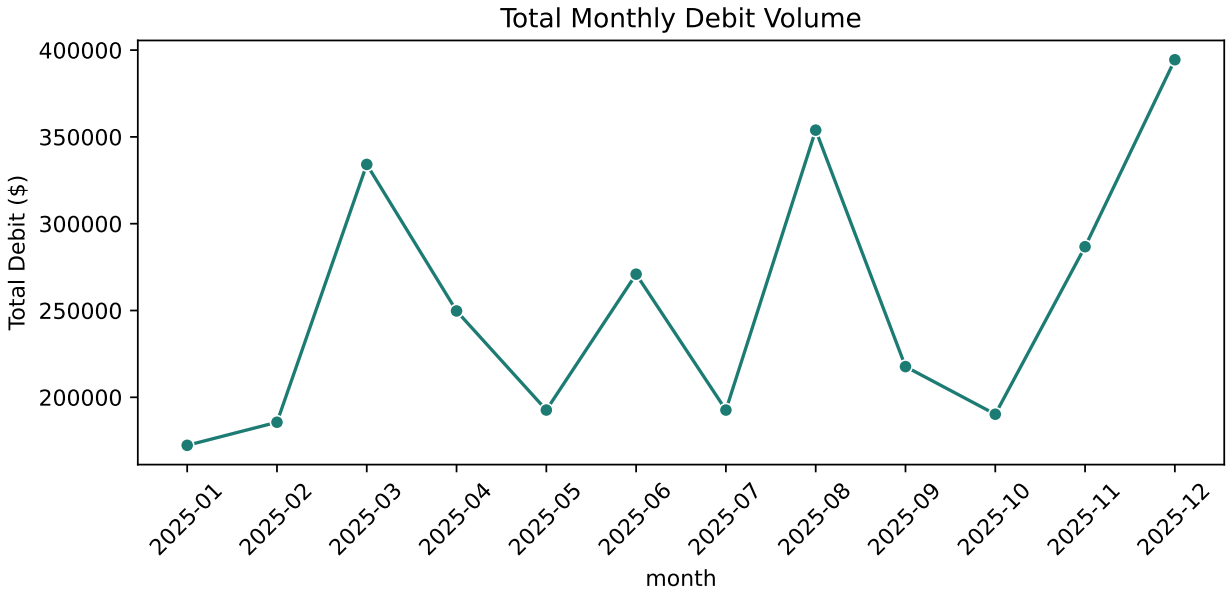
```
debit["month"] = debit["entry_date"].dt.to_period("M").astype(str)
monthly = debit.groupby("month", as_index=False)["debit"].sum()
monthly
```

	month	debit
0	2025-01	172341.25
1	2025-02	185630.06
2	2025-03	334149.58
3	2025-04	249734.34
4	2025-05	192663.53
5	2025-06	270924.54

	month	debit
6	2025-07	192653.19
7	2025-08	353903.51
8	2025-09	217687.87
9	2025-10	190253.40
10	2025-11	286748.33
11	2025-12	394447.38

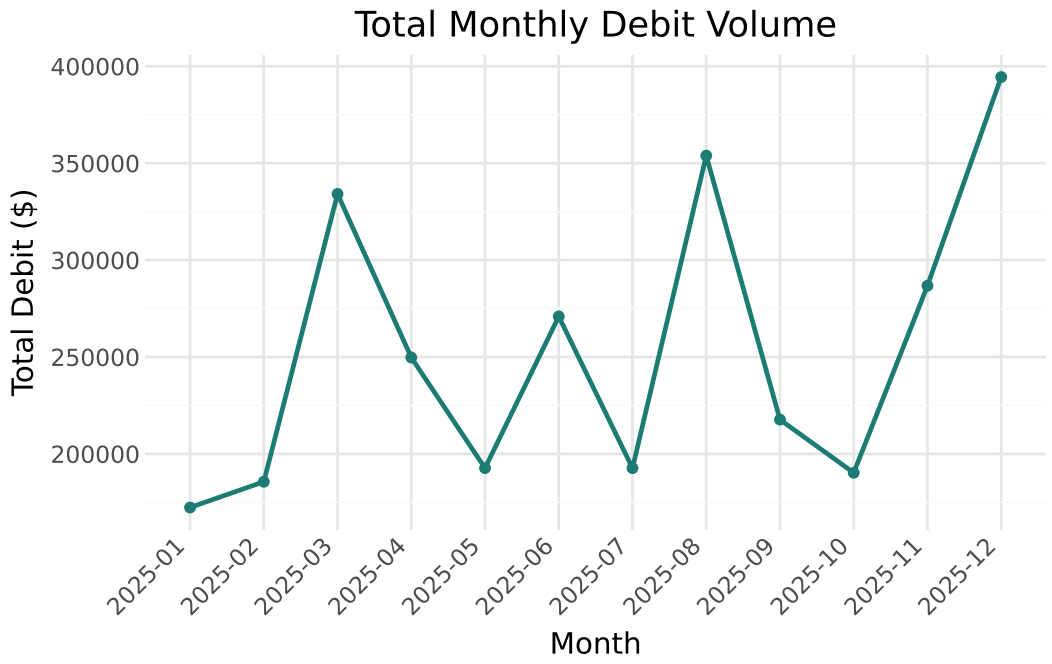
6.7. seaborn

```
plt.figure(figsize=(8, 4))
sns.lineplot(data=monthly, x="month", y="debit", marker="o",
 ↪  color="#1c7c73")
plt.xticks(rotation=45)
plt.title("Total Monthly Debit Volume")
plt.ylabel("Total Debit ($)")
plt.tight_layout()
plt.show()
```



6.8. plotnine

```
(
    ggplot(monthly, aes(x="month", y="debit", group=1))
    + geom_line(color="#1c7c73", size=1)
    + geom_point(color="#1c7c73")
    + labs(title="Total Monthly Debit Volume", x="Month", y="Total
     ↪  Debit ($)")
    + theme_minimal()
    + theme(axis_text_x=element_text(rotation=45, hjust=1))
)
```



Three months — March, August, and December — stand out with noticeably higher totals. If you recall from Chapter 5, this is exactly where several of our seven z-score-flagged outlier transactions landed. A line chart makes that timing pattern immediately visible in a way that a table of monthly totals would not.

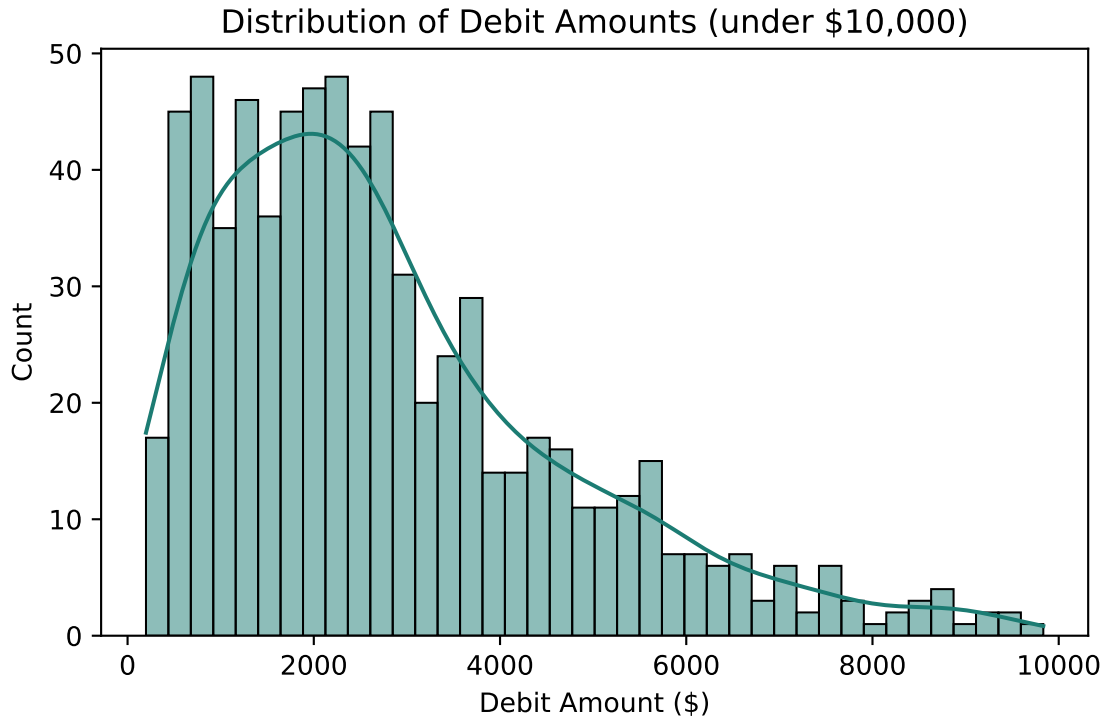
6.9. Distribution Plots: Revisiting Chapter 5, Properly Styled

Chapter 5 showed that transaction amounts are heavily right-skewed, and that zooming in on amounts under $10,000 revealed the underlying shape more clearly. Let's rebuild that finding with proper statistical plotting tools.

```
zoomed = debit[debit["debit"] < 10000]
```

6.10. seaborn

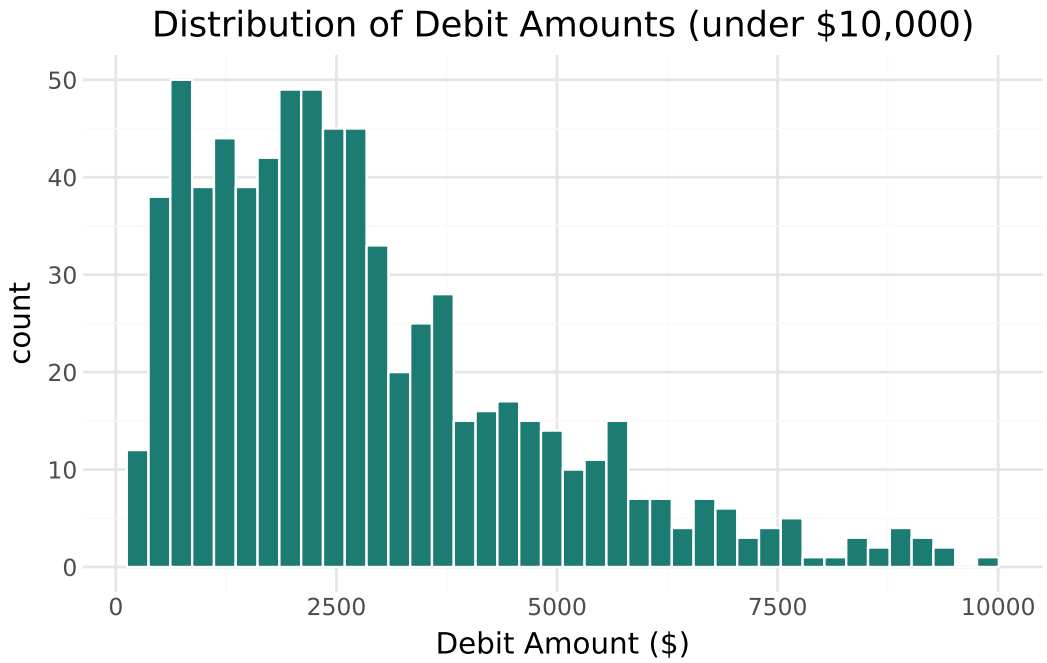
```
plt.figure(figsize=(6, 4))
sns.histplot(data=zoomed, x="debit", bins=40, kde=True,
 ↪  color="#1c7c73")
plt.title("Distribution of Debit Amounts (under $10,000)")
plt.xlabel("Debit Amount ($)")
plt.tight_layout()
plt.show()
```



6.11. plotnine

```
(
    ggplot(zoomed, aes(x="debit"))
    + geom_histogram(bins=40, fill="#1c7c73", color="white")
```

```
    + labs(title="Distribution of Debit Amounts (under $10,000)",
     ↪  x="Debit Amount ($)")
    + theme_minimal()
)
```

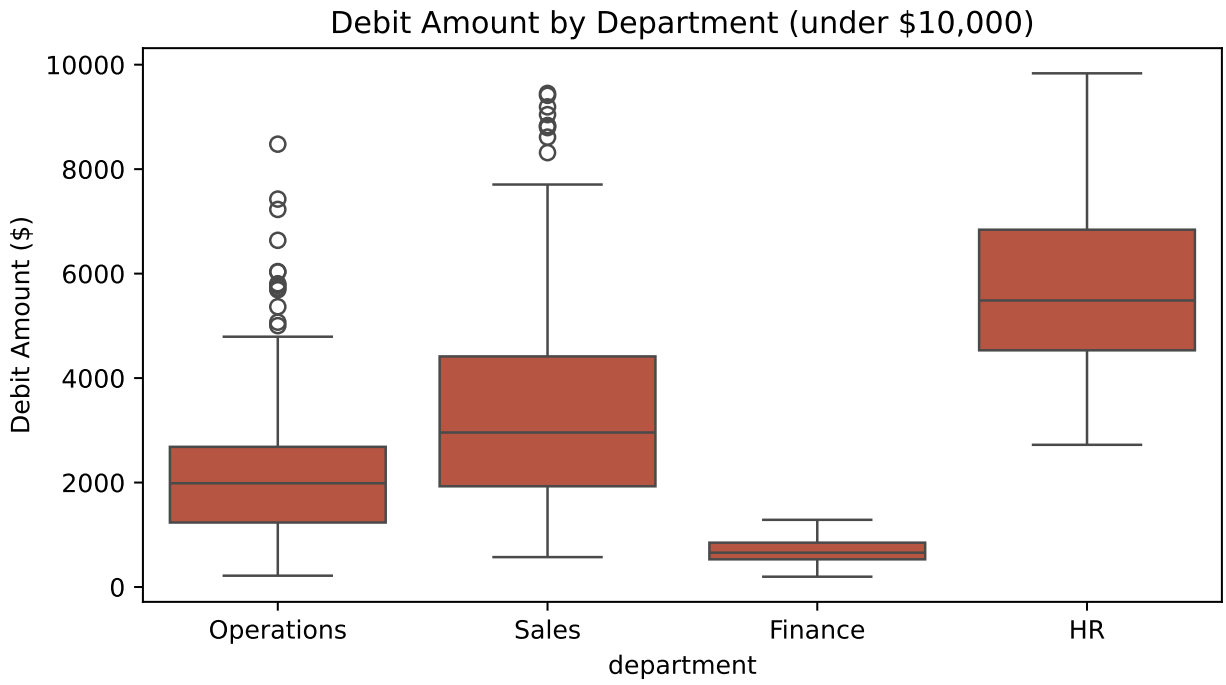


Seaborn's `kde=True` option overlays a smoothed density curve directly on the histogram — a nice way to see the distribution's overall shape without being distracted by individual bin heights.

Let's also compare distributions across departments with a boxplot and a violin plot, which shows both the summary statistics (boxplot) and the full shape of the distribution (violin).

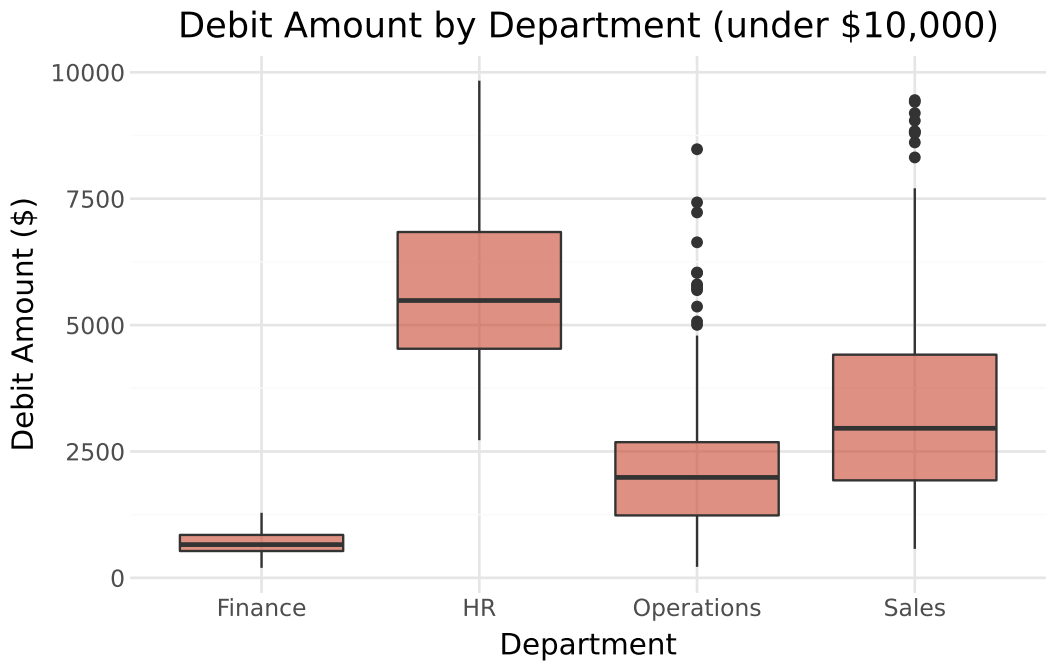
6.12. seaborn

```
plt.figure(figsize=(7, 4))
sns.boxplot(data=zoomed, x="department", y="debit", color="#c9482f")
plt.title("Debit Amount by Department (under $10,000)")
plt.ylabel("Debit Amount ($)")
plt.tight_layout()
plt.show()
```



6.13. plotnine

```
from plotnine import geom_boxplot
(
    ggplot(zoomed, aes(x="department", y="debit"))
    + geom_boxplot(fill="#c9482f", alpha=0.6)
    + labs(title="Debit Amount by Department (under $10,000)",
     ↪  x="Department", y="Debit Amount ($)")
    + theme_minimal()
)
```



> **Connect to Practice**
>
> Finance's violin/box shape is noticeably narrower and lower than the others — consistent with the low coefficient of variation we calculated for Finance in Chapter 5. Seeing this visually, rather than just as a single CV number, makes the point land faster for a non-technical audience.

6.14. Scatter Plots: Spotting Outliers Visually

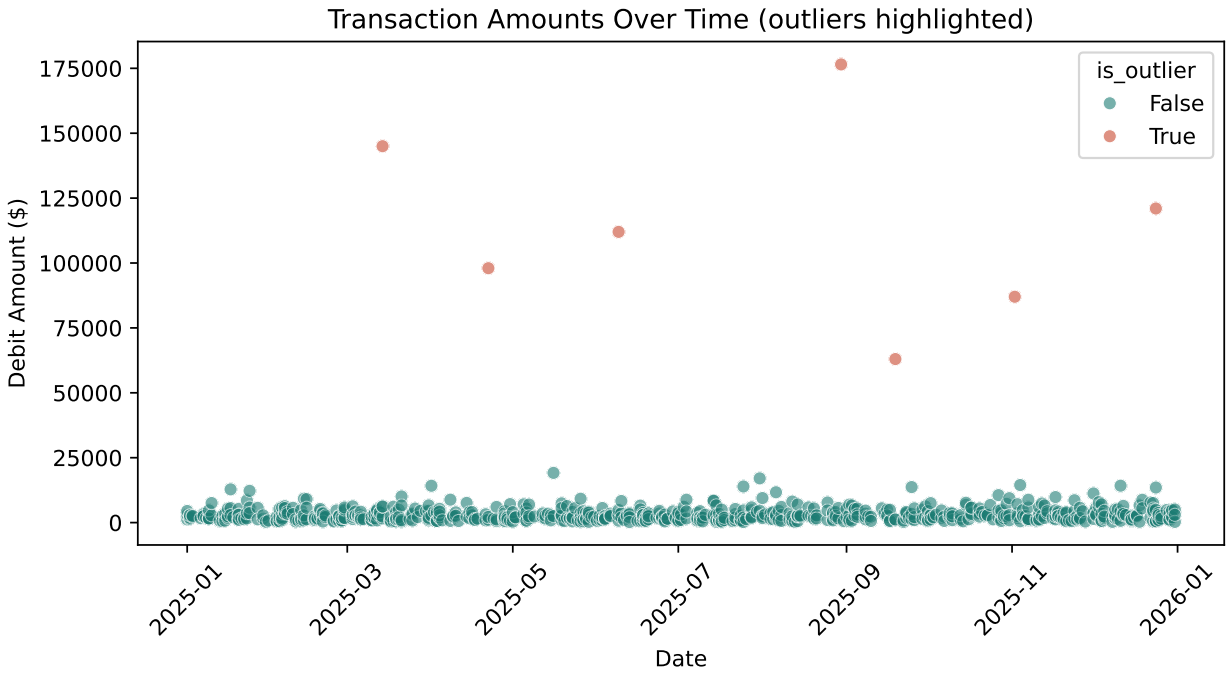
Chapter 5 flagged seven transactions using a z-score threshold. Let's plot every transaction over time and highlight those outliers directly.

```
mean_debit, std_debit = debit["debit"].mean(), debit["debit"].std()
debit["z_score"] = (debit["debit"] - mean_debit) / std_debit
debit["is_outlier"] = debit["z_score"].abs() > 3
```

6.15. seaborn

```
plt.figure(figsize=(8, 4.5))
sns.scatterplot(
```

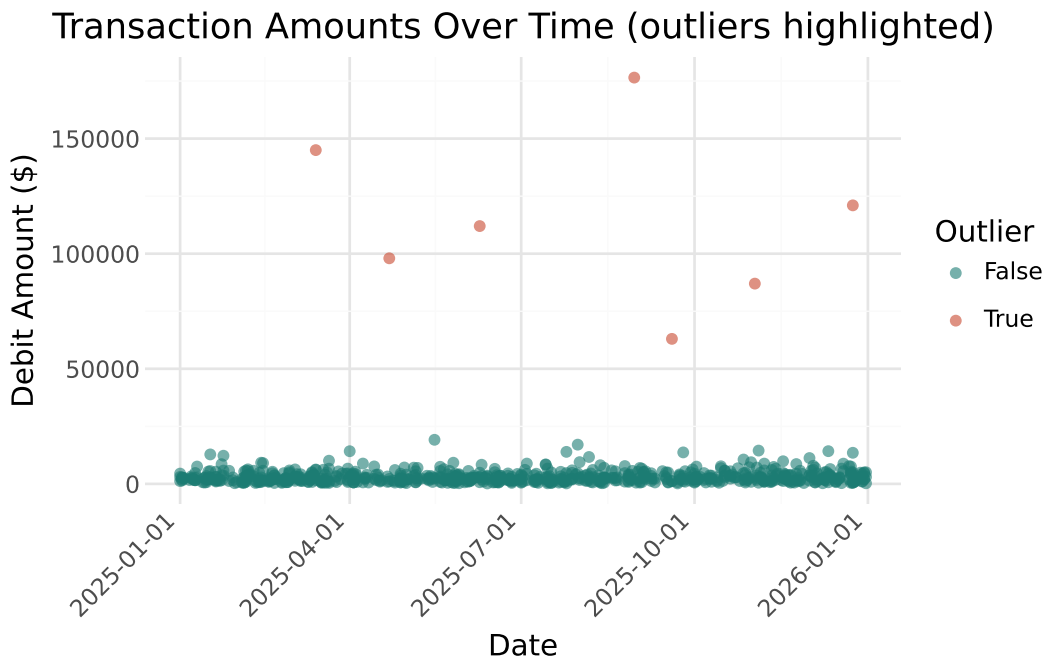
```
    data=debit, x="entry_date", y="debit", hue="is_outlier",
    palette={True: "#c9482f", False: "#1c7c73"}, alpha=0.6
)
plt.title("Transaction Amounts Over Time (outliers highlighted)")
plt.ylabel("Debit Amount ($)")
plt.xlabel("Date")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



6.16. plotnine

```
(
    ggplot(debit, aes(x="entry_date", y="debit", color="is_outlier"))
    + geom_point(alpha=0.6)
    + scale_color_manual(values={True: "#c9482f", False: "#1c7c73"})
    + labs(title="Transaction Amounts Over Time (outliers
     ↪  highlighted)",
           x="Date", y="Debit Amount ($)", color="Outlier")
    + theme_minimal()
    + theme(axis_text_x=element_text(rotation=45, hjust=1))
```

```
)
```



The seven outliers are immediately obvious, sitting far above the dense cluster of routine transactions near the bottom of the chart — this is a far faster way to communicate "here's what stands out" to a reviewer than a filtered table alone.

6.17. Heatmaps: Cross-Tabulated Data

A heatmap is well-suited to showing a two-way summary — here, total debit volume by department **and** account type at once.

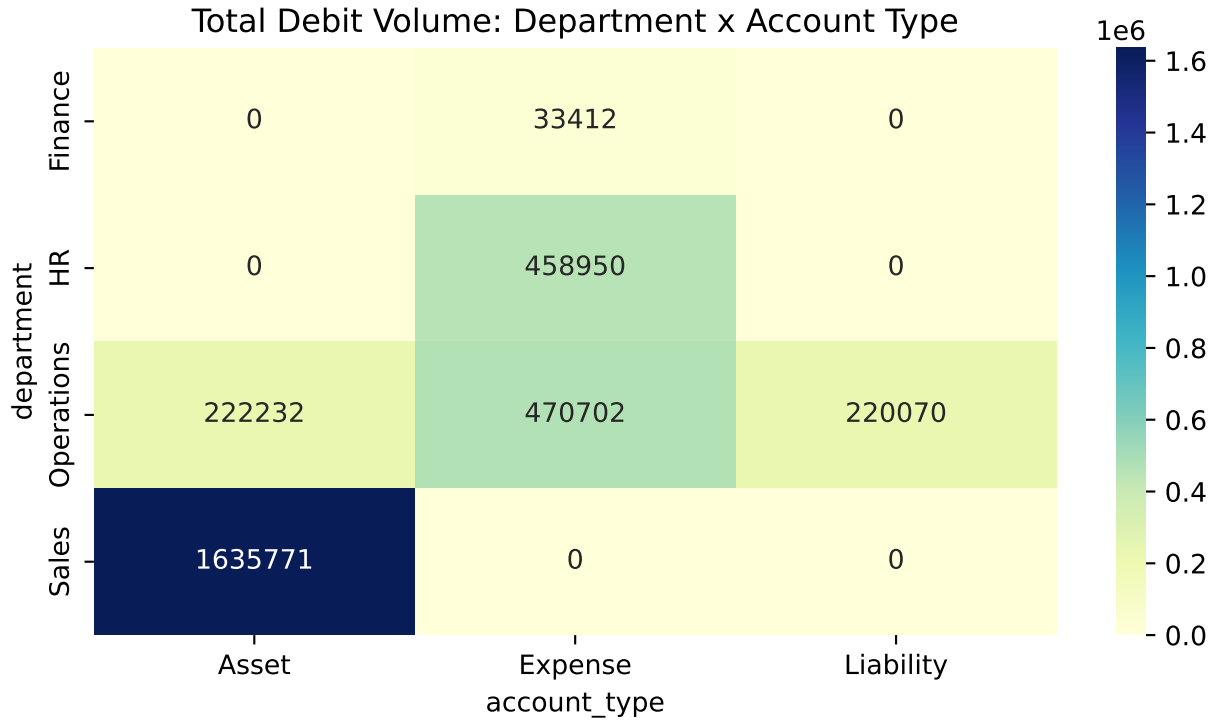
```
pivot = debit.pivot_table(
    index="department", columns="account_type", values="debit",
    aggfunc="sum", fill_value=0
)
pivot
```

account_type department	Asset	Expense	Liability
Finance	0.00	33411.56	0.00
HR	0.00	458950.41	0.00

account_type department	Asset	Expense	Liability
Operations	222231.90	470702.06	220069.76
Sales	1635771.29	0.00	0.00

6.18. seaborn

```
plt.figure(figsize=(7, 4))
sns.heatmap(pivot, annot=True, fmt=".0f", cmap="YlGnBu")
plt.title("Total Debit Volume: Department x Account Type")
plt.tight_layout()
plt.show()
```

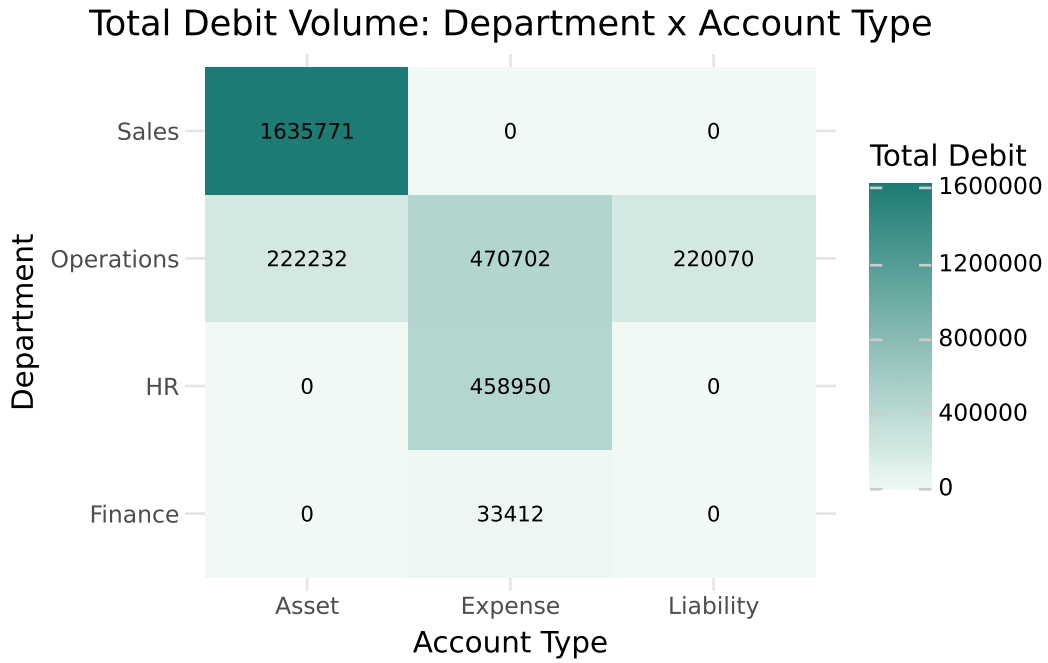


6.19. plotnine

```
long_form = pivot.reset_index().melt(
    id_vars="department", var_name="account_type",
↪   value_name="total_debit"
```

```
)

(
    ggplot(long_form, aes(x="account_type", y="department",
    fill="total_debit"))
    + geom_tile()
    + geom_text(aes(label="total_debit.round(0).astype(int)"), size=8)
    + scale_fill_gradient(low="#f0f9f4", high="#1c7c73")
    + labs(title="Total Debit Volume: Department x Account Type",
           x="Account Type", y="Department", fill="Total Debit")
    + theme_minimal()
)
```



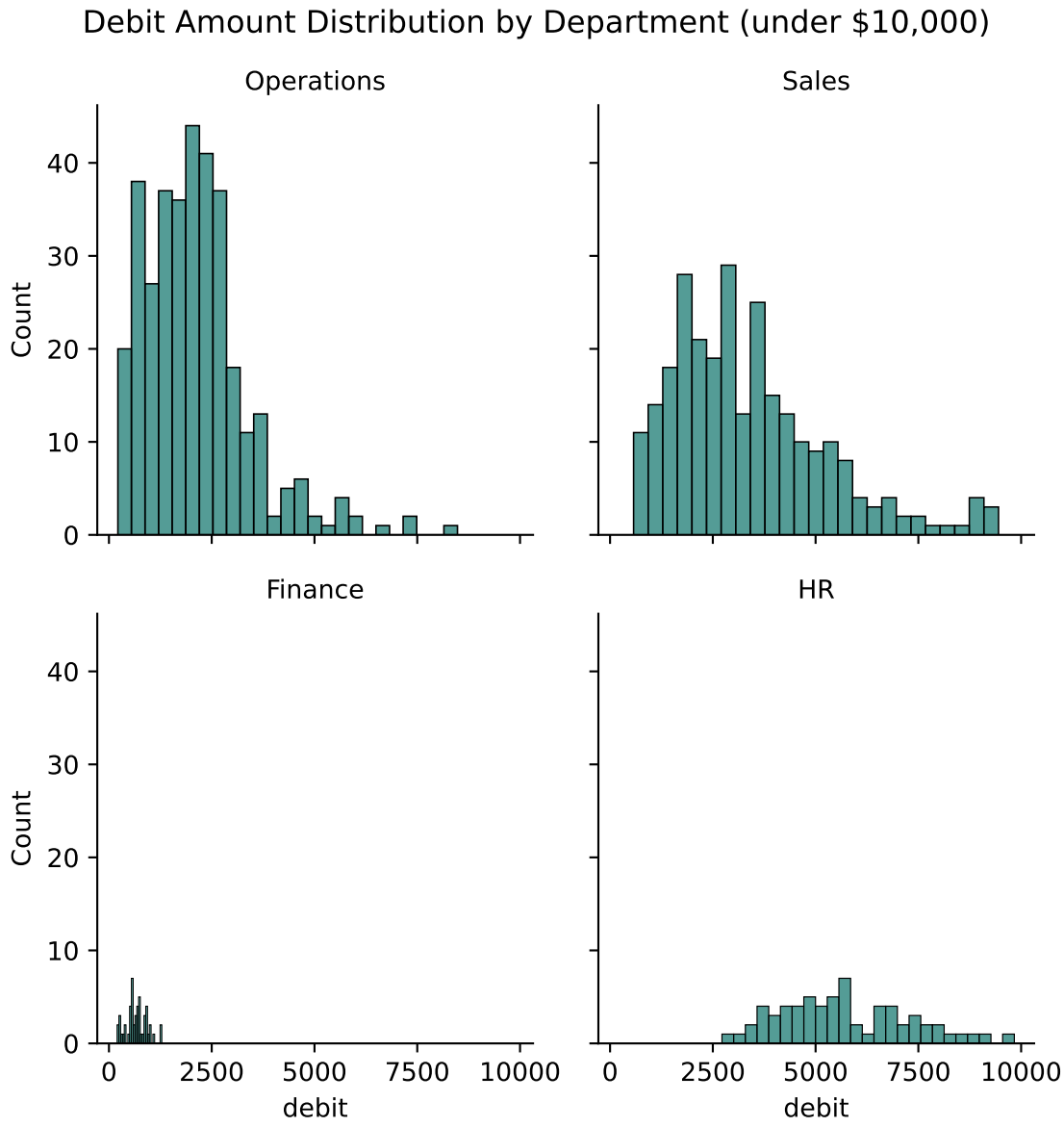
This view immediately shows which department-account combinations don't occur at all (the empty/zero cells) versus where activity concentrates — Sales transactions are almost entirely Asset-type accounts (Cash, Accounts Receivable), while Operations activity is spread across Asset, Expense, and Liability accounts.

6.20. Faceting: Small Multiples

Faceting splits one chart into a grid of smaller panels, one per category — a powerful way to compare distribution shapes across groups at a glance, rather than overlaying them all on one crowded chart.

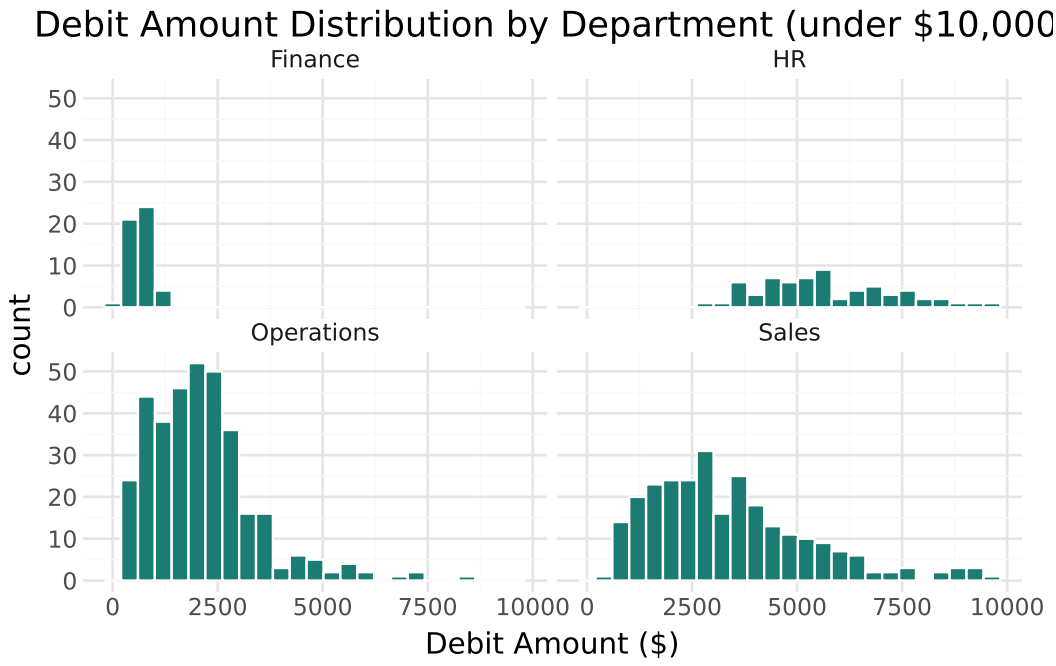
6.21. seaborn

```
g = sns.FacetGrid(zoomed, col="department", col_wrap=2, height=3)
g.map_dataframe(sns.histplot, x="debit", bins=25, color="#1c7c73")
g.set_titles("{col_name}")
g.figure.suptitle("Debit Amount Distribution by Department (under
 ↪  $10,000)", y=1.03)
plt.show()
```



6.22. plotnine

```
(
    ggplot(zoomed, aes(x="debit"))
    + geom_histogram(bins=25, fill="#1c7c73", color="white")
    + facet_wrap("~department")
    + labs(title="Debit Amount Distribution by Department (under
     ↪  $10,000)",
           x="Debit Amount ($)")
    + theme_minimal()
)
```



Faceting makes it easy to see, for example, that Finance's transaction volume is both much smaller in count and much more tightly clustered near zero than the other three departments — a pattern that would be much harder to notice in a single combined histogram.

6.23. When to Reach for Matplotlib

Seaborn and plotnine handle the overwhelming majority of accounting charts you'll need. Occasionally, though, you want a **custom multi-panel layout** — combining several unrelated chart types into a single "dashboard" figure — where matplotlib's low-level control is genuinely the easier path.

```
import matplotlib.gridspec as gridspec

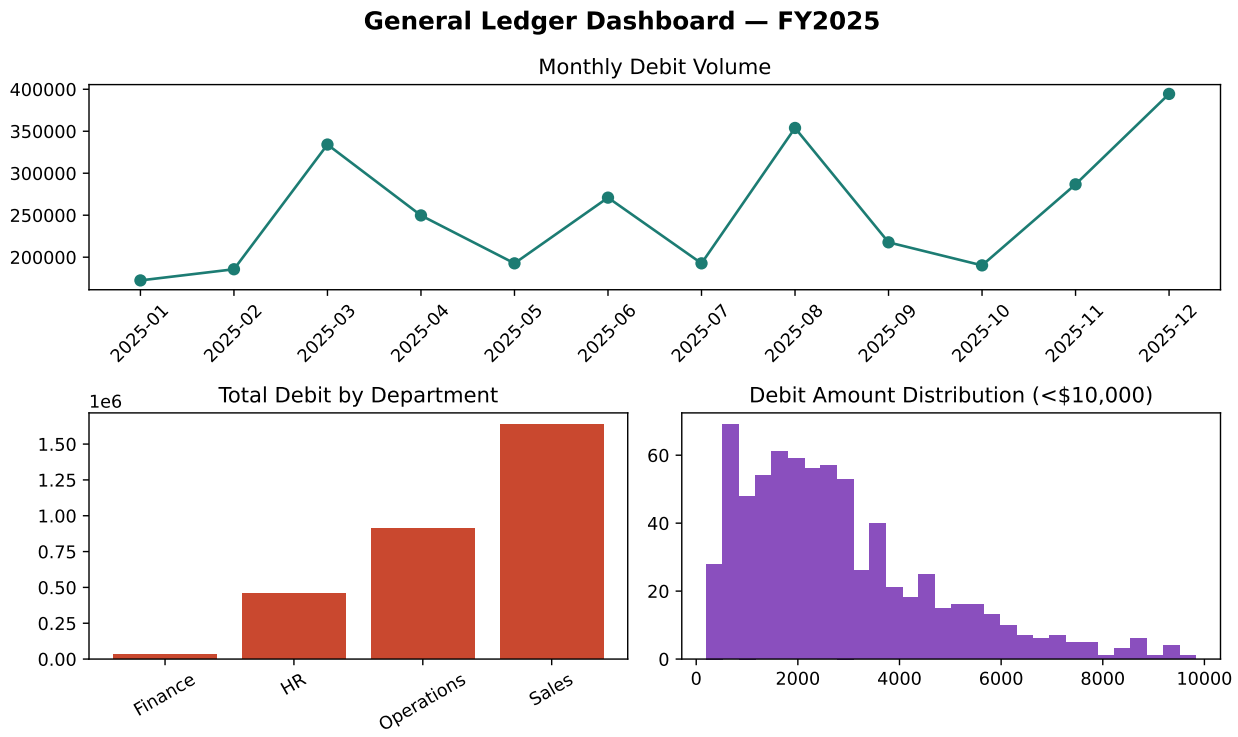
fig = plt.figure(figsize=(10, 6))
gs = gridspec.GridSpec(2, 2, figure=fig, height_ratios=[1, 1.2])

ax1 = fig.add_subplot(gs[0, :])
ax1.plot(monthly["month"], monthly["debit"], marker="o",
 ↪  color="#1c7c73")
ax1.set_title("Monthly Debit Volume")
ax1.tick_params(axis="x", rotation=45)

ax2 = fig.add_subplot(gs[1, 0])
ax2.bar(dept_totals["department"], dept_totals["debit"],
 ↪  color="#c9482f")
ax2.set_title("Total Debit by Department")
ax2.tick_params(axis="x", rotation=30)

ax3 = fig.add_subplot(gs[1, 1])
ax3.hist(zoomed["debit"], bins=30, color="#8a4fbe")
ax3.set_title("Debit Amount Distribution (<$10,000)")

fig.suptitle("General Ledger Dashboard - FY2025", fontsize=14,
 ↪  fontweight="bold")
plt.tight_layout()
plt.show()
```



> **Connect to Practice**
>
> This kind of combined dashboard figure — several related charts arranged in one image — is a common deliverable format for a management reporting package. Matplotlib's `GridSpec` gives you precise control over panel sizes and positions that would be awkward to achieve directly in seaborn or plotnine, which is why it's worth knowing even if you use it rarely.

6.24. Design Principles for Accounting Visualizations

A technically correct chart can still mislead a reader if it's designed carelessly. A few principles worth internalizing:

6.24.1. Truncated Axes Exaggerate Change

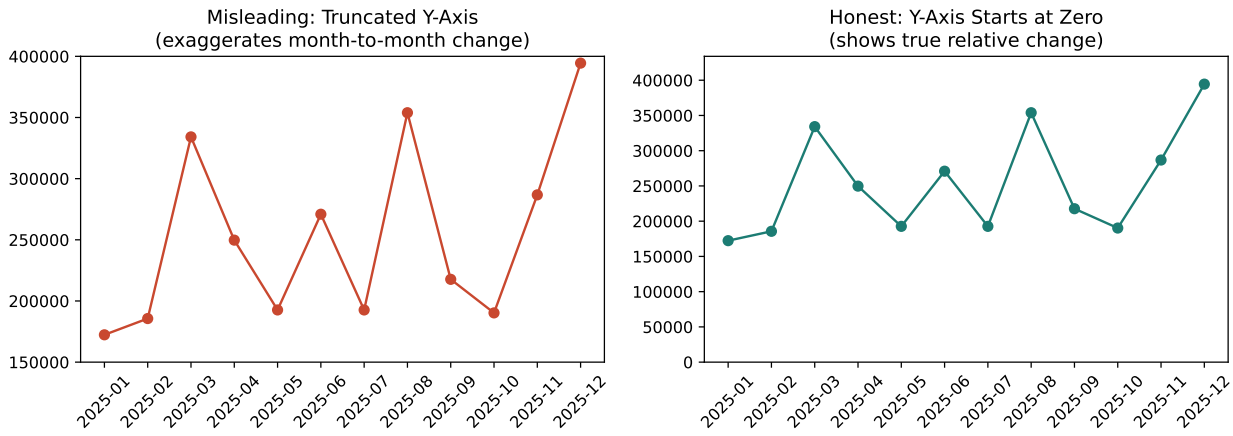
```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].plot(monthly["month"], monthly["debit"], marker="o",
 ↪  color="#c9482f")
axes[0].set_ylim(150000, 400000)
```

```
axes[0].set_title("Misleading: Truncated Y-Axis\n(exaggerates
 ↪  month-to-month change)")
axes[0].tick_params(axis="x", rotation=45)

axes[1].plot(monthly["month"], monthly["debit"], marker="o",
 ↪  color="#1c7c73")
axes[1].set_ylim(0, monthly["debit"].max() * 1.1)
axes[1].set_title("Honest: Y-Axis Starts at Zero\n(shows true relative
 ↪  change)")
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```



Both charts show *identical* data. The left version, with its y-axis starting at $150,000 instead of zero, makes the dip between March and May look like a dramatic collapse. The right version, with a y-axis starting at zero, shows the same fluctuation as the far more modest change it actually is. This is one of the most common ways financial charts mislead — intentionally or not — and one of the easiest to check for: **look at where the y-axis starts.**

6.24.2. Other Design Principles

- **Color choice matters.** Avoid relying on red/green alone to signal "bad/good" — a meaningful share of viewers have red-green color vision deficiency and won't be able to distinguish them. Pair color with position, labels, or a different visual encoding (e.g., an icon or explicit text) where the distinction matters.
- **Label clearly for your actual audience.** A chart for an audit committee or controller should have a clear title, labeled axes with units (e.g., "$", not just numbers), and should not assume the reader already knows what "debit" and "credit" refer to in this specific context.

- **Match the chart type to the question.** A line chart implies a trend over a continuous variable (usually time) — using one to connect unrelated categories (like departments) implies a false sense of order or continuity. A bar chart is almost always the better choice for comparing discrete categories.

6.25. Chapter Summary

- Visualization complements numeric EDA (Chapter 5) by making patterns — trends, skew, clusters, outliers — visible at a glance, especially for non-technical audiences.
- Seaborn offers ready-made statistical plotting functions; plotnine builds charts by layering grammar-of-graphics components — both accomplish the same tasks with different syntax philosophies.
- Bar charts suit category comparisons, line charts suit trends over time, histograms/boxplots/violins suit distributions, scatter plots suit relationships (including visually spotting outliers), heatmaps suit two-way cross-tabulated summaries, and faceting enables side-by-side comparison across groups.
- Matplotlib remains useful for custom multi-panel layouts that don't fit neatly into seaborn's or plotnine's higher-level interfaces.
- Truncated y-axes are one of the most common ways financial charts mislead, by exaggerating the apparent magnitude of change — always check where a y-axis starts, both in your own work and when reviewing others'.
- Good design (color choice, clear labeling, appropriate chart type) is not a cosmetic afterthought — it directly affects whether your audience draws an accurate conclusion.

6.26. Discussion Questions

1. Choose one chart from this chapter and recreate it in the *other* library (if you focused on the seaborn tab, try plotnine, and vice versa). Which did you find more intuitive, and why?
2. Look back at the misleading vs. honest y-axis example. Can you think of a real-world scenario in accounting or finance reporting where someone might be tempted to truncate an axis, intentionally or not?
3. Why might a violin plot reveal something a boxplot alone would miss, even though both summarize the same underlying data?

6.27. Exercises

1. Build a faceted distribution plot (like the one in this chapter) using `posted_by` instead of `department`. Are any employees' transaction patterns visually distinct from the others?
2. Recreate the department bar chart from this chapter, but sort the bars from largest to smallest total debit rather than alphabetically. (Hint: sort the underlying DataFrame before plotting.)
3. Find or construct a "bad" chart — either one you build yourself with a deliberately truncated axis or misleading color choice, or one you find in the wild — and write a short critique (150 words) explaining what's misleading and how you would redesign it.
4. Using the heatmap section as a starting point, build a similar department x account-type heatmap, but for `credit` instead of `debit`. How does the pattern compare to the debit-side heatmap?

Part III.

Part III: Applied Accounting Analytics

7. Financial Statement Analytics

Learning Objectives

By the end of this chapter, you should be able to:

- Load and structure multi-year financial statement data for analysis
- Perform horizontal (trend) analysis and vertical (common-size) analysis
- Compute liquidity, profitability, leverage, and efficiency ratios
- Build a reusable function that automates ratio calculations across any set of financial statements
- Apply DuPont decomposition to explain *why* return on equity changed, not just that it changed
- Interpret ratio trends critically, rather than treating any single ratio as a verdict on its own
- Understand the siginicance of financial statments for capital markets efficiency

7.1. From General Ledger to Financial Statements

Chapters 3 and 4 traced how transactions flow from the general ledger into a trial balance. Financial statements are the next and final step in that chain: the income statement summarizes revenue and expense activity for a period, and the balance sheet summarizes assets, liabilities, and equity at a point in time.

This chapter works with **five years of financial statements (2021□2025)** for a fictional company, "Ledger & Co.," stored in `income_statement.csv` and `balance_sheet.csv`. Unlike the transaction-level data from earlier chapters, this data is already fully summarized — each row is one fiscal year, and each column is a financial statement line item.

7.2. pandas

```
import pandas as pd

inc = pd.read_csv("data/income_statement.csv")
bs = pd.read_csv("data/balance_sheet.csv")

inc
```

	year	Revenue	Cost of Goods Sold	Gross Profit	Operating Expenses (SG&A)	Depreciation and A
0	2021	4200000	2604000	1596000	882000	95000
1	2022	4850000	2982750	1867250	994250	105000
2	2023	5450000	3270000	2180000	1090000	118000
3	2024	6100000	3629500	2470500	1189500	128000
4	2025	6900000	4036500	2863500	1311000	140000

7.3. polars

```
import polars as pl
inc_pl = pl.read_csv("data/income_statement.csv")
bs_pl = pl.read_csv("data/balance_sheet.csv")
bs_pl
```

year i64	Cash i64	Accounts Receivable i64	Inventory i64	Property, Plant & Equipment (net) i64	Accounts Payable i64
2021	461051	483288	392384	675000	249699
2022	763486	531507	424940	730000	294189
2023	1186389	582329	447945	787000	331479
2024	1719566	635068	477304	844000	377866
2025	2412155	699452	508710	904000	431297

7.4. pandas

```
bs[["year", "Cash", "Total Current Assets", "Total Assets", "Total
 ↪  Liabilities", "Total Equity"]]
```

	year	Cash	Total Current Assets	Total Assets	Total Liabilities	Total Equity
0	2021	461051	1336723	2011723	779699	1232024
1	2022	763486	1719933	2449933	769189	1680744
2	2023	1186389	2216663	3003663	741479	2262184
3	2024	1719566	2831938	3675938	712866	2963072
4	2025	2412155	3620317	4524317	691297	3833020

7.5. polars

```
bs_pl.select("year", "Cash", "Total Current Assets", "Total Assets",
 ↪  "Total Liabilities", "Total Equity")
```

year i64	Cash i64	Total Current Assets i64	Total Assets i64	Total Liabilities i64	Total Equity i64
2021	461051	1336723	2011723	779699	1232024
2022	763486	1719933	2449933	769189	1680744
2023	1186389	2216663	3003663	741479	2262184
2024	1719566	2831938	3675938	712866	2963072
2025	2412155	3620317	4524317	691297	3833020

> **A Note on Tools for This Chapter**
>
> This chapter uses pandas only. Financial statement data is small by nature — five rows, a few dozen columns — so polars' performance advantages (Chapter 4) don't come into play here the way they did with transaction-level general ledger data. Feel free to redo any of this chapter's calculations in polars as practice, but there's no analytical reason to prefer it for a dataset this size.

Before calculating anything, it's worth confirming the balance sheet actually balances — a basic sanity check you should run on any real financial statement data before trusting it:

7.6. pandas

```
imbalance = bs["Total Assets"] - bs["Total Liabilities and Equity"]
imbalance
```

```
0    0
1    0
2    0
3    0
4    0
dtype: int64
```

7.7. polars

```
imbalance_pl = bs_pl.select("Total Assets") - bs_pl.select("Total
 ↪  Liabilities and Equity")
imbalance_pl
```

Total Assets
i64
0
0
0
0
0

Every year balances exactly, as it should.

7.8. Horizontal (Trend) Analysis

Horizontal analysis compares a line item across time, showing dollar and percentage change from one period to the next.

7.9. pandas

```
income_by_year = inc.set_index("year")

dollar_change = income_by_year.diff()
dollar_change[["Revenue", "Net Income"]]
```

year	Revenue	Net Income
2021	NaN	NaN
2022	650000.0	120870.0
2023	600000.0	165900.0
2024	650000.0	149310.0
2025	800000.0	211325.0

```
pct_change = income_by_year.pct_change() * 100
pct_change[["Revenue", "Net Income"]].round(1)
```

year	Revenue	Net Income
2021	NaN	NaN
2022	15.5	27.5
2023	12.4	29.6
2024	11.9	20.5
2025	13.1	24.1

7.10. polars

```
dollar_change_pl = (
    inc_pl
    .sort("year")
    .with_columns(
        pl.col(["Revenue", "Net Income"]).diff()
    )
    .select(["year", "Revenue", "Net Income"])
```

```
)

dollar_change_pl
```

year i64	Revenue i64	Net Income i64
2021	null	null
2022	650000	120870
2023	600000	165900
2024	650000	149310
2025	800000	211325

```
pct_change_pl = (
    inc_pl
    .sort("year")
    .with_columns(
        (pl.col(["Revenue", "Net Income"]).pct_change() *
 ↪  100).round(1)
    )
    .select(["year", "Revenue", "Net Income"])
)

pct_change_pl
```

year i64	Revenue f64	Net Income f64
2021	null	null
2022	15.5	27.5
2023	12.4	29.6
2024	11.9	20.5
2025	13.1	24.1

Revenue grew between 12% and 15% annually, but notice that **net income grew faster than revenue every year** (e.g., 27.5% net income growth vs. 15.5% revenue growth in 2022) — a sign that profitability is improving, not just sales volume. We'll confirm this more precisely with margin ratios shortly.

7.10.1. Indexed Trend Analysis

A related technique sets the earliest year to 100 and expresses every later year relative to that baseline — useful for comparing the *relative* growth rate of several line items on one consistent scale, regardless of their different starting dollar amounts.

7.11. pandas

```
indexed = income_by_year / income_by_year.iloc[0] * 100
indexed[["Revenue", "Cost of Goods Sold", "Net Income"]].round(1)
```

year	Revenue	Cost of Goods Sold	Net Income
2021	100.0	100.0	100.0
2022	115.5	114.5	127.5
2023	129.8	125.6	165.2
2024	145.2	139.4	199.1
2025	164.3	155.0	247.1

7.12. polars

```
indexed_pl = (
    inc_pl
    .sort("year")
    .with_columns(
        (
            pl.col(["Revenue", "Cost of Goods Sold", "Net Income"])
            / pl.col(["Revenue", "Cost of Goods Sold", "Net
              ↪  Income"]).first()
            * 100
        ).round(1)
    )
    .select(["Revenue", "Cost of Goods Sold", "Net Income"])
)

indexed_pl
```

Revenue f64	Cost of Goods Sold f64	Net Income f64
100.0	100.0	100.0
115.5	114.5	127.5
129.8	125.6	165.2
145.2	139.4	199.1
164.3	155.0	247.1

By 2025, revenue has grown to 164% of its 2021 level, while net income has grown to 247% of its 2021 level — again showing that profitability is compounding faster than the top line.

7.13. Vertical (Common-Size) Analysis

Vertical analysis expresses every line item as a percentage of a single base figure within the same year — typically revenue for the income statement, and total assets for the balance sheet. This is especially useful for comparing structure across years (or across companies of different sizes) without dollar amounts getting in the way.

7.14. pandas

```
common_size_is = income_by_year.div(income_by_year["Revenue"], axis=0)
 ↪  * 100
common_size_is[["Cost of Goods Sold", "Gross Profit", "Operating
 ↪  Income", "Net Income"]].round(1)
```

year	Cost of Goods Sold	Gross Profit	Operating Income	Net Income
2021	62.0	38.0	14.7	10.5
2022	61.5	38.5	15.8	11.6
2023	60.0	40.0	17.8	13.3
2024	59.5	40.5	18.9	14.4
2025	58.5	41.5	20.5	15.8

7.15. polars

```
common_size_is_pl = (
    inc_pl
    .sort("year")
    .with_columns(
        (
            pl.exclude("year") / pl.col("Revenue") * 100
        ).round(1)
    )
    .select([
        "year",
        "Cost of Goods Sold",
        "Gross Profit",
        "Operating Income",
        "Net Income",
    ])
)

common_size_is_pl
```

year i64	Cost of Goods Sold f64	Gross Profit f64	Operating Income f64	Net Income f64
2021	62.0	38.0	14.7	10.5
2022	61.5	38.5	15.8	11.6
2023	60.0	40.0	17.8	13.3
2024	59.5	40.5	18.9	14.4
2025	58.5	41.5	20.5	15.8

Cost of Goods Sold has fallen from 62.0% of revenue in 2021 to 58.5% in 2025 — a steady margin improvement, likely reflecting economies of scale or improved purchasing terms.

7.16. pandas

```
balance_by_year = bs.set_index("year")
common_size_bs = balance_by_year.div(balance_by_year["Total Assets"],
 ↪  axis=0) * 100
```

```
common_size_bs[["Cash", "Accounts Receivable", "Inventory", "Property,
 ↪  Plant & Equipment (net)"]].round(1)
```

year	Cash	Accounts Receivable	Inventory	Property, Plant & Equipment (net)
2021	22.9	24.0	19.5	33.6
2022	31.2	21.7	17.3	29.8
2023	39.5	19.4	14.9	26.2
2024	46.8	17.3	13.0	23.0
2025	53.3	15.5	11.2	20.0

7.17. polars

```
common_size_bs_pl = (
    bs_pl
    .sort("year")
    .with_columns(
        (
            pl.exclude("year") / pl.col("Total Assets") * 100
        ).round(1)
    )
    .select([
        "year",
        "Cash",
        "Accounts Receivable",
        "Inventory",
        "Property, Plant & Equipment (net)",
    ])
)

common_size_bs_pl
```

year i64	Cash f64	Accounts Receivable f64	Inventory f64	Property, Plant & Equipment (net) f64
2021	22.9	24.0	19.5	33.6
2022	31.2	21.7	17.3	29.8

year i64	Cash f64	Accounts Receivable f64	Inventory f64	Property, Plant & Equipment (net) f64
2023	39.5	19.4	14.9	26.2
2024	46.8	17.3	13.0	23.0
2025	53.3	15.5	11.2	20.0

! Reading the Balance Sheet Structure

Cash has grown from 22.9% to 53.3% of total assets over five years, while receivables, inventory, and PP&E have all *shrunk* as a share of the balance sheet. On its own, this looks like a strong cash position — but as we'll discuss later in this chapter, a rapidly growing cash pile can also raise a legitimate question: is management deploying capital efficiently, or simply letting cash accumulate?

7.18. Ratio Analysis

Ratios distill financial statement data into standardized measures that are comparable across time periods and even across companies. We'll compute four common categories.

7.18.1. Liquidity Ratios

Liquidity ratios measure a company's ability to meet short-term obligations. Liquidity ratios compare what a business owns against what it owes in the near term. Liquidity ratios include current ratio, quick ratio[1] (acid-test ratio), and cash ratio. Investors use these ratios to assess if a company is at risk of bankruptcy or financial distress.

7.19. pandas

```
merged = inc.merge(bs, on="year")

current_ratio = merged["Total Current Assets"] / merged["Total Current
 ↪  Liabilities"]
quick_ratio = (merged["Cash"] + merged["Accounts Receivable"]) /
 ↪  merged["Total Current Liabilities"]
```

[1] The difference between current ratio and quick ratio is that quick ratio is stricter, excluding inventory because it is the least liquid current asset

```
pd.DataFrame({"year": merged["year"], "Current Ratio": current_ratio,
 ↪  "Quick Ratio": quick_ratio}).round(2)
```

	year	Current Ratio	Quick Ratio
0	2021	4.05	2.86
1	2022	4.66	3.51
2	2023	5.52	4.41
3	2024	6.39	5.32
4	2025	7.37	6.33

7.20. polars

```
ratios_pl = (
    inc_pl
    .join(bs_pl, on="year", how="inner")
    .select(
        "year",
        (
            pl.col("Total Current Assets")
            / pl.col("Total Current Liabilities")
        ).round(2).alias("Current Ratio"),
        (
            (pl.col("Cash") + pl.col("Accounts Receivable"))
            / pl.col("Total Current Liabilities")
        ).round(2).alias("Quick Ratio"),
    )
)

ratios_pl
```

year i64	Current Ratio f64	Quick Ratio f64
2021	4.05	2.86
2022	4.66	3.51
2023	5.52	4.41

year i64	Current Ratio f64	Quick Ratio f64
2024	6.39	5.32
2025	7.37	6.33

A current ratio above 1.0 generally indicates a company can cover its short-term liabilities with short-term assets. Here, both ratios are comfortably above 1.0 and rising every year — worth a closer look later, since *very* high liquidity isn't automatically a good sign either.

7.20.1. Profitability Ratios

Profitability ratios evaluate a company's ability to generate earnings relative to its revenue, operating costs, assets, and shareholder equity. Generally, higher ratios indicate stronger efficiency, cost control, and financial health.

Profitability ratios are primarily divided into two categories: **Margin Ratios** (which show how much profit is kept from sales) and **Return Ratios** (which show how well resources are used to generate returns for investors). Margin ratios include gross profit margin, operating profit margin, and net profit margin. Return ratios include return on assets (ROA), return on Equity (ROE), and return on invested capital (ROIC).

7.21. pandas

```
gross_margin = merged["Gross Profit"] / merged["Revenue"]
operating_margin = merged["Operating Income"] / merged["Revenue"]
net_margin = merged["Net Income"] / merged["Revenue"]
roa = merged["Net Income"] / merged["Total Assets"]
roe = merged["Net Income"] / merged["Total Equity"]

pd.DataFrame({
    "year": merged["year"],
    "Gross Margin": gross_margin,
    "Operating Margin": operating_margin,
    "Net Margin": net_margin,
    "ROA": roa,
    "ROE": roe,
}).round(3)
```

	year	Gross Margin	Operating Margin	Net Margin	ROA	ROE
0	2021	0.380	0.147	0.105	0.219	0.357
1	2022	0.385	0.158	0.116	0.229	0.334
2	2023	0.400	0.178	0.133	0.242	0.321
3	2024	0.405	0.189	0.144	0.238	0.296
4	2025	0.415	0.205	0.158	0.240	0.284

7.22. polars

```
merged_pl = (
    inc_pl
    .join(bs_pl, on="year", how="inner")
)
```

```
profitability_ratios = (
    merged_pl
    .select(
        "year",
        (
            pl.col("Gross Profit") / pl.col("Revenue")
        ).round(3).alias("Gross Margin"),
        (
            pl.col("Operating Income") / pl.col("Revenue")
        ).round(3).alias("Operating Margin"),
        (
            pl.col("Net Income") / pl.col("Revenue")
        ).round(3).alias("Net Margin"),
        (
            pl.col("Net Income") / pl.col("Total Assets")
        ).round(3).alias("ROA"),
        (
            pl.col("Net Income") / pl.col("Total Equity")
        ).round(3).alias("ROE"),
    )
)

profitability_ratios
```

year i64	Gross Margin f64	Operating Margin f64	Net Margin f64	ROA f64	ROE f64
2021	0.38	0.147	0.105	0.219	0.357
2022	0.385	0.158	0.116	0.229	0.334
2023	0.4	0.178	0.133	0.242	0.321
2024	0.405	0.189	0.144	0.238	0.296
2025	0.415	0.205	0.158	0.24	0.284

Every profitability margin improved steadily from 2021 to 2025. But look closely at **ROE** — despite improving margins, ROE actually *declines* over the same period (35.7% down to 28.4%). This looks contradictory at first glance. We'll resolve exactly why using **DuPont analysis** later in this chapter.

7.22.1. Leverage Ratios

Leverage ratios measure how much a company relies on debt financing, and its ability to service that debt. By comparing debt to equity, assets, or earnings, these metrics help investors and analysts assess a business's solvency, long-term stability, and overall financial risk. Leverage ratios include debt-to-equity ratio, debt-to-asset ratio, debt-to-EBITDA ratio, and interest coverage ratio.

A **high leverage ratio** indicates a heavy reliance on debt. While this can amplify profits when a company performs well, it also significantly increases the risk of default during economic downturns or periods of low cash flow. A **low leverage ratio** points to a conservative capital structure with less debt reliance. This often suggests greater financial stability and lower risk of bankruptcy, though it can also signal an inability or reluctance to borrow for growth.

Because acceptable thresholds vary widely by industry—for example, capital-intensive manufacturing or utility companies naturally require more debt than service-oriented firms—these metrics should always be benchmarked against competitors in the same sector.

7.23. pandas

```
debt_to_equity = (merged["Short-term Debt"] + merged["Long-term
 ↪  Debt"]) / merged["Total Equity"]
interest_coverage = merged["Operating Income"] / merged["Interest
 ↪  Expense"]
```

```
pd.DataFrame({
    "year": merged["year"],
    "Debt-to-Equity": debt_to_equity,
    "Interest Coverage": interest_coverage,
}).round(2)
```

	year	Debt-to-Equity	Interest Coverage
0	2021	0.43	9.98
1	2022	0.28	13.24
2	2023	0.18	18.69
3	2024	0.11	26.20
4	2025	0.07	39.24

7.24. polars

```
leverage_ratios_pl = (
    merged_pl
    .select(
        "year",
        (
            (pl.col("Short-term Debt") + pl.col("Long-term Debt"))
            / pl.col("Total Equity")
        ).round(2).alias("Debt-to-Equity"),
        (
            pl.col("Operating Income")
            / pl.col("Interest Expense")
        ).round(2).alias("Interest Coverage"),
    )
)

leverage_ratios_pl
```

year i64	Debt-to-Equity f64	Interest Coverage f64
2021	0.43	9.98
2022	0.28	13.24

year i64	Debt-to-Equity f64	Interest Coverage f64
2023	0.18	18.69
2024	0.11	26.2
2025	0.07	39.24

Debt-to-equity has fallen sharply (from 0.43 to 0.18), and interest coverage has risen from about 10x to nearly 40x — the company has been paying down debt aggressively while growing operating income, substantially reducing its financial risk.

7.24.1. Efficiency Ratios

Efficiency ratios measure how well a company uses its assets to generate revenue. They compare a company's expenses to its sales or track how quickly it turns assets into cash. Higher turnover ratios and lower expense ratios usually signal strong operational health. Efficiency ratios include asset turnover ratio, inventory turnover ratio, and accounts receivable turnover ratio.

7.25. pandas

```
asset_turnover = merged["Revenue"] / merged["Total Assets"]
dso = merged["Accounts Receivable"] / merged["Revenue"] * 365
dio = merged["Inventory"] / merged["Cost of Goods Sold"] * 365

pd.DataFrame({
    "year": merged["year"],
    "Asset Turnover": asset_turnover,
    "Days Sales Outstanding": dso,
    "Days Inventory Outstanding": dio,
}).round(1)
```

	year	Asset Turnover	Days Sales Outstanding	Days Inventory Outstanding
0	2021	2.1	42.0	55.0
1	2022	2.0	40.0	52.0
2	2023	1.8	39.0	50.0
3	2024	1.7	38.0	48.0
4	2025	1.5	37.0	46.0

7.26. polars

```
asset_efficiency_pl = (
    merged_pl
    .select(
        "year",
        (
            pl.col("Revenue")
            / pl.col("Total Assets")
        ).round(1).alias("Asset Turnover"),
        (
            pl.col("Accounts Receivable")
            / pl.col("Revenue")
            * 365
        ).round(1).alias("Days Sales Outstanding"),
        (
            pl.col("Inventory")
            / pl.col("Cost of Goods Sold")
            * 365
        ).round(1).alias("Days Inventory Outstanding"),
    )
)

asset_efficiency_pl
```

year i64	Asset Turnover f64	Days Sales Outstanding f64	Days Inventory Outstanding f64
2021	2.1	42.0	55.0
2022	2.0	40.0	52.0
2023	1.8	39.0	50.0
2024	1.7	38.0	48.0
2025	1.5	37.0	46.0

Days Sales Outstanding and Days Inventory Outstanding have both **improved** (fallen) — the company collects receivables faster and turns over inventory faster in 2025 than in 2021. Yet asset turnover has *declined* (from 2.09 down to 1.53) — total assets are growing even faster than revenue, largely because of that growing cash balance. This is another thread we'll pick up in the DuPont section.

7.27. Automating Ratio Calculations

Rather than recomputing each ratio by hand every time you get a new year of data, it's worth writing a single reusable function.

7.28. pandas

```
def compute_ratios(income_df: pd.DataFrame, balance_df: pd.DataFrame)
 ↪  -> pd.DataFrame:
    """Compute standard liquidity, profitability, leverage, and
     ↪  efficiency ratios
    from an income statement and balance sheet sharing a common 'year'
 ↪  column."""
    df = income_df.merge(balance_df, on="year")
    r = pd.DataFrame({"year": df["year"]})

    # Liquidity
    r["Current Ratio"] = df["Total Current Assets"] / df["Total
 ↪  Current Liabilities"]
    r["Quick Ratio"] = (df["Cash"] + df["Accounts Receivable"]) /
 ↪  df["Total Current Liabilities"]

    # Profitability
    r["Gross Margin"] = df["Gross Profit"] / df["Revenue"]
    r["Operating Margin"] = df["Operating Income"] / df["Revenue"]
    r["Net Margin"] = df["Net Income"] / df["Revenue"]
    r["Return on Assets"] = df["Net Income"] / df["Total Assets"]
    r["Return on Equity"] = df["Net Income"] / df["Total Equity"]

    # Leverage
    r["Debt-to-Equity"] = (df["Short-term Debt"] + df["Long-term
 ↪  Debt"]) / df["Total Equity"]
    r["Interest Coverage"] = df["Operating Income"] / df["Interest
 ↪  Expense"]

    # Efficiency
    r["Asset Turnover"] = df["Revenue"] / df["Total Assets"]
    r["Days Sales Outstanding"] = df["Accounts Receivable"] /
 ↪  df["Revenue"] * 365
```

```
    r["Days Inventory Outstanding"] = df["Inventory"] / df["Cost of
↪   Goods Sold"] * 365

    return r.set_index("year")
```

```
ratios = compute_ratios(inc, bs)
ratios.round(3)
```

year	Current Ratio	Quick Ratio	Gross Margin	Operating Margin	Net Margin	Return on Assets
2021	4.054	2.864	0.380	0.147	0.105	0.219
2022	4.659	3.508	0.385	0.158	0.116	0.229
2023	5.521	4.406	0.400	0.178	0.133	0.242
2024	6.395	5.317	0.405	0.189	0.144	0.238
2025	7.369	6.333	0.415	0.205	0.158	0.240

7.29. polars

```
def compute_ratios_pl(income_df: pl.DataFrame, balance_df:
↪   pl.DataFrame) -> pl.DataFrame:
    """Compute standard liquidity, profitability, leverage, and
    ↪  efficiency ratios
    from an income statement and balance sheet sharing a common 'year'
↪   column."""

    df = income_df.join(balance_df, on="year", how="inner")

    r = df.select(
        "year",

        # Liquidity
        (
            pl.col("Total Current Assets")
            / pl.col("Total Current Liabilities")
        ).alias("Current Ratio"),

        (
```

```
        (pl.col("Cash") + pl.col("Accounts Receivable"))
        / pl.col("Total Current Liabilities")
    ).alias("Quick Ratio"),


    # Profitability
    (
        pl.col("Gross Profit")
        / pl.col("Revenue")
    ).alias("Gross Margin"),

    (
        pl.col("Operating Income")
        / pl.col("Revenue")
    ).alias("Operating Margin"),

    (
        pl.col("Net Income")
        / pl.col("Revenue")
    ).alias("Net Margin"),

    (
        pl.col("Net Income")
        / pl.col("Total Assets")
    ).alias("Return on Assets"),

    (
        pl.col("Net Income")
        / pl.col("Total Equity")
    ).alias("Return on Equity"),


    # Leverage
    (
        (pl.col("Short-term Debt") + pl.col("Long-term Debt"))
        / pl.col("Total Equity")
    ).alias("Debt-to-Equity"),

    (
        pl.col("Operating Income")
        / pl.col("Interest Expense")
```

```
        ).alias("Interest Coverage"),


        # Efficiency
        (
            pl.col("Revenue")
            / pl.col("Total Assets")
        ).alias("Asset Turnover"),

        (
            pl.col("Accounts Receivable")
            / pl.col("Revenue")
            * 365
        ).alias("Days Sales Outstanding"),

        (
            pl.col("Inventory")
            / pl.col("Cost of Goods Sold")
            * 365
        ).alias("Days Inventory Outstanding"),
    )

    return r
```

```
ratios_pl = compute_ratios_pl(inc_pl, bs_pl)

ratios_pl = ratios_pl.with_columns(
    pl.exclude("year").round(3)
)

ratios_pl
```

year i64	Current Ratio f64	Quick Ratio f64	Gross Margin f64	Operating Margin f64	Net Margin f64	Return on Assets f64
2021	4.054	2.864	0.38	0.147	0.105	0.219
2022	4.659	3.508	0.385	0.158	0.116	0.229
2023	5.521	4.406	0.4	0.178	0.133	0.242
2024	6.395	5.317	0.405	0.189	0.144	0.238
2025	7.369	6.333	0.415	0.205	0.158	0.24

> **Connect to Practice**
>
> Writing analysis as a reusable function like this — rather than one-off code you rerun and edit each period — is exactly how analytics work scales in practice. Next quarter's financial statements can be run through the same `compute_ratios()` function with zero changes, as long as the column names match. This is also foreshadows the automated reporting techniques in Chapter 14.

7.30. DuPont Decomposition: Explaining *Why* ROE Changed

The DuPont framework breaks **return on equity (ROE)** into three multiplied components, each capturing a different lever of performance:

$$\text{ROE} = \underbrace{\frac{\text{Net Income}}{\text{Revenue}}}_{\text{Net Margin}} \times \underbrace{\frac{\text{Revenue}}{\text{Total Assets}}}_{\text{Asset Turnover}} \times \underbrace{\frac{\text{Total Assets}}{\text{Total Equity}}}_{\text{Equity Multiplier}}$$

7.31. pandas

```
dupont = pd.DataFrame({"year": merged["year"]})
dupont["Net Margin"] = merged["Net Income"] / merged["Revenue"]
dupont["Asset Turnover"] = merged["Revenue"] / merged["Total Assets"]
dupont["Equity Multiplier"] = merged["Total Assets"] / merged["Total
 ↪  Equity"]
dupont["ROE (DuPont)"] = dupont["Net Margin"] * dupont["Asset
 ↪  Turnover"] * dupont["Equity Multiplier"]

dupont.set_index("year").round(3)
```

year	Net Margin	Asset Turnover	Equity Multiplier	ROE (DuPont)
2021	0.105	2.088	1.633	0.357
2022	0.116	1.980	1.458	0.334
2023	0.133	1.814	1.328	0.321
2024	0.144	1.659	1.241	0.296
2025	0.158	1.525	1.180	0.284

7.32. polars

```
dupont_pl = (
    merged_pl
    .select(
        "year",

        (
            pl.col("Net Income") / pl.col("Revenue")
        ).alias("Net Margin"),

        (
            pl.col("Revenue") / pl.col("Total Assets")
        ).alias("Asset Turnover"),

        (
            pl.col("Total Assets") / pl.col("Total Equity")
        ).alias("Equity Multiplier"),

        (
            (pl.col("Net Income") / pl.col("Revenue"))
            *
            (pl.col("Revenue") / pl.col("Total Assets"))
            *
            (pl.col("Total Assets") / pl.col("Total Equity"))
        ).alias("ROE (DuPont)"),
    )
    .with_columns(
        pl.exclude("year").round(3)
    )
)

dupont_pl
```

year i64	Net Margin f64	Asset Turnover f64	Equity Multiplier f64	ROE (DuPont) f64
2021	0.105	2.088	1.633	0.357
2022	0.116	1.98	1.458	0.334
2023	0.133	1.814	1.328	0.321
2024	0.144	1.659	1.241	0.296

year i64	Net Margin f64	Asset Turnover f64	Equity Multiplier f64	ROE (DuPont) f64
2025	0.158	1.525	1.18	0.284

This confirms exactly the same ROE figures as the direct calculation earlier — but now we can see *why* ROE fell even as profitability rose:

- **Net Margin improved** (10.5% → 15.8%) — profitability genuinely got better.
- **Asset Turnover fell** (2.09 → 1.53) — assets (mostly that growing cash pile) grew faster than revenue.
- **Equity Multiplier fell** (1.63 → 1.18) — the company relies on much less debt financing than before.

The margin improvement was real, but it was **more than offset** by the combined effect of falling asset turnover and falling leverage. In other words: the company is becoming more profitable per dollar of sales, but less efficient at *using* its (growing) asset base, and increasingly equity-financed rather than debt-financed — both of which mechanically pull ROE down, even while the business is arguably getting healthier in other respects.

> **!** Why This Matters
>
> Reporting "ROE declined from 35.7% to 28.4%" without DuPont context could easily be read as bad news. DuPont decomposition shows that the *decline* is actually a byproduct of paying down debt and building cash reserves — both generally prudent financial decisions — rather than declining profitability. This is a good example of why a single ratio, in isolation, can mislead; understanding its components tells the real story.

7.33. Visualizing the Trends

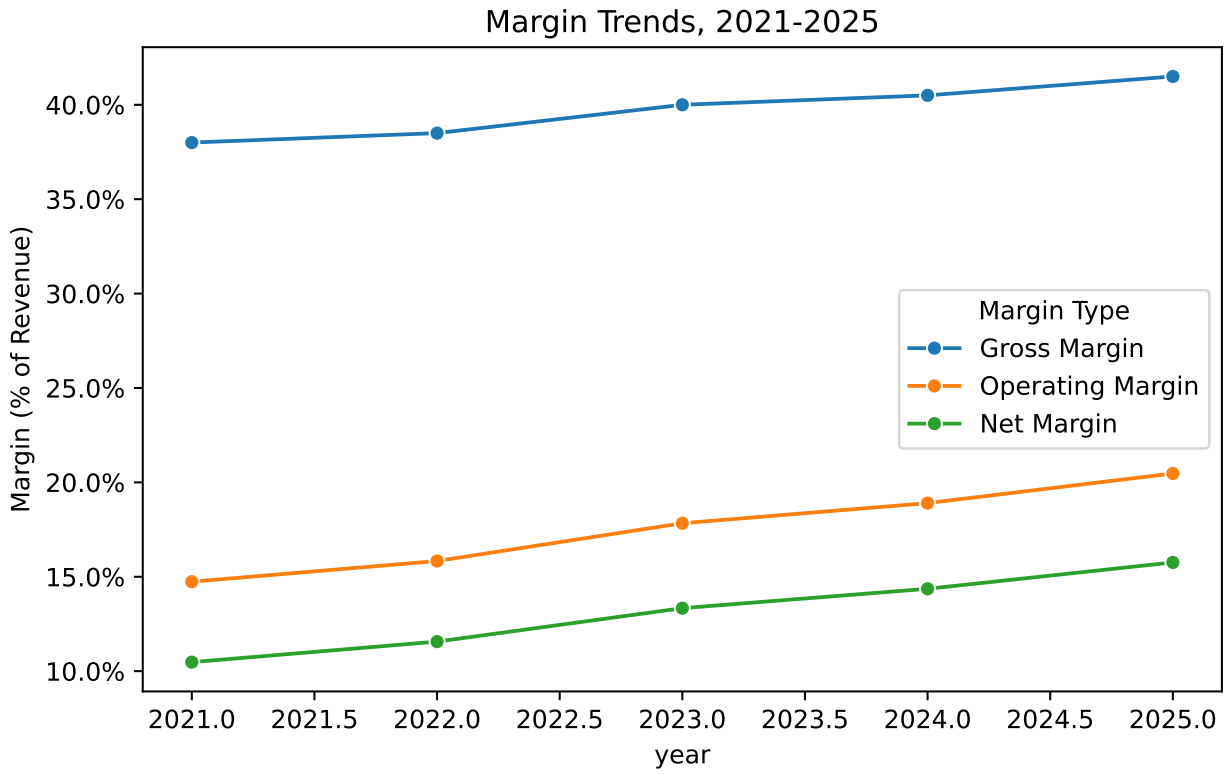
A trend that takes several tables to describe in words is often obvious in one chart.

7.34. seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
```

```
margin_long = ratios.reset_index()[["year", "Gross Margin", "Operating
 ↪  Margin", "Net Margin"]].melt(
    id_vars="year", var_name="Margin Type", value_name="Value"
)

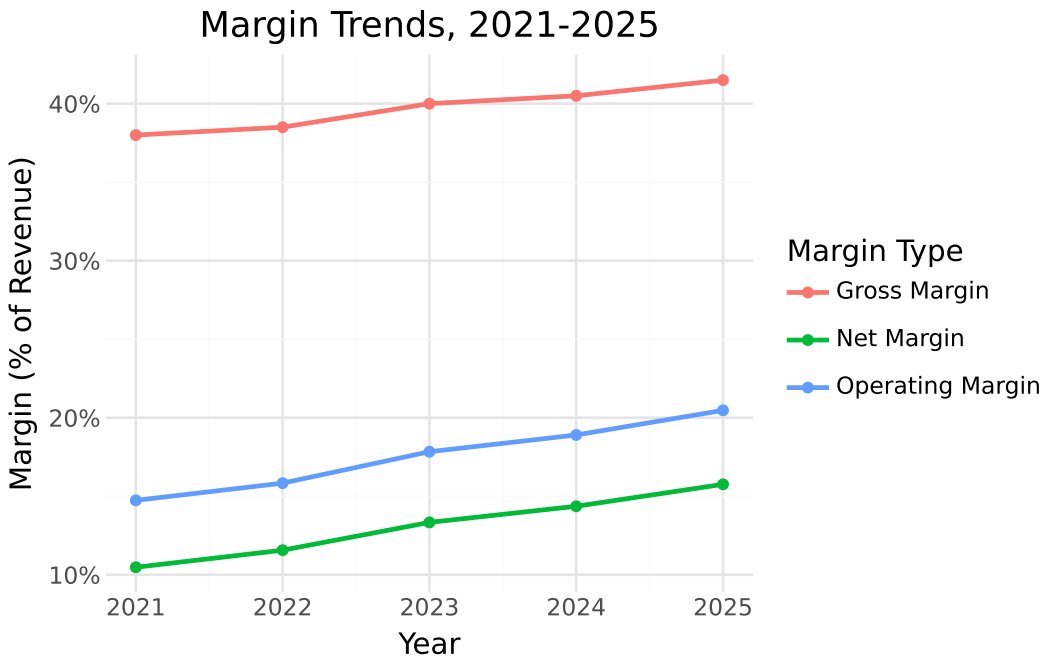
plt.figure(figsize=(7, 4.5))
sns.lineplot(data=margin_long, x="year", y="Value", hue="Margin Type",
 ↪  marker="o")
plt.title("Margin Trends, 2021-2025")
plt.ylabel("Margin (% of Revenue)")
plt.gca().yaxis.set_major_formatter(mticker.PercentFormatter(1.0))
plt.tight_layout()
plt.show()
```



7.35. plotnine

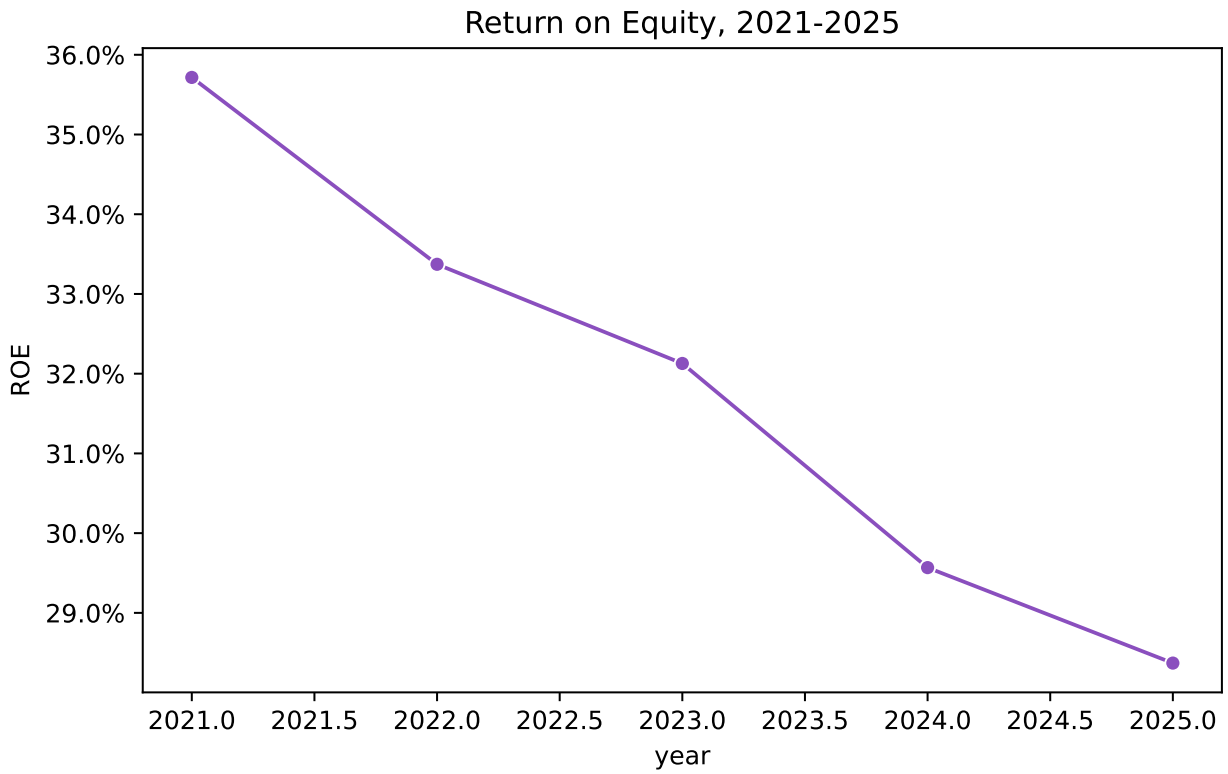
```
from plotnine import *
```

```
(
    ggplot(margin_long, aes(x="year", y="Value", color="Margin Type"))
    + geom_line(size=1)
    + geom_point()
    + scale_y_continuous(labels=lambda l: [f"{v:.0%}" for v in l])
    + labs(title="Margin Trends, 2021-2025", x="Year", y="Margin (% of
     ↪  Revenue)")
    + theme_minimal()
)
```



All three margins trend upward and roughly in parallel — visual confirmation that the profitability improvement is broad-based (starting at the gross profit level) rather than coming from just one line item, like a one-time tax benefit.

```
plt.figure(figsize=(7, 4.5))
sns.lineplot(data=dupont, x="year", y="ROE (DuPont)", marker="o",
 ↪  color="#8a4fbe")
plt.title("Return on Equity, 2021-2025")
plt.ylabel("ROE")
plt.gca().yaxis.set_major_formatter(mticker.PercentFormatter(1.0))
plt.tight_layout()
plt.show()
```



Seeing the steady ROE decline plotted directly against the improving margins from the chart above makes the apparent contradiction visually obvious — exactly the kind of pattern that should prompt a DuPont-style investigation rather than a surface-level "ROE is getting worse" conclusion.

7.36. Putting It Together: A Financial Health Memo

A typical output of this kind of analysis isn't just a table of ratios — it's a short written interpretation for a decision-maker. Here's a brief example, written the way you might summarize this exact analysis for a manager:

Sample Memo Excerpt

Ledger & Co.'s profitability has improved steadily over the past five years, with net margin rising from 10.5% to 15.8%, driven by both improving gross margins and declining interest expense. The company has also meaningfully reduced financial risk, cutting debt-to-equity from 0.43 to 0.18 while more than tripling interest coverage. Return on equity has declined over the same period (35.7% to 28.4%), but DuPont analysis attributes this to the combined effect of debt paydown and a growing cash balance — both generally prudent — rather than any underlying weakness in operations. One area worth management's attention: the current

> *ratio has risen to 7.4x, well above what's typically needed for liquidity purposes, suggesting cash may be accumulating faster than it's being deployed into growth, debt reduction, or shareholder returns.*

Notice that this memo doesn't just repeat the numbers — it interprets what they mean *together*, flags a genuine open question (the growing cash pile) rather than treating every metric as unambiguously good news, and avoids the trap of reporting the ROE decline without its DuPont explanation.

7.37. Why These Numbers Matter to Capital Markets

Every ratio computed in this chapter has a purpose beyond internal analysis: financial statements are the primary channel through which a company communicates its performance to the outside investors, lenders, and analysts who allocate capital in public and private markets. It's worth stepping back to understand *why* that communication role matters, and how the specific numbers in this chapter connect to it.

7.37.1. Financial Statements as an Information Bridge

Company managers know far more about their business than outside investors ever can — a gap economists call **information asymmetry**. Audited financial statements exist largely to narrow that gap: they give outside capital providers a standardized, independently verified basis for judging a company's performance without needing direct access to its internal records. This is precisely why the audit techniques in Chapter 9 matter so much to capital markets — a number nobody trusts is a number capital markets can't efficiently price.

7.37.2. Market Reactions to Financial Information

When a company releases financial results, especially quarterly earnings, market prices often move — sometimes sharply — in response. Researchers and practitioners study this using **event studies**: measuring a stock's price movement in a narrow window around an announcement to isolate the market's reaction to the *new* information specifically, separate from broader market movements. A company that reports earnings **above** what analysts expected (a positive "earnings surprise") tends to see its stock price rise; one that misses expectations tends to see it fall — often regardless of whether the absolute numbers were "good" in isolation, since markets are reacting to information *relative to what was already priced in*.

Connect to Practice

This is exactly why the *trend* and *DuPont* analysis in this chapter matter more than any single year's numbers in isolation. A market that already expects margin improvement won't react much to a company merely delivering it — but the same result would move markets substantially if it arrived as a surprise. Context, not just the raw figure, drives market reaction.

7.37.3. How Ratios Connect to Valuation and the Cost of Capital

The specific ratio categories from this chapter map directly onto how capital markets participants evaluate a company:

- **Profitability ratios** (margins, ROA, ROE) feed directly into valuation multiples like the price-to-earnings (P/E) ratio — equity analysts routinely build forecasts of future margins from exactly the kind of trend analysis performed earlier in this chapter.
- **Leverage ratios** (debt-to-equity, interest coverage) are central to how credit rating agencies assess default risk, which in turn determines a company's **cost of debt** — Ledger & Co.'s improving interest coverage (10x to nearly 40x) is exactly the kind of trend that would support a credit rating upgrade and cheaper future borrowing.
- **Liquidity ratios** (current ratio, quick ratio) inform short-term credit risk assessments — lenders extending short-term credit look at these first.
- **Efficiency ratios** (asset turnover, DSO, DIO) help analysts judge whether management is deploying capital productively, directly relevant to whether a stock trades at a premium or discount to its peers.

7.37.4. A Word of Caution: Markets Use More Than Ratios

It would be a mistake to conclude that capital markets simply mechanically calculate ratios and price securities accordingly. A few important caveats:

- **Financial statements are backward-looking.** They report what already happened; markets price what's expected to happen next. This is why forward guidance, analyst forecasts, and macroeconomic conditions all matter alongside historical ratios.
- **Accounting numbers involve judgment and estimates**, not pure objective fact — recall Chapter 11's revenue recognition red flags. A market's confidence in a company's *reporting quality*, not just its reported numbers, affects how much weight investors place on them.
- **Market efficiency is a matter of ongoing academic and practical debate.** The Efficient Market Hypothesis, in its various forms, argues that prices reflect available information

to different degrees — but the extent to which this holds in practice, and for which markets, remains genuinely contested among finance researchers and practitioners. This book takes no position on that debate; the point relevant here is narrower and less controversial: financial statement information demonstrably influences prices, whatever the underlying market efficiency actually is.

! Why This Matters for Your Career

Whether you end up in equity research, credit analysis, corporate finance, or audit, the techniques in this chapter are the same ones capital markets participants use every day to translate financial statements into investment, lending, and valuation decisions. Understanding *why* a ratio matters to a market participant — not just how to calculate it — is what separates mechanical number-crunching from analysis that actually informs a decision.

7.38. Chapter Summary

- Horizontal analysis compares a line item across time (dollar and percent change, or indexed to a base year); vertical (common-size) analysis expresses every line item as a percentage of a base figure within the same year.
- Liquidity, profitability, leverage, and efficiency ratios each capture a different dimension of financial performance, and no single ratio tells the whole story on its own.
- Wrapping ratio calculations in a reusable function (like `compute_ratios()`) means the same analysis can be rerun on any future period's data without rewriting code.
- DuPont decomposition breaks ROE into net margin, asset turnover, and equity multiplier — essential for explaining *why* ROE changed, not just that it did, and for avoiding misleading single-ratio conclusions.
- A rising ratio isn't automatically good news (e.g., an extremely high current ratio can signal underused cash) any more than a falling ratio is automatically bad news (e.g., falling ROE driven by prudent deleveraging) — ratio analysis requires interpretation, not just calculation.
- These same ratios are the language capital markets use to price securities and assess risk: profitability ratios inform valuation multiples, leverage ratios inform credit ratings and the cost of debt, and market reactions to financial statement releases depend on results relative to expectations, not just the absolute numbers.

7.39. Discussion Questions

1. Ledger & Co.'s current ratio rose from 4.05 to 7.37 over five years. At what point, if any, does "more liquidity" stop being a good thing? What would you want to know before

deciding?
2. Explain in your own words why ROE declined even though every margin (gross, operating, net) improved over the same period.
3. If you were advising this company's CFO, what would you recommend they consider doing with the growing cash balance, and what additional information would you want before making that recommendation?
4. If Ledger & Co. were a public company and reported this chapter's 2025 results in line with what analysts already expected, how might the market's reaction differ from a scenario where these same results arrived as a surprise? What does this imply about the relationship between "good numbers" and "stock price reaction"?

7.40. Exercises

1. Using `compute_ratios()`, add one additional ratio of your choice (e.g., **Return on Invested Capital**, **Cash Ratio**, or **Fixed Asset Turnover**) and compute it for all five years.
2. Perform vertical analysis on the *liabilities and equity* side of the balance sheet (not just assets, which this chapter covered). How has the company's financing mix shifted between 2021 and 2025?
3. Recreate the margin trend chart from this chapter, but plot Return on Assets and Return on Equity together instead of the three margins. What does the widening or narrowing gap between ROA and ROE tell you about leverage over time?
4. Write your own 150-word financial health memo based on the leverage and efficiency ratios computed in this chapter (rather than the profitability-focused example memo shown above).

8. Management Accounting Analytics

Learning Objectives

By the end of this chapter, you should be able to:

- Distinguish management accounting analytics from the financial accounting analytics covered in Chapter 7
- Compute and interpret standard costing variances for materials, labor, and overhead
- Perform cost-volume-profit (CVP) analysis, including break-even point, margin of safety, and target profit
- Compare traditional overhead allocation to activity-based costing (ABC), and explain when the two produce meaningfully different product costs
- Build management-facing visualizations for internal decision-making
- Perform all of the above using both pandas and polars, and visualize results with both seaborn and plotnine

8.1. Financial vs. Management Accounting Analytics

Chapter 7 analyzed financial statements the way an outside investor or lender would: ratios computed from GAAP-based, externally reported numbers, governed by accounting standards and audited for accuracy. **Management accounting** serves a different audience entirely — the people running the business day to day — and follows different rules, or more precisely, no externally mandated rules at all. A budget variance report, a break-even analysis, or an internal product costing model never gets filed with the SEC or audited by an external firm; it exists purely to help managers make better decisions, which means it can be structured however is most useful internally.

This chapter covers three of the most common management accounting analytics tasks: **standard costing variance analysis** (did we spend what we expected to, and why not?), **cost-volume-profit analysis** (how many units do we need to sell to be profitable?), and **activity-based costing** (are we allocating overhead to products in a way that reflects how they actually consume resources?).

We'll use a fictional manufacturer's data: `manufacturing_variance_data.csv` (12 months of standard costing detail), `cvp_monthly_data.csv` (monthly unit economics), and `abc_cost_data.csv` (a five-product overhead allocation comparison).

8.2. pandas

```
import pandas as pd
variance_pd = pd.read_csv("data/manufacturing_variance_data.csv")
variance_pd.head()
```

	month	units_produced	std_materials_qty_per_unit	std_materials_price	actual_materials_qty_to
0	2025-01	3943	3.0	4.5	11453.4
1	2025-02	4022	3.0	4.5	11680.0
2	2025-03	4882	3.0	4.5	14888.6
3	2025-04	4162	3.0	4.5	12359.6
4	2025-05	3915	3.0	4.5	12050.9

8.3. polars

```
import polars as pl
variance_pl = pl.read_csv("data/manufacturing_variance_data.csv")
variance_pl.head()
```

month str	units_produced i64	std_materials_qty_per_unit f64	std_materials_price f64	actual_materials_qty_tota f64
"2025-01"	3943	3.0	4.5	11453.4
"2025-02"	4022	3.0	4.5	11680.0
"2025-03"	4882	3.0	4.5	14888.6
"2025-04"	4162	3.0	4.5	12359.6
"2025-05"	3915	3.0	4.5	12050.9

8.4. Standard Costing Variance Analysis

A **standard cost** is the cost a company *expects* to incur per unit under normal, efficient operating conditions — a predetermined benchmark set before the period begins. Comparing actual costs to this standard, and breaking the difference into specific causes, is the foundation of management accounting's cost control toolkit.

8.4.1. Materials Variances

The total difference between actual and standard materials cost splits into two independent causes: did we pay a different **price** per unit of material, and did we use a different **quantity** than the standard allows for the output we actually produced? Figure 8.1 shows the total material variance.

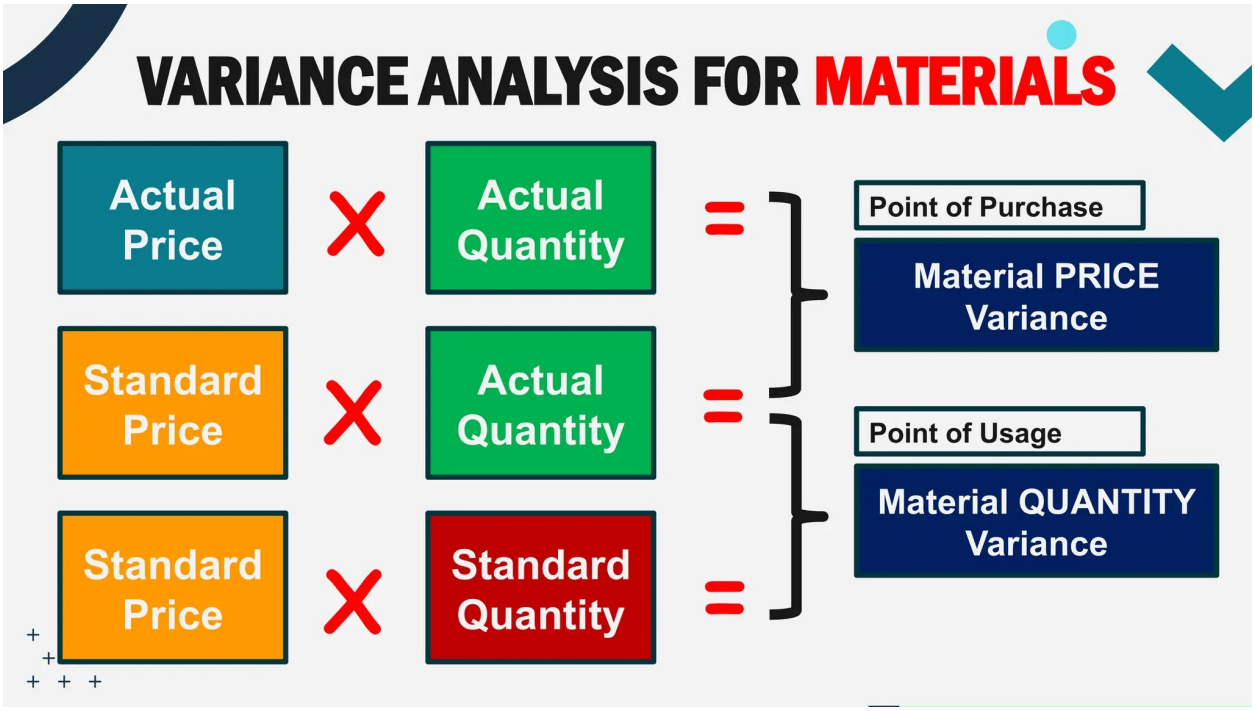


Figure 8.1.: Materials Variance

8.5. pandas

```
# Materials price variance = (Actual Price - Standard Price) x Actual
 ↪  Quantity
variance_pd["materials_price_variance"] = (
```

```
    (variance_pd["actual_materials_price"] -
 ↪  variance_pd["std_materials_price"])
    * variance_pd["actual_materials_qty_total"]
)

# Materials quantity variance = (Actual Qty - Standard Qty Allowed) x
 ↪  Standard Price
variance_pd["std_qty_allowed"] =
 ↪  variance_pd["std_materials_qty_per_unit"] *
 ↪  variance_pd["units_produced"]
variance_pd["materials_qty_variance"] = (
    (variance_pd["actual_materials_qty_total"] -
 ↪  variance_pd["std_qty_allowed"])
    * variance_pd["std_materials_price"]
)

variance_pd[["month", "materials_price_variance",
 ↪  "materials_qty_variance"]].round(0)
```

	month	materials_price_variance	materials_qty_variance
0	2025-01	-412.0	-1690.0
1	2025-02	1402.0	-1737.0
2	2025-03	-15.0	1092.0
3	2025-04	593.0	-569.0
4	2025-05	880.0	1377.0
5	2025-06	1142.0	2859.0
6	2025-07	2585.0	538.0
7	2025-08	1063.0	2038.0
8	2025-09	1750.0	-521.0
9	2025-10	515.0	-1292.0
10	2025-11	1534.0	991.0
11	2025-12	2583.0	-423.0

8.6. polars

```
variance_pl = variance_pl.with_columns([
    ((pl.col("actual_materials_price") -
 ↪  pl.col("std_materials_price")) *
 ↪  pl.col("actual_materials_qty_total"))
```

```
        .alias("materials_price_variance"),
    (pl.col("std_materials_qty_per_unit") *
 ↪  pl.col("units_produced")).alias("std_qty_allowed"),
]).with_columns(
    ((pl.col("actual_materials_qty_total") -
 ↪  pl.col("std_qty_allowed")) * pl.col("std_materials_price"))
        .alias("materials_qty_variance")
)

variance_pl.select(["month", "materials_price_variance",
 ↪  "materials_qty_variance"])
```

month str	materials_price_variance f64	materials_qty_variance f64
"2025-01"	-412.3224	-1690.2
"2025-02"	1401.6	-1737.0
"2025-03"	-14.8886	1091.7
"2025-04"	593.2608	-568.8
"2025-05"	879.7157	1376.55
…	…	…
"2025-08"	1062.7833	2038.05
"2025-09"	1750.4384	-520.65
"2025-10"	514.6011	-1291.95
"2025-11"	1534.4496	990.9
"2025-12"	2583.2154	-422.55

Reading Variance Signs

By convention, a **positive** variance here means actual cost exceeded standard cost — **unfavorable**. A **negative** variance means actual cost was less than standard — **favorable**. This convention (rather than always reporting an absolute value) preserves the direction of the difference, which is exactly the information a manager needs to know whether to investigate a problem or recognize an improvement.

```
print("Annual materials price variance:",
 ↪  round(variance_pd["materials_price_variance"].sum(), 0))
print("Annual materials quantity variance:",
 ↪  round(variance_pd["materials_qty_variance"].sum(), 0))
```

```
Annual materials price variance: 13620.0
Annual materials quantity variance: 2663.0
```

The materials **price** variance is unfavorable by about $13,600 for the year — actual material prices ran above standard, consistent with a gradual price increase over the year. The materials **quantity** variance is a smaller unfavorable $2,700 — usage was close to standard, with only modest waste.

8.6.1. Labor Variances

Labor variances follow the same logic: a **rate** variance (did we pay a different wage than standard?) and an **efficiency** variance (did it take more or fewer hours than standard for the output produced?).

8.7. pandas

```
variance_pd["labor_rate_variance"] = (
    (variance_pd["actual_labor_rate"] - variance_pd["std_labor_rate"])
 ↪  * variance_pd["actual_labor_hours_total"]
)

variance_pd["std_hours_allowed"] =
 ↪  variance_pd["std_labor_hours_per_unit"] *
 ↪  variance_pd["units_produced"]
variance_pd["labor_efficiency_variance"] = (
    (variance_pd["actual_labor_hours_total"] -
 ↪  variance_pd["std_hours_allowed"]) * variance_pd["std_labor_rate"]
)

variance_pd[["month", "labor_rate_variance",
 ↪  "labor_efficiency_variance"]].round(0)
```

	month	labor_rate_variance	labor_efficiency_variance
0	2025-01	553.0	-1028.0
1	2025-02	-1904.0	-2906.0
2	2025-03	968.0	1390.0
3	2025-04	1487.0	-4525.0
4	2025-05	-1181.0	2036.0
5	2025-06	-104.0	-3975.0

	month	labor_rate_variance	labor_efficiency_variance
6	2025-07	1725.0	-3870.0
7	2025-08	590.0	-5662.0
8	2025-09	-1681.0	-3356.0
9	2025-10	-273.0	796.0
10	2025-11	1283.0	-4632.0
11	2025-12	-1127.0	-4898.0

8.8. polars

```
variance_pl = variance_pl.with_columns([
    ((pl.col("actual_labor_rate") - pl.col("std_labor_rate")) *
 ↪  pl.col("actual_labor_hours_total"))
        .alias("labor_rate_variance"),
    (pl.col("std_labor_hours_per_unit") *
 ↪  pl.col("units_produced")).alias("std_hours_allowed"),
]).with_columns(
    ((pl.col("actual_labor_hours_total") -
 ↪  pl.col("std_hours_allowed")) * pl.col("std_labor_rate"))
        .alias("labor_efficiency_variance")
)

variance_pl.select(["month", "labor_rate_variance",
 ↪  "labor_efficiency_variance"])
```

month str	labor_rate_variance f64	labor_efficiency_variance f64
"2025-01"	552.995	-1028.5
"2025-02"	-1903.704	-2906.2
"2025-03"	968.422	1390.4
"2025-04"	1487.058	-4525.4
"2025-05"	-1181.232	2036.1
…	…	…
"2025-08"	590.348	-5661.7
"2025-09"	-1680.552	-3356.1
"2025-10"	-272.536	796.4
"2025-11"	1282.972	-4632.1

month str	labor_rate_variance f64	labor_efficiency_variance f64
"2025-12"	-1127.44	-4898.3

```
print("Annual labor rate variance:",
 ↪  round(variance_pd["labor_rate_variance"].sum(), 0))
print("Annual labor efficiency variance:",
 ↪  round(variance_pd["labor_efficiency_variance"].sum(), 0))
```

```
Annual labor rate variance: 338.0
Annual labor efficiency variance: -30631.0
```

The labor **efficiency** variance is favorable by roughly $30,600 — the workforce is completing units in meaningfully fewer hours than the standard allows, consistent with a learning-curve effect as workers become more practiced over the year. This is the single largest variance in the whole dataset, and it's favorable — a genuinely positive operational story.

8.8.1. Overhead Variances

Overhead variances measure the difference between actual manufacturing overhead costs and **applied** standard costs. Variable overhead splits into *spending* and *efficiency* variances based on activity levels. Variable overhead variances are very similar in terms of computation to materials and labor variances. Fixed overhead splits into spending (budget) and volume variances based on capacity and production.

$$\text{VC Spending Variance} = \text{Actual Hours} \times (\text{Actual Rate} - \text{Standard Rate})$$

$$\text{VC Efficiency Variance} = \text{Standard Rate} \times (\text{Actual Hours} - \text{Standard Hours})$$

Fixed overhead costs remain constant in total regardless of changes in production volume within a relevant range. Figure 8.2 shows total fixed cost overhead variance breakdown.

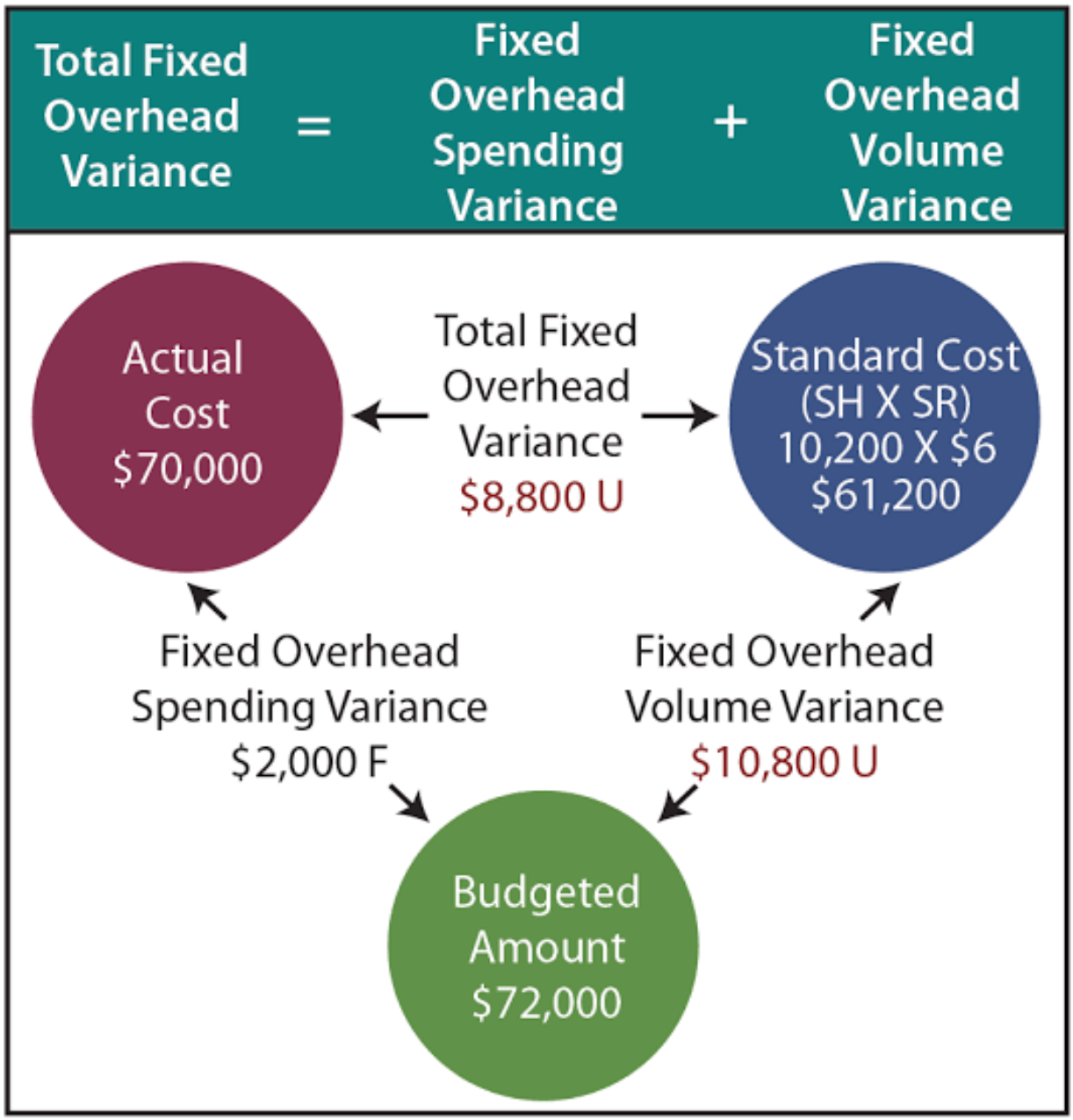


Figure 8.2.: Fixed Overhead Variance

```
variance_pd["var_oh_applied"] =
 ↪  variance_pd["actual_labor_hours_total"] *
 ↪  variance_pd["std_var_oh_rate_per_labor_hr"]
variance_pd["var_oh_spending_variance"] =
 ↪  variance_pd["actual_var_oh_total"] - variance_pd["var_oh_applied"]
variance_pd["fixed_oh_budget_variance"] =
 ↪  variance_pd["actual_fixed_oh"] - variance_pd["budgeted_fixed_oh"]
```

```
print("Annual variable overhead spending variance:",
 ↪  round(variance_pd["var_oh_spending_variance"].sum(), 0))
print("Annual fixed overhead budget variance:",
 ↪  round(variance_pd["fixed_oh_budget_variance"].sum(), 0))
```

```
Annual variable overhead spending variance: 1218.0
Annual fixed overhead budget variance: 5070.0
```

8.8.2. Visualizing the Full Variance Picture

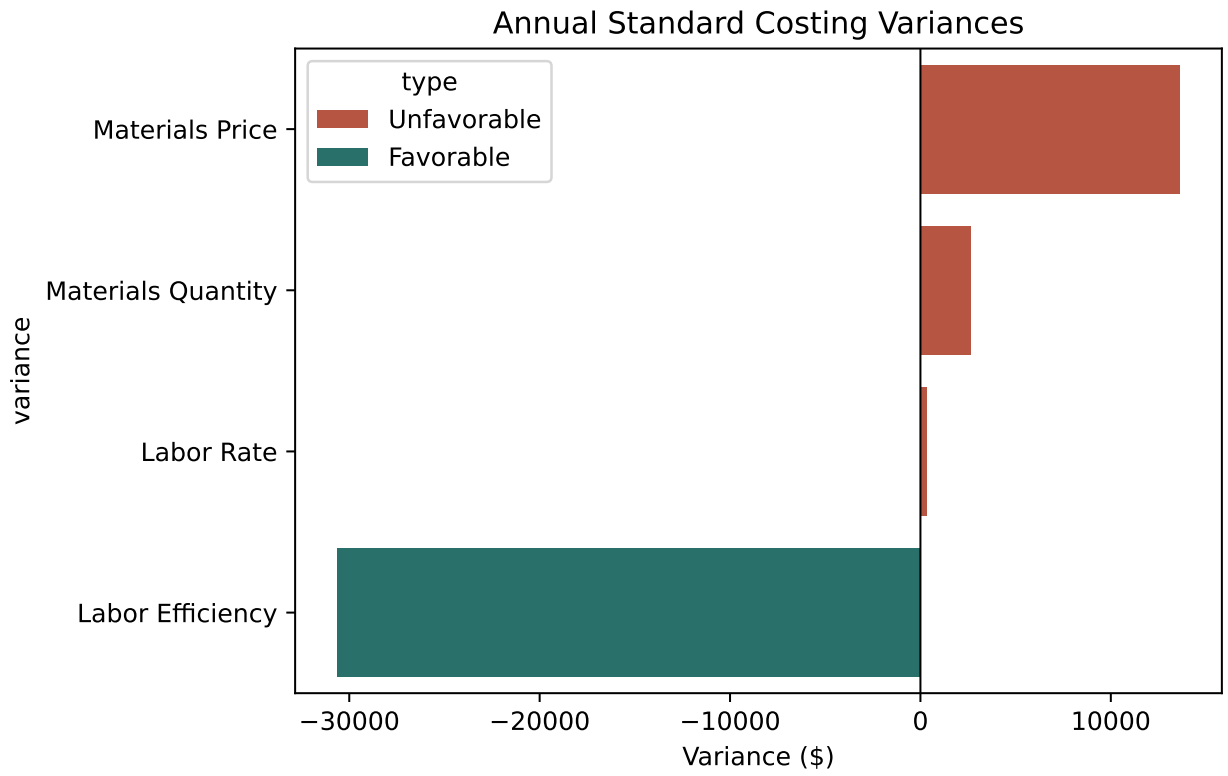
```
variance_summary = pd.DataFrame({
    "variance": ["Materials Price", "Materials Quantity", "Labor
     ↪  Rate", "Labor Efficiency"],
    "amount": [
        variance_pd["materials_price_variance"].sum(),
        variance_pd["materials_qty_variance"].sum(),
        variance_pd["labor_rate_variance"].sum(),
        variance_pd["labor_efficiency_variance"].sum(),
    ],
})
variance_summary["type"] = variance_summary["amount"].apply(lambda x:
 ↪  "Unfavorable" if x > 0 else "Favorable")
```

8.9. seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 4.5))
sns.barplot(
    data=variance_summary, x="amount", y="variance", hue="type",
    palette={"Unfavorable": "#c9482f", "Favorable": "#1c7c73"}
)
plt.title("Annual Standard Costing Variances")
plt.xlabel("Variance ($)")
plt.axvline(0, color="black", linewidth=0.8)
```

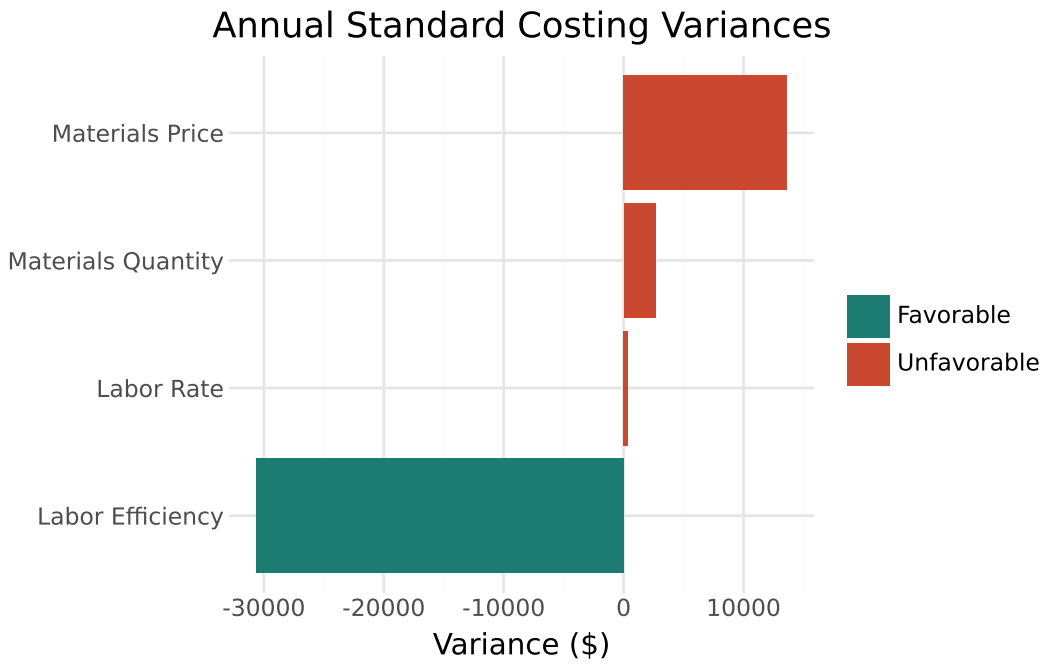
```
plt.tight_layout()
plt.show()
```



8.10. plotnine

```
from plotnine import *

(
    ggplot(variance_summary, aes(x="reorder(variance, amount)",
 ↪  y="amount", fill="type"))
    + geom_col()
    + coord_flip()
    + scale_fill_manual(values={"Unfavorable": "#c9482f", "Favorable":
     ↪  "#1c7c73"})
    + labs(title="Annual Standard Costing Variances", x="",
     ↪  y="Variance ($)", fill="")
    + theme_minimal()
)
```



! Reading the Full Picture Together

No single variance tells the whole story. Rising material prices (unfavorable) are being more than offset by a strong labor efficiency gain (favorable) — a pattern only visible by looking at all four variances side by side. A manager who only reviewed the materials price variance might conclude the year was purely a story of rising input costs, missing the larger, favorable efficiency story happening in labor. This is the same lesson from Chapter 7's DuPont analysis: a single number in isolation can mislead, and decomposition reveals the real drivers.

8.11. Cost-Volume-Profit (CVP) Analysis

CVP analysis answers a different question: given our **cost structure**[1], how many units do we need to sell to break even, and how much cushion do we currently have above that point?

8.12. pandas

```
cvp_pd = pd.read_csv("data/cvp_monthly_data.csv")
```

[1]Cost structure refers to the proportion of variable and fixed cost in total cost. For example, if total cost is $100, variable cost could be $40 and fixed cost then is $60 or vice versa. Then, cost structure is variable cost is 40% and fixed cost is 60%. Cost structure has critical implications while managers make decisions.

```
cvp_pd.head()
```

	month	units_sold	selling_price_per_unit	variable_cost_per_unit	fixed_costs
0	2025-01	3783	52.0	35.58	44003.67
1	2025-02	3866	52.0	34.26	43327.21
2	2025-03	4809	52.0	36.96	43944.08
3	2025-04	4079	52.0	34.73	39148.50
4	2025-05	3760	52.0	36.74	43761.73

8.13. polars

```
cvp_pl = pl.read_csv("data/cvp_monthly_data.csv")
cvp_pl.head()
```

month str	units_sold i64	selling_price_per_unit f64	variable_cost_per_unit f64	fixed_costs f64
"2025-01"	3783	52.0	35.58	44003.67
"2025-02"	3866	52.0	34.26	43327.21
"2025-03"	4809	52.0	36.96	43944.08
"2025-04"	4079	52.0	34.73	39148.5
"2025-05"	3760	52.0	36.74	43761.73

8.13.1. Contribution Margin and Break-Even

The **contribution margin per unit** represents the amount of money each individual item sold contributes toward covering your company's fixed costs. It isolates profitability at the product level by stripping away fixed constraints, subtracting only the variable costs—such as raw materials, direct labor, and sales commissions—from the unit's selling price.

The **contribution margin ratio** expresses your profit margins as a clean percentage of total sales revenue rather than a flat dollar amount. By dividing the contribution margin by total sales, you instantly see what percentage of each dollar generated is available to pay down fixed overhead and fund bottom-line profits. This ratio is a vital tool for managers forecasting profitability, as it allows you to quickly calculate how a sudden 10% increase or decrease in gross sales volume will mathematically impact net income.

The **break-even point in units** calculates the exact physical quantity of product your business must manufacture and sell to achieve a net income of zero. To find this number, total fixed costs are divided by the contribution margin per unit, revealing the baseline production target required to avoid losing money. Tracking this metric gives operations managers a clear, tangible daily or monthly sales quota that production teams must hit before the business can claim true profitability.

The **break-even point in dollars** translates your operational baseline into a concrete financial revenue target, dictating the total sales volume needed to perfectly balance the books. By dividing total fixed costs by the *contribution margin ratio*, this formula determines the gross cash inflow required to cover all combined variable and fixed expenses. This dollar-centric metric is highly prized by executives and lenders, as it provides a universal benchmark to evaluate financial risk and measure safety margins against actual sales forecasts.

The **Margin of Safety (or safety margin)** is a financial metric that measures the cushion a business has between its current or projected sales volume and its break-even point. It represents the amount by which sales can drop before the business begins to lose money, serving as a critical indicator of financial risk. A high margin of safety means the business is well-protected against economic downturns or sales drops, while a low margin indicates high risk.

$$\begin{aligned}\text{Contribution Margin per Unit} = &\text{Selling Price per Unit} \\ &- \text{Variable Cost per Unit}\end{aligned}$$

$$\text{Contribution Margin Ratio} = \frac{\text{Contribution Margin per Unit}}{\text{Selling Price per Unit}}$$

$$\text{Break-Even Point (Units)} = \frac{\text{Fixed Costs}}{\text{Contribution Margin per Unit}}$$

$$\text{Break-Even Point (Dollars)} = \frac{\text{Fixed Costs}}{\text{Contribution Margin Ratio}}$$

$$\begin{aligned}\text{Margin of Safety (Dollars)} = &\text{Current or Budgeted Sales} \\ &- \text{Break-Even Sales}\end{aligned}$$

$$\begin{aligned}\text{Margin of Safety (Units)} = &\text{Current or Budgeted Units Sold} \\ &- \text{Break-Even Units}\end{aligned}$$

$$\text{Margin of Safety Percentage} = \frac{\text{Margin of Safety (Dollars)}}{\text{Current or Budgeted Sales}} \times 100$$

8.14. pandas

```
cvp_pd["contribution_margin_per_unit"] =
 ↪  cvp_pd["selling_price_per_unit"] -
 ↪  cvp_pd["variable_cost_per_unit"]
cvp_pd["cm_ratio"] = cvp_pd["contribution_margin_per_unit"] /
 ↪  cvp_pd["selling_price_per_unit"]
cvp_pd["breakeven_units"] = cvp_pd["fixed_costs"] /
 ↪  cvp_pd["contribution_margin_per_unit"]
cvp_pd["margin_of_safety_pct"] = (cvp_pd["units_sold"] -
 ↪  cvp_pd["breakeven_units"]) / cvp_pd["units_sold"] * 100

cvp_pd[["month", "contribution_margin_per_unit", "cm_ratio",
 ↪  "breakeven_units", "margin_of_safety_pct"]].round(2)
```

	month	contribution_margin_per_unit	cm_ratio	breakeven_units	margin_of_safety_pct
0	2025-01	16.42	0.32	2679.88	29.16
1	2025-02	17.74	0.34	2442.35	36.82
2	2025-03	15.04	0.29	2921.81	39.24
3	2025-04	17.27	0.33	2266.85	44.43
4	2025-05	15.26	0.29	2867.74	23.73
5	2025-06	16.39	0.32	2592.71	46.06
6	2025-07	16.41	0.32	2590.94	34.51
7	2025-08	16.51	0.32	2485.33	34.80
8	2025-09	17.04	0.33	2518.38	43.92
9	2025-10	15.71	0.30	2633.82	40.49
10	2025-11	16.50	0.32	2534.58	40.15
11	2025-12	17.56	0.34	2433.35	38.98

8.15. polars

```
cvp_pl = cvp_pl.with_columns([
    (pl.col("selling_price_per_unit") -
 ↪  pl.col("variable_cost_per_unit")).alias("contribution_margin_per_unit"),
]).with_columns([
    (pl.col("contribution_margin_per_unit") /
 ↪  pl.col("selling_price_per_unit")).alias("cm_ratio"),
```

```
    (pl.col("fixed_costs") /
 ↪  pl.col("contribution_margin_per_unit")).alias("breakeven_units"),
]).with_columns(
    ((pl.col("units_sold") - pl.col("breakeven_units")) /
 ↪  pl.col("units_sold") * 100).alias("margin_of_safety_pct")
)

cvp_pl.select(["month", "contribution_margin_per_unit", "cm_ratio",
 ↪  "breakeven_units", "margin_of_safety_pct"])
```

month str	contribution_margin_per_unit f64	cm_ratio f64	breakeven_units f64	margin_of_safety_pct f64
"2025-01"	16.42	0.315769	2679.88246	29.159861
"2025-02"	17.74	0.341154	2442.345547	36.824999
"2025-03"	15.04	0.289231	2921.81383	39.242798
"2025-04"	17.27	0.33211 5	2266.850029	44.426329
"2025-05"	15.26	0.293462	2867.741153	23.730288
…	…	…	…	…
"2025-08"	16.51	0.3175	2485.327075	34.802543
"2025-09"	17.04	0.327692	2518.38439	43.92375
"2025-10"	15.71	0.302115	2633.81795	40.492138
"2025-11"	16.5	0.317308	2534.575758	40.151694
"2025-12"	17.56	0.337692	2433.346811	38.98328

The contribution margin ratio sits consistently around 30-33%, and the margin of safety — how far actual sales exceed the break-even point — ranges from roughly 24% to 46% across months. Every month in this dataset operated safely above break-even, but with meaningfully more cushion in some months (June, September) than others (January, May).

8.15.1. The Break-Even Chart

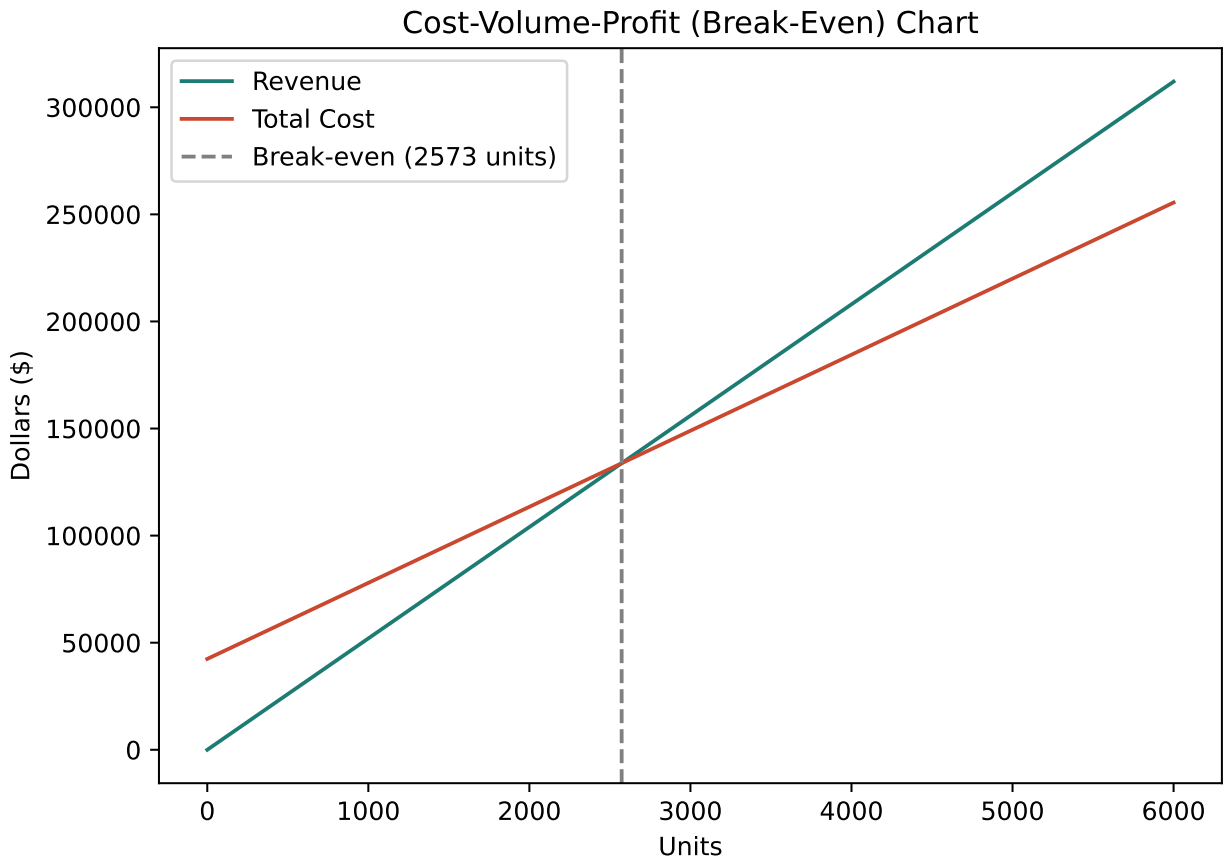
```
import numpy as np

avg_price = cvp_pd["selling_price_per_unit"].mean()
avg_vc = cvp_pd["variable_cost_per_unit"].mean()
avg_fc = cvp_pd["fixed_costs"].mean()
breakeven_units = avg_fc / (avg_price - avg_vc)
```

```
units_range = np.linspace(0, 6000, 100)
be_df = pd.DataFrame({
    "units": units_range,
    "revenue": units_range * avg_price,
    "total_cost": avg_fc + units_range * avg_vc,
})
```

8.16. seaborn

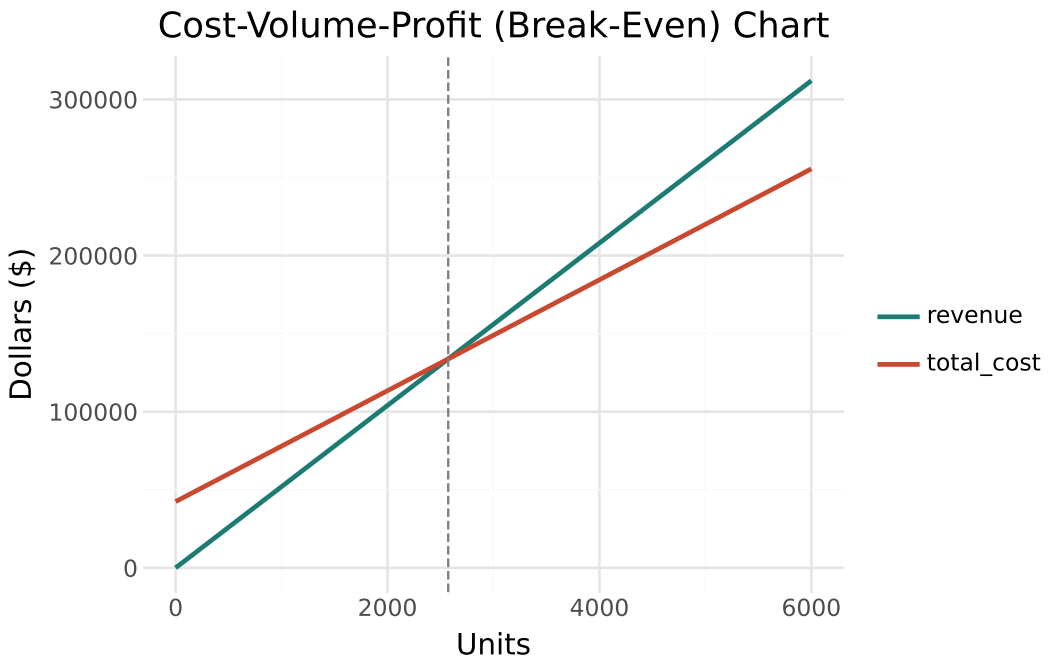
```
plt.figure(figsize=(7, 5))
plt.plot(be_df["units"], be_df["revenue"], label="Revenue",
 ↪  color="#1c7c73")
plt.plot(be_df["units"], be_df["total_cost"], label="Total Cost",
 ↪  color="#c9482f")
plt.axvline(breakeven_units, linestyle="--", color="gray",
 ↪  label=f"Break-even ({breakeven_units:.0f} units)")
plt.title("Cost-Volume-Profit (Break-Even) Chart")
plt.xlabel("Units")
plt.ylabel("Dollars ($)")
plt.legend()
plt.tight_layout()
plt.show()
```



8.17. plotnine

```
be_long = be_df.melt(id_vars="units", value_vars=["revenue",
 ↪  "total_cost"], var_name="line", value_name="dollars")

(
    ggplot(be_long, aes(x="units", y="dollars", color="line"))
    + geom_line(size=1)
    + geom_vline(xintercept=breakeven_units, linetype="dashed",
     ↪  color="gray")
    + scale_color_manual(values={"revenue": "#1c7c73", "total_cost":
     ↪  "#c9482f"})
    + labs(title="Cost-Volume-Profit (Break-Even) Chart", x="Units",
     ↪  y="Dollars ($)", color="")
    + theme_minimal()
)
```



The break-even point sits at roughly 2,570 units per month — well below every month's actual sales volume in this dataset (all above 3,700 units), consistent with the healthy margin of safety percentages computed above.

8.17.1. Target Profit Analysis

CVP logic extends naturally to a different question: how many units are needed to hit a specific profit target, not just to break even?

```
target_profit = 30000
target_units = (avg_fc + target_profit) / (avg_price - avg_vc)
print(f"Units needed for a ${target_profit:,} monthly target profit:
 ↪  {target_units:.0f}")
```

```
Units needed for a $30,000 monthly target profit: 4393
```

8.18. Activity-Based Costing (ABC)

Traditional overhead allocation spreads all manufacturing overhead across products using a single base — often machine hours or direct labor hours. This works reasonably well when products consume overhead resources in **roughly the same proportions**. When products differ substantially in complexity, it can systematically distort product costs.

8.19. pandas

```
products_pd = pd.read_csv("data/abc_cost_data.csv")
products_pd[["product", "units_produced", "machine_hours",
 ↪  "num_setups", "num_inspections"]]
```

	product	units_produced	machine_hours	num_setups	num_inspections
0	Standard-A	18000	3600	12	40
1	Standard-B	15000	3000	10	35
2	Custom-C	1200	480	45	90
3	Custom-D	900	360	40	85
4	Specialty-E	300	180	30	60

8.20. polars

```
products_pl = pl.read_csv("data/abc_cost_data.csv")
products_pl.select(["product", "units_produced", "machine_hours",
 ↪  "num_setups", "num_inspections"])
```

product str	units_produced i64	machine_hours i64	num_setups i64	num_inspections i64
”Standard-A”	18000	3600	12	40
”Standard-B”	15000	3000	10	35
”Custom-C”	1200	480	45	90
”Custom-D”	900	360	40	85
”Specialty-E”	300	180	30	60

Notice the pattern: **Standard-A** and **Standard-B** are produced in large volumes with relatively few setups or inspections per unit — simple, high-volume products. **Custom-C**, **Custom-D**, and **Specialty-E** are produced in much smaller volumes but require a disproportionate number of setups and inspections — complex, low-volume products.

8.20.1. Traditional Allocation (Single Base: Machine Hours)

8.21. pandas

```
TOTAL_OVERHEAD = 300_000

trad_rate_per_mh = TOTAL_OVERHEAD / products_pd["machine_hours"].sum()
products_pd["traditional_oh_allocated"] = products_pd["machine_hours"]
 ↪  * trad_rate_per_mh

products_pd[["product", "machine_hours",
 ↪  "traditional_oh_allocated"]].round(0)
```

	product	machine_hours	traditional_oh_allocated
0	Standard-A	3600	141732.0
1	Standard-B	3000	118110.0
2	Custom-C	480	18898.0
3	Custom-D	360	14173.0
4	Specialty-E	180	7087.0

8.22. polars

```
total_machine_hours =
 ↪  products_pl.select(pl.col("machine_hours").sum()).item()
trad_rate_per_mh_pl = TOTAL_OVERHEAD / total_machine_hours

products_pl = products_pl.with_columns(
    (pl.col("machine_hours") *
 ↪  trad_rate_per_mh_pl).alias("traditional_oh_allocated")
)

products_pl.select(["product", "machine_hours",
 ↪  "traditional_oh_allocated"])
```

product str	machine_hours i64	traditional_oh_allocated f64
"Standard-A"	3600	141732.283465
"Standard-B"	3000	118110.23622
"Custom-C"	480	18897.637795
"Custom-D"	360	14173.228346
"Specialty-E"	180	7086.614173

8.22.1. Activity-Based Allocation (Three Cost Pools)

ABC instead splits overhead into pools tied to specific activities, each allocated using the cost driver that actually causes that cost — setups, inspections, and machine time each get their own rate.

8.23. pandas

```
setup_pool, inspection_pool, machine_pool = TOTAL_OVERHEAD * 0.35,
 ↪  TOTAL_OVERHEAD * 0.25, TOTAL_OVERHEAD * 0.40

setup_rate = setup_pool / products_pd["num_setups"].sum()
inspection_rate = inspection_pool /
 ↪  products_pd["num_inspections"].sum()
machine_rate = machine_pool / products_pd["machine_hours"].sum()

products_pd["abc_setup_cost"] = products_pd["num_setups"] * setup_rate
products_pd["abc_inspection_cost"] = products_pd["num_inspections"] *
 ↪  inspection_rate
products_pd["abc_machine_cost"] = products_pd["machine_hours"] *
 ↪  machine_rate
products_pd["abc_oh_allocated"] = (
    products_pd["abc_setup_cost"] + products_pd["abc_inspection_cost"]
 ↪  + products_pd["abc_machine_cost"]
)

products_pd[["product", "abc_setup_cost", "abc_inspection_cost",
 ↪  "abc_machine_cost", "abc_oh_allocated"]].round(0)
```

	product	abc_setup_cost	abc_inspection_cost	abc_machine_cost	abc_oh_allocated
0	Standard-A	9197.0	9677.0	56693.0	75567.0
1	Standard-B	7664.0	8468.0	47244.0	63376.0
2	Custom-C	34489.0	21774.0	7559.0	63822.0
3	Custom-D	30657.0	20565.0	5669.0	56891.0
4	Specialty-E	22993.0	14516.0	2835.0	40343.0

8.24. polars

```
setup_pool_pl, inspection_pool_pl, machine_pool_pl = TOTAL_OVERHEAD *
 ↪  0.35, TOTAL_OVERHEAD * 0.25, TOTAL_OVERHEAD * 0.40

setup_rate_pl = setup_pool_pl /
 ↪  products_pl.select(pl.col("num_setups").sum()).item()
inspection_rate_pl = inspection_pool_pl /
 ↪  products_pl.select(pl.col("num_inspections").sum()).item()
machine_rate_pl = machine_pool_pl /
 ↪  products_pl.select(pl.col("machine_hours").sum()).item()

products_pl = products_pl.with_columns([
    (pl.col("num_setups") * setup_rate_pl).alias("abc_setup_cost"),
    (pl.col("num_inspections") *
 ↪  inspection_rate_pl).alias("abc_inspection_cost"),
    (pl.col("machine_hours") *
 ↪  machine_rate_pl).alias("abc_machine_cost"),
]).with_columns(
    (pl.col("abc_setup_cost") + pl.col("abc_inspection_cost") +
 ↪  pl.col("abc_machine_cost")).alias("abc_oh_allocated")
)

products_pl.select(["product", "abc_setup_cost",
 ↪  "abc_inspection_cost", "abc_machine_cost", "abc_oh_allocated"])
```

product str	abc_setup_cost f64	abc_inspection_cost f64	abc_machine_cost f64	abc_oh_allocated f64
”Standard-A”	9197.080292	9677.419355	56692.913386	75567.413033
”Standard-B”	7664.233577	8467.741935	47244.094488	63376.07

product str	abc_setup_cost f64	abc_inspection_cost f64	abc_machine_cost f64	abc_oh_allocated f64
"Custom-C"	34489.051095	21774.193548	7559.055118	63822.299761
"Custom-D"	30656.934307	20564.516129	5669.291339	56890.741774
"Specialty-E"	22992.70073	14516.129032	2834.645669	40343.475431

Both methods allocate the same $300,000 total — the only question is *how* it's distributed across the five products.

```
products_pd["traditional_cost_per_unit"] = (
    products_pd["direct_materials_cost"] +
 ↪  products_pd["direct_labor_cost"] +
 ↪  products_pd["traditional_oh_allocated"]
) / products_pd["units_produced"]

products_pd["abc_cost_per_unit"] = (
    products_pd["direct_materials_cost"] +
 ↪  products_pd["direct_labor_cost"] + products_pd["abc_oh_allocated"]
) / products_pd["units_produced"]

products_pd["cost_per_unit_difference"] =
 ↪  products_pd["abc_cost_per_unit"] -
 ↪  products_pd["traditional_cost_per_unit"]
products_pd[["product", "traditional_cost_per_unit",
 ↪  "abc_cost_per_unit", "cost_per_unit_difference"]].round(2)
```

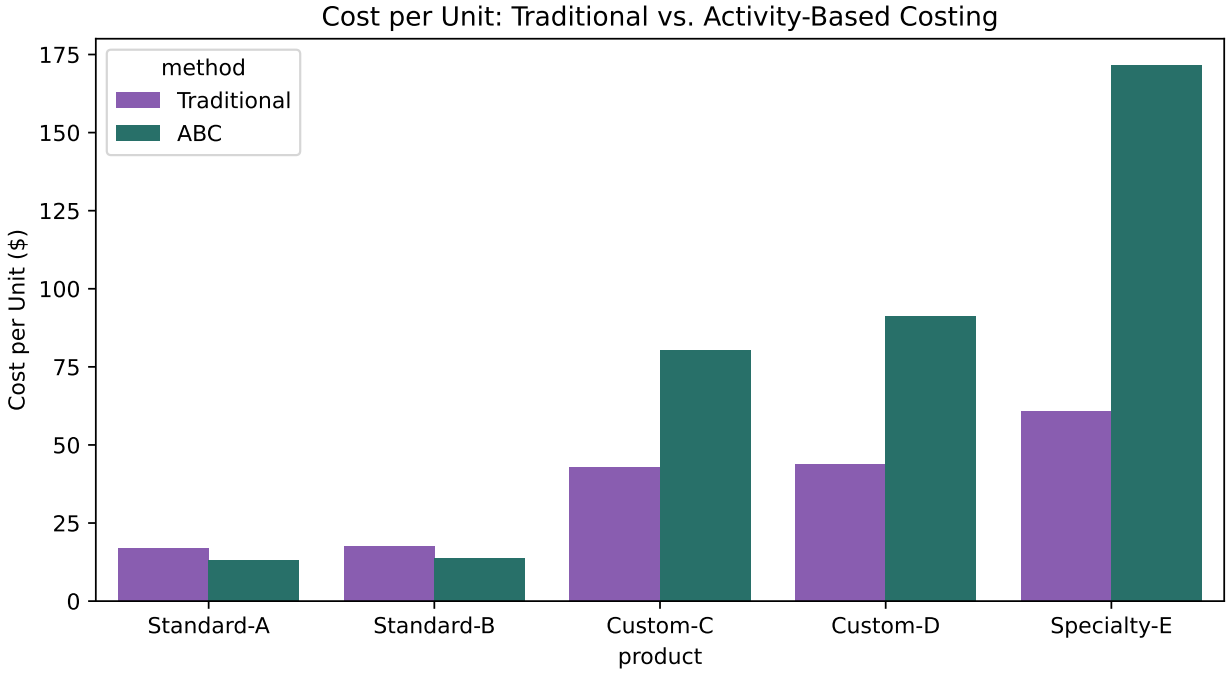
	product	traditional_cost_per_unit	abc_cost_per_unit	cost_per_unit_difference
0	Standard-A	16.87	13.20	-3.68
1	Standard-B	17.37	13.73	-3.65
2	Custom-C	42.75	80.19	37.44
3	Custom-D	43.75	91.21	47.46
4	Specialty-E	60.62	171.48	110.86

8.25. seaborn

```
comparison = products_pd.melt(
```

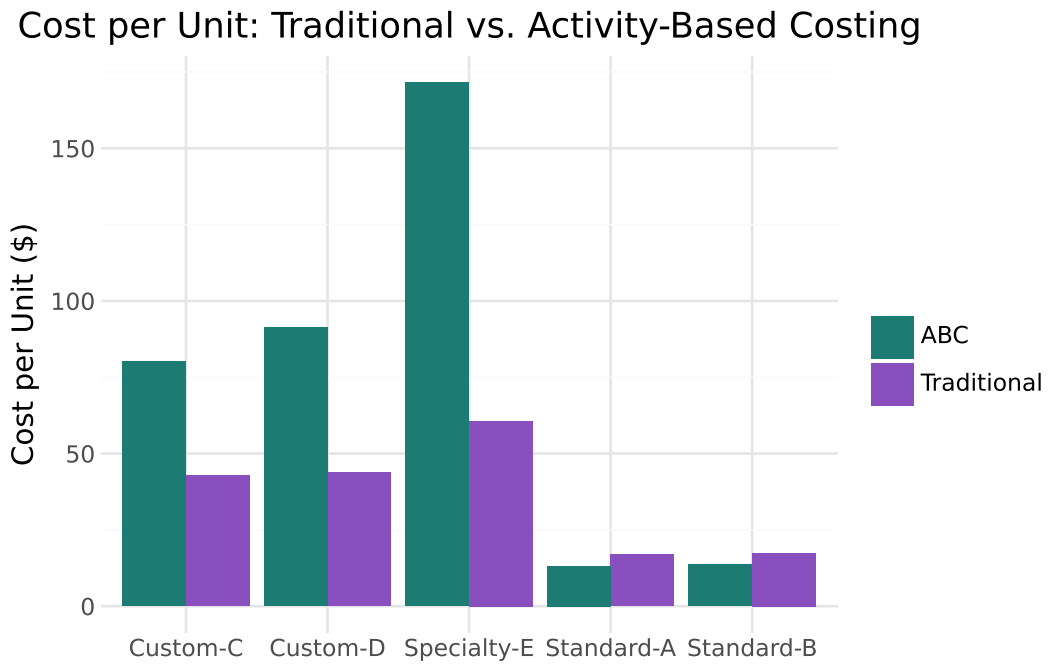
```
    id_vars="product", value_vars=["traditional_cost_per_unit",
 ↪  "abc_cost_per_unit"],
    var_name="method", value_name="cost_per_unit"
)
comparison["method"] = comparison["method"].map({
    "traditional_cost_per_unit": "Traditional", "abc_cost_per_unit":
     ↪  "ABC"
})

plt.figure(figsize=(8, 4.5))
sns.barplot(data=comparison, x="product", y="cost_per_unit",
 ↪  hue="method",
            palette={"Traditional": "#8a4fbe", "ABC": "#1c7c73"})
plt.title("Cost per Unit: Traditional vs. Activity-Based Costing")
plt.ylabel("Cost per Unit ($)")
plt.tight_layout()
plt.show()
```



8.26. plotnine

```
(
    ggplot(comparison, aes(x="product", y="cost_per_unit",
    ↪ fill="method"))
    + geom_col(position="dodge")
    + scale_fill_manual(values={"Traditional": "#8a4fbe", "ABC":
    ↪  "#1c7c73"})
    + labs(title="Cost per Unit: Traditional vs. Activity-Based
    ↪  Costing", x="", y="Cost per Unit ($)", fill="")
    + theme_minimal()
)
```



! The Classic ABC Finding, Reproduced Here

Look at the direction of every difference: **Standard-A** and **Standard-B** — the high-volume, simple products — cost *less* per unit under ABC than under traditional allocation. **Custom-C**, **Custom-D**, and especially **Specialty-E** — the low-volume, complex products — cost dramatically *more* per unit under ABC, with Specialty-E's cost increasing by nearly $111 per unit.

This is precisely the textbook ABC finding: traditional single-base allocation systematically **undercosts** low-volume, complex products (since they don't consume much machine time, even though they generate disproportionate setup and inspection activity) while **overcost-**

ing high-volume, simple products. A company pricing Specialty-E based on its traditional-costing number could be selling it at a loss without realizing it — exactly the kind of decision-relevant insight management accounting analytics is meant to surface.

8.27. Chapter Summary

- Management accounting analytics serves internal decision-makers rather than external stakeholders, and isn't bound by GAAP or subject to external audit — its only test is whether it helps managers make better decisions.
- Standard costing variances decompose the gap between actual and expected cost into specific, actionable causes: price/rate variances (did we pay more or less?) and quantity/efficiency variances (did we use more or less than the standard allows?).
- No single variance should be read in isolation — in this chapter, a real cost increase (materials price) was more than offset by a real efficiency gain (labor), a pattern only visible when every variance is viewed together.
- CVP analysis (contribution margin, break-even point, margin of safety, target profit) answers different but related operational questions than variance analysis, focused on volume and profitability rather than cost control.
- Traditional overhead allocation and activity-based costing can produce meaningfully different product costs whenever products vary substantially in production complexity — ABC's multiple cost-driver approach corrects for a bias that a single allocation base cannot.
- Every technique in this chapter was implemented in both pandas and polars, and visualized in both seaborn and plotnine, following the same dual-tooling approach used throughout this book.

8.28. Discussion Questions

1. The labor efficiency variance was the largest single variance in this chapter, and it was favorable. What follow-up questions would you want answered before concluding this reflects genuine, sustainable operational improvement rather than, say, a one-time factor?
2. If a manager only reviewed the materials price variance without also looking at the labor efficiency variance, what incorrect conclusion might they draw about the year's cost performance?
3. Specialty-E's cost per unit rises by nearly $111 under ABC. If this company had been pricing Specialty-E based on the traditional cost figure, what business risk does that create, and what would you recommend management do next?

8.29. Exercises

1. Recompute the fixed overhead **volume variance** (the difference between budgeted fixed overhead and fixed overhead applied based on standard hours allowed for actual production) — a variance not covered in this chapter — using `manufacturing_variance_data.csv`.
2. Using `cvp_monthly_data.csv`, calculate the sales volume (in units) needed to increase monthly target profit by 20% compared to this chapter's $30,000 example, and explain whether that volume looks achievable given the historical range of `units_sold` in the dataset.
3. Add a sixth, hypothetical product to `abc_cost_data.csv` with very high volume but also very high setup/inspection activity (unlike any current product). Recompute both traditional and ABC costs for it — does the usual "high volume means ABC lowers cost" pattern still hold?
4. Using either pandas or polars, calculate each product's **overhead allocation as a percentage of total cost** under both traditional and ABC methods. Which product's overhead percentage changes the most, and does that match your intuition from the cost-per-unit chart in this chapter?

9. Audit Analytics

Learning Objectives

By the end of this chapter, you should be able to:

- Apply Benford's Law as a first-digit test on transaction amounts, and correctly interpret what a deviation does and doesn't tell you
- Design and run common journal entry tests: weekend postings, round-dollar amounts, period-end cutoff timing, duplicate entries, and unusual posting activity
- Explain and implement simple random, systematic, stratified, and monetary unit sampling
- Combine multiple red flags into a composite risk score to prioritize audit review
- Interpret analytics output the way an auditor would: as a starting point for investigation, not a final verdict

9.1. From EDA to Audit Testing

Chapter 1 introduced the idea that modern audits increasingly test **entire populations** of transactions rather than small manual samples. This chapter turns that idea into practice, applying the EDA techniques from Chapter 5 (distributions, outliers) and the visualization skills from Chapter 6 to a specifically audit-oriented set of tests.

We'll use two datasets in this chapter:

- `general_ledger_large.csv` (from Chapter 5) — 752 transactions, useful for Benford's Law (which needs a reasonably large sample) and for demonstrating sampling techniques
- `audit_test_transactions.csv` (new) — a smaller, purpose-built set of 117 journal entries with several deliberately embedded red flags, used for journal entry testing and red-flag scoring

9.2. pandas

```
import pandas as pd
import numpy as np

gl = pd.read_csv("data/general_ledger_large.csv")
audit_gl = pd.read_csv("data/audit_test_transactions.csv")
audit_gl["entry_date"] = pd.to_datetime(audit_gl["entry_date"])

print(gl.shape, audit_gl.shape)
```

```
(1504, 7) (234, 7)
```

9.3. polars

```
import polars as pl

gl_pl = pl.read_csv("data/general_ledger_large.csv")
audit_gl_pl = pl.read_csv("data/audit_test_transactions.csv")
audit_gl_pl = audit_gl_pl.with_columns(
    pl.col("entry_date").str.to_date())
```

9.4. Benford□s Law[1]

Benford□s Law describes the relative frequency distribution for leading digits of numbers in datasets. Leading digits with smaller values occur more frequently than larger values, meaning that in many naturally occurring datasets, the leading digit of numbers is not uniformly distributed — the digit 1 appears as the first digit about 30% of the time, far more often than 9 (which appears about 4.6% of the time). Significant deviations from this expected pattern are sometimes used as a red flag for further investigation, particularly in datasets of financial transaction amounts.

Analysis of datasets shows that many data follow Benford's law. For example, analysts have found that stock prices, population numbers, death rates, sports statistics, TikTok likes, financial and tax information, and billing amounts often have leading digits that follow this distribution. Figure 9.1 shows the distribution of first digit number in Benford's Law.

[1] It is also called First Digit Law

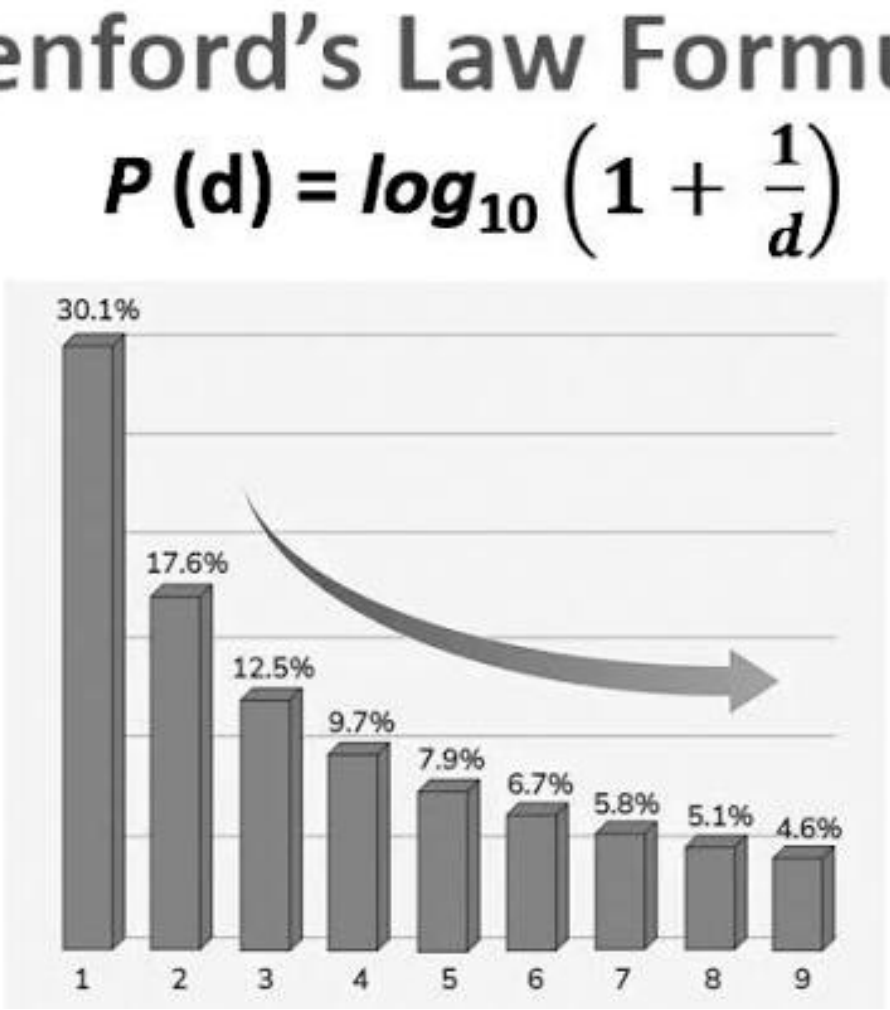


Figure 9.1.: Benford's Law for the First Digit

9.5. pandas

```
debit = gl[gl["debit"] > 0]["debit"]

# extract the first significant digit of each amount
first_digits = (
    debit.astype(str)
    .str.replace(".", "", regex=False)
    .str.lstrip("0")
    .str[0]
    .astype(int)
)
```

```
observed = first_digits.value_counts(normalize=True).sort_index()

benford_expected = pd.Series({d: np.log10(1 + 1/d) for d in range(1,
 ↪  10)})

comparison = pd.DataFrame({"Observed": observed, "Benford Expected":
 ↪  benford_expected})
comparison.round(3)
```

	Observed	Benford Expected
1	0.250	0.301
2	0.247	0.176
3	0.129	0.125
4	0.097	0.097
5	0.090	0.079
6	0.064	0.067
7	0.049	0.058
8	0.036	0.051
9	0.037	0.046

9.6. polars

```
import polars as pl
import math

# Filter positive debit amounts
debit_pl = gl_pl.filter(
    pl.col("debit") > 0
)

# Extract the first significant digit
first_digits_pl = (
    debit_pl
    .select(
        pl.col("debit")
        .cast(pl.String)
```

```
        .str.replace(".", "", literal=True)
        .str.strip_chars_start("0")
        .str.slice(0, 1)
        .cast(pl.Int8)
        .alias("first_digit")
    )
)

# Observed proportions
observed_pl = (
    first_digits_pl
    .group_by("first_digit")
    .len()
    .with_columns(
        (pl.col("len") / pl.col("len").sum()).alias("Observed")
    )
    .select("first_digit", "Observed")
    .sort("first_digit")
)

# Benford expected proportions
benford_expected_pl = pl.DataFrame({
    "first_digit": list(range(1, 10)),
    "Benford Expected": [math.log10(1 + 1 / d) for d in range(1, 10)]
})

# Combine the two tables
comparison_pl = (
    observed_pl
    .join(benford_expected_pl, on="first_digit", how="outer")
    .sort("first_digit")
    .with_columns(
        pl.exclude("first_digit").round(3)
    )
)

comparison_pl
```

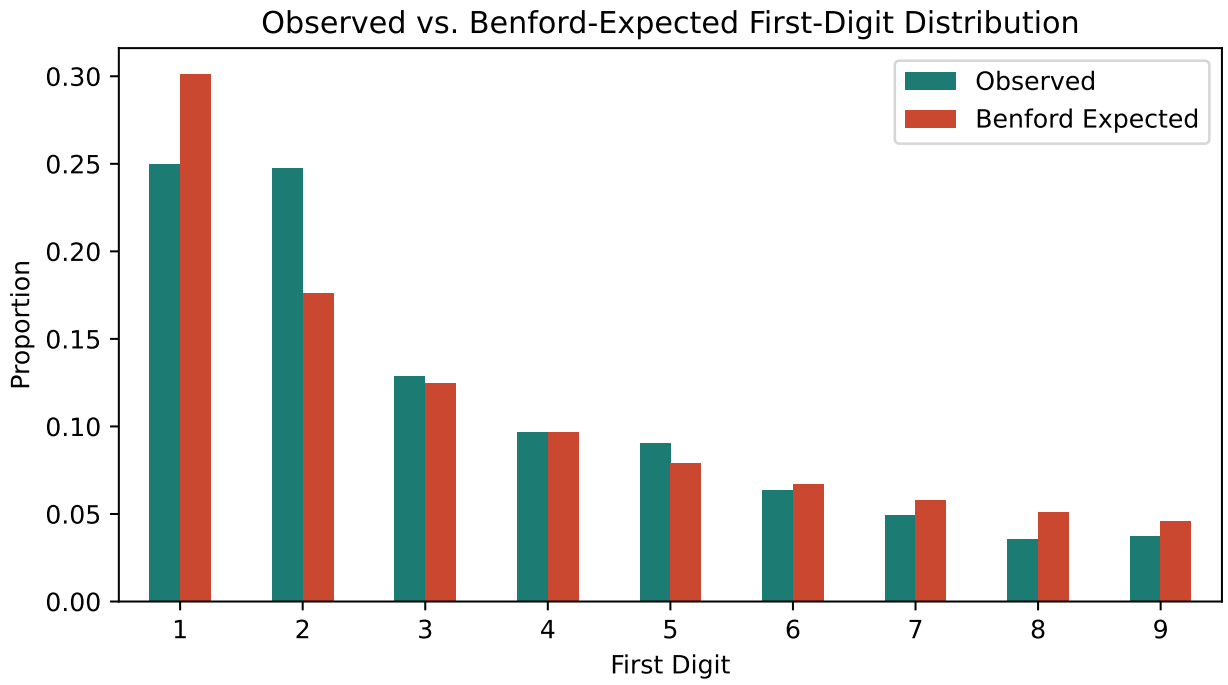
first_digit i8	Observed f64	first_digit_right i64	Benford Expected f64
1	0.25	1	0.301
2	0.247	2	0.176
3	0.129	3	0.125
4	0.097	4	0.097
5	0.09	5	0.079
6	0.064	6	0.067
7	0.049	7	0.058
8	0.036	8	0.051
9	0.037	9	0.046

Now we will visualize the actual distribution and the expected distribution of first digit from Benford's Law using `matplotib`, `seaborn`, and `plotnine` modules of python.

9.7. matplotlib

```
import matplotlib.pyplot as plt

comparison.plot(kind="bar", figsize=(7, 4), color=["#1c7c73",
 ↪  "#c9482f"])
plt.title("Observed vs. Benford-Expected First-Digit Distribution")
plt.xlabel("First Digit")
plt.ylabel("Proportion")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



9.8. seaborn

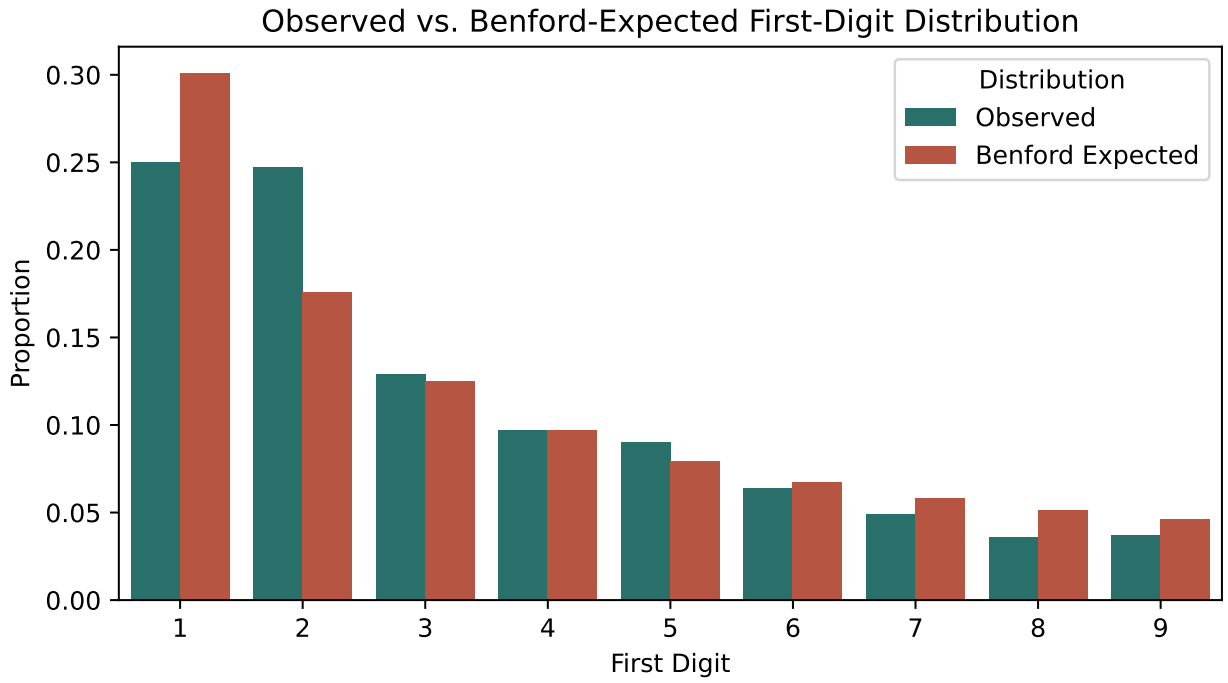
```
import seaborn as sns

# Reshape to long format for Seaborn
comparison_long = comparison_pl.unpivot(
    index="first_digit",
    on=["Observed", "Benford Expected"],
    variable_name="Distribution",
    value_name="Proportion"
)

plt.figure(figsize=(7, 4))

sns.barplot(
    data=comparison_long,
    x="first_digit",
    y="Proportion",
    hue="Distribution",
    palette=["#1c7c73", "#c9482f"]
)
```

```
plt.title("Observed vs. Benford-Expected First-Digit Distribution")
plt.xlabel("First Digit")
plt.ylabel("Proportion")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



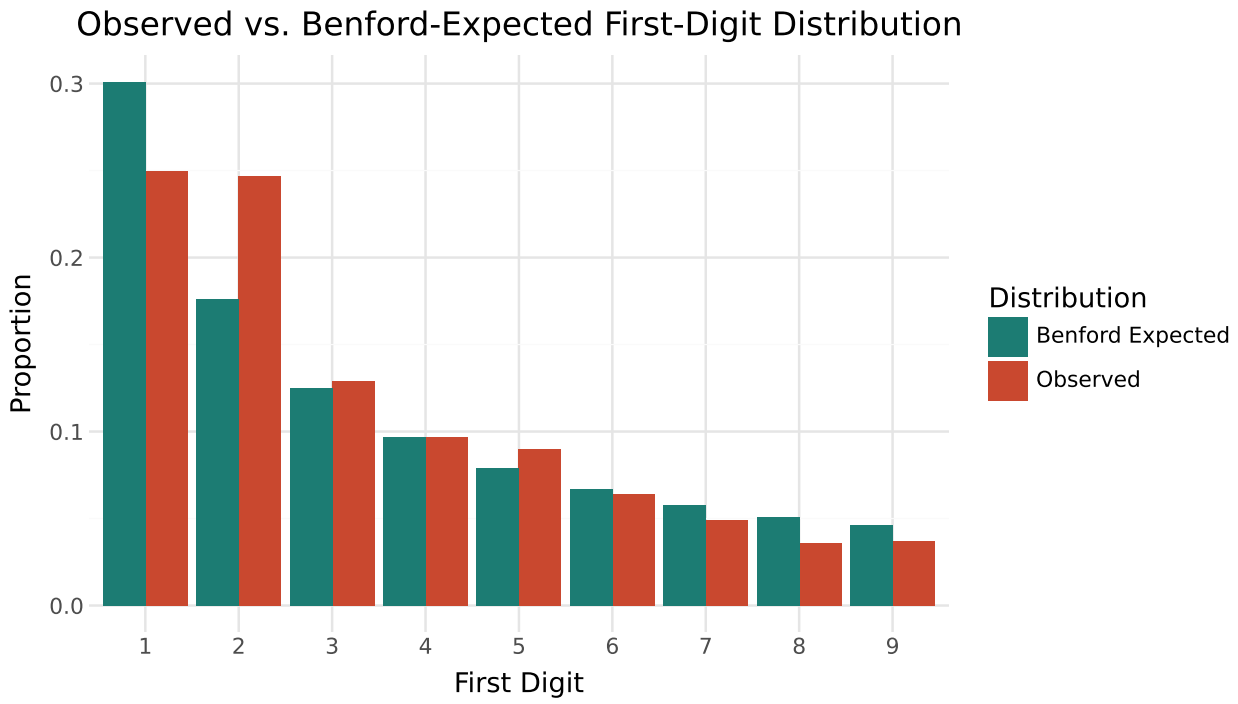
9.9. plotnine

```
import polars as pl
from plotnine import (
    ggplot,
    aes,
    geom_col,
    labs,
    scale_fill_manual,
    theme_minimal,
    theme,
    element_text
```

```
)

# Reshape to long format (done in Polars)
comparison_long = (
    comparison_pl
    .unpivot(
        index="first_digit",
        on=["Observed", "Benford Expected"],
        variable_name="Distribution",
        value_name="Proportion"
    )
)

# Create the plot
(
    ggplot(
        comparison_long.to_pandas(),
        aes(
            x="factor(first_digit)",
            y="Proportion",
            fill="Distribution"
        )
    )
    + geom_col(position="dodge")
    + scale_fill_manual(values=["#1c7c73", "#c9482f"])
    + labs(
        title="Observed vs. Benford-Expected First-Digit
 ↪  Distribution",
        x="First Digit",
        y="Proportion"
    )
    + theme_minimal()
    + theme(
        figure_size=(7, 4),
        axis_text_x=element_text(rotation=0)
    )
)
```



9.9.1. Testing Statistical Significance

A visual comparison suggests some deviation, particularly around digits 1 and 2. A chi-square goodness-of-fit test quantifies whether this deviation is larger than we'd expect from random chance alone.

9.10. pandas

```
from scipy.stats import chisquare

observed_counts =
 ↪  first_digits.value_counts().sort_index().reindex(range(1, 10),
 ↪  fill_value=0)
expected_counts = benford_expected * len(first_digits)

chi2, p_value = chisquare(observed_counts, expected_counts)
print(f"Chi-square statistic: {chi2:.2f}")
print(f"P-value: {p_value:.6f}")
```

```
Chi-square statistic: 35.21
```

```
P-value: 0.000025
```

9.11. polars

```
from scipy.stats import chisquare
import polars as pl
import math

# Observed counts
observed_counts_pl = (
    first_digits_pl
    .group_by("first_digit")
    .len()
    .rename({"len": "Observed"})
    .join(
        pl.DataFrame({"first_digit": range(1, 10)}),
        on="first_digit",
        how="right",
    )
    .fill_null(0)
    .sort("first_digit")
)

# Total number of observations
n = observed_counts_pl["Observed"].sum()


# Expected counts
expected_counts_pl = pl.Series(
    "Expected",
    [math.log10(1 + 1 / d) * n for d in range(1, 10)]
)

# Chi-square test
chi2_pl, p_value_pl = chisquare(
    observed_counts_pl["Observed"].to_numpy(),
    expected_counts_pl.to_numpy()
)

print(f"Chi-square statistic: {chi2_pl:.2f}")
```

```
print(f"P-value: {p_value_pl:.6f}")
```

```
Chi-square statistic: 35.21
P-value: 0.000025
```

The p-value is well below 0.05, indicating the deviation from Benford's Law is statistically significant — this dataset's first-digit distribution does **not** match Benford's expected pattern.

> **!** Does This Mean Fraud? No — and This Is the Real Lesson
>
> It's tempting to treat a failed Benford's Law test as evidence of manipulation. It isn't, at least not on its own, and this dataset is a good illustration of why.
> Benford's Law works best on data that spans **several orders of magnitude** and arises from many unrelated multiplicative processes — think city populations, or river lengths, or a company's transactions spanning $1 to $1,000,000. Our data does span a wide dollar range ($197 to $176,500), but it's built from a fairly small number of **transaction-type templates**, each with its own fairly narrow, specific typical size (e.g., rent is always close to $2,700; payroll is always close to $5,000). This kind of structure — amounts clustering around a handful of "typical" values per transaction type — will deviate from Benford's Law even with no manipulation at all, simply because the data doesn't have the scale-invariant, multiplicative structure Benford's Law assumes.
> **The real lesson:** a Benford's Law deviation is a prompt to ask *why* the data deviates, not an automatic red flag. In practice, an auditor would want to understand the client's transaction mix before concluding anything from this test — and might apply Benford's Law separately to a single account or transaction type with more naturally varied amounts (e.g., across all miscellaneous vendor invoices) rather than a full general ledger mixing many structurally different transaction types together.

9.12. Journal Entry Testing

Beyond Benford's Law, auditors commonly run a set of standard rule-based tests directly on general ledger data. We'll apply five of the most common tests to `audit_test_transactions.csv`.

9.13. pandas

```
audit_gl["day_of_week"] = audit_gl["entry_date"].dt.day_name()
audit_gl["amount"] = audit_gl[["debit", "credit"]].max(axis=1)
audit_gl.head()
```

	entry_id	entry_date	account	debit	credit	department	posted_by	day_of_we
0	JE5137	2025-01-01	Cash	1754.15	0.00	Sales	dwilson	Wednesda
1	JE5137	2025-01-01	Accounts Receivable	0.00	1754.15	Sales	dwilson	Wednesda
2	JE5106	2025-01-02	Inventory	2648.94	0.00	Operations	agarcia	Thursday
3	JE5106	2025-01-02	Accounts Payable	0.00	2648.94	Operations	agarcia	Thursday
4	JE5132	2025-01-06	Cash	2680.60	0.00	Sales	jsmith	Monday

9.14. polars

```
audit_gl_pl = (
    audit_gl_pl
    .with_columns(
        pl.col("entry_date").dt.strftime("%A").alias("day_of_week"),
        pl.max_horizontal("debit", "credit").alias("amount"),
    )
)

audit_gl_pl.head()
```

entry_id str	entry_date date	account str	debit f64	credit f64	department str	posted_by str	day_of_we str
”JE5137”	2025-01-01	”Cash”	1754.15	0.0	”Sales”	”dwilson”	”Wednesd
”JE5137”	2025-01-01	”Accounts Receivable”	0.0	1754.15	”Sales”	”dwilson”	”Wednesd
”JE5106”	2025-01-02	”Inventory”	2648.94	0.0	”Operations”	”agarcia”	”Thursday
”JE5106”	2025-01-02	”Accounts Payable”	0.0	2648.94	”Operations”	”agarcia”	”Thursday
”JE5132”	2025-01-06	”Cash”	2680.6	0.0	”Sales”	”jsmith”	”Monday”

9.14.1. Test 1: Weekend Postings

Legitimate business activity is usually concentrated on weekdays. Entries posted on a weekend are worth a second look, especially in departments that don’t normally operate then.

9.15. pandas

```
audit_gl["is_weekend"] = audit_gl["entry_date"].dt.weekday >= 5
audit_gl[audit_gl["is_weekend"]][["entry_id", "entry_date",
↪  "day_of_week", "account", "amount",
↪  "posted_by"]].drop_duplicates("entry_id")
```

	entry_id	entry_date	day_of_week	account	amount	posted_by
26	JE5179	2025-02-22	Saturday	Utilities Expense	503.11	rpatel
100	JE5900	2025-06-14	Saturday	Accounts Receivable	50000.00	jsmith
174	JE5905	2025-09-21	Sunday	Salaries Expense	41000.00	ceverett
184	JE5108	2025-10-19	Sunday	Cost of Goods Sold	1098.04	dwilson
200	JE5105	2025-11-15	Saturday	Cash	1851.67	rpatel
226	JE5906	2025-12-28	Sunday	Accounts Receivable	75000.00	ceverett

9.16. polars

```
audit_gl_pl = audit_gl_pl.with_columns(
    (pl.col("entry_date").dt.weekday() > 5).alias("is_weekend")
)

weekend_entries = (
    audit_gl_pl
    .filter(pl.col("is_weekend"))
    .select(
        "entry_id",
        "entry_date",
        "day_of_week",
        "account",
        "amount",
        "posted_by",
    )
    .unique(subset="entry_id")
)

weekend_entries
```

entry_id str	entry_date date	day_of_week str	account str	amount f64	posted_by str
"JE5108"	2025-10-19	"Sunday"	"Cost of Goods Sold"	1098.04	"dwilson"
"JE5105"	2025-11-15	"Saturday"	"Cash"	1851.67	"rpatel"
"JE5900"	2025-06-14	"Saturday"	"Accounts Receivable"	50000.0	"jsmith"
"JE5906"	2025-12-28	"Sunday"	"Accounts Receivable"	75000.0	"ceverett"
"JE5179"	2025-02-22	"Saturday"	"Utilities Expense"	503.11	"rpatel"
"JE5905"	2025-09-21	"Sunday"	"Salaries Expense"	41000.0	"ceverett"

9.16.1. Test 2: Round-Dollar Amounts

Genuine transactions (a specific invoice, a calculated tax amount) rarely land on a perfectly round number. A cluster of exact round-dollar entries can indicate estimated, fabricated, or manually adjusted amounts.

9.17. pandas

```
audit_gl["is_round_dollar"] = (audit_gl["amount"] % 500 == 0) &
 ↪  (audit_gl["amount"] > 0)
audit_gl[audit_gl["is_round_dollar"]][["entry_id", "entry_date",
 ↪  "account", "amount"]].drop_duplicates("entry_id")
```

	entry_id	entry_date	account	amount
60	JE5901	2025-04-08	Accounts Payable	10000.0
100	JE5900	2025-06-14	Accounts Receivable	50000.0
152	JE5903	2025-08-19	Rent Expense	3000.0
154	JE5904	2025-08-19	Rent Expense	3000.0
174	JE5905	2025-09-21	Salaries Expense	41000.0
226	JE5906	2025-12-28	Accounts Receivable	75000.0
232	JE5902	2025-12-30	Accounts Receivable	92500.0

9.18. polars

```
audit_gl_pl = audit_gl_pl.with_columns(
    (
        (pl.col("amount") % 500 == 0)
        & (pl.col("amount") > 0)
    ).alias("is_round_dollar")
)

round_dollar_entries = (
    audit_gl_pl
    .filter(pl.col("is_round_dollar"))
    .select(
        "entry_id",
        "entry_date",
        "account",
        "amount",
    )
    .unique(subset="entry_id")
)

round_dollar_entries
```

entry_id str	entry_date date	account str	amount f64
"JE5906"	2025-12-28	"Accounts Receivable"	75000.0
"JE5905"	2025-09-21	"Salaries Expense"	41000.0
"JE5900"	2025-06-14	"Accounts Receivable"	50000.0
"JE5903"	2025-08-19	"Rent Expense"	3000.0
"JE5901"	2025-04-08	"Accounts Payable"	10000.0
"JE5904"	2025-08-19	"Rent Expense"	3000.0
"JE5902"	2025-12-30	"Accounts Receivable"	92500.0

9.18.1. Test 3: Period-End Cutoff Timing

Transactions posted in the last few days of a fiscal year deserve extra scrutiny, since this is a common window for manipulating which period revenue or expenses land in (a "cutoff" issue).

9.19. pandas

```
year_end_cutoff = pd.Timestamp("2025-12-27")
audit_gl["is_year_end"] = audit_gl["entry_date"] >= year_end_cutoff
audit_gl[audit_gl["is_year_end"]][["entry_id", "entry_date",
 ↪  "account", "amount"]].drop_duplicates("entry_id")
```

	entry_id	entry_date	account	amount
226	JE5906	2025-12-28	Accounts Receivable	75000.00
228	JE5119	2025-12-29	Rent Expense	2828.43
230	JE5116	2025-12-30	Rent Expense	2860.71
232	JE5902	2025-12-30	Accounts Receivable	92500.00

9.20. polars

```
from datetime import datetime
import polars as pl

year_end_cutoff = datetime(2025, 12, 27)

audit_gl_pl = audit_gl_pl.with_columns(
    (pl.col("entry_date") >= year_end_cutoff).alias("is_year_end")
)

year_end_entries = (
    audit_gl_pl
    .filter(pl.col("is_year_end"))
    .select(
        "entry_id",
        "entry_date",
        "account",
        "amount",
    )
    .unique(subset="entry_id")
)

year_end_entries
```

entry_id str	entry_date date	account str	amount f64
"JE5902"	2025-12-30	"Accounts Receivable"	92500.0
"JE5906"	2025-12-28	"Accounts Receivable"	75000.0
"JE5116"	2025-12-30	"Rent Expense"	2860.71
"JE5119"	2025-12-29	"Rent Expense"	2828.43

9.20.1. Test 4: Duplicate Entries

Two entries with an identical date, account, and amount may represent an accidental double-posting — or, less innocently, a deliberate duplication.

9.21. pandas

```
dupe_mask = audit_gl.duplicated(subset=["entry_date", "account",
 ↪  "debit", "credit"], keep=False)
audit_gl[dupe_mask][["entry_id", "entry_date", "account", "debit",
 ↪  "credit", "posted_by"]]
```

	entry_id	entry_date	account	debit	credit	posted_by
152	JE5903	2025-08-19	Rent Expense	3000.0	0.0	mchen
153	JE5903	2025-08-19	Cash	0.0	3000.0	mchen
154	JE5904	2025-08-19	Rent Expense	3000.0	0.0	mchen
155	JE5904	2025-08-19	Cash	0.0	3000.0	mchen

9.22. polars

```
duplicate_entries = (
    audit_gl_pl
    .with_columns(
        pl.len().over(
            ["entry_date", "account", "debit", "credit"]
        ).alias("duplicate_count")
    )
```

```
    .filter(pl.col("duplicate_count") > 1)
    .select(
        "entry_id",
        "entry_date",
        "account",
        "debit",
        "credit",
        "posted_by",
    )
)

duplicate_entries
```

entry_id str	entry_date date	account str	debit f64	credit f64	posted_by str
”JE5903”	2025-08-19	”Rent Expense”	3000.0	0.0	”mchen”
”JE5903”	2025-08-19	”Cash”	0.0	3000.0	”mchen”
”JE5904”	2025-08-19	”Rent Expense”	3000.0	0.0	”mchen”
”JE5904”	2025-08-19	”Cash”	0.0	3000.0	”mchen”

9.22.1. Test 5: Unusual Posting Activity

An employee who rarely posts journal entries suddenly posting one — especially a large one — is worth investigating, since it may indicate access that should have been restricted, or an entry made outside of normal responsibilities.

9.23. pandas

```
poster_counts =
 ↪  audit_gl.groupby("posted_by")["entry_id"].nunique().sort_values()
poster_counts
```

```
posted_by
ceverett     2
ktaylor     15
rpatel      17
agarcia     19
```

```
dwilson     19
jsmith      21
mchen       24
Name: entry_id, dtype: int64
```

9.24. polars

```
poster_counts_pl = (
    audit_gl_pl
    .group_by("posted_by")
    .agg(
        pl.col("entry_id").n_unique().alias("entry_count")
    )
    .sort("entry_count")
)

poster_counts_pl
```

posted_by	entry_count
str	u32
"ceverett"	2
"ktaylor"	15
"rpatel"	17
"agarcia"	19
"dwilson"	19
"jsmith"	21
"mchen"	24

9.25. pandas

```
rare_threshold = poster_counts.quantile(0.1)
rare_posters = poster_counts[poster_counts <=
 ↪  rare_threshold].index.tolist()
print("Posters flagged as unusually infrequent:", rare_posters)

audit_gl[audit_gl["posted_by"].isin(rare_posters)][["entry_id",
 ↪  "entry_date", "account", "amount",
 ↪  "posted_by"]].drop_duplicates("entry_id")
```

```
Posters flagged as unusually infrequent: ['ceverett']
```

	entry_id	entry_date	account	amount	posted_by
174	JE5905	2025-09-21	Salaries Expense	41000.0	ceverett
226	JE5906	2025-12-28	Accounts Receivable	75000.0	ceverett

9.26. polars

```
# Compute the 10th percentile of entry counts
rare_threshold_pl = (
    poster_counts_pl
    .select(pl.col("entry_count").quantile(0.1,
 ↪  interpolation="linear"))
    .item()
)

# Get the list of rare posters
rare_posters_pl = (
    poster_counts_pl
    .filter(pl.col("entry_count") <= rare_threshold_pl)
    .get_column("posted_by")
    .to_list()
)

print("Posters flagged as unusually infrequent:", rare_posters_pl)

# Retrieve their journal entries
rare_entries = (
    audit_gl_pl
    .filter(pl.col("posted_by").is_in(rare_posters_pl))
    .select(
        "entry_id",
        "entry_date",
        "account",
        "amount",
        "posted_by",
    )
```

```
    .unique(subset="entry_id")
)

rare_entries
```

```
Posters flagged as unusually infrequent: ['ceverett']
```

entry_id	entry_date	account	amount	posted_by
str	date	str	f64	str
”JE5905”	2025-09-21	”Salaries Expense”	41000.0	”ceverett”
”JE5906”	2025-12-28	”Accounts Receivable”	75000.0	”ceverett”

Connect to Practice

Notice that several of these tests flagged *different* transactions from each other — the weekend test and the round-dollar test overlap on some entries but not others. This is exactly why real audit programs run multiple tests rather than relying on just one: each test is designed to catch a different kind of anomaly, and combining them (next section) produces a much more informative signal than any single test alone.

9.27. Combining Red Flags into a Composite Risk Score

Rather than reviewing five separate flagged lists, it's more useful to combine them into a single **risk score** per journal entry — the more red flags an entry trips, the higher its priority for manual review.

9.28. pandas

```
# Roll each transaction line up to one row per journal entry
entry_level = audit_gl.groupby("entry_id").agg(
    entry_date=("entry_date", "first"),
    account=("account", lambda x: " / ".join(x)),
    amount=("amount", "max"),
    department=("department", "first"),
    posted_by=("posted_by", "first"),
).reset_index()
```

```
entry_level["is_weekend"] = entry_level["entry_date"].dt.weekday >= 5
entry_level["is_round_dollar"] = (entry_level["amount"] % 500 == 0) &
 ↪  (entry_level["amount"] > 0)
entry_level["is_year_end"] = entry_level["entry_date"] >=
 ↪  pd.Timestamp("2025-12-27")
entry_level["is_rare_poster"] =
 ↪  entry_level["posted_by"].map(poster_counts) <= rare_threshold

dupe_entry_ids = audit_gl[dupe_mask]["entry_id"].unique()
entry_level["is_duplicate"] =
 ↪  entry_level["entry_id"].isin(dupe_entry_ids)

mean_amt, std_amt = entry_level["amount"].mean(),
 ↪  entry_level["amount"].std()
entry_level["z_score"] = (entry_level["amount"] - mean_amt) / std_amt
entry_level["is_large_amount"] = entry_level["z_score"].abs() > 3

flag_columns = ["is_weekend", "is_round_dollar", "is_year_end",
 ↪  "is_rare_poster", "is_duplicate", "is_large_amount"]
entry_level["risk_score"] = entry_level[flag_columns].sum(axis=1)

ranked = entry_level.sort_values("risk_score", ascending=False)
ranked[["entry_id", "entry_date", "account", "amount", "posted_by",
 ↪  "risk_score"] + flag_columns].head(10)
```

	entry_id	entry_date	account	amount	posted_by	risk_score	is_
116	JE5906	2025-12-28	Accounts Receivable / Sales Revenue	75000.00	ceverett	5	Tru
115	JE5905	2025-09-21	Salaries Expense / Cash	41000.00	ceverett	4	Tru
112	JE5902	2025-12-30	Accounts Receivable / Sales Revenue	92500.00	rpatel	3	Fal
110	JE5900	2025-06-14	Accounts Receivable / Sales Revenue	50000.00	jsmith	3	Tru
113	JE5903	2025-08-19	Rent Expense / Cash	3000.00	mchen	2	Fal
114	JE5904	2025-08-19	Rent Expense / Cash	3000.00	mchen	2	Fal
79	JE5179	2025-02-22	Utilities Expense / Accounts Payable	503.11	rpatel	1	Tru
111	JE5901	2025-04-08	Accounts Payable / Cash	10000.00	mchen	1	Fal
5	JE5105	2025-11-15	Cash / Accounts Receivable	1851.67	rpatel	1	Tru
16	JE5116	2025-12-30	Rent Expense / Cash	2860.71	dwilson	1	Fal

9.29. polars

```
import polars as pl
from datetime import datetime

# Roll each transaction line up to one row per journal entry
entry_level_pl = (
    audit_gl_pl
    .group_by("entry_id")
    .agg(
        pl.col("entry_date").first().alias("entry_date"),
        pl.col("account").str.join(" / ").alias("account"),
        pl.col("amount").max().alias("amount"),
        pl.col("department").first().alias("department"),
        pl.col("posted_by").first().alias("posted_by"),
    )
)

# Duplicate entry IDs
dupe_entry_ids = (
    audit_gl_pl
    .with_columns(
        pl.len().over(
            ["entry_date", "account", "debit", "credit"]
        ).alias("duplicate_count")
    )
    .filter(pl.col("duplicate_count") > 1)  # from your earlier
 ↪  duplicate calculation
    .select("entry_id")
    .unique()
)

# Add risk indicators
entry_level_pl = (
    entry_level_pl
    .with_columns(
        # Weekend flag (Polars weekday: Sat=6, Sun=7)
        (pl.col("entry_date").dt.weekday() >= 6)
        .alias("is_weekend"),

        # Round dollar flag
```

```
        (
            (pl.col("amount") % 500 == 0)
            & (pl.col("amount") > 0)
        ).alias("is_round_dollar"),

        # Year end flag
        (
            pl.col("entry_date") >= datetime(2025, 12, 27)
        ).alias("is_year_end"),
    )
    .join(
        poster_counts_pl,
        on="posted_by",
        how="left",
    )
    .with_columns(
        (
            pl.col("entry_count") <= rare_threshold
        ).alias("is_rare_poster")
    )
    .drop("entry_count")
    .join(
        dupe_entry_ids.with_columns(
            pl.lit(True).alias("is_duplicate")
        ),
        on="entry_id",
        how="left",
    )
    .with_columns(
        pl.col("is_duplicate")
        .fill_null(False)
    )
)


# Calculate z-score
mean_amt = entry_level_pl.select(
    pl.col("amount").mean()
).item()

std_amt = entry_level_pl.select(
```

```
    pl.col("amount").std()
).item()


entry_level_pl = (
    entry_level_pl
    .with_columns(
        (
            (pl.col("amount") - mean_amt)
            / std_amt
        ).alias("z_score")
    )
    .with_columns(
        (
            pl.col("z_score").abs() > 3
        ).alias("is_large_amount")
    )
)


# Calculate risk score
flag_columns = [
    "is_weekend",
    "is_round_dollar",
    "is_year_end",
    "is_rare_poster",
    "is_duplicate",
    "is_large_amount",
]


ranked_pl = (
    entry_level_pl
    .with_columns(
        pl.sum_horizontal(
            flag_columns
        ).alias("risk_score")
    )
    .sort(
        "risk_score",
        descending=True
```

```
    )
)


# Top 10 risky journal entries
top10 = (
    ranked_pl
    .select(
        [
            "entry_id",
            "entry_date",
            "account",
            "amount",
            "posted_by",
            "risk_score",
        ]
        + flag_columns
    )
    .head(10)
)

top10
```

entry_id str	entry_date date	account str	amount f64	posted_by str	risk_score u32	is_weekend bool
"JE5906"	2025-12-28	"Accounts Receivable / Sales Re…	75000.0	"ceverett"	5	true
"JE5905"	2025-09-21	"Salaries Expense / Cash"	41000.0	"ceverett"	4	true
"JE5902"	2025-12-30	"Accounts Receivable / Sales Re…	92500.0	"rpatel"	3	false
"JE5900"	2025-06-14	"Accounts Receivable / Sales Re…	50000.0	"jsmith"	3	true
"JE5903"	2025-08-19	"Rent Expense / Cash"	3000.0	"mchen"	2	false
"JE5904"	2025-08-19	"Rent Expense / Cash"	3000.0	"mchen"	2	false
"JE5119"	2025-12-29	"Rent Expense / Cash"	2828.43	"rpatel"	1	false
"JE5179"	2025-02-22	"Utilities Expense / Accounts P…	503.11	"rpatel"	1	true
"JE5116"	2025-12-30	"Rent Expense / Cash"	2860.71	"dwilson"	1	false
"JE5901"	2025-04-08	"Accounts Payable / Cash"	10000.0	"mchen"	1	false

The entries at the top of this ranking are exactly the ones deliberately constructed with multiple overlapping red flags — a weekend, near-year-end, round-dollar entry posted by a rarely active employee sits at the very top, while entries triggering only one incidental flag (say, a normal transaction that simply happened to land on a Saturday) rank much lower. This is precisely the kind

of prioritized list a risk-based audit program would use to decide where to spend limited review time first.

> **!** A Risk Score Prioritizes — It Doesn't Convict
>
> A high risk score means "review this first," not "this is fraudulent." Some entries flagged here may turn out to have a completely legitimate explanation (a real large weekend sale during a holiday promotion, for example). The value of a composite score is triage — making sure the entries most worth a human's attention actually get it, out of a population too large to review manually in full.

9.30. Sampling Techniques

Even with full-population analytics, auditors often still need to draw a **sample** for certain procedures — for example, when a test requires physically confirming a transaction with a third party. We'll cover four common sampling approaches using `general_ledger_large.csv`.

9.31. pandas

```
debit_gl = gl[gl["debit"] > 0].reset_index(drop=True)
```

9.32. polars

```
debit_gl_pl = gl_pl.filter(
    pl.col("debit") > 0
)
```

9.32.1. Simple Random Sampling

Every transaction has an equal chance of selection.

9.33. pandas

```
simple_sample = debit_gl.sample(n=30, random_state=42)
print("Sample size:", len(simple_sample))
print("Coverage of total population value:",
 ↪  round(simple_sample["debit"].sum() / debit_gl["debit"].sum() *
 ↪  100, 1), "%")
```

```
Sample size: 30
Coverage of total population value: 2.7 %
```

9.34. polars

```
simple_sample_pl = debit_gl_pl.sample(
    n=30,
    seed=42
)

print("Sample size:", simple_sample_pl.height)

print(
    "Coverage of total population value:",
    round(
        simple_sample_pl["debit"].sum()
        / debit_gl_pl["debit"].sum()
        * 100,
        1
    ),
    "%"
)
```

```
Sample size: 30
Coverage of total population value: 2.5 %
```

9.34.1. Systematic Sampling

Select every k-th transaction after sorting, which is easy to execute and audit-defensible, though it can be biased if the data has a hidden periodic pattern.

9.35. pandas

```
k = len(debit_gl) // 30
systematic_sample = debit_gl.iloc[::k].head(30)
print("Sample size:", len(systematic_sample))
```

```
Sample size: 30
```

9.36. polars

```
k_pl = debit_gl_pl.height // 30

systematic_sample_pl = (
    debit_gl_pl
    .with_row_index("row_nr")
    .filter(pl.col("row_nr") % k == 0)
    .head(30)
    .drop("row_nr")
)

print("Sample size:", systematic_sample_pl.height)
```

```
Sample size: 30
```

9.36.1. Stratified Sampling

Divide the population into meaningful groups (here, department) and sample proportionally from each, ensuring smaller departments aren't overlooked entirely by chance.

9.37. pandas

```
strata_sizes = debit_gl.groupby("department").size()
allocation = (strata_sizes / strata_sizes.sum() *
 ↪  30).round().astype(int)
allocation
```

```
department
Finance         2
HR              3
Operations     14
Sales          11
dtype: int64
```

9.38. polars

```
strata_sizes_pl = (
    debit_gl_pl
    .group_by("department")
    .agg(
        pl.len().alias("size")
    )
)

allocation_pl = (
    strata_sizes_pl
    .with_columns(
        (
            pl.col("size")
            / pl.col("size").sum()
            * 30
        )
        .round()
        .cast(pl.Int64)
        .alias("allocation")
    )
)

allocation_pl
```

department	size	allocation
str	u32	i64
”Operations”	350	14
”Finance”	50	2
”HR”	66	3
”Sales”	286	11

department	size	allocation
str	u32	i64

9.39. pandas

```
stratified_sample = (
    debit_gl.groupby("department", group_keys=False)
    .apply(lambda x: x.sample(n=min(len(x), allocation[x.name]),
 ↪  random_state=42))
)
print("Sample size:", len(stratified_sample))

stratified_sample
```

```
Sample size: 30
```

	entry_id	entry_date	account	debit	credit	posted_by
207	JE3006	2025-04-05	Insurance Expense	869.40	0.0	rpatel
536	JE3724	2025-09-22	Insurance Expense	782.39	0.0	ktaylor
643	JE3167	2025-11-17	Salaries Expense	9834.50	0.0	agarcia
723	JE3390	2025-12-23	Salaries Expense	7533.10	0.0	rpatel
32	JE3014	2025-01-17	Salaries Expense	5582.38	0.0	agarcia
321	JE3380	2025-06-04	Accounts Payable	1973.60	0.0	dwilson
730	JE3370	2025-12-24	Utilities Expense	475.52	0.0	dwilson
668	JE3121	2025-11-26	Inventory	1326.19	0.0	jsmith
498	JE3547	2025-08-28	Cost of Goods Sold	2598.49	0.0	jsmith
319	JE3465	2025-06-03	Inventory	2067.13	0.0	dwilson
592	JE3521	2025-10-22	Rent Expense	2729.07	0.0	jsmith
648	JE3072	2025-11-18	Cost of Goods Sold	3054.41	0.0	mchen
481	JE3334	2025-08-19	Rent Expense	2597.73	0.0	jsmith
597	JE3237	2025-10-27	Cost of Goods Sold	1955.03	0.0	mchen
371	JE3344	2025-06-25	Inventory	1771.81	0.0	dwilson
152	JE3532	2025-03-12	Accounts Payable	840.07	0.0	agarcia
251	JE3012	2025-05-01	Utilities Expense	362.10	0.0	agarcia
340	JE3685	2025-06-12	Inventory	2460.36	0.0	mchen
258	JE3244	2025-05-06	Cost of Goods Sold	674.78	0.0	dwilson
30	JE3516	2025-01-16	Accounts Receivable	2466.22	0.0	rpatel
711	JE3316	2025-12-15	Cash	1236.81	0.0	jsmith

	entry_id	entry_date	account	debit	credit	posted_by
391	JE3095	2025-07-08	Cash	1279.24	0.0	rpatel
552	JE3341	2025-09-30	Accounts Receivable	6278.19	0.0	agarcia
598	JE3424	2025-10-27	Accounts Receivable	4648.77	0.0	mchen
419	JE3332	2025-07-18	Accounts Receivable	984.15	0.0	agarcia
748	JE3733	2025-12-30	Accounts Receivable	4650.07	0.0	dwilson
194	JE3207	2025-04-01	Accounts Receivable	14194.17	0.0	jsmith
511	JE3089	2025-09-04	Accounts Receivable	5065.06	0.0	agarcia
93	JE3258	2025-02-17	Accounts Receivable	1862.37	0.0	mchen
477	JE3074	2025-08-15	Cash	2356.13	0.0	mchen

9.40. polars

```
stratified_sample_pl = (
    debit_gl_pl
    # Add the allocated sample size for each department
    .join(
        allocation_pl,
        on="department",
        how="left"
    )
    # Randomize rows within each department
    .with_columns(
        pl.int_range(0, pl.len())
        .shuffle()
        .over("department")
        .alias("random_order")
    )
    # Keep only the allocated number from each department
    .filter(
        pl.col("random_order") < pl.col("allocation")
    )
    .drop(
        ["size", "allocation", "random_order"]
    )
)

print("Sample size:", stratified_sample_pl.height)
stratified_sample_pl
```

```
Sample size: 30
```

entry_id str	entry_date str	account str	debit f64	credit f64	department str	posted_by str
"JE3050"	"2025-01-10"	"Accounts Receivable"	2822.34	0.0	"Sales"	"dwilson"
"JE3649"	"2025-01-22"	"Accounts Payable"	1277.11	0.0	"Operations"	"agarcia"
"JE3728"	"2025-02-09"	"Accounts Receivable"	3278.1	0.0	"Sales"	"dwilson"
"JE3437"	"2025-02-13"	"Accounts Receivable"	9192.06	0.0	"Sales"	"ktaylor"
"JE3019"	"2025-02-20"	"Salaries Expense"	3773.34	0.0	"HR"	"agarcia"
…	…	…	…	…	…	…
"JE3671"	"2025-10-17"	"Accounts Receivable"	5775.5	0.0	"Sales"	"ktaylor"
"JE3399"	"2025-10-28"	"Cost of Goods Sold"	1313.05	0.0	"Operations"	"rpatel"
"JE3109"	"2025-12-01"	"Accounts Receivable"	3549.11	0.0	"Sales"	"mchen"
"JE3714"	"2025-12-11"	"Utilities Expense"	693.3	0.0	"Operations"	"rpatel"
"JE3128"	"2025-12-24"	"Accounts Receivable"	4923.77	0.0	"Sales"	"agarcia"

9.40.1. Monetary Unit Sampling (MUS)

MUS selects transactions with probability proportional to their dollar value, so a $50,000 transaction is far more likely to be selected than a $500 one. This concentrates audit effort on the transactions most likely to contain a material misstatement in dollar terms.

A key feature of proper MUS is handling □**key items**□: any single transaction larger than the sampling interval (population value ÷ sample size) is automatically tested with certainty, since a random draw could otherwise miss it entirely despite its size.

9.41. pandas

```
sample_size = 30
total_value = debit_gl["debit"].sum()
sampling_interval = total_value / sample_size
print("Sampling interval: $", round(sampling_interval, 2))

key_items = debit_gl[debit_gl["debit"] >= sampling_interval]
key_items[["entry_id", "account", "debit"]]
```

```
Sampling interval: $ 101371.23
```

	entry_id	account	debit
161	JE3900	Accounts Receivable	145000.0
334	JE3902	Cost of Goods Sold	112000.0
501	JE3903	Accounts Receivable	176500.0
733	JE3906	Cash	121000.0

9.42. polars

```
sample_size_pl = 30

total_value_pl = debit_gl_pl["debit"].sum()

sampling_interval_pl = total_value_pl / sample_size_pl

print("Sampling interval: $", round(sampling_interval_pl, 2))

key_items_pl = (
    debit_gl_pl
    .filter(
        pl.col("debit") >= sampling_interval_pl
    )
    .select(
        "entry_id",
        "account",
        "debit"
    )
)

key_items_pl
```

```
Sampling interval: $ 101371.23
```

entry_id	account	debit
str	str	f64
"JE3900"	"Accounts Receivable"	145000.0
"JE3902"	"Cost of Goods Sold"	112000.0

entry_id	account	debit
str	str	f64
"JE3903"	"Accounts Receivable"	176500.0
"JE3906"	"Cash"	121000.0

These four transactions are exactly the largest of the seven outliers we identified back in Chapter 5 — each one exceeds the sampling interval on its own, so proper MUS pulls every one of them into the sample automatically, rather than leaving their inclusion to chance.

9.43. pandas

```
remainder = debit_gl[debit_gl["debit"] <
 ↪  sampling_interval].reset_index(drop=True)
remaining_sample_size = sample_size - len(key_items)
remaining_interval = remainder["debit"].sum() / remaining_sample_size

rng = np.random.default_rng(42)
random_start = rng.uniform(0, remaining_interval)
cum_values = remainder["debit"].cumsum()

selection_points = random_start + remaining_interval *
 ↪  np.arange(remaining_sample_size)
selected_idx = np.searchsorted(cum_values.values, selection_points)
selected_idx = np.clip(selected_idx, 0, len(remainder) - 1)

mus_remainder_sample = remainder.iloc[selected_idx]
mus_sample = pd.concat([key_items,
 ↪  mus_remainder_sample]).drop_duplicates(subset="entry_id")

print("Total MUS sample size:", len(mus_sample))
print("Coverage of total population value:",
 ↪  round(mus_sample["debit"].sum() / total_value * 100, 1), "%")
```

```
Total MUS sample size: 30
Coverage of total population value: 28.0 %
```

9.44. polars

```
import polars as pl
import numpy as np

# Remaining population below sampling interval
remainder_pl = (
    debit_gl_pl
    .filter(pl.col("debit") < sampling_interval_pl)
)

# Remaining sample size
remaining_sample_size_pl = sample_size_pl - key_items_pl.height

# Calculate remaining interval
remaining_interval_pl = (
    remainder_pl
    .select(pl.col("debit").sum())
    .item()
    / remaining_sample_size_pl
)

# Generate random starting point
rng = np.random.default_rng(42)
random_start_pl = rng.uniform(0, remaining_interval_pl)

# Add cumulative debit value
remainder_pl = (
    remainder_pl
    .with_columns(
        pl.col("debit")
        .cum_sum()
        .alias("cum_values")
    )
)

# Create selection points
selection_points = (
    random_start_pl
    + remaining_interval_pl * np.arange(remaining_sample_size_pl)
)
```

```
# Find matching rows
selected_indices_pl = np.searchsorted(
    remainder_pl["cum_values"].to_numpy(),
    selection_points
)

selected_indices_pl = np.clip(
    selected_indices_pl,
    0,
    remainder_pl.height - 1
)

# Select rows using indices
mus_remainder_sample_pl = (
    remainder_pl
    .with_row_index("row_nr")
    .filter(
        pl.col("row_nr").is_in(selected_indices_pl)
    )
    .drop("row_nr", "cum_values")
)


# Combine key items and MUS remainder sample
mus_sample_pl = (
    pl.concat(
        [
            key_items_pl,
            mus_remainder_sample_pl.select(
        "entry_id",
        "account",
        "debit"
    )
        ]
    )
    .unique(subset="entry_id")
)


print("Total MUS sample size:", mus_sample_pl.height)
```

```
coverage_pl = (
    mus_sample_pl["debit"].sum()
    / total_value_pl
    * 100
)

print(
    "Coverage of total population value:",
    round(coverage_pl, 1),
    "%"
)
```

```
Total MUS sample size: 30
Coverage of total population value: 28.0 %
```

9.44.1. Comparing Coverage Across Methods

9.45. pandas

```
comparison = pd.DataFrame({
    "Method": ["Simple Random", "Monetary Unit Sampling"],
    "Sample Size": [len(simple_sample), len(mus_sample)],
    "% of Population Value Covered": [
        round(simple_sample["debit"].sum() / total_value * 100, 1),
        round(mus_sample["debit"].sum() / total_value * 100, 1),
    ],
})
comparison
```

	Method	Sample Size	% of Population Value Covered
0	Simple Random	30	2.7
1	Monetary Unit Sampling	30	28.0

9.46. polars

```
comparison_pl = pl.DataFrame(
    {
        "Method": [
            "Simple Random",
            "Monetary Unit Sampling"
        ],
        "Sample Size": [
            simple_sample_pl.height,
            mus_sample_pl.height
        ],
        "% of Population Value Covered": [
            round(
                simple_sample_pl["debit"].sum()
                / total_value
                * 100,
                1
            ),
            round(
                mus_sample_pl["debit"].sum()
                / total_value
                * 100,
                1
            ),
        ],
    }
)

comparison_pl
```

Method str	Sample Size i64	% of Population Value Covered f64
”Simple Random”	30	2.5
”Monetary Unit Sampling”	30	28.0

With the same sample size (30 transactions), MUS covers roughly ten times more of the population’s total dollar value than simple random sampling — a direct illustration of why MUS is a standard technique for **substantive testing** of account balances, where the auditor’s primary

concern is whether the *dollar total* is materially misstated, not just whether any given transaction looks unusual.

> **Connect to Practice**
>
> Sampling methods and journal entry testing serve different purposes and are often used together in practice: full-population rule-based tests (like the red-flag scoring above) are well-suited to finding *specific unusual transactions* regardless of size, while sampling methods like MUS are better suited to testing whether an *account balance as a whole* is fairly stated. A real audit program typically uses both.

9.47. Chapter Summary

- Benford's Law compares the observed first-digit distribution of transaction amounts to a well-established theoretical pattern; a significant deviation is a prompt to investigate *why*, not proof of manipulation — data built from a small number of "typical amount" transaction types can fail this test with no wrongdoing involved.
- Standard journal entry tests — weekend postings, round-dollar amounts, period-end cutoff timing, duplicates, and rare posting activity — each catch a different kind of anomaly, and different tests will often flag different (if overlapping) transactions.
- Combining multiple red flags into a single risk score prioritizes limited review time toward the transactions most worth a human's attention, without claiming to prove anything on its own.
- Simple random, systematic, stratified, and monetary unit sampling each serve different purposes; MUS in particular concentrates coverage on dollar value by combining probability-proportional-to-size selection with automatic inclusion of any "key item" larger than the sampling interval.
- Every technique in this chapter produces a *starting point for investigation*, not a final conclusion — audit analytics narrows down where to look; it doesn't replace the judgment of determining what a flagged item actually means.

9.48. Discussion Questions

1. Why did the general ledger data in this chapter fail the Benford's Law test, and what would you want to know about a *real* company's data before concluding anything from a similar result?
2. Look at the top-ranked entries from the composite risk score. If you were the auditor, what specific follow-up question would you ask about the highest-scoring entry?

3. Why does Monetary Unit Sampling automatically include any transaction larger than the sampling interval, rather than leaving its inclusion to chance like the rest of the population?

9.49. Exercises

1. Apply Benford's Law separately to just the **Sales Revenue** transactions in `general_ledger_large.csv` (a single, more homogeneous account) rather than the whole general ledger. Does the fit improve compared to the whole-population test in this chapter? Why might that be?
2. Add a sixth journal entry test of your own design (for example: entries with a missing or unusually short description, or entries where the debit and credit amounts on the same entry don't match). Apply it to `audit_test_transactions.csv` and report what it flags.
3. Recompute the composite risk score using only four of the six flags (your choice of which two to drop). How much does the top-10 ranking change? What does this tell you about how sensitive a composite score is to the specific flags chosen?
4. Using `general_ledger_large.csv`, build a stratified sample allocated by **posted_by** instead of department, and compare its dollar-value coverage to the department-stratified sample built in this chapter.

10. Fraud and Anomaly Detection

Learning Objectives

By the end of this chapter, you should be able to:

- Engineer features from transaction data suitable for statistical and machine learning anomaly detection
- Apply K-means clustering to identify groups of unusual transactions, and explain how feature choice shapes clustering results
- Apply Isolation Forest to flag anomalous transactions, and explain the intuition behind how it works
- Evaluate unsupervised detection methods using precision and recall against known anomalies
- Explain why rule-based tests (Chapter 8) and model-based anomaly detection each catch patterns the other misses, and why real fraud detection programs use both

10.1. From Red Flags to Statistical Anomaly Detection

Chapter 8 built rule-based tests: weekend postings, round-dollar amounts, cutoff timing, duplicates, rare posters. Each rule looks for one specific, pre-defined pattern. This chapter introduces a different approach: **unsupervised anomaly detection**[1], where a **statistical** or **machine learning** method looks for transactions that are unusual across *several dimensions at once*, without being told in advance exactly what pattern to look for.

We'll use two datasets already familiar from earlier chapters:

- `general_ledger_large.csv` (Chapter 5) — 752 transactions, including the 7 **known outliers** we identified there using a z-score threshold, which we can now use as a benchmark to evaluate these new methods against
- `audit_test_transactions.csv` (Chapter 8) — 117 journal entries with deliberately embedded red-flag scenarios

[1] We call it unsupervised because *ex ante* we do not know whether it is a fraud, meaning that it is not labeled from the beginning.

10.2. Z Score – A Statistical Method for Anomaly Detection

A Z-score is a statistical measurement that tells you how many standard deviations a specific data point is from the average (mean). To find a Z-score, you subtract the dataset's average from your data point, and then divide that number by the standard deviation:

- Z-score of 0: The data point is exactly average.
- Positive Z-score: The data point is above average.
- Negative Z-score: The data point is below average

$$Z = \frac{x - \mu}{\sigma}$$

Where, x = Your data point, μ = The average of the dataset, and σ = The standard deviation.

Anomaly detection is the process of finding rare or unexpected events in data (like a sudden spike in website traffic or credit card fraud). Z-scores help find these events using a simple math rule. In normal data, about 99.7% of all data points fall between a Z-score of -3 and 3. This is often called the "3-sigma rule". Any data point that lands outside this -3 to 3 range (meaning its Z-score is greater than 3 or less than -3) is so rare that it is usually flagged as an anomaly or outlier. Figure 10.1 shows the distribution of normal data points using z scores. Z-score curve is also called normal distribution curve or bell shaped curve.

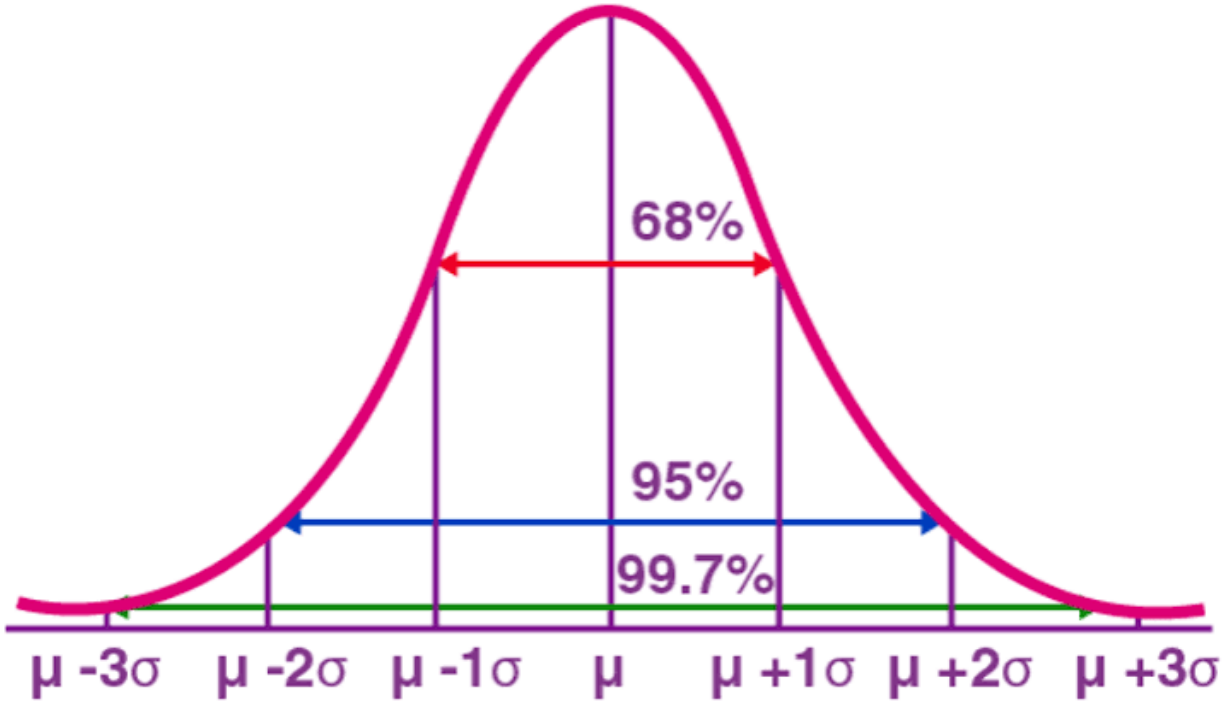


Figure 10.1.: Z Score and Distribution of Data Points

10.3. pandas

```
import pandas as pd
import numpy as np

gl = pd.read_csv("data/general_ledger_large.csv")
gl["entry_date"] = pd.to_datetime(gl["entry_date"])
debit = gl[gl["debit"] > 0].copy().reset_index(drop=True)

# Recall the 7 known outliers from Chapter 5 -- we'll treat these as a
# ground-truth benchmark for evaluating the new methods in this
 ↪  chapter
mean_debit, std_debit = debit["debit"].mean(), debit["debit"].std()
debit["z_score"] = (debit["debit"] - mean_debit) / std_debit
debit["known_outlier"] = debit["z_score"].abs() > 3

print("Known outliers:", debit["known_outlier"].sum())
```

```
Known outliers: 7
```

10.4. polars

```
import polars as pl

# Read the data
gl_pl = pl.read_csv(
    "data/general_ledger_large.csv",
    try_parse_dates=True
)

# Keep only debit transactions
debit_pl = (
    gl_pl
    .filter(pl.col("debit") > 0)
)

# Compute mean and standard deviation
mean_debit = debit_pl.select(
    pl.col("debit").mean()
```

```
).item()

std_debit = debit_pl.select(
    pl.col("debit").std()
).item()

# Compute z-score and identify known outliers
debit_pl = (
    debit_pl
    .with_columns(
        (
            (pl.col("debit") - mean_debit)
            / std_debit
        ).alias("z_score")
    )
    .with_columns(
        (
            pl.col("z_score").abs() > 3
        ).alias("known_outlier")
    )
)

print(
    "Known outliers:",
    debit_pl["known_outlier"].sum()
)
```

```
Known outliers: 7
```

10.5. Feature Engineering for Anomaly Detection

Feature Engineering is the process of selecting, creating or modifying features like input variables or data to help machine learning models learn patterns more effectively. It involves transforming raw data into meaningful inputs that improve model accuracy and performance. Figure 10.2 shows the architecture of feature engineering.

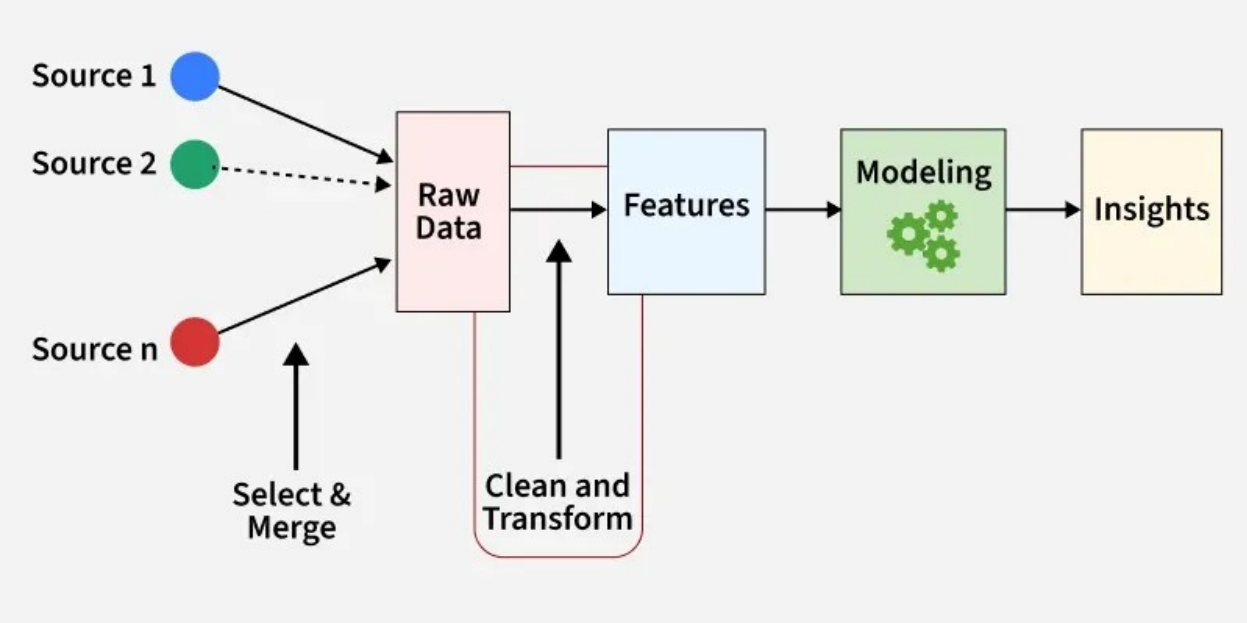


Figure 10.2.: Feature Engineering Architecture

Feature engineering process involves handling missing values, encoding categories, scaling numbers, creating new features or combining existing ones. It helps turn messy real-world data into a form that models can understand and use for better predictions. Figure 10.3 shows activities that are typically performed in feature engineering.

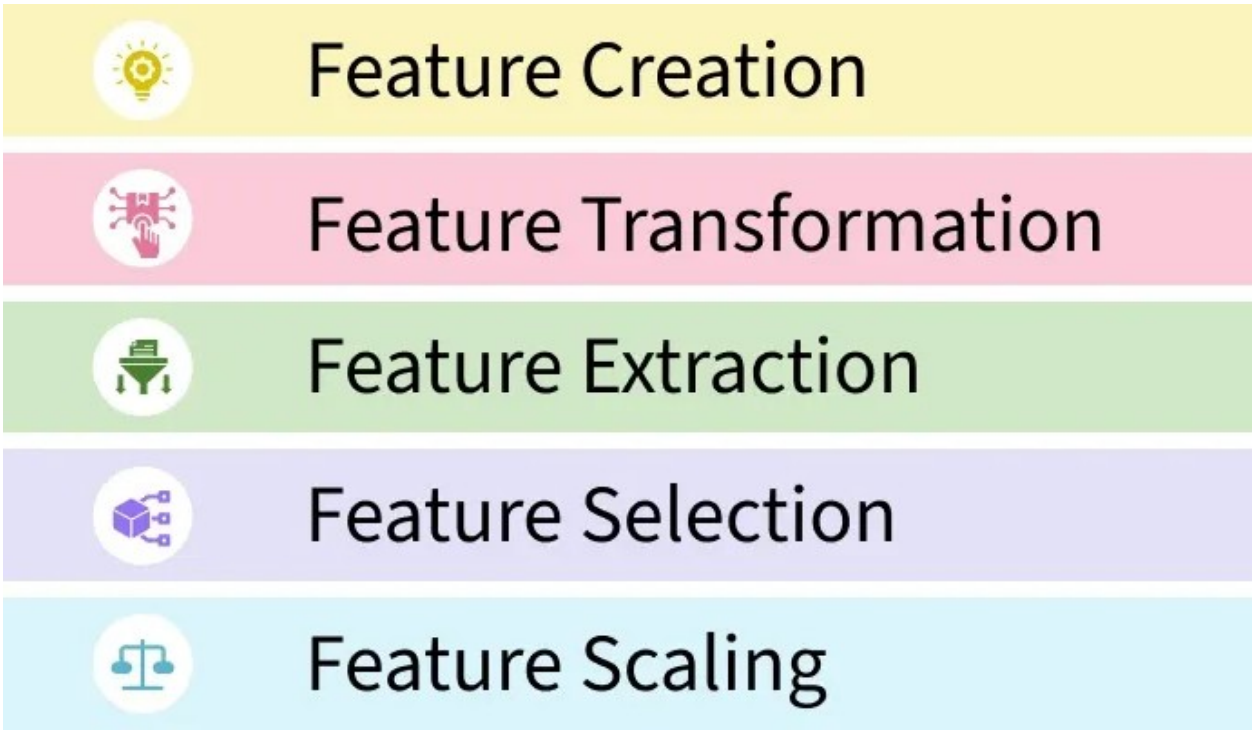


Figure 10.3.: Feature Engineering Process

- **Feature Creation**: Feature creation involves generating new features from domain knowledge or by observing patterns in the data.

- **Feature Transformation**: Feature transformation adjusts features to improve model learning, such as encoding categorical variables, logarithimic transformation of skewed data, or adjusting the range of feature for consistency.

- **Feature Extraction**: Feature extraction is transform existing features into a lower-dimensional or more informative representation. For example, Principal Component Analysis (PCA) is a feature extraction technique.

- **Feature Selection**: Feature selection involves choosing a subset of relevant features to use in the model. For feature selection, we can use filter methods - statistical measures like correlation; wrapper methods - select feature based on model performance; and embedded methods - feature selection embedded in the model.

- **Feature Scaling**: Feature scaling ensures all features contribute equally to the model. It involves min-max scaling or standard scaling.

Both methods in this chapter need transactions represented as **numeric features**, not raw dates and text. A few useful features for transaction-level fraud detection:

10.6. pandas

```
debit["log_amount"] = np.log1p(debit["debit"])
debit["is_weekend"] = (debit["entry_date"].dt.weekday >=
 ↪  5).astype(int)
debit["day_of_month"] = debit["entry_date"].dt.day

debit[["entry_id", "debit", "log_amount", "is_weekend",
 ↪  "day_of_month"]].head()
```

	entry_id	debit	log_amount	is_weekend	day_of_month
0	JE3253	1231.76	7.117011	0	1
1	JE3413	3024.76	8.014918	0	1
2	JE3476	4476.03	8.406715	0	1
3	JE3030	1488.48	7.306182	0	2
4	JE3066	2705.81	7.903526	0	2

10.7. polars

```
debit_pl = (
    debit_pl
    .with_columns(
        # Natural log of (1 + debit)
        pl.col("debit").log1p().alias("log_amount"),

        # Weekend indicator (Saturday=6, Sunday=7 in Polars)
        (pl.col("entry_date").dt.weekday() >= 6)
            .cast(pl.Int8)
            .alias("is_weekend"),

        # Day of the month
        pl.col("entry_date").dt.day().alias("day_of_month"),
    )
)

debit_pl.select(
    "entry_id",
```

```
    "debit",
    "log_amount",
    "is_weekend",
    "day_of_month",
).head()
```

entry_id str	debit f64	log_amount f64	is_weekend i8	day_of_month i8
"JE3253"	1231.76	7.117011	0	1
"JE3413"	3024.76	8.014918	0	1
"JE3476"	4476.03	8.406715	0	1
"JE3030"	1488.48	7.306182	0	2
"JE3066"	2705.81	7.903526	0	2

- **`log_amount`** compresses the wide range of transaction sizes (recall the heavy right skew from Chapter 5), which helps distance-based methods like clustering treat amount differences more sensibly.
- **`is_weekend`** encodes a categorical pattern (Chapter 8's weekend test) as a numeric feature.
- **`day_of_month`** can help surface period-end clustering patterns.

Which features you *choose* has a *major* effect on what these methods find — as the next section shows directly.

10.8. K-Means Clustering for Anomaly Detection

K-means clustering groups similar observations together based on distance in feature space. For anomaly detection, the idea is simple: unusual transactions often end up isolated in their own small cluster, separate from the bulk of "normal" activity.

10.8.1. Attempt 1: Clustering on Amount and Weekend Indicator

10.9. pandas

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```
features_v1 = debit[["log_amount", "is_weekend"]]
X_scaled_v1 = StandardScaler().fit_transform(features_v1)

kmeans_v1 = KMeans(n_clusters=4, random_state=42, n_init=10)
debit["cluster_v1"] = kmeans_v1.fit_predict(X_scaled_v1)

debit["cluster_v1"].value_counts()
```

```
cluster_v1
0    372
3    182
1    175
2     23
Name: count, dtype: int64
```

10.10. polars

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import polars as pl

# Select the features and convert to NumPy
features_v1 = debit_pl.select(
    "log_amount",
    "is_weekend"
).to_numpy()

# Standardize the features
X_scaled_v1 = StandardScaler().fit_transform(features_v1)

# Fit KMeans
kmeans_v1 = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=10
)

clusters = kmeans_v1.fit_predict(X_scaled_v1)
```

```
# Add cluster labels back to the Polars DataFrame
debit_pl = debit_pl.with_columns(
    pl.Series("cluster_v1", clusters)
)

# Equivalent of value_counts()
cluster_counts = (
    debit_pl
    .group_by("cluster_v1")
    .len()
    .sort("cluster_v1")
)

print(cluster_counts)
```

```
shape: (4, 2)

 cluster_v1   len
 ---          ---
 i32          u32

 0            372
 1            175
 2            23
 3            182
```

10.11. pandas

```
smallest_cluster_v1 = debit["cluster_v1"].value_counts().idxmin()
flagged_v1 = debit[debit["cluster_v1"] == smallest_cluster_v1]

print("Smallest cluster size:", len(flagged_v1))
print("Known outliers captured:", flagged_v1["known_outlier"].sum(),
 ↪  "/ 7")
```

```
Smallest cluster size: 23
Known outliers captured: 2 / 7
```

10.12. polars

```
# Find the smallest cluster
smallest_cluster_v1 = (
    debit_pl
    .group_by("cluster_v1")
    .len()
    .sort("len")
    .get_column("cluster_v1")[0]
)

# Filter observations belonging to the smallest cluster
flagged_v1 = (
    debit_pl
    .filter(pl.col("cluster_v1") == smallest_cluster_v1)
)

print("Smallest cluster size:", flagged_v1.height)
print("Known outliers captured:", flagged_v1["known_outlier"].sum(),
 ↪  "/ 7")
```

```
Smallest cluster size: 23
Known outliers captured: 2 / 7
```

This result is disappointing — the smallest cluster only captures 2 of the 7 known outliers.

⚠ Why Did This Fail?

Check how many transactions in the whole dataset occurred on a weekend:

```
debit["is_weekend"].sum()
```

```
np.int64(23)
```

Exactly 23 — and our smallest cluster also had 23 members. **The clustering essentially just separated weekend from weekday transactions**, because `is_weekend` is a binary feature that creates a very clean, easy-to-separate split, while `log_amount` differences (even for our large outliers) look comparatively small after scaling. Only the outliers that *happened* to also land on a weekend got captured — a coincidence of our injected data design, not a genuine amount-based detection.
This is an important lesson: **clustering results are entirely shaped by which features you**

> **include and how they□re scaled.** A single dominant categorical feature can hijack the clustering structure away from the pattern you actually care about.

10.12.1. Attempt 2: Clustering on Amount-Focused Features Only

10.13. pandas

```
features_v2 = debit[["log_amount", "z_score"]]
X_scaled_v2 = StandardScaler().fit_transform(features_v2)

kmeans_v2 = KMeans(n_clusters=4, random_state=42, n_init=10)
debit["cluster_v2"] = kmeans_v2.fit_predict(X_scaled_v2)

debit["cluster_v2"].value_counts()
```

```
cluster_v2
3    360
1    235
0    150
2      7
Name: count, dtype: int64
```

10.14. polars

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import polars as pl

# Select the features and convert to NumPy
features_v2 = debit_pl.select(
    "log_amount",
    "z_score"
).to_numpy()

# Standardize the features
X_scaled_v2 = StandardScaler().fit_transform(features_v2)
```

```
# Fit KMeans
kmeans_v2 = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=10
)

clusters = kmeans_v2.fit_predict(X_scaled_v2)

# Add cluster labels back to the Polars DataFrame
debit_pl = debit_pl.with_columns(
    pl.Series("cluster_v2", clusters)
)

# Equivalent of value_counts()
cluster_counts_v2 = (
    debit_pl
    .group_by("cluster_v2")
    .len()
    .sort("cluster_v2")
)

print(cluster_counts_v2)
```

```
shape: (4, 2)

 cluster_v2  len
 ---         ---
 i32         u32

 0           150
 1           235
 2           7
 3           360
```

10.15. pandas

```
smallest_cluster_v2 = debit["cluster_v2"].value_counts().idxmin()
```

```
flagged_v2 = debit[debit["cluster_v2"] == smallest_cluster_v2]

print("Smallest cluster size:", len(flagged_v2))
flagged_v2[["entry_id", "debit",
 ↪  "known_outlier"]].sort_values("debit", ascending=False)
```

```
Smallest cluster size: 7
```

	entry_id	debit	known_outlier
501	JE3903	176500.0	True
161	JE3900	145000.0	True
733	JE3906	121000.0	True
334	JE3902	112000.0	True
229	JE3901	98000.0	True
611	JE3904	87000.0	True
534	JE3905	63000.0	True

10.16. polars

```
# Find the cluster with the fewest observations
smallest_cluster_v2 = (
    debit_pl
    .group_by("cluster_v2")
    .len()
    .sort("len")
    .get_column("cluster_v2")[0]
)

# Filter rows belonging to the smallest cluster
flagged_v2 = (
    debit_pl
    .filter(pl.col("cluster_v2") == smallest_cluster_v2)
)

print("Smallest cluster size:", flagged_v2.height)

(
    flagged_v2
```

```
    .select(
        "entry_id",
        "debit",
        "known_outlier",
    )
    .sort("debit", descending=True)
)
```

```
Smallest cluster size: 7
```

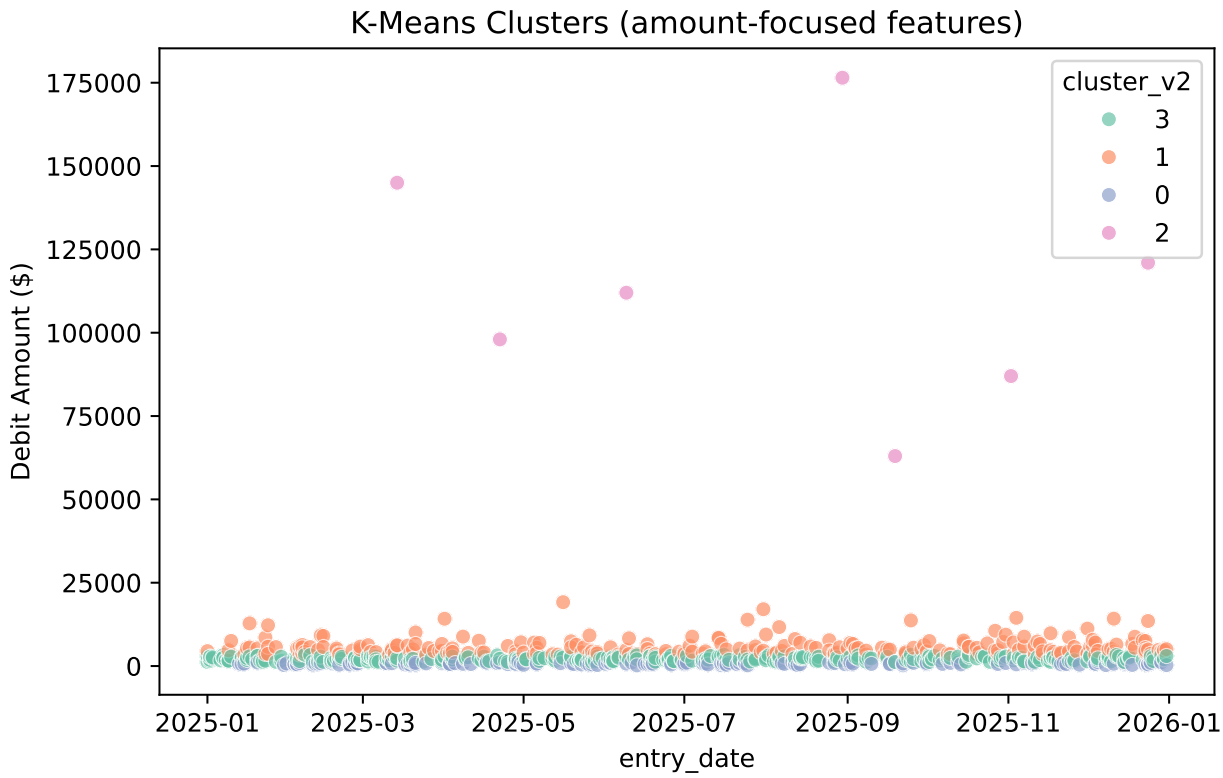
entry_id	debit	known_outlier
str	f64	bool
"JE3903"	176500.0	true
"JE3900"	145000.0	true
"JE3906"	121000.0	true
"JE3902"	112000.0	true
"JE3901"	98000.0	true
"JE3904"	87000.0	true
"JE3905"	63000.0	true

By dropping the dominant `is_weekend` feature and clustering purely on magnitude-related features, the smallest cluster now contains **exactly the 7 known outliers □ no more, no less.**

10.17. seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 4.5))
sns.scatterplot(
    data=debit, x="entry_date", y="debit",
    hue=debit["cluster_v2"].astype(str), palette="Set2", alpha=0.7
)
plt.title("K-Means Clusters (amount-focused features)")
plt.ylabel("Debit Amount ($)")
plt.tight_layout()
plt.show()
```



10.18. plotnine

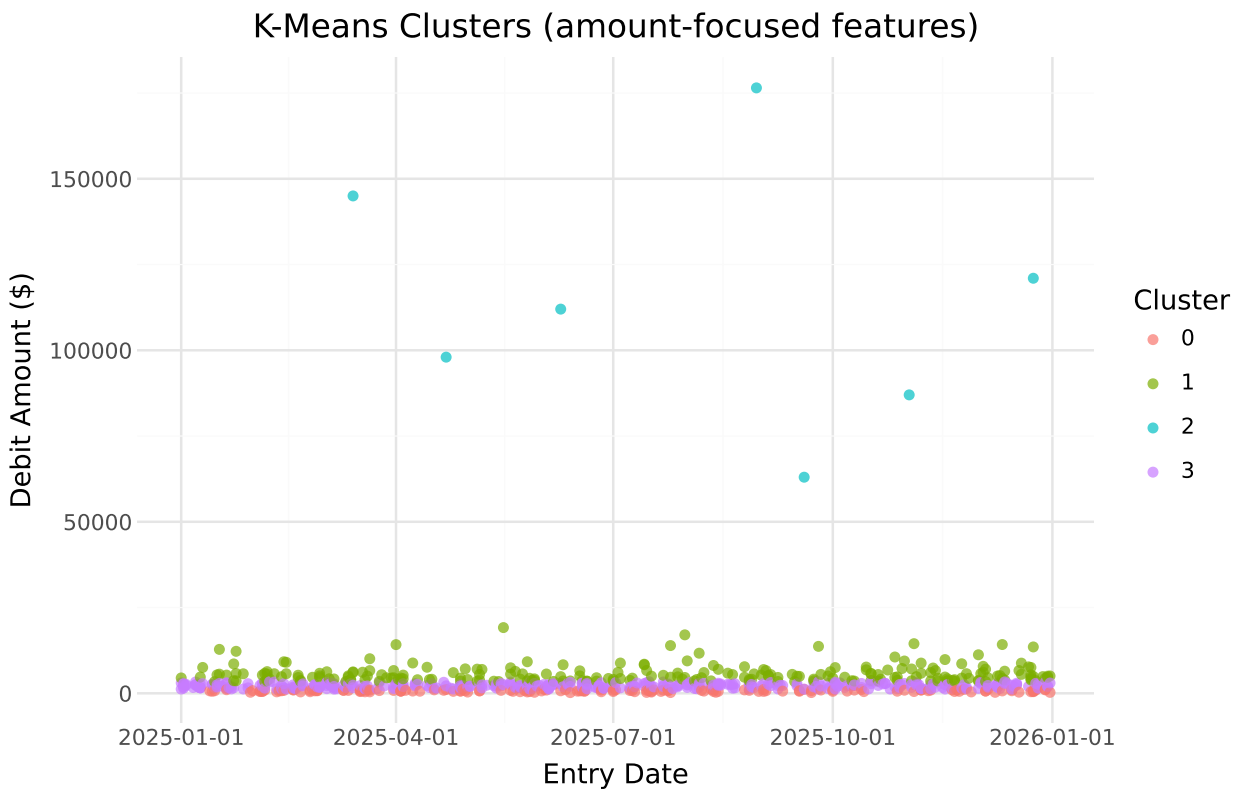
```
from plotnine import (
    ggplot,
    aes,
    geom_point,
    labs,
    theme_minimal,
    theme,
    element_text
)

# Convert Polars to pandas
debit_pd = debit_pl.to_pandas()

# Convert cluster labels to strings for discrete coloring
debit_pd["cluster_v2"] = debit_pd["cluster_v2"].astype(str)

(
```

```
    ggplot(
        debit_pd,
        aes(
            x="entry_date",
            y="debit",
            color="cluster_v2"
        )
    )
    + geom_point(alpha=0.7)
    + labs(
        title="K-Means Clusters (amount-focused features)",
        x="Entry Date",
        y="Debit Amount ($)",
        color="Cluster"
    )
    + theme_minimal()
    + theme(
        figure_size=(7, 4.5),
        plot_title=element_text(ha="center")
    )
)
```



> **Connect to Practice**
>
> This side-by-side comparison is worth remembering beyond this chapter: an unsupervised method isn't "wrong" when it produces an unexpected result — it's finding *some* real structure in the data, just not necessarily the structure you were hoping for. Before trusting a clustering result, always sanity-check what's actually driving the groupings, the way we did here by checking the weekend transaction count.

10.19. Isolation Forest

Isolation Forest takes a different approach: rather than grouping similar points together, it tries to *isolate* each observation with a series of random splits. The intuition is that anomalies — points that are few and different — take **fewer splits** to isolate than normal points buried in a dense cluster of similar observations. Points that get isolated quickly, across many random trees, receive a low (more anomalous) score.

10.20. pandas

```
from sklearn.ensemble import IsolationForest

iso_features = debit[["debit", "is_weekend", "day_of_month"]]

iso_model = IsolationForest(n_estimators=200, contamination=0.02,
 ↪  random_state=42)
debit["anomaly_pred"] = iso_model.fit_predict(iso_features)   # -1 =
 ↪  anomaly, 1 = normal
debit["anomaly_score"] = iso_model.decision_function(iso_features)  #
 ↪  lower = more anomalous

flagged_iso = debit[debit["anomaly_pred"] ==
 ↪  -1].sort_values("anomaly_score")
print("Flagged:", len(flagged_iso))
flagged_iso[["entry_id", "debit", "anomaly_score", "known_outlier"]]
```

```
Flagged: 16
```

	entry_id	debit	anomaly_score	known_outlier
501	JE3903	176500.00	-0.121960	True
611	JE3904	87000.00	-0.104764	True
161	JE3900	145000.00	-0.080345	True
733	JE3906	121000.00	-0.070663	True
334	JE3902	112000.00	-0.069159	True
229	JE3901	98000.00	-0.051642	True
314	JE3047	689.53	-0.021666	False
502	JE3299	552.35	-0.016016	False
76	JE3507	5701.67	-0.015681	False
524	JE3315	5514.90	-0.014020	False
188	JE3026	4433.84	-0.008993	False
448	JE3350	3160.54	-0.006935	False
313	JE3199	2287.49	-0.006081	False
294	JE3426	5651.14	-0.004339	False
207	JE3006	869.40	-0.002835	False
534	JE3905	63000.00	-0.000022	True

10.21. polars

```
from sklearn.ensemble import IsolationForest
import polars as pl

# Select features and convert to NumPy
iso_features = (
    debit_pl
    .select(
        "debit",
        "is_weekend",
        "day_of_month"
    )
    .to_numpy()
)

# Fit Isolation Forest
iso_model = IsolationForest(
    n_estimators=200,
    contamination=0.02,
    random_state=42
)

anomaly_pred = iso_model.fit_predict(iso_features)          # -1 =
 ↪  anomaly, 1 = normal
anomaly_score = iso_model.decision_function(iso_features)   # Lower =
 ↪  more anomalous

# Add results back to the Polars DataFrame
debit_pl = debit_pl.with_columns(
    [
        pl.Series("anomaly_pred", anomaly_pred),
        pl.Series("anomaly_score", anomaly_score),
    ]
)

# Filter flagged anomalies
flagged_iso = (
    debit_pl
    .filter(pl.col("anomaly_pred") == -1)
    .sort("anomaly_score")
```

```
)

print("Flagged:", flagged_iso.height)

flagged_iso.select(
    "entry_id",
    "debit",
    "anomaly_score",
    "known_outlier"
)
```

```
Flagged: 16
```

entry_id str	debit f64	anomaly_score f64	known_outlier bool
"JE3903"	176500.0	-0.12196	true
"JE3904"	87000.0	-0.104764	true
"JE3900"	145000.0	-0.080345	true
"JE3906"	121000.0	-0.070663	true
"JE3902"	112000.0	-0.069159	true
…	…	…	…
"JE3350"	3160.54	-0.006935	false
"JE3199"	2287.49	-0.006081	false
"JE3426"	5651.14	-0.004339	false
"JE3006"	869.4	-0.002835	false
"JE3905"	63000.0	-0.000022	true

Unlike our first clustering attempt, Isolation Forest — even with `is_weekend` included as a feature — successfully captures all 7 known outliers, plus a few additional transactions. Isolation Forest tends to be more robust to this kind of feature-dominance issue than K-means, since it isolates based on how *easy* a point is to separate from the rest, across many random trees, rather than relying on a single distance calculation.

10.21.1. The `contamination` Parameter Controls the Precision-Recall Tradeoff

`contamination` tells the model what fraction of the data to expect as anomalous. This directly trades off **precision** (of what you flag, how much is real) against **recall** (of what's real, how much you catch).

10.22. pandas

```
from sklearn.metrics import precision_score, recall_score

results = []
for contamination in [0.01, 0.02, 0.05]:
    model = IsolationForest(n_estimators=200,
    ↪  contamination=contamination, random_state=42)
    pred = model.fit_predict(iso_features)
    flagged = pred == -1
    results.append({
        "contamination": contamination,
        "num_flagged": flagged.sum(),
        "precision": round(precision_score(debit["known_outlier"],
        ↪  flagged), 3),
        "recall": round(recall_score(debit["known_outlier"], flagged),
        ↪  3),
    })

pd.DataFrame(results)
```

	contamination	num_flagged	precision	recall
0	0.01	8	0.750	0.857
1	0.02	16	0.438	1.000
2	0.05	38	0.184	1.000

10.23. polars

```
from sklearn.ensemble import IsolationForest
from sklearn.metrics import precision_score, recall_score
import polars as pl

results = []

# Convert ground truth labels from Polars to NumPy
y_true = debit_pl["known_outlier"].to_numpy()
```

```
# iso_features is already a NumPy array from the previous step
for contamination in [0.01, 0.02, 0.05]:

    model = IsolationForest(
        n_estimators=200,
        contamination=contamination,
        random_state=42
    )

    pred = model.fit_predict(iso_features)

    # -1 = anomaly, convert to True/False
    flagged = pred == -1

    results.append(
        {
            "contamination": contamination,
            "num_flagged": flagged.sum(),
            "precision": round(
                precision_score(y_true, flagged),
                3
            ),
            "recall": round(
                recall_score(y_true, flagged),
                3
            ),
        }
    )

# Create Polars DataFrame
results_pl = pl.DataFrame(results)

results_pl
```

contamination f64	num_flagged i64	precision f64	recall f64
0.01	8	0.75	0.857
0.02	16	0.438	1.0
0.05	38	0.184	1.0

> **!** Reading This Table
>
> At `contamination=0.01`, the model flags only 8 transactions, catching 6 of the 7 outliers (86% recall) with high precision (75%) — a tight, high-confidence list. At `contamination=0.05`, it flags 38 transactions and catches all 7 outliers (100% recall), but precision drops to just 18% — meaning roughly 4 out of every 5 flagged transactions turn out not to be one of our known outliers.
> Neither setting is "correct" in the abstract. A team with limited review capacity might prefer the tighter, higher-precision list even if it risks missing one real anomaly; a team performing a high-stakes investigation might prefer to cast a wider net and accept more false positives in exchange for near-certain coverage. This tradeoff, not just the algorithm itself, is a central design decision in any real fraud detection program.

10.24. Combining Model-Based and Rule-Based Detection

Do Isolation Forest and the rule-based tests from Chapter 8 actually agree? Let's apply Isolation Forest to `audit_test_transactions.csv` — the dataset with known injected scenarios — and compare.

10.25. pandas

```
audit_gl = pd.read_csv("data/audit_test_transactions.csv")
audit_gl["entry_date"] = pd.to_datetime(audit_gl["entry_date"])
audit_gl["amount"] = audit_gl[["debit", "credit"]].max(axis=1)

entry_level = audit_gl.groupby("entry_id").agg(
    entry_date=("entry_date", "first"),
    amount=("amount", "max"),
    posted_by=("posted_by", "first"),
).reset_index()

entry_level["is_weekend"] = (entry_level["entry_date"].dt.weekday >=
 ↪  5).astype(int)
entry_level["is_year_end"] = (entry_level["entry_date"] >=
 ↪  pd.Timestamp("2025-12-27")).astype(int)

poster_counts = audit_gl.groupby("posted_by")["entry_id"].nunique()
entry_level["poster_frequency"] =
 ↪  entry_level["posted_by"].map(poster_counts)
```

```
iso_features_audit = entry_level[["amount", "is_weekend",
 ↪  "is_year_end", "poster_frequency"]]

iso_audit = IsolationForest(n_estimators=200, contamination=0.1,
 ↪  random_state=42)
entry_level["anomaly_pred"] =
 ↪  iso_audit.fit_predict(iso_features_audit)

known_injected = ["JE5900", "JE5901", "JE5902", "JE5903", "JE5904",
 ↪  "JE5905", "JE5906"]
entry_level["is_known_injected"] =
 ↪  entry_level["entry_id"].isin(known_injected)

entry_level[entry_level["is_known_injected"]][["entry_id", "amount",
 ↪  "anomaly_pred", "is_known_injected"]]
```

	entry_id	amount	anomaly_pred	is_known_injected
110	JE5900	50000.0	-1	True
111	JE5901	10000.0	-1	True
112	JE5902	92500.0	-1	True
113	JE5903	3000.0	1	True
114	JE5904	3000.0	1	True
115	JE5905	41000.0	-1	True
116	JE5906	75000.0	-1	True

10.26. polars

```
import polars as pl
from sklearn.ensemble import IsolationForest

# Read data
audit_gl_pl = pl.read_csv(
    "data/audit_test_transactions.csv",
    try_parse_dates=True
)
```

```
# Create amount column (max of debit and credit)
audit_gl_pl = audit_gl_pl.with_columns(
    pl.max_horizontal(
        ["debit", "credit"]
    ).alias("amount")
)

# Roll transaction lines up to journal-entry level
entry_level_pl = (
    audit_gl_pl
    .group_by("entry_id")
    .agg(
        pl.col("entry_date").first().alias("entry_date"),
        pl.col("amount").max().alias("amount"),
        pl.col("posted_by").first().alias("posted_by"),
    )
)

# Add date-based features
entry_level_pl = entry_level_pl.with_columns(
    [
        (
            (pl.col("entry_date").dt.weekday() >= 6)
            .cast(pl.Int8)
            .alias("is_weekend")
        ),

        (
            (pl.col("entry_date") >= pl.date(2025, 12, 27))
            .cast(pl.Int8)
            .alias("is_year_end")
        ),
    ]
)

# Calculate poster frequency
poster_counts_pl = (
    audit_gl_pl
    .group_by("posted_by")
    .agg(
        pl.col("entry_id")
```

```
        .n_unique()
        .alias("poster_frequency")
    )
)

# Add poster frequency to entry-level data
entry_level_pl = (
    entry_level_pl
    .join(
        poster_counts_pl,
        on="posted_by",
        how="left"
    )
)

# Prepare Isolation Forest features
iso_features_audit = (
    entry_level_pl
    .select(
        "amount",
        "is_weekend",
        "is_year_end",
        "poster_frequency"
    )
    .to_numpy()
)

# Fit Isolation Forest
iso_audit = IsolationForest(
    n_estimators=200,
    contamination=0.1,
    random_state=42
)

anomaly_pred = iso_audit.fit_predict(
    iso_features_audit
)

# Add predictions
entry_level_pl = entry_level_pl.with_columns(
    pl.Series(
```

```
        "anomaly_pred",
        anomaly_pred
    )
)

# Known injected anomalies
known_injected = [
    "JE5900",
    "JE5901",
    "JE5902",
    "JE5903",
    "JE5904",
    "JE5905",
    "JE5906",
]

entry_level_pl = entry_level_pl.with_columns(
    pl.col("entry_id")
    .is_in(known_injected)
    .alias("is_known_injected")
)

# Show known injected entries
(
    entry_level_pl
    .filter(
        pl.col("is_known_injected")
    )
    .select(
        "entry_id",
        "amount",
        "anomaly_pred",
        "is_known_injected"
    )
)
```

entry_id str	amount f64	anomaly_pred i64	is_known_injected bool
"JE5900"	50000.0	-1	true
"JE5905"	41000.0	-1	true

entry_id str	amount f64	anomaly_pred i64	is_known_injected bool
"JE5903"	3000.0	1	true
"JE5901"	10000.0	-1	true
"JE5904"	3000.0	1	true
"JE5906"	75000.0	-1	true
"JE5902"	92500.0	-1	true

Isolation Forest catches 5 of our 7 deliberately injected scenarios — but **misses both duplicate entries (JE5903 and JE5904)** entirely.

> **!** Why the Model Misses the Duplicates — and Why That's the Whole Point
>
> Look at what features we gave the model: `amount`, `is_weekend`, `is_year_end`, `poster_frequency`. Nothing about the duplicate entries is unusual along *any* of those dimensions — they're a completely ordinary $3,000 rent payment, on a weekday, by a normal poster. The only thing wrong with them is that an **identical entry appears twice**, which isn't a pattern any of our numeric features can capture.
> This is exactly why Chapter 8's rule-based duplicate test still matters even after adding machine learning to the toolkit: **a model can only find what its features make visible.** Isolation Forest is excellent at catching multivariate, magnitude-based anomalies without needing to specify the exact pattern in advance — but it cannot catch a pattern like "this exact combination of fields appeared before," because we never engineered a feature that would represent that relationship. A specific rule-based test, by contrast, was purpose-built to catch exactly that.
> **The practical conclusion:** rule-based tests and model-based anomaly detection are **complements**, not *competitors*. A real fraud detection program runs both, precisely because each one has systematic blind spots the other doesn't share.

10.27. Evaluating Unsupervised Methods Without Labels

Everything in this chapter has had an advantage real fraud detection rarely has: we *know* which transactions were the true anomalies, because we built the data ourselves. In practice, you almost never have confirmed fraud labels to check your precision and recall against — that's the whole reason detection is hard.

A few practical approaches used when true labels aren't available:

- **Manual review of a sample of flagged items**, to informally estimate a precision rate before scaling a method up to a full population

- **Stability checks**: does the same method, run with a slightly different random seed or a slightly different feature set, flag roughly the same transactions? Wildly different results across runs suggest the model is picking up noise rather than a real signal
- **Cross-referencing with rule-based flags** (as we just did): if a transaction is flagged by *both* an unsupervised model and an independent rule-based test, that agreement itself is evidence worth weighing more heavily than either signal alone
- **Escalating gradually**: starting with a small, high-precision flagged set (like our `contamination=0.01` result), reviewing it, and only widening the net if that initial review turns up genuine issues

10.28. Chapter Summary

- Anomaly detection methods require numeric features engineered from transaction data; the choice and scaling of features has a *major* effect on what these methods find, sometimes more than the choice of algorithm itself.
- K-means clustering can be hijacked by a single dominant categorical feature (like `is_weekend`), producing clusters that reflect that feature rather than the anomaly pattern you actually care about — always sanity-check what's driving a clustering result.
- Isolation Forest isolates anomalies based on how few random splits it takes to separate them from the rest of the data, and tends to be more robust than K-means to less-informative features being included.
- The `contamination` parameter controls a real precision-recall tradeoff: a stricter setting yields a smaller, higher-confidence list; a looser setting catches more true anomalies at the cost of many more false positives.
- Model-based and rule-based detection have different blind spots — in this chapter, Isolation Forest missed duplicate entries entirely, because "this is a duplicate" was never represented in the feature set. Real fraud detection programs use both approaches together for exactly this reason.
- Without confirmed fraud labels (the normal situation in practice), evaluating an unsupervised method relies on manual review, stability checks, and cross-referencing with independent rule-based signals rather than precision/recall against ground truth.

10.29. Discussion Questions

1. In the first K-means attempt, the model effectively just separated weekend from weekday transactions. How would you have discovered this problem if you *hadn't* already known the true outlier count from Chapter 5?

2. If you were setting the `contamination` parameter for a real audit engagement with limited staff time, how would you decide between a stricter (higher precision) and looser (higher recall) setting?
3. Isolation Forest missed the duplicate-entry scenario entirely. Can you think of another type of fraud or error pattern that a purely numeric, feature-based model would likely miss, and what kind of rule-based test could catch it instead?

10.30. Exercises

1. Add `department` (one-hot encoded) as an additional feature to the Isolation Forest model on `general_ledger_large.csv`. Does this change which transactions get flagged at `contamination=0.02`?
2. Rerun the K-means clustering from "Attempt 2" with `n_clusters` set to 3 and to 6 instead of 4. Does the smallest cluster still capture exactly the 7 known outliers in both cases?
3. Using `audit_test_transactions.csv`, engineer one additional feature that *would* help a model detect the duplicate-entry scenario (JE5903/JE5904), add it to the Isolation Forest feature set, and check whether the model now flags both entries.
4. Combine this chapter's Isolation Forest flags with Chapter 8's composite risk score (rule-based) for `audit_test_transactions.csv` into a single comparison table showing, for each of the 7 known injected entries, whether it was caught by (a) the rule-based score, (b) Isolation Forest, (c) both, or (d) neither.

11. Revenue Recognition and Transaction-Level Analysis

Learning Objectives

By the end of this chapter, you should be able to:

- Explain why revenue recognition timing is a common area of financial reporting risk
- Detect quarter-end revenue concentration patterns often associated with channel stuffing
- Identify "bill-and-hold" style anomalies by comparing invoice dates to shipment dates
- Build an accounts receivable aging schedule and compute Days Sales Outstanding (DSO) by period
- Recognize survivorship bias in time-to-payment analysis, and explain why it can distort period-over-period comparisons
- Perform all of the above using both pandas and polars

11.1. Why Revenue Recognition Is a Common Risk Area

Revenue is usually the single largest number on the income statement, and the timing of when it's recognized has a direct, often material, effect on reported profitability. Accounting standards (ASC 606 in the U.S.) require revenue to be recognized when control of a good or service actually transfers to the customer — not simply when a contract is signed or an invoice is generated (FASB 2014). In addition, auditing standards consider revenue recognition a high fraud risk area (PCAOB 2024a).

Because revenue timing has such a large impact on reported results, it's also one of the most common areas where financial reporting goes wrong — sometimes through honest error, sometimes through deliberate manipulation. Two patterns show up often enough that they're standard audit analytics tests in their own right:

- **Channel stuffing / cutoff risk**: recognizing an unusually large share of a period's revenue right at period-end, potentially borrowing sales from the next period to inflate the current one
- **Bill-and-hold arrangements**: invoicing a customer before goods actually ship, recognizing revenue before control has genuinely transferred

This chapter builds detection techniques for both, using a new dataset: `sales_transactions.csv`, containing 427 invoices across 35 customers for fiscal year 2025.

11.2. pandas

```
import pandas as pd
sales_pd = pd.read_csv("data/sales_transactions.csv",
 ↪  parse_dates=["invoice_date", "ship_date", "payment_date"])
sales_pd.head()
```

	invoice_id	customer_id	invoice_date	ship_date	amount	payment_terms_days	quarter	payme
0	INV2058	CUST103	2025-01-01	2024-12-30	5544.89	45	1	2025-0
1	INV2074	CUST117	2025-01-02	2024-12-31	6579.43	60	1	2025-0
2	INV2010	CUST113	2025-01-03	2025-01-01	2640.39	60	1	2025-0
3	INV2060	CUST122	2025-01-03	2025-01-01	7605.61	30	1	2025-0
4	INV2072	CUST112	2025-01-04	2025-01-02	4942.04	60	1	2025-0

11.3. polars

```
import polars as pl
sales_pl = pl.read_csv("data/sales_transactions.csv",
 ↪  try_parse_dates=True)
sales_pl.head()
```

invoice_id str	customer_id str	invoice_date date	ship_date date	amount f64	payment_terms_days i64	quarter i64	paymen date
"INV2058"	"CUST103"	2025-01-01	2024-12-30	5544.89	45	1	2025-01-
"INV2074"	"CUST117"	2025-01-02	2024-12-31	6579.43	60	1	2025-04-
"INV2010"	"CUST113"	2025-01-03	2025-01-01	2640.39	60	1	2025-02-

invoice_id str	customer_id str	invoice_date date	ship_date date	amount f64	payment_terms_days i64	quarter i64	paymen date
"INV2060"	"CUST122"	2025-01-03	2025-01-01	7605.61	30	1	2025-01-
"INV2072"	"CUST112"	2025-01-04	2025-01-02	4942.04	60	1	2025-02-

Each invoice has an `invoice_date` (when revenue was recorded), a `ship_date` (when goods actually shipped), a `payment_date` (when the customer paid, if they have as of year-end), and `payment_terms_days` (the customer's agreed payment window).

11.4. Detecting Quarter-End Revenue Concentration

If revenue were recognized at a steady pace throughout each quarter, we'd expect roughly even daily volume. A disproportionate share of revenue landing in the final days of a quarter is a red flag worth investigating.

11.5. pandas

```
sales_pd["quarter_end_date"] =
 ↪  sales_pd.groupby("quarter")["invoice_date"].transform("max")
sales_pd["days_before_qend"] = (sales_pd["quarter_end_date"] -
 ↪  sales_pd["invoice_date"]).dt.days
sales_pd["is_quarter_end_spike"] = sales_pd["days_before_qend"] <= 3

spike_summary_pd = sales_pd.groupby(["quarter",
 ↪  "is_quarter_end_spike"])["amount"].agg(["count", "sum"])
spike_summary_pd
```

quarter	is_quarter_end_spike	count	sum
1	False	82	348991.90
	True	25	153873.10
2	False	79	399689.58
	True	26	176263.00
3	False	83	403877.06
	True	22	123481.36
4	False	87	427565.46

quarter	is_quarter_end_spike	count	sum
	True	23	203325.24

11.6. polars

```
sales_pl = sales_pl.with_columns(

 ↪  pl.col("invoice_date").max().over("quarter").alias("quarter_end_date")
).with_columns(
    (pl.col("quarter_end_date") -
 ↪  pl.col("invoice_date")).dt.total_days().alias("days_before_qend")
).with_columns(
    (pl.col("days_before_qend") <= 3).alias("is_quarter_end_spike")
)

spike_summary_pl = (
    sales_pl.group_by(["quarter", "is_quarter_end_spike"])
    .agg([pl.len().alias("count"),
 ↪  pl.col("amount").sum().alias("sum")])
    .sort(["quarter", "is_quarter_end_spike"])
)
spike_summary_pl
```

quarter i64	is_quarter_end_spike bool	count u32	sum f64
1	false	82	348991.9
1	true	25	153873.1
2	false	79	399689.58
2	true	26	176263.0
3	false	83	403877.06
3	true	22	123481.36
4	false	87	427565.46
4	true	23	203325.24

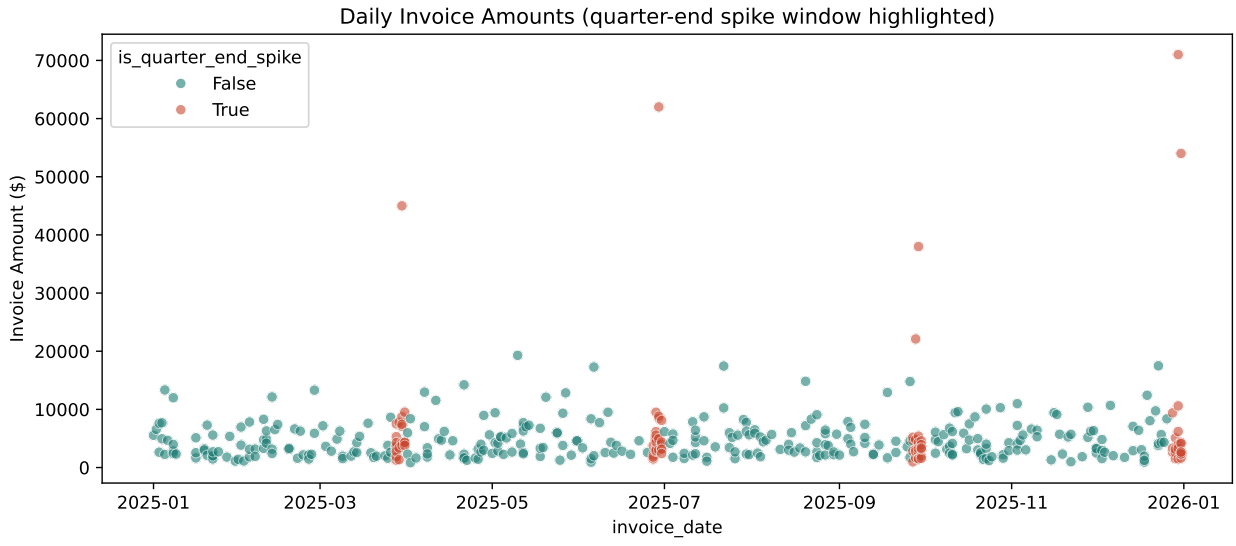
The last **4 days** of each quarter — out of roughly 90 days total — account for **20% to 32% of that quarter□s total revenue.** That's a striking concentration: if revenue were recognized evenly, those

4 days should account for only about 4% of the quarter's total. This kind of pattern, on its own, doesn't prove anything improper happened — some genuine seasonality or sales-team incentive structures can produce a legitimate end-of-period push — but it's exactly the kind of pattern that warrants a closer look.

11.7. seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 4.5))
sns.scatterplot(
    data=sales_pd, x="invoice_date", y="amount",
 ↪  hue="is_quarter_end_spike",
    palette={True: "#c9482f", False: "#1c7c73"}, alpha=0.6
)
plt.title("Daily Invoice Amounts (quarter-end spike window
 ↪  highlighted)")
plt.ylabel("Invoice Amount ($)")
plt.tight_layout()
plt.show()
```



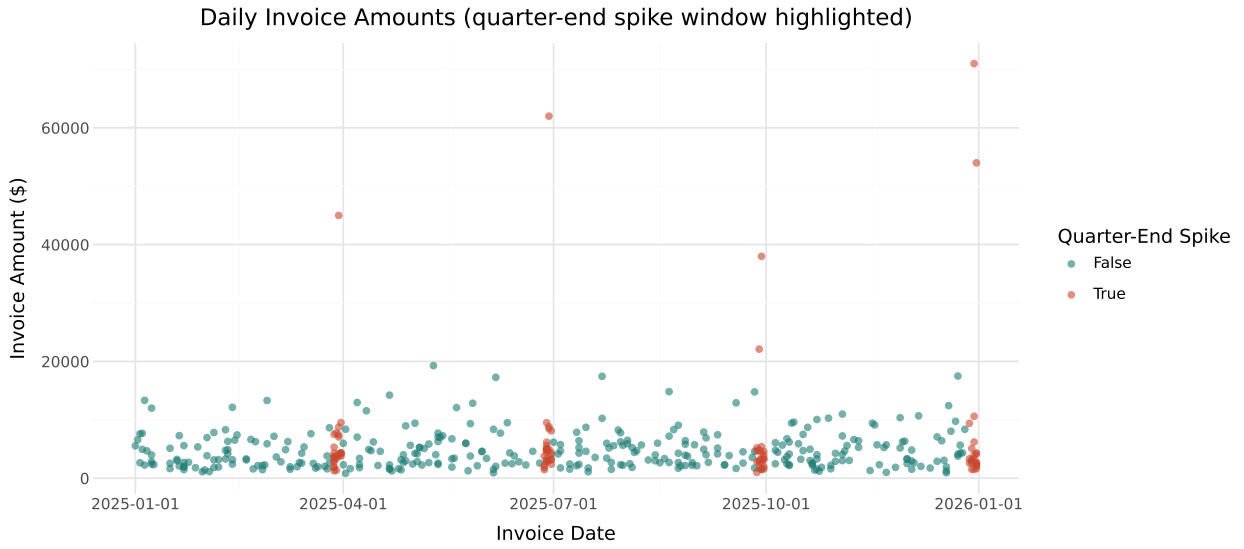
11.8. plotnine

```
from plotnine import (
    ggplot,
    aes,
    geom_point,
    scale_color_manual,
    labs,
    theme_minimal,
    theme,
    element_text
)

# Convert Polars DataFrame to pandas
sales_pd = sales_pl.to_pandas()

(
    ggplot(
        sales_pd,
        aes(
            x="invoice_date",
            y="amount",
            color="is_quarter_end_spike"
        )
    )
    + geom_point(alpha=0.6)
    + scale_color_manual(
        values={
            True: "#c9482f",
            False: "#1c7c73"
        }
    )
    + labs(
        title="Daily Invoice Amounts (quarter-end spike window
 ↪  highlighted)",
        x="Invoice Date",
        y="Invoice Amount ($)",
        color="Quarter-End Spike"
    )
    + theme_minimal()
    + theme(
```

```
        figure_size=(10, 4.5),
        plot_title=element_text(ha="center")
    )
)
```



The red points cluster visibly at the right edge of each quarter — exactly where we'd expect to see them if revenue is being pushed into period-end.

11.8.1. Which Customers Drive the Concentration?

11.9. pandas

```
customer_summary = sales_pd.groupby("customer_id").apply(
    lambda x: pd.Series({
        "total_revenue": x["amount"].sum(),
        "spike_revenue": x.loc[x["is_quarter_end_spike"],
        ↪  "amount"].sum(),
    }),
    include_groups=False,
)
customer_summary["pct_from_spike"] = customer_summary["spike_revenue"]
 ↪  / customer_summary["total_revenue"] * 100

customer_summary.sort_values("pct_from_spike",
 ↪  ascending=False).head(8).round(1)
```

customer_id	total_revenue	spike_revenue	pct_from_spike
CUST130	98124.9	77197.8	78.7
CUST118	114430.5	76820.0	67.1
CUST112	188043.0	119988.8	63.8
CUST124	88800.1	44995.6	50.7
CUST104	27418.7	12466.0	45.5
CUST128	97684.5	42647.0	43.7
CUST107	55173.0	19555.3	35.4
CUST106	54509.0	18589.0	34.1

11.10. polars

```
customer_summary_pl = (
    sales_pl
    .group_by("customer_id")
    .agg(
        pl.col("amount").sum().alias("total_revenue"),

        pl.col("amount")
        .filter(pl.col("is_quarter_end_spike"))
        .sum()
        .alias("spike_revenue"),
    )
    .with_columns(
        (
            pl.col("spike_revenue")
            / pl.col("total_revenue")
            * 100
        )
        .round(1)
        .alias("pct_from_spike")
    )
    .sort("pct_from_spike", descending=True)
    .head(8)
)
```

```
customer_summary_pl
```

customer_id str	total_revenue f64	spike_revenue f64	pct_from_spike f64
"CUST130"	98124.94	77197.82	78.7
"CUST118"	114430.52	76820.02	67.1
"CUST112"	188043.02	119988.81	63.8
"CUST124"	88800.06	44995.59	50.7
"CUST104"	27418.69	12465.98	45.5
"CUST128"	97684.52	42647.0	43.7
"CUST107"	55173.03	19555.28	35.4
"CUST106"	54509.05	18589.01	34.1

Four customers stand out with well over half their annual revenue concentrated in quarter-end spike windows — worth remembering, since we'll come back to these exact customers in the next section.

11.11. Detecting Bill-and-Hold Patterns

A **bill-and-hold** arrangement — invoicing a customer before goods actually ship — is a classic revenue recognition red flag, since control of the goods hasn't transferred to the customer yet. We can detect this directly by comparing `invoice_date` to `ship_date`.

11.12. pandas

```
sales_pd["ship_lag_days"] = (sales_pd["ship_date"] -
 ↪  sales_pd["invoice_date"]).dt.days

bill_and_hold_pd = sales_pd[sales_pd["ship_lag_days"] > 10]
bill_and_hold_pd[["invoice_id", "customer_id", "invoice_date",
 ↪  "ship_date", "ship_lag_days", "amount"]]
```

	invoice_id	customer_id	invoice_date	ship_date	ship_lag_days	amount
96	INV2423	CUST112	2025-03-30	2025-04-24	25	45000.0

	invoice_id	customer_id	invoice_date	ship_date	ship_lag_days	amount
201	INV2424	CUST118	2025-06-29	2025-07-29	30	62000.0
307	INV2425	CUST124	2025-09-29	2025-10-19	20	38000.0
412	INV2426	CUST130	2025-12-30	2026-02-03	35	71000.0
426	INV2427	CUST112	2025-12-31	2026-01-28	28	54000.0

11.13. polars

```
sales_pl = sales_pl.with_columns(
    (pl.col("ship_date") -
 ↪  pl.col("invoice_date")).dt.total_days().alias("ship_lag_days")
)

bill_and_hold_pl = sales_pl.filter(pl.col("ship_lag_days") > 10)
bill_and_hold_pl.select(["invoice_id", "customer_id", "invoice_date",
 ↪  "ship_date", "ship_lag_days", "amount"])
```

invoice_id str	customer_id str	invoice_date date	ship_date date	ship_lag_days i64	amount f64
”INV2423”	”CUST112”	2025-03-30	2025-04-24	25	45000.0
”INV2424”	”CUST118”	2025-06-29	2025-07-29	30	62000.0
”INV2425”	”CUST124”	2025-09-29	2025-10-19	20	38000.0
”INV2426”	”CUST130”	2025-12-30	2026-02-03	35	71000.0
”INV2427”	”CUST112”	2025-12-31	2026-01-28	28	54000.0

Five invoices ship 20 to 35 days *after* the invoice date, in sharp contrast to the rest of the population, where goods almost always ship on or slightly before the invoice date (a normal “ship, then bill” sequence).

11.14. pandas

```
sales_pd["ship_lag_days"].describe()
```

```
count    427.000000
mean      -0.681499
```

```
std         3.238082
min        -2.000000
25%        -2.000000
50%        -1.000000
75%         0.000000
max        35.000000
Name: ship_lag_days, dtype: float64
```

11.15. polars

```
sales_pl.select("ship_lag_days").describe()
```

statistic	ship_lag_days
str	f64
"count"	427.0
"null_count"	0.0
"mean"	-0.681499
"std"	3.238082
"min"	-2.0
"25%"	-2.0
"50%"	-1.0
"75%"	0.0
"max"	35.0

The median ship lag across the whole population is *negative one day* (goods typically ship a day before invoicing) — making a 20+ day lag unmistakably unusual.

> **!** Connecting the Two Findings
>
> Look back at the customer-level spike concentration table from the previous section. **CUST112, CUST118, CUST124, and CUST130** — the four customers with the highest share of revenue from quarter-end spikes — are the *same* customers behind every bill-and-hold invoice flagged here. Two independent tests, run on completely different logic (timing concentration vs. invoice/ship date mismatch), converge on the same small set of customers. This kind of convergence between unrelated tests is a much stronger signal than either finding alone, and is exactly the kind of pattern that should move a transaction from "worth a look" to "worth a real investigation."

11.16. Accounts Receivable Aging

An **AR aging schedule** aging is an accounting method that categorizes a company's unpaid customer invoices based on the length of time they have been due. It groups outstanding balances into specific time buckets (e.g., 0-30, 31-60, 61-90, and 90+ days) to help businesses track collections, assess credit risk, and manage cash flow.

11.17. pandas

```
as_of_date = pd.Timestamp("2025-12-31")

unpaid_pd = sales_pd[sales_pd["payment_date"].isna()].copy()
unpaid_pd["days_outstanding"] = (as_of_date -
 ↪  unpaid_pd["invoice_date"]).dt.days

bins = [-1, 30, 60, 90, float("inf")]
labels = ["0-30", "31-60", "61-90", "90+"]
unpaid_pd["aging_bucket"] = pd.cut(unpaid_pd["days_outstanding"],
 ↪  bins=bins, labels=labels)

aging_summary_pd = unpaid_pd.groupby("aging_bucket",
 ↪  observed=True)["amount"].agg(["count", "sum"])
aging_summary_pd
```

aging_bucket	count	sum
0-30	48	334474.04
31-60	10	55776.25
61-90	3	13833.75
90+	6	27595.55

11.18. polars

```
as_of_pl = pl.date(2025, 12, 31)

unpaid_pl =
 ↪  sales_pl.filter(pl.col("payment_date").is_null()).with_columns(
```

```
    (as_of_pl -
 ↪  pl.col("invoice_date")).dt.total_days().alias("days_outstanding")
)

unpaid_pl = unpaid_pl.with_columns(
    pl.when(pl.col("days_outstanding") <= 30).then(pl.lit("0-30"))
    .when(pl.col("days_outstanding") <= 60).then(pl.lit("31-60"))
    .when(pl.col("days_outstanding") <= 90).then(pl.lit("61-90"))
    .otherwise(pl.lit("90+"))
    .alias("aging_bucket")
)

aging_summary_pl = (
    unpaid_pl.group_by("aging_bucket")
    .agg([pl.len().alias("count"),
 ↪  pl.col("amount").sum().alias("sum")])
    .sort("aging_bucket")
)
aging_summary_pl
```

aging_bucket str	count u32	sum f64
"0-30"	48	334474.04
"31-60"	10	55776.25
"61-90"	3	13833.75
"90+"	6	27595.55



A Bin-Boundary Bug Worth Knowing About

The pandas bins deliberately start at **-1**, not **0**. Several invoices had exactly 0 days outstanding (invoiced on the very last day of the year), and pandas' `pd.cut()` uses *left-open* intervals by default — meaning a bin of `(0, 30]` would silently **exclude** a value of exactly 0. Using `-1` as the lower bound avoids this trap. This is a classic, easy-to-miss bug: the first version of this code (using `bins=[0, 30, 60, 90, inf]`) silently dropped 9 real invoices from the aging schedule with no error or warning — always double-check your bucket totals sum back to your actual unpaid count.

Roughly $27,600 sits in the 90+ day bucket — the balances most at risk of never being collected, and the ones a real audit or collections team would prioritize reviewing first.

11.19. Days Sales Outstanding (DSO) by Quarter

Days Sales Outstanding (DSO) is a financial metric that measures the average number of days it takes a company to collect payment after making a sale. A lower DSO indicates healthy cash flow and efficient collections, while a higher DSO points to potential liquidity risks and delayed payments.

11.20. pandas

```
paid_pd = sales_pd[sales_pd["is_paid"]].copy()
paid_pd["days_to_collect"] = (paid_pd["payment_date"] -
 ↪  paid_pd["invoice_date"]).dt.days

dso_by_quarter_pd =
 ↪  paid_pd.groupby("quarter")["days_to_collect"].mean()
dso_by_quarter_pd.round(1)
```

```
quarter
1    62.2
2    52.2
3    48.7
4    31.1
Name: days_to_collect, dtype: float64
```

11.21. polars

```
paid_pl = sales_pl.filter(pl.col("is_paid")).with_columns(
    (pl.col("payment_date") -
 ↪  pl.col("invoice_date")).dt.total_days().alias("days_to_collect")
)

dso_by_quarter_pl = (
    paid_pl.group_by("quarter")

 ↪  .agg(pl.col("days_to_collect").mean().alias("avg_days_to_collect"))
    .sort("quarter")
)
```

```
dso_by_quarter_pl
```

quarter i64	avg_days_to_collect f64
1	62.205607
2	52.152381
3	48.727273
4	31.142857

At first glance, this looks like a clear success story: average collection time falls from 62 days in Q1 to just 31 days in Q4.

> ! Before You Celebrate: Survivorship Bias
>
> This apparent improvement is at least partly an illusion, and it's worth understanding exactly why. Our data was captured **as of December 31, 2025**. A Q1 invoice has had up to a full year to get paid before that cutoff — plenty of time for even slow payers to show up in our "paid" data. A Q4 invoice, by contrast, might have been issued on, say, December 20th — only 11 days before our cutoff. **Only the fastest-paying Q4 customers could possibly show up as □paid□ in this snapshot; every slow-paying Q4 customer is still sitting in accounts receivable, invisible to this calculation.**
> In other words, the Q4 "paid" group isn't a random or representative sample of Q4 invoices — it's systematically biased toward whichever invoices happened to be paid fastest. This is a specific case of **survivorship bias**: drawing a conclusion from only the observations that "survived" to the point of being measurable, while ignoring the ones that didn't (yet).
> **The fix**, if you needed a genuinely fair quarter-over-quarter comparison, would be to only compare invoices that have all had an *equal* amount of time to be collected (for example, restricting every quarter's calculation to invoices at least 90 days old as of the measurement date) — something you're encouraged to try in this chapter's exercises.

11.22. Chapter Summary

- Revenue recognition timing is one of the highest-risk areas in financial reporting, since a large share of reported profitability depends on *when* revenue is recorded, not just whether the underlying sale is genuine.
- Quarter-end revenue concentration (a disproportionate share of a period's revenue landing in its final days) is a standard analytics test associated with channel stuffing risk.

- Comparing invoice dates to ship dates directly detects bill-and-hold patterns, where revenue is recognized before goods have actually transferred to the customer.
- When two independently-designed tests flag the same set of transactions or customers, that convergence is stronger evidence than either test alone.
- AR aging schedules require careful attention to bin boundaries — an off-by-one error in bucket definitions can silently drop real invoices from your analysis with no visible error.
- DSO and other time-to-event metrics computed near a data cutoff are vulnerable to survivorship bias: recent periods will systematically appear to collect faster, simply because slow payers haven't had time to be captured yet, not because collection genuinely improved.

11.23. Discussion Questions

1. A 20-30% revenue concentration in the last 4 days of a quarter isn't proof of a problem by itself. What additional information would you want before deciding whether this pattern reflects a real business practice versus a reporting concern?
2. Why does convergence between the quarter-end spike analysis and the bill-and-hold analysis (both flagging the same four customers) matter more than either finding alone?
3. Explain, in your own words, why the declining DSO trend from Q1 to Q4 is at least partly misleading, and describe a concrete way you could measure DSO that would avoid this bias.

11.24. Exercises

1. Recompute DSO by quarter, but restrict the analysis to only invoices at least 120 days old as of December 31, 2025 (i.e., invoiced on or before September 3, 2025). Does the "improving DSO" trend still appear once this restriction removes the survivorship bias?
2. Using the customer-level spike concentration table, identify any customer with more than 30% of their revenue from quarter-end spikes who was *not* one of the five bill-and-hold invoices. What follow-up would you recommend for that customer specifically?
3. Rebuild the AR aging schedule using 15-day buckets instead of 30-day buckets (0-15, 16-30, 31-45, 46-60, 61-90, 90+). Does any bucket reveal a concentration of risk that the coarser 30-day buckets obscured?
4. Using either pandas or polars, calculate the average `ship_lag_days` by customer (not just by invoice) across the full dataset. Does this reveal any customers with a mildly unusual average shipping pattern that fell below this chapter's 10-day flagging threshold for any single invoice?

12. Tax Analytics

Learning Objectives

By the end of this chapter, you should be able to:

- Compute effective tax rates (ETR) at the jurisdiction and consolidated level
- Build a rate reconciliation bridging statutory to effective tax rates
- Identify disproportionate profit allocation across jurisdictions using profit-per-employee and profit-to-revenue metrics
- Recompute expected sales tax from transaction-level data and flag compliance discrepancies
- Apply both pandas and polars to multi-jurisdiction and transaction-level tax data
- Recognize the limits of a chosen materiality threshold when flagging compliance exceptions

12.1. Two Kinds of Tax Analytics

This chapter covers two related but distinct areas of tax analytics:

1. **Income tax analytics** — analyzing effective tax rates across jurisdictions and over time, and understanding what drives the gap between a company's statutory and effective tax rate
2. **Indirect tax compliance testing** — recomputing sales tax at the transaction level to verify it was calculated and collected correctly

We'll use two datasets: `tax_provision_by_jurisdiction.csv` (a multinational company's income tax provision by country and year) and `sales_tax_test_transactions.csv` (300 transaction-level sales with sales tax detail).

12.2. pandas

```
import pandas as pd
tax_pd = pd.read_csv("data/tax_provision_by_jurisdiction.csv")
tax_pd
```

	year	jurisdiction	revenue	employees	pretax_income	statutory_rate	tax_expense	reven
0	2023	United States	21329009	420	3673682	0.210	726076	5078
1	2023	United Kingdom	7460194	160	1115781	0.250	286045	4662
2	2023	Germany	6204232	130	1097064	0.300	309781	4772
3	2023	Ireland	2294657	35	7349868	0.125	886646	6556
4	2023	Singapore	3203999	60	400186	0.170	73632	5340
5	2023	Bermuda	208878	4	5964247	0.000	3310	5221
6	2024	United States	20910072	420	3165520	0.210	607806	4978
7	2024	United Kingdom	7471163	160	1034235	0.250	259165	4669
8	2024	Germany	6447472	130	842126	0.300	244953	4959
9	2024	Ireland	2609897	35	8353045	0.125	1043015	7456
10	2024	Singapore	3714296	60	515627	0.170	88072	6190
11	2024	Bermuda	187779	4	5712833	0.000	0	4694
12	2025	United States	23445443	420	4022515	0.210	868518	5582
13	2025	United Kingdom	8220750	160	1229443	0.250	324860	5138
14	2025	Germany	7095661	130	1200687	0.300	338194	5458
15	2025	Ireland	2332291	35	7921309	0.125	972589	6663
16	2025	Singapore	3781970	60	671177	0.170	107459	6303
17	2025	Bermuda	266682	4	8037523	0.000	0	6667

12.3. polars

```
import polars as pl
tax_pl = pl.read_csv("data/tax_provision_by_jurisdiction.csv")
```

Our fictional company operates in six jurisdictions with very different statutory tax rates, from Bermuda (0%) to Germany (30%).

12.4. Effective Tax Rate by Jurisdiction

The **effective tax rate (ETR)** is tax expense divided by pretax income — the tax rate a company actually experiences, which can differ substantially from any jurisdiction's statutory rate due to

credits, deductions, and other adjustments.

```
tax_pd[["jurisdiction", "year", "statutory_rate",
 ↪  "effective_tax_rate"]].round(3)
```

	jurisdiction	year	statutory_rate	effective_tax_rate
0	United States	2023	0.210	0.198
1	United Kingdom	2023	0.250	0.256
2	Germany	2023	0.300	0.282
3	Ireland	2023	0.125	0.121
4	Singapore	2023	0.170	0.184
5	Bermuda	2023	0.000	0.001
6	United States	2024	0.210	0.192
7	United Kingdom	2024	0.250	0.251
8	Germany	2024	0.300	0.291
9	Ireland	2024	0.125	0.125
10	Singapore	2024	0.170	0.171
11	Bermuda	2024	0.000	0.000
12	United States	2025	0.210	0.216
13	United Kingdom	2025	0.250	0.264
14	Germany	2025	0.300	0.282
15	Ireland	2025	0.125	0.123
16	Singapore	2025	0.170	0.160
17	Bermuda	2025	0.000	0.000

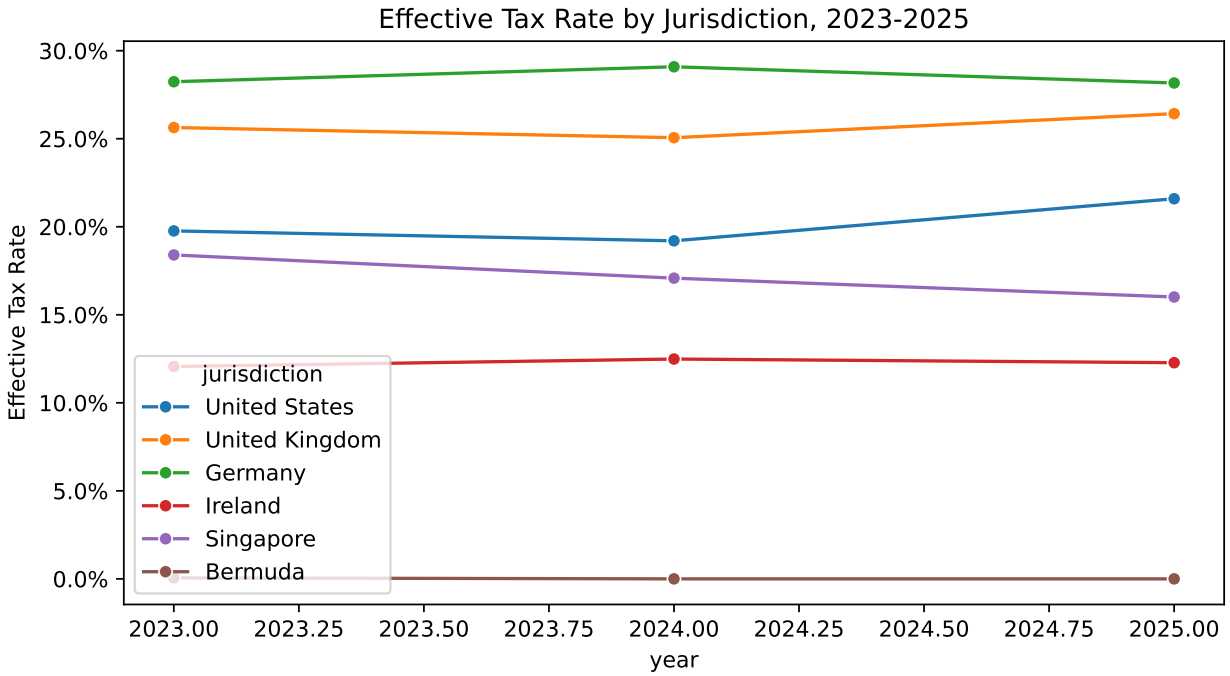
Most jurisdictions show an effective rate reasonably close to their statutory rate. Ireland and Bermuda, however, show *very* low effective rates — but that alone isn't unusual, since their statutory rates are also low (12.5% and 0%, respectively). The more interesting question is how much *profit* is being taxed at those low rates, relative to the actual business activity happening there.

12.5. seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4.5))
sns.lineplot(data=tax_pd, x="year", y="effective_tax_rate",
 ↪  hue="jurisdiction", marker="o")
```

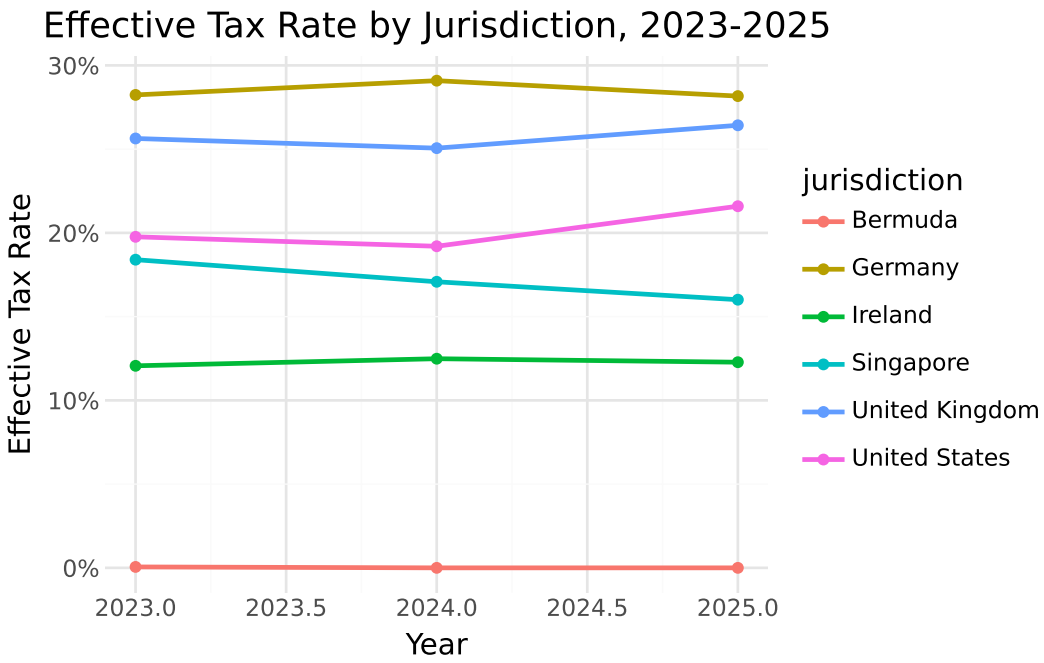
```
plt.title("Effective Tax Rate by Jurisdiction, 2023-2025")
plt.ylabel("Effective Tax Rate")
plt.gca().yaxis.set_major_formatter(plt.matplotlib.ticker.PercentFormatter(1.0))
plt.tight_layout()
plt.show()
```



12.6. plotnine

```
from plotnine import *

(
    ggplot(tax_pd, aes(x="year", y="effective_tax_rate",
    color="jurisdiction"))
    + geom_line(size=1)
    + geom_point()
    + scale_y_continuous(labels=lambda l: [f"{v:.0%}" for v in l])
    + labs(title="Effective Tax Rate by Jurisdiction, 2023-2025",
      x="Year", y="Effective Tax Rate")
    + theme_minimal()
)
```



Every major operating jurisdiction (US, UK, Germany, Singapore) tracks fairly close to a stable band each year. Ireland and Bermuda sit visibly apart, near the bottom of the chart — consistent with their low statutory rates, but worth keeping in mind as we look at profit allocation next.

12.7. Consolidated Effective Tax Rate

12.8. pandas

```
consolidated_pd = tax_pd.groupby("year",
 ↪  as_index=False)[["pretax_income", "tax_expense"]].sum()
consolidated_pd["consolidated_etr"] = consolidated_pd["tax_expense"] /
 ↪  consolidated_pd["pretax_income"]
consolidated_pd.round(3)
```

	year	pretax_income	tax_expense	consolidated_etr
0	2023	19600828	2285490	0.117
1	2024	19623386	2243011	0.114
2	2025	23082654	2611620	0.113

12.9. polars

```
consolidated_pl = (
    tax_pl.group_by("year")
    .agg([pl.col("pretax_income").sum(), pl.col("tax_expense").sum()])
    .with_columns((pl.col("tax_expense") /
 ↪  pl.col("pretax_income")).alias("consolidated_etr"))
    .sort("year")
)
consolidated_pl
```

year i64	pretax_income i64	tax_expense i64	consolidated_etr f64
2023	19600828	2285490	0.116602
2024	19623386	2243011	0.114303
2025	23082654	2611620	0.113142

The consolidated ETR is roughly **11–12%** — noticeably lower than *any* individual major jurisdiction's statutory rate (21% to 30%). This gap is the first clue that something in the jurisdictional mix is pulling the blended rate down substantially.

12.10. Spotting Disproportionate Profit Allocation

A standard analytical test for profit-shifting risk compares each jurisdiction's **share of profit** to its **share of real economic activity** — typically proxied by revenue or headcount. A jurisdiction booking far more profit than its business substance would suggest is a classic red flag.

12.11. pandas

```
totals_pd = tax_pd.groupby("year").agg(
    total_revenue=("revenue", "sum"),
    total_profit=("pretax_income", "sum"),
    total_employees=("employees", "sum"),
)
```

```
low_substance_pd = (
    tax_pd[tax_pd["jurisdiction"].isin(["Ireland", "Bermuda"])]
    .groupby("year")
    .agg(ls_revenue=("revenue", "sum"), ls_profit=("pretax_income",
 ↪  "sum"), ls_employees=("employees", "sum"))
)

combined_pd = totals_pd.join(low_substance_pd)
combined_pd["pct_of_revenue"] = combined_pd["ls_revenue"] /
 ↪  combined_pd["total_revenue"] * 100
combined_pd["pct_of_employees"] = combined_pd["ls_employees"] /
 ↪  combined_pd["total_employees"] * 100
combined_pd["pct_of_profit"] = combined_pd["ls_profit"] /
 ↪  combined_pd["total_profit"] * 100

combined_pd[["pct_of_revenue", "pct_of_employees",
 ↪  "pct_of_profit"]].round(1)
```

year	pct_of_revenue	pct_of_employees	pct_of_profit
2023	6.2	4.8	67.9
2024	6.8	4.8	71.7
2025	5.8	4.8	69.1

12.12. polars

```
totals_pl = tax_pl.group_by("year").agg([
    pl.col("revenue").sum().alias("total_revenue"),
    pl.col("pretax_income").sum().alias("total_profit"),
    pl.col("employees").sum().alias("total_employees"),
])

low_substance_pl = (
    tax_pl.filter(pl.col("jurisdiction").is_in(["Ireland",
 ↪  "Bermuda"]))
    .group_by("year")
    .agg([
```

```
        pl.col("revenue").sum().alias("ls_revenue"),
        pl.col("pretax_income").sum().alias("ls_profit"),
        pl.col("employees").sum().alias("ls_employees"),
    ])
)

combined_pl = totals_pl.join(low_substance_pl,
 ↪  on="year").with_columns([
    (pl.col("ls_revenue") / pl.col("total_revenue") *
 ↪  100).alias("pct_of_revenue"),
    (pl.col("ls_employees") / pl.col("total_employees") *
 ↪  100).alias("pct_of_employees"),
    (pl.col("ls_profit") / pl.col("total_profit") *
 ↪  100).alias("pct_of_profit"),
]).sort("year")

combined_pl.select(["year", "pct_of_revenue", "pct_of_employees",
 ↪  "pct_of_profit"])
```

year i64	pct_of_revenue f64	pct_of_employees f64	pct_of_profit f64
2023	6.151045	4.820766	67.926289
2024	6.767368	4.820766	71.679159
2025	5.757226	4.820766	69.137769

! A Textbook Profit-Shifting Pattern

Ireland and Bermuda together account for only about **6% of consolidated revenue** and **under 5% of employees** — yet they generate roughly **68–72% of consolidated pretax profit**. This is exactly the kind of mismatch that international tax authorities (and analysts studying a company's tax footprint) look for: profit concentrated in low-tax jurisdictions far out of proportion to the actual business substance (people, sales activity) located there. It directly explains why the consolidated ETR is so much lower than any major operating jurisdiction's statutory rate — a small, low-taxed slice of the business is generating most of the reported profit.

12.12.1. Profit per Employee: A Sharper View of the Same Pattern

12.13. pandas

```
tax_pd["profit_per_employee"].groupby(tax_pd["jurisdiction"]).mean().sort_values(
```

```
jurisdiction
Bermuda           1642884.0
Ireland            224993.0
Singapore            8817.0
United States        8620.0
Germany              8051.0
United Kingdom       7041.0
Name: profit_per_employee, dtype: float64
```

12.14. polars

```
(
    tax_pl
    .group_by("jurisdiction")
    .agg(
        pl.col("profit_per_employee")
        .mean()
        .round(0)
        .alias("profit_per_employee")
    )
    .sort("profit_per_employee", descending=True)
)
```

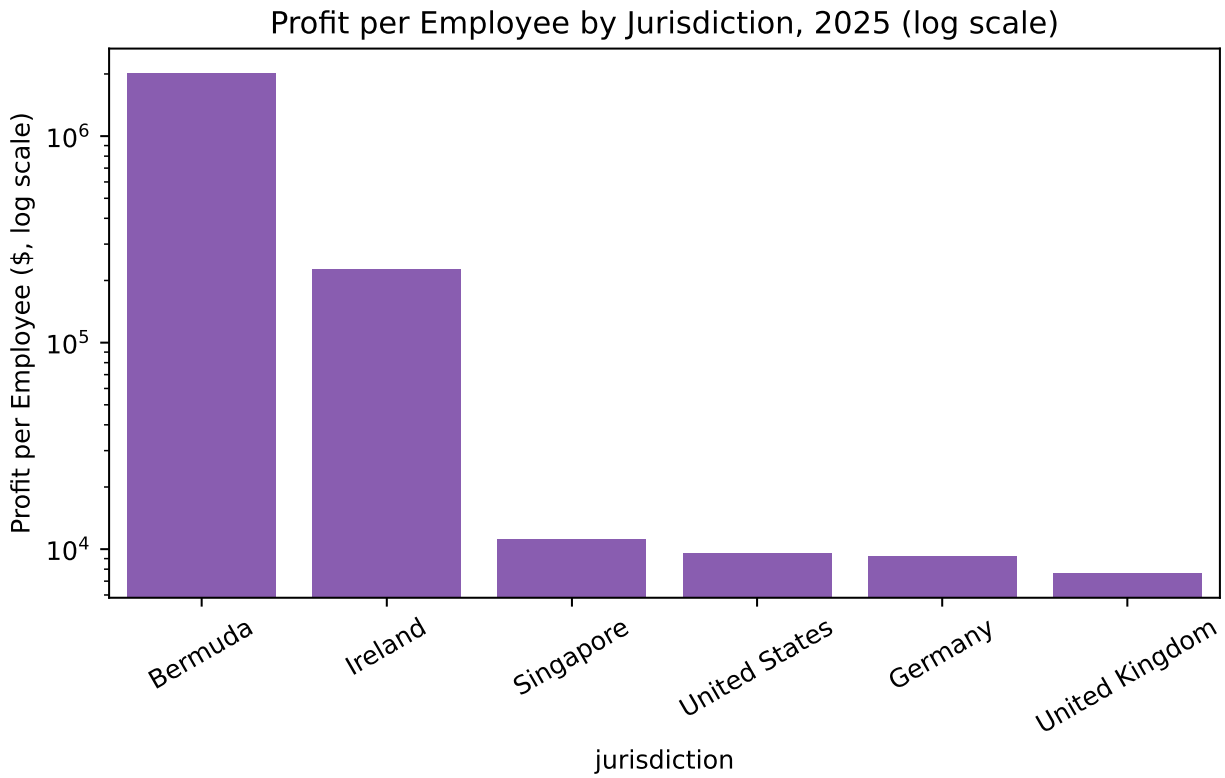
jurisdiction str	profit_per_employee f64
"Bermuda"	1.642884e6
"Ireland"	224993.0
"Singapore"	8817.0
"United States"	8620.0
"Germany"	8051.0
"United Kingdom"	7041.0

Bermuda's average profit per employee is roughly **$1.6 million** — around 150-200 times higher than the US, UK, or Germany, where profit per employee sits in the $7,000-$9,500 range. No plausible business model explains a handful of employees in Bermuda generating that much profit relative to hundreds of employees running substantial operations elsewhere; this ratio, on its own, is one of the strongest quantitative signals in this chapter.

```
snapshot_2025 = tax_pd[tax_pd["year"] ==
 ↪  2025].sort_values("profit_per_employee", ascending=False)
```

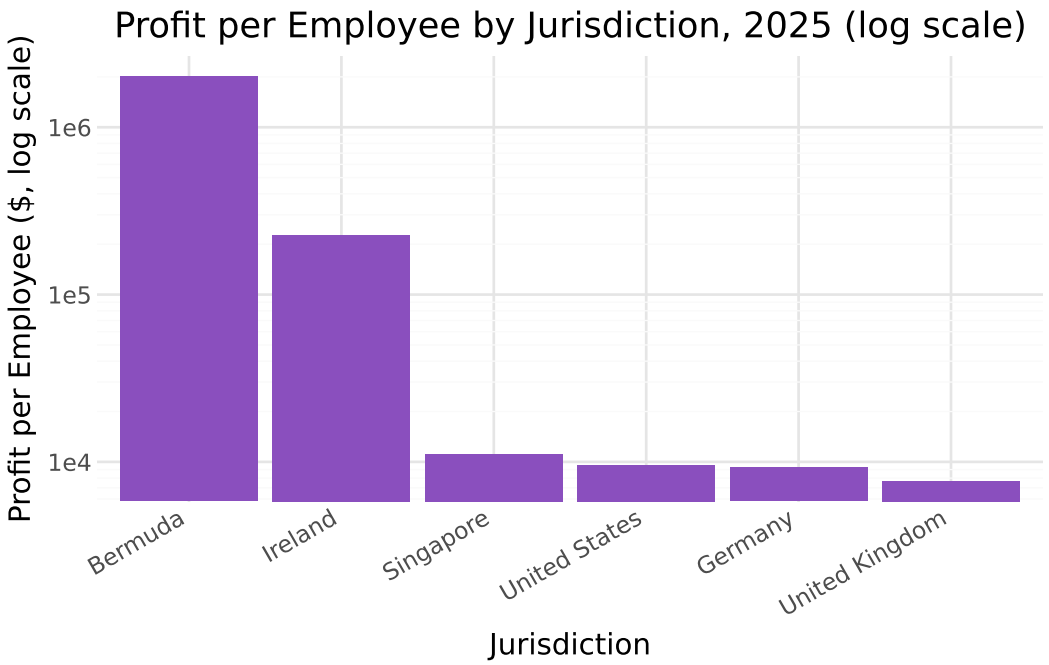
12.15. seaborn

```
plt.figure(figsize=(7, 4.5))
sns.barplot(data=snapshot_2025, x="jurisdiction",
 ↪  y="profit_per_employee", color="#8a4fbe")
plt.yscale("log")
plt.title("Profit per Employee by Jurisdiction, 2025 (log scale)")
plt.ylabel("Profit per Employee ($, log scale)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



12.16. plotnine

```
(
    ggplot(snapshot_2025, aes(x="reorder(jurisdiction,
 ↪  -profit_per_employee)", y="profit_per_employee"))
    + geom_col(fill="#8a4fbe")
    + scale_y_log10()
    + labs(title="Profit per Employee by Jurisdiction, 2025 (log
     ↪  scale)",
           x="Jurisdiction", y="Profit per Employee ($, log scale)")
    + theme_minimal()
    + theme(axis_text_x=element_text(rotation=30, hjust=1))
)
```



Note the **log scale** on this chart — without it, Bermuda's bar would be so tall that every other jurisdiction's bar would be visually indistinguishable from zero, which itself illustrates just how extreme the gap is.

12.17. Rate Reconciliation

Rate reconciliation is a formal financial statement disclosure that explains the mathematical difference between a company's domestic statutory income tax rate and its effective tax rate (ETR). For multinational corporations, it highlights how global operations, foreign tax differentials, and international provisions shift tax burdens.

Public companies disclose a **rate reconciliation** — a bridge explaining the difference between tax expense at the home-country statutory rate and the actual tax expense reported. Let's build a simplified version, using the U.S. statutory rate (21%) as the baseline.

12.18. pandas

```
us_statutory_rate = 0.21

tax_pd["tax_at_us_rate"] = tax_pd["pretax_income"] * us_statutory_rate
tax_pd["foreign_rate_benefit"] = tax_pd["tax_at_us_rate"] -
 ↪  tax_pd["tax_expense"]
```

```
contribution = (
    tax_pd.groupby("jurisdiction")["foreign_rate_benefit"]
    .sum()
    .sort_values(ascending=False)
)
contribution.round(0)
```

```
jurisdiction
Bermuda           4136757.0
Ireland           2058837.0
United States       78561.0
Singapore           64105.0
United Kingdom    -160384.0
Germany           -233554.0
Name: foreign_rate_benefit, dtype: float64
```

12.19. polars

```
import polars as pl
us_statutory_rate = 0.21
# Add calculated columns
tax_pl = (
    tax_pl
    .with_columns(
        [
            (
                pl.col("pretax_income") * us_statutory_rate
            ).alias("tax_at_us_rate"),

            (
                pl.col("pretax_income") * us_statutory_rate
                - pl.col("tax_expense")
            ).alias("foreign_rate_benefit"),
        ]
    )
)

# Summarize by jurisdiction
```

```
contribution_pl = (
    tax_pl
    .group_by("jurisdiction")
    .agg(
        pl.col("foreign_rate_benefit")
        .sum()
        .round(0)
        .alias("foreign_rate_benefit")
    )
    .sort("foreign_rate_benefit", descending=True)
)

contribution_pl
```

jurisdiction str	foreign_rate_benefit f64
”Bermuda”	4.136757e6
”Ireland”	2.058837e6
”United States”	78561.0
”Singapore”	64105.0
”United Kingdom”	-160384.0
”Germany”	-233554.0

Over the three years in this dataset, booking profit in Bermuda and Ireland rather than at the US statutory rate reduced consolidated tax expense by roughly **$4.1 million** and **$2.1 million**, respectively — figures that would typically appear as a “foreign rate differential” line item in a real rate reconciliation footnote.

12.20. pandas

```
consolidated_pd["tax_at_us_statutory"] =
 ↪  consolidated_pd["pretax_income"] * us_statutory_rate
consolidated_pd["total_rate_differential_benefit"] = (
    consolidated_pd["tax_at_us_statutory"] -
 ↪  consolidated_pd["tax_expense"]
)
consolidated_pd[["year", "consolidated_etr",
 ↪  "total_rate_differential_benefit"]].round(3)
```

	year	consolidated_etr	total_rate_differential_benefit
0	2023	0.117	1830683.88
1	2024	0.114	1877900.06
2	2025	0.113	2235737.34

12.21. polars

```
import polars as pl

us_statutory_rate = 0.21

consolidated_pl = (
    consolidated_pl
    .with_columns(
        [
            (
                pl.col("pretax_income") * us_statutory_rate
            ).alias("tax_at_us_statutory"),

            (
                pl.col("pretax_income") * us_statutory_rate
                - pl.col("tax_expense")
            ).alias("total_rate_differential_benefit"),
        ]
    )
)

(
    consolidated_pl
    .select(
        "year",
        "consolidated_etr",
        "total_rate_differential_benefit",
    )
    .with_columns(
        [
            pl.col("consolidated_etr").round(3),
```

```
            pl.col("total_rate_differential_benefit").round(3),
        ]
    )
)
```

year i64	consolidated_etr f64	total_rate_differential_benefit f64
2023	0.117	1.8307e6
2024	0.114	1.8779e6
2025	0.113	2.2357e6

12.22. Sales Tax Compliance Testing

Switching to indirect tax: `sales_tax_test_transactions.csv` contains 300 transactions across 8 U.S. states, each with a taxable amount, the applicable statutory rate, and the tax actually collected. We can recompute the *expected* tax and compare it to what was actually charged.

12.23. pandas

```
sales_pd = pd.read_csv("data/sales_tax_test_transactions.csv")
sales_pd.head()
```

	transaction_id	state	sale_amount	taxable_amount	statutory_rate	tax_collected
0	TXN4000	WA	47.46	47.46	0.0650	3.08
1	TXN4001	IL	396.50	0.00	0.0625	0.00
2	TXN4002	PA	210.94	210.94	0.0600	12.66
3	TXN4003	WA	697.70	697.70	0.0650	45.35
4	TXN4004	TX	236.63	236.63	0.0625	14.79

12.24. polars

```
sales_pl = pl.read_csv("data/sales_tax_test_transactions.csv")
sales_pl.head()
```

transaction_id str	state str	sale_amount f64	taxable_amount f64	statutory_rate f64	tax_collected f64
"TXN4000"	"WA"	47.46	47.46	0.065	3.08
"TXN4001"	"IL"	396.5	0.0	0.0625	0.0
"TXN4002"	"PA"	210.94	210.94	0.06	12.66
"TXN4003"	"WA"	697.7	697.7	0.065	45.35
"TXN4004"	"TX"	236.63	236.63	0.0625	14.79

12.25. pandas

```
sales_pd["expected_tax"] = (sales_pd["taxable_amount"] *
 ↪  sales_pd["statutory_rate"]).round(2)
sales_pd["tax_variance"] = sales_pd["tax_collected"] -
 ↪  sales_pd["expected_tax"]
sales_pd["is_discrepancy"] = sales_pd["tax_variance"].abs() > 0.5

print("Discrepancies found:", sales_pd["is_discrepancy"].sum())
sales_pd[sales_pd["is_discrepancy"]][
    ["transaction_id", "state", "taxable_amount", "statutory_rate",
 ↪  "expected_tax", "tax_collected", "tax_variance"]
].head(10)
```

```
Discrepancies found: 14
```

	transaction_id	state	taxable_amount	statutory_rate	expected_tax	tax_collected	tax_variance
5	TXN4005	NY	527.48	0.0400	21.10	33.54	12.44
31	TXN4031	CA	71.70	0.0725	5.20	2.55	-2.65
34	TXN4034	WA	201.03	0.0650	13.07	6.03	-7.04
62	TXN4062	OH	99.79	0.0575	5.74	2.51	-3.23
77	TXN4077	WA	131.91	0.0650	8.57	13.28	4.71
127	TXN4127	CA	261.13	0.0725	18.93	0.00	-18.93
191	TXN4191	CA	40.92	0.0725	2.97	3.64	0.67
201	TXN4201	NY	128.41	0.0400	5.14	3.67	-1.47
202	TXN4202	TX	139.36	0.0625	8.71	13.30	4.59
212	TXN4212	FL	415.04	0.0600	24.90	11.77	-13.13

12.26. polars

```
sales_pl = sales_pl.with_columns(
    (pl.col("taxable_amount") *
 ↪  pl.col("statutory_rate")).round(2).alias("expected_tax")
).with_columns(
    (pl.col("tax_collected") -
 ↪  pl.col("expected_tax")).alias("tax_variance")
).with_columns(
    (pl.col("tax_variance").abs() > 0.5).alias("is_discrepancy")
)

print("Discrepancies found:",
 ↪  sales_pl.filter(pl.col("is_discrepancy")).height)
sales_pl.filter(pl.col("is_discrepancy")).select(
    ["transaction_id", "state", "taxable_amount", "statutory_rate",
 ↪  "expected_tax", "tax_collected", "tax_variance"]
).head(10)
```

```
Discrepancies found: 14
```

transaction_id str	state str	taxable_amount f64	statutory_rate f64	expected_tax f64	tax_collected f64	tax_variance f64
"TXN4005"	"NY"	527.48	0.04	21.1	33.54	12.44
"TXN4031"	"CA"	71.7	0.0725	5.2	2.55	-2.65
"TXN4034"	"WA"	201.03	0.065	13.07	6.03	-7.04
"TXN4062"	"OH"	99.79	0.0575	5.74	2.51	-3.23
"TXN4077"	"WA"	131.91	0.065	8.57	13.28	4.71
"TXN4127"	"CA"	261.13	0.0725	18.93	0.0	-18.93
"TXN4191"	"CA"	40.92	0.0725	2.97	3.64	0.67
"TXN4201"	"NY"	128.41	0.04	5.14	3.67	-1.47
"TXN4202"	"TX"	139.36	0.0625	8.71	13.3	4.59
"TXN4212"	"FL"	415.04	0.06	24.9	11.77	-13.13

Both libraries agree: 14 transactions show a tax variance exceeding our $0.50 threshold, covering undercharged tax, overcharged tax, and at least one transaction where no tax was collected at all despite a taxable sale.

12.26.1. Quantifying the Exposure

12.27. pandas

```
total_undercharged = sales_pd[sales_pd["tax_variance"] <
 ↪  0]["tax_variance"].sum()
total_overcharged = sales_pd[sales_pd["tax_variance"] >
 ↪  0]["tax_variance"].sum()

print(f"Total understated tax (potential liability exposure):
 ↪  ${total_undercharged:.2f}")
print(f"Total overcharged tax (potential customer refund exposure):
 ↪  ${total_overcharged:.2f}")
```

```
Total understated tax (potential liability exposure): $-70.71
Total overcharged tax (potential customer refund exposure): $28.59
```

12.28. polars

```
# Total understated tax (negative variances)
total_undercharged = (
    sales_pl
    .filter(pl.col("tax_variance") < 0)
    .select(pl.col("tax_variance").sum())
    .item()
)

# Total overcharged tax (positive variances)
total_overcharged = (
    sales_pl
    .filter(pl.col("tax_variance") > 0)
    .select(pl.col("tax_variance").sum())
    .item()
)

print(
    f"Total understated tax (potential liability exposure): "
    f"${total_undercharged:.2f}"
```

```
)

print(
    f"Total overcharged tax (potential customer refund exposure): "
    f"${total_overcharged:.2f}"
)
```

```
Total understated tax (potential liability exposure): $-70.71
Total overcharged tax (potential customer refund exposure): $28.59
```

12.28.1. The Materiality Threshold Matters

12.29. pandas

```
sales_pd["abs_variance"] = sales_pd["tax_variance"].abs()
near_threshold = sales_pd[(sales_pd["abs_variance"] > 0.01) &
 ↪  (sales_pd["abs_variance"] <= 0.5)]
near_threshold[["transaction_id", "state", "taxable_amount",
 ↪  "tax_variance"]]
```

	transaction_id	state	taxable_amount	tax_variance
252	TXN4252	OH	110.62	0.28
292	TXN4292	OH	84.69	0.21

12.30. polars

```
# Create the absolute variance column
sales_pl = sales_pl.with_columns(
    pl.col("tax_variance")
    .abs()
    .alias("abs_variance")
)

# Filter transactions with variances near the threshold
near_threshold = (
```

```
    sales_pl
    .filter(
        (pl.col("abs_variance") > 0.01)
        & (pl.col("abs_variance") <= 0.5)
    )
    .select(
        "transaction_id",
        "state",
        "taxable_amount",
        "tax_variance",
    )
)

near_threshold
```

transaction_id str	state str	taxable_amount f64	tax_variance f64
"TXN4252"	"OH"	110.62	0.28
"TXN4292"	"OH"	84.69	0.21

! Choosing a Threshold Is a Judgment Call, Not Just a Number

These two transactions have a small but genuine tax variance (20-30 cents) that falls just under our $0.50 flagging threshold. A stricter threshold (say, $0.05) would catch them — but would likely also flag a large number of transactions where the "discrepancy" is nothing more than ordinary rounding in how tax is calculated at the point of sale. There's no universally correct threshold; it depends on transaction volume, the cost of investigating false positives, and the client's own risk tolerance. This is the same tension we saw with Isolation Forest's `contamination` parameter in Chapter 9 — every anomaly detection method, statistical or rule-based, requires a threshold decision that trades off precision against recall.

12.31. Chapter Summary

- Effective tax rate (ETR) is tax expense divided by pretax income, and can differ substantially from statutory rates due to credits, deductions, and jurisdictional mix.
- A consolidated ETR well below every major jurisdiction's statutory rate is a signal to investigate *where* profit is being reported, not just *how much* tax is being paid overall.

- Comparing a jurisdiction's share of profit to its share of revenue or headcount is a standard test for detecting disproportionate profit allocation — in this chapter, two jurisdictions with under 5% of headcount generated roughly 70% of consolidated profit.
- Profit-per-employee, viewed on a log scale given how extreme the gaps can be, is a particularly sharp way to visualize this kind of mismatch.
- A rate reconciliation quantifies exactly how much each jurisdiction's rate difference from a home-country baseline contributed to total tax savings — the same disclosure format used in real 10-K filings.
- Sales tax compliance testing follows the same recompute-and-compare logic used throughout this book: calculate the expected value independently, compare it to the recorded value, and flag material differences.
- Every flagging threshold, whether for sales tax variances or statistical anomalies, is a genuine tradeoff between catching more real issues and generating more false positives — there's no threshold that eliminates this tradeoff entirely.

12.32. Discussion Questions

1. Why is comparing a jurisdiction's share of *profit* to its share of *revenue or employees* a more informative test than simply looking at its effective tax rate in isolation?
2. If you were a financial statement analyst (not an auditor or tax authority), why might you still care about the profit-per-employee pattern shown in this chapter, even though tax minimization within the law is not illegal?
3. The sales tax materiality threshold in this chapter ($0.50 per transaction) is a judgment call. How might your answer change if you were testing 300 transactions versus 3 million?

12.33. Exercises

1. Recompute the rate reconciliation using the **UK** statutory rate (25%) as the baseline instead of the US rate. Does Bermuda and Ireland's combined contribution to the rate differential change in a way you'd expect?
2. Using `tax_provision_by_jurisdiction.csv`, compute each jurisdiction's 3-year *average* effective tax rate and compare it to its statutory rate. Which jurisdiction shows the largest gap, and is that gap consistent with what you'd expect from its business substance?
3. Lower the sales tax discrepancy threshold from $0.50 to $0.10 and rerun the compliance test. How many additional transactions get flagged, and does a quick look at them suggest genuine errors or just rounding noise?
4. Using `sales_tax_test_transactions.csv`, group discrepancies by `state` rather than looking at them transaction-by-transaction. Is any single state's tax calculation logic

disproportionately likely to be wrong, which might suggest a system configuration issue rather than isolated data entry errors?

Part IV.

Part IV: Advanced Analytics and Artificial Intelligence

13. Introduction to Predictive Modeling

Learning Objectives

By the end of this chapter, you should be able to:

- Explain the difference between regression and classification, and identify which fits a given accounting question
- Split data into training and test sets, and explain why this matters for evaluating a model honestly
- Fit and interpret a linear regression model for a continuous accounting outcome
- Fit and interpret a logistic regression model for a binary accounting outcome (financial distress)
- Evaluate regression models with MAE, RMSE, and R^2, and classification models with accuracy, precision, recall, F1, and ROC-AUC
- Recognize the "accuracy trap" that arises with imbalanced outcomes, and avoid overtrusting a single metric
- Perform all of the above using both pandas and polars, and visualize results with both seaborn and plotnine

13.1. From Descriptive to Predictive Analytics

Recall the four types of analytics from Chapter 1: descriptive, diagnostic, predictive, and prescriptive. Every chapter so far has lived mostly in the first two — summarizing what happened (Chapter 5), and investigating why (Chapters 8-11). This chapter takes the first step into **predictive analytics**: using historical data to estimate an outcome we don't yet know.

We'll build two models on two different accounting questions:

1. **Regression** — predicting a continuous outcome: how many days will it take to collect a given invoice? (using `sales_transactions.csv` from Chapter 10)
2. **Classification** — predicting a binary outcome: is this company financially distressed? (using a new dataset, `company_financial_distress.csv`)

13.2. pandas

```
import pandas as pd
import numpy as np

sales_pd = pd.read_csv("data/sales_transactions.csv",
 ↪  parse_dates=["invoice_date", "ship_date", "payment_date"])
distress_pd = pd.read_csv("data/company_financial_distress.csv")
print(sales_pd.shape, distress_pd.shape)
```

```
(427, 9) (320, 10)
```

13.3. polars

```
import polars as pl

sales_pl = pl.read_csv("data/sales_transactions.csv",
 ↪  try_parse_dates=True)
distress_pl = pl.read_csv("data/company_financial_distress.csv")
print(sales_pl.shape, distress_pl.shape)
```

```
(427, 9) (320, 10)
```

13.4. Regression: Predicting Days to Collect

13.4.1. Feature Engineering

13.5. pandas

```
paid_pd = sales_pd[sales_pd["is_paid"]].copy()
paid_pd["days_to_collect"] = (paid_pd["payment_date"] -
 ↪  paid_pd["invoice_date"]).dt.days
paid_pd["ship_lag_days"] = (paid_pd["ship_date"] -
 ↪  paid_pd["invoice_date"]).dt.days
paid_pd["log_amount"] = np.log1p(paid_pd["amount"])
```

```
paid_pd[["amount", "payment_terms_days", "quarter", "ship_lag_days",
 ↪  "days_to_collect"]].describe()
```

	amount	payment_terms_days	quarter	ship_lag_days	days_to_collect
count	360.000000	360.000000	360.000000	360.000000	360.000000
mean	5014.964194	47.791667	2.250000	-0.822222	51.338889
std	5143.852818	11.809700	1.028155	2.543560	36.141653
min	841.050000	30.000000	1.000000	-2.000000	5.000000
25%	2426.720000	45.000000	1.000000	-2.000000	27.000000
50%	3892.305000	45.000000	2.000000	-1.000000	41.000000
75%	6102.247500	60.000000	3.000000	0.000000	61.000000
max	62000.000000	60.000000	4.000000	30.000000	209.000000

13.6. polars

```
paid_pl = sales_pl.filter(pl.col("is_paid")).with_columns([
    (pl.col("payment_date") -
 ↪  pl.col("invoice_date")).dt.total_days().alias("days_to_collect"),
    (pl.col("ship_date") -
 ↪  pl.col("invoice_date")).dt.total_days().alias("ship_lag_days"),
    pl.col("amount").log1p().alias("log_amount"),
])

paid_pl.select(["amount", "payment_terms_days", "quarter",
 ↪  "ship_lag_days", "days_to_collect"]).describe()
```

statistic str	amount f64	payment_terms_days f64	quarter f64	ship_lag_days f64	days_to_collect f64
"count"	360.0	360.0	360.0	360.0	360.0
"null_count"	0.0	0.0	0.0	0.0	0.0
"mean"	5014.964194	47.791667	2.25	-0.822222	51.338889
"std"	5143.852818	11.8097	1.028155	2.54356	36.141653
"min"	841.05	30.0	1.0	-2.0	5.0
"25%"	2429.3	45.0	1.0	-2.0	27.0
"50%"	3931.69	45.0	2.0	-1.0	41.0
"75%"	6079.72	60.0	3.0	0.0	61.0

statistic str	amount f64	payment_terms_days f64	quarter f64	ship_lag_days f64	days_to_collect f64
"max"	62000.0	60.0	4.0	30.0	209.0

> **Connect to Practice**
>
> Recall from Chapter 10 that restricting to *paid* invoices introduces survivorship bias when comparing DSO across recent periods. The same caution applies here: this model is trained only on invoices that have already been collected, which may not perfectly represent the invoices still outstanding. Keep this in mind when interpreting the model's real-world usefulness later in this chapter.

13.6.1. Train/Test Split

Before fitting any model, we set aside a portion of the data the model never sees during training, so we can honestly evaluate how well it generalizes to new invoices.

```
from sklearn.model_selection import train_test_split

features_reg = ["log_amount", "payment_terms_days", "quarter",
 ↪  "ship_lag_days"]
X = paid_pd[features_reg]
y = paid_pd["days_to_collect"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
 ↪  test_size=0.25, random_state=42)
print(f"Training set: {len(X_train)} invoices, Test set: {len(X_test)}
 ↪  invoices")
```

```
Training set: 270 invoices, Test set: 90 invoices
```

> **Why Not Just Use All the Data?**
>
> If we fit a model on all 360 invoices and evaluated it on those same 360 invoices, we'd be measuring how well the model *memorized* this specific data, not how well it will perform on the next new invoice. Setting aside a test set the model never sees during training is the standard way to get an honest estimate of real-world performance.

13.6.2. Fitting and Interpreting the Model

13.7. scikit-learn

```
from sklearn.linear_model import LinearRegression

reg_model = LinearRegression()
reg_model.fit(X_train, y_train)

for feature, coef in zip(features_reg, reg_model.coef_):
    print(f"{feature}: {coef:.3f}")
print(f"Intercept: {reg_model.intercept_:.3f}")
```

```
log_amount: -3.263
payment_terms_days: 0.971
quarter: -6.214
ship_lag_days: 0.135
Intercept: 44.940
```

13.8. pyfixest

```
import pyfixest as pf
from pyfixest.estimation import feols

model = feols("days_to_collect ~ log_amount + payment_terms_days +
 ↪  quarter + ship_lag_days", data = paid_pd)

pf.etable([model])
```

	days_to_collect (1)
coef	
log_amount	-4.814
	(2.840)
payment_terms_days	0.903***
	(0.152)

	days_to_collect (1)
quarter	-7.059***
	(1.740)
ship_lag_days	0.220
	(0.734)
Intercept	64.088*
	(25.452)
stats	
Observations	360
R^2	0.158

Significance levels: * p < 0.05, ** p < 0.01, *** p < 0.001. Format of coefficient cell: Coefficient (Std. Error)

```
model.summary()
```

```
###

Estimation:  OLS
Dep. var.: days_to_collect
sample: None = all
Inference:  iid
Observations:  360

| Coefficient        |   Estimate |   Std. Error |   t value |
Pr(>|t|) |    2.5% |   97.5% |
|:-------------------|-----------:|-------------:|----------:|-----------:|------
| Intercept          |     64.088 |       25.452 |     2.518 |
0.012 |  14.031 | 114.144 |
| log_amount         |     -4.814 |        2.840 |    -1.695 |
0.091 | -10.399 |   0.772 |
| payment_terms_days |      0.903 |        0.152 |     5.957 |
0.000 |   0.605 |   1.201 |
| quarter            |     -7.059 |        1.740 |    -4.057 |
0.000 | -10.480 |  -3.637 |
| ship_lag_days      |      0.220 |        0.734 |     0.300 |
0.764 |  -1.223 |   1.663 |
---
RMSE: 33.116 R2: 0.158
```

```
pf.summary(model)
```

```
###

Estimation:  OLS
Dep. var.: days_to_collect
sample: None = all
Inference:  iid
Observations:  360

| Coefficient         |   Estimate |   Std. Error |   t value |
Pr(>|t|) |    2.5% |   97.5% |
|:--------------------|-----------:|-------------:|----------:|-----------:|-----------:|------
| Intercept           |     64.088 |       25.452 |     2.518 |
0.012 |  14.031 | 114.144 |
| log_amount          |     -4.814 |        2.840 |    -1.695 |
0.091 | -10.399 |   0.772 |
| payment_terms_days |      0.903 |        0.152 |     5.957 |
0.000 |   0.605 |   1.201 |
| quarter             |     -7.059 |        1.740 |    -4.057 |
0.000 | -10.480 |  -3.637 |
| ship_lag_days       |      0.220 |        0.734 |     0.300 |
0.764 |  -1.223 |   1.663 |
---
RMSE: 33.116 R2: 0.158
```

```
model.tidy()
```

Coefficient	Estimate	Std. Error	t value	Pr(>|t|)	2.5%	97.5%
Intercept	64.087655	25.452490	2.517933	1.224390e-02	14.031034	114.144276
log_amount	-4.813551	2.840305	-1.694730	9.100358e-02	-10.399491	0.772389
payment_terms_days	0.902613	0.151513	5.957333	6.175350e-09	0.604638	1.200589
quarter	-7.058792	1.739809	-4.057223	6.109975e-05	-10.480419	-3.637164
ship_lag_days	0.220201	0.733691	0.300128	7.642549e-01	-1.222725	1.663128

```
model.coef()
model.se()
model.tstat()
```

```
model.confint()
```

	2.5%	97.5%
Intercept	14.031034	114.144276
log_amount	-10.399491	0.772389
payment_terms_days	0.604638	1.200589
quarter	-10.480419	-3.637164
ship_lag_days	-1.222725	1.663128

```
model.coefplot()
# or pf.coefplot(model)
```

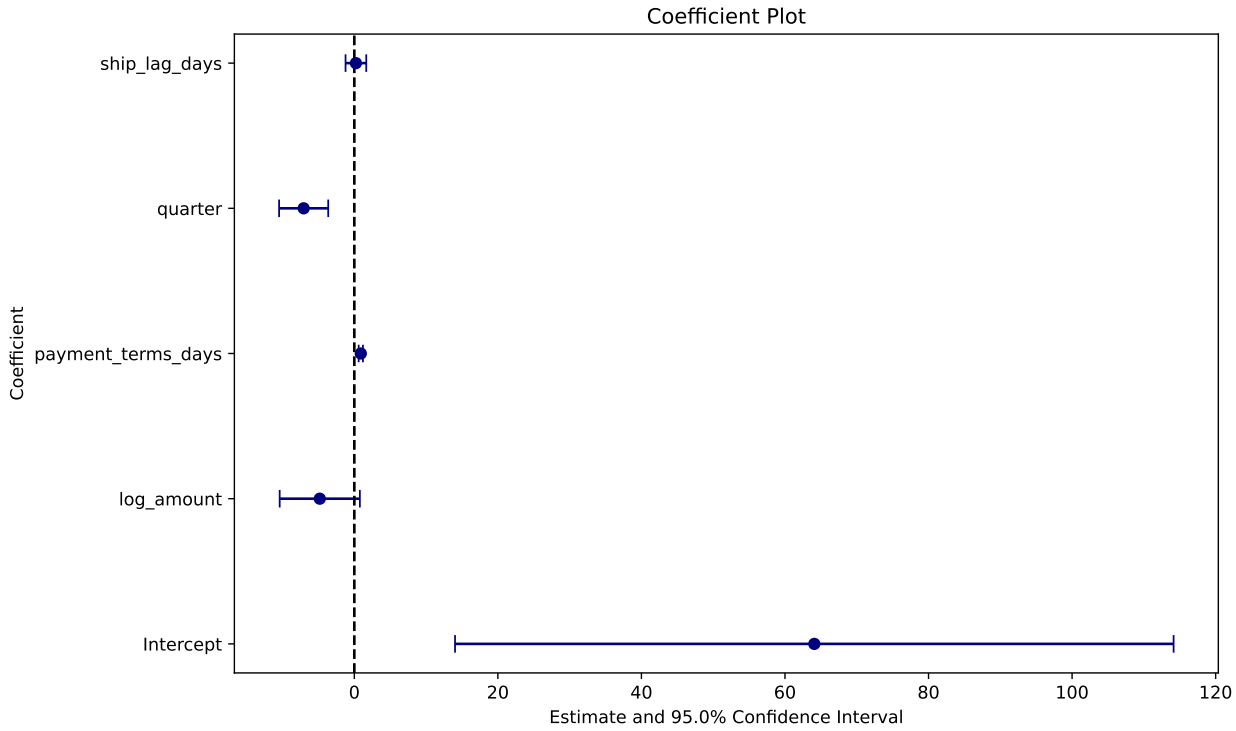


Figure 13.1.: Factors Affecting Days to Collect (DTC)

A few coefficients are worth reading carefully:

- **`payment_terms_days` (+0.97)**: each additional day of payment terms offered is associated with almost exactly one additional day to collect — customers largely pay according to the terms they're given, which is reassuring and expected.

- **`quarter`** (**-6.2**): later quarters are associated with *faster* collection. This should look familiar — it's the same declining-DSO pattern from Chapter 10, which we already know is partly a survivorship bias artifact from measuring near a fixed cutoff date, not necessarily a genuine operational improvement.
- **`log_amount`** (**-3.3**): larger invoices tend to be collected slightly faster, plausibly because larger balances get more collections attention.

Figure 13.1 presents the importance of coefficients.

13.8.1. Evaluating the Model

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
 ↪  r2_score

y_pred = reg_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"MAE:  {mae:.1f} days")
print(f"RMSE: {rmse:.1f} days")
print(f"R²:   {r2:.3f}")
```

```
MAE:  23.7 days
RMSE: 34.8 days
R²:   0.141
```

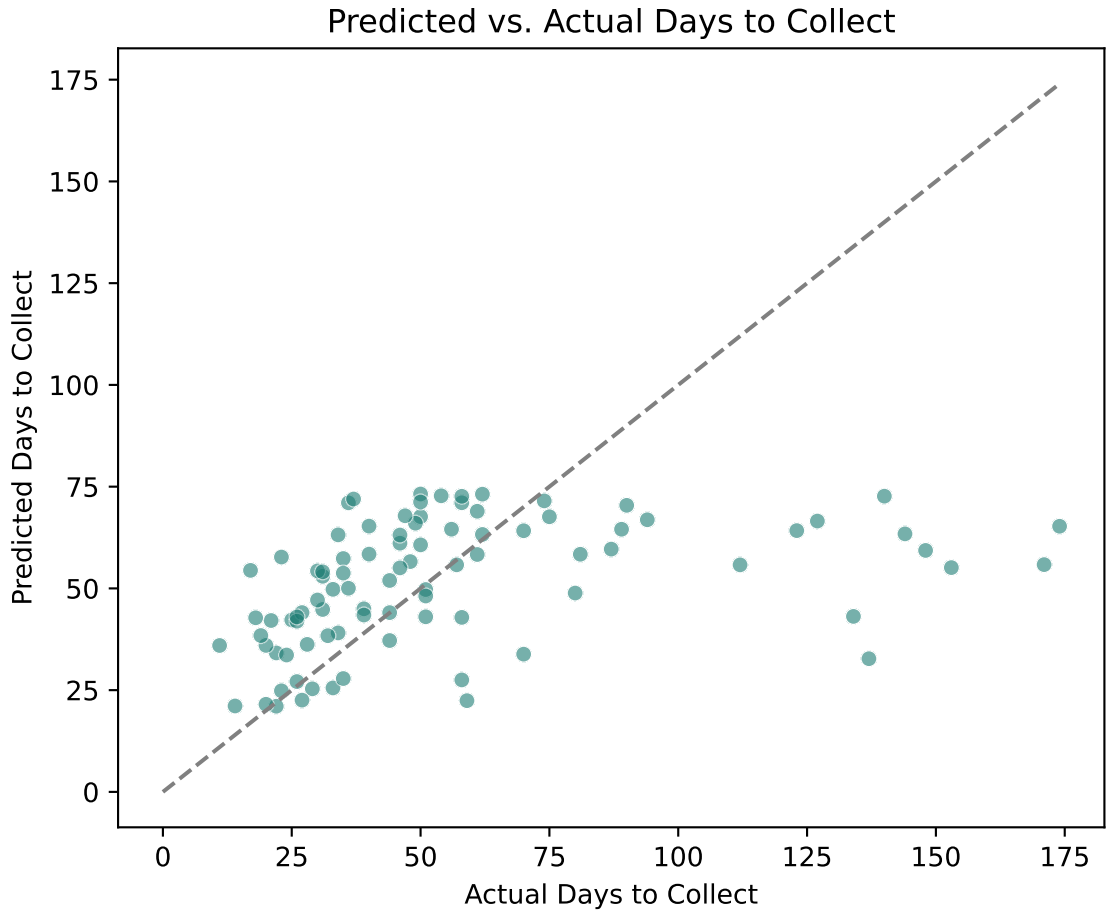
```
import seaborn as sns
import matplotlib.pyplot as plt

pred_df = pd.DataFrame({"actual": y_test, "predicted": y_pred})
```

13.9. seaborn

```
plt.figure(figsize=(6, 5))
sns.scatterplot(data=pred_df, x="actual", y="predicted", alpha=0.6,
 ↪  color="#1c7c73")
```

```
lims = [0, max(pred_df["actual"].max(), pred_df["predicted"].max())]
plt.plot(lims, lims, "--", color="gray")
plt.title("Predicted vs. Actual Days to Collect")
plt.xlabel("Actual Days to Collect")
plt.ylabel("Predicted Days to Collect")
plt.tight_layout()
plt.show()
```

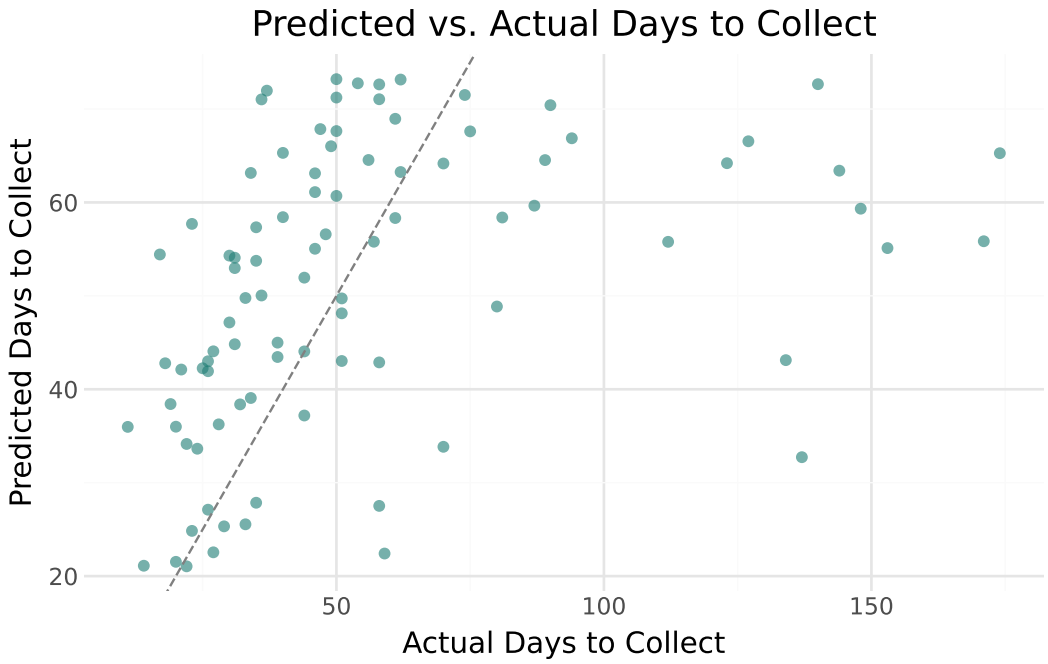


13.10. plotnine

```
from plotnine import *

(
    ggplot(pred_df, aes(x="actual", y="predicted"))
    + geom_point(alpha=0.6, color="#1c7c73")
```

```
    + geom_abline(slope=1, intercept=0, linetype="dashed",
     ↪  color="gray")
    + labs(title="Predicted vs. Actual Days to Collect", x="Actual
     ↪  Days to Collect", y="Predicted Days to Collect")
    + theme_minimal()
)
```



! An Honest Look at a Weak Model

An R^2 of roughly 0.14 means this model explains only about **14% of the variation** in how long invoices take to collect — the rest is essentially unmodeled from these four features alone. The scatter plot makes this concrete: predictions cluster in a fairly narrow band regardless of the actual outcome, which swings far more widely. This is a realistic and important result, not a failure of the chapter. **Not every accounting prediction problem yields a strong model**, and reporting a weak R^2 honestly is far more useful than cherry-picking features until a chart looks better. Collection timing likely depends on factors we haven't captured here at all — a specific customer's cash position, an ongoing dispute, or a change in the customer's own collections priorities.

13.11. Classification: Predicting Financial Distress

Our second dataset, `company_financial_distress.csv`, contains financial ratios for 320 companies along with a binary outcome: whether each company experienced **financial distress** in the following year. The ratios mirror those used in classic bankruptcy prediction research (e.g., the Altman Z-score): liquidity, leverage, profitability, and efficiency measures.

```
distress_pd.head()
```

	company_id	current_ratio	debt_to_equity	roa	net_margin	asset_turnover	interest_coverag
0	CO1000	3.07	0.29	0.1017	0.1413	1.15	8.65
1	CO1001	1.29	0.72	-0.1067	0.0564	0.20	14.20
2	CO1002	1.04	0.65	0.1469	-0.1050	2.17	-1.08
3	CO1003	1.52	0.89	-0.0288	0.0602	1.36	5.83
4	CO1004	0.99	0.70	-0.1236	-0.1677	1.29	7.05

```
print("Distress rate:",
 ↪  distress_pd["financial_distress"].mean().round(3))
distress_pd["financial_distress"].value_counts()
```

```
Distress rate: 0.169

financial_distress
0    266
1     54
Name: count, dtype: int64
```

Only about 17% of companies in this dataset are labeled distressed — an imbalance worth keeping in mind, since it will matter a great deal when we evaluate the model shortly.

13.11.1. Comparing Distressed vs. Healthy Companies

13.12. pandas

```
distress_pd.groupby("financial_distress")[
    ["current_ratio", "debt_to_equity", "roa", "interest_coverage"]
].mean().round(3)
```

financial_distress	current_ratio	debt_to_equity	roa	interest_coverage
0	1.845	0.857	0.063	6.614
1	1.538	1.354	0.019	5.231

13.13. polars

```
distress_pl.group_by("financial_distress").agg([
    pl.col("current_ratio").mean(),
    pl.col("debt_to_equity").mean(),
    pl.col("roa").mean(),
    pl.col("interest_coverage").mean(),
]).sort("financial_distress")
```

financial_distress i64	current_ratio f64	debt_to_equity f64	roa f64	interest_coverage f64
0	1.845489	0.856729	0.062801	6.614211
1	1.537593	1.353704	0.018669	5.231296

The direction of every difference matches financial theory: distressed companies show a lower current ratio (less liquidity), higher debt-to-equity (more leverage), lower ROA (less profitable), and lower interest coverage (less cushion to service debt).

13.13.1. Train/Test Split and Standardization

```
features_clf = [
    "current_ratio", "debt_to_equity", "roa", "net_margin",
    "asset_turnover", "interest_coverage",
     ↪  "working_capital_to_assets", "retained_earnings_to_assets",
]

Xc = distress_pd[features_clf]
yc = distress_pd["financial_distress"]

Xc_train, Xc_test, yc_train, yc_test = train_test_split(
```

```
    Xc, yc, test_size=0.25, random_state=42, stratify=yc
)
print("Train distress rate:", yc_train.mean().round(3), " Test
 ↪  distress rate:", yc_test.mean().round(3))
```

```
Train distress rate: 0.171  Test distress rate: 0.162
```

Notice `stratify=yc` — this ensures the train and test sets both preserve the same 17% distress rate, rather than risking a split where, by chance, the test set ends up with very few (or zero) distressed companies.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
Xc_train_scaled = scaler.fit_transform(Xc_train)
Xc_test_scaled = scaler.transform(Xc_test)
```

> **Why Standardize for Logistic Regression?**
>
> Our features span very different scales — `debt_to_equity` typically ranges from 0 to 4, while `interest_coverage` ranges from -5 to 30. Standardizing (subtracting the mean, dividing by the standard deviation) puts every feature on a comparable scale, which makes the resulting coefficients directly comparable to each other — a larger coefficient genuinely means a stronger effect, not just a feature with bigger raw numbers.

13.13.2. Fitting the Model

13.14. scikit-learn

```
from sklearn.linear_model import LogisticRegression

clf_model = LogisticRegression(random_state=42)
clf_model.fit(Xc_train_scaled, yc_train)

coef_df = pd.DataFrame({"feature": features_clf, "coefficient":
 ↪  clf_model.coef_[0]}).sort_values("coefficient")
coef_df["direction"] = coef_df["coefficient"].apply(lambda c:
 ↪  "Increases Risk" if c > 0 else "Decreases Risk")
```

```
coef_df
```

	feature	coefficient	direction
0	current_ratio	-0.738514	Decreases Risk
2	roa	-0.656879	Decreases Risk
3	net_margin	-0.614102	Decreases Risk
7	retained_earnings_to_assets	-0.376256	Decreases Risk
5	interest_coverage	-0.300631	Decreases Risk
6	working_capital_to_assets	-0.179103	Decreases Risk
4	asset_turnover	0.011675	Increases Risk
1	debt_to_equity	1.206548	Increases Risk

13.15. pyfixest

```
model_2 = pf.feglm("financial_distress ~ current_ratio +
 ↪  debt_to_equity + roa + net_margin + asset_turnover +
 ↪  interest_coverage +  working_capital_to_assets +
 ↪  retained_earnings_to_assets", data = distress_pd, family =
 ↪  "logit")

pf.etable([model_2])
```

	financial_distress (1)
coef	
current_ratio	-0.885***
	(0.258)
debt_to_equity	2.100***
	(0.365)
roa	-9.956***
	(2.487)
net_margin	-8.285***
	(2.217)
asset_turnover	0.017
	(0.497)

	financial_distress (1)
interest_coverage	-0.086*
	(0.042)
working_capital_to_assets	-2.079
	(1.247)
retained_earnings_to_assets	-2.011*
	(0.958)
Intercept	-0.567
	(0.851)
stats	
Observations	320
R^2	-

Significance levels: * p < 0.05, ** p < 0.01, *** p < 0.001. Format of coefficient cell: Coefficient (Std. Error)

```
pf.summary(model_2)
```

```
###

Estimation:  Logit
Dep. var.: financial_distress
sample: None = all
Inference:  iid
Observations:  320

| Coefficient                 |   Estimate |   Std. Error |   t value
|   Pr(>|t|) |    2.5% |   97.5% |
|:---------------------------|----------:|-------------:|----------:|----------
| Intercept                   |     -0.567 |        0.851 |    -0.666
|      0.505 |  -2.235 |   1.101 |
| current_ratio               |     -0.885 |        0.258 |    -3.426
|      0.001 |  -1.392 |  -0.379 |
| debt_to_equity              |      2.100 |        0.365 |     5.761
|      0.000 |   1.386 |   2.815 |
| roa                         |     -9.956 |        2.487 |    -4.002
|      0.000 | -14.831 |  -5.080 |
| net_margin                  |     -8.285 |        2.217 |    -3.737
|      0.000 | -12.631 |  -3.940 |
```

```
| asset_turnover               |      0.017 |        0.497 |     0.035
|      0.972 |  -0.956 |   0.991 |
| interest_coverage            |     -0.086 |        0.042 |    -2.075
|      0.038 |  -0.168 |  -0.005 |
| working_capital_to_assets    |     -2.079 |        1.247 |    -1.667
|      0.096 |  -4.524 |   0.366 |
| retained_earnings_to_assets |     -2.011 |        0.958 |    -2.098
|      0.036 |  -3.889 |  -0.133 |
---
Deviance: 198.241
```

```
model_2.tidy()
```

Coefficient	Estimate	Std. Error	t value	Pr(>|t|)	2.5%	97.5%
Intercept	-0.566756	0.850955	-0.666024	5.053960e-01	-2.234597	1.101085
current_ratio	-0.885488	0.258488	-3.425646	6.133381e-04	-1.392115	-0.378861
debt_to_equity	2.100229	0.364566	5.760905	8.366416e-09	1.385693	2.814765
roa	-9.955580	2.487427	-4.002361	6.271354e-05	-14.830847	-5.080313
net_margin	-8.285320	2.217174	-3.736883	1.863154e-04	-12.630901	-3.939739
asset_turnover	0.017232	0.496592	0.034700	9.723188e-01	-0.956070	0.990533
interest_coverage	-0.086300	0.041594	-2.074849	3.800056e-02	-0.167822	-0.004778
working_capital_to_assets	-2.079107	1.247389	-1.666767	9.556068e-02	-4.523944	0.365730
retained_earnings_to_assets	-2.010975	0.958423	-2.098213	3.588634e-02	-3.889448	-0.132501

```
model_2.coefplot()
```

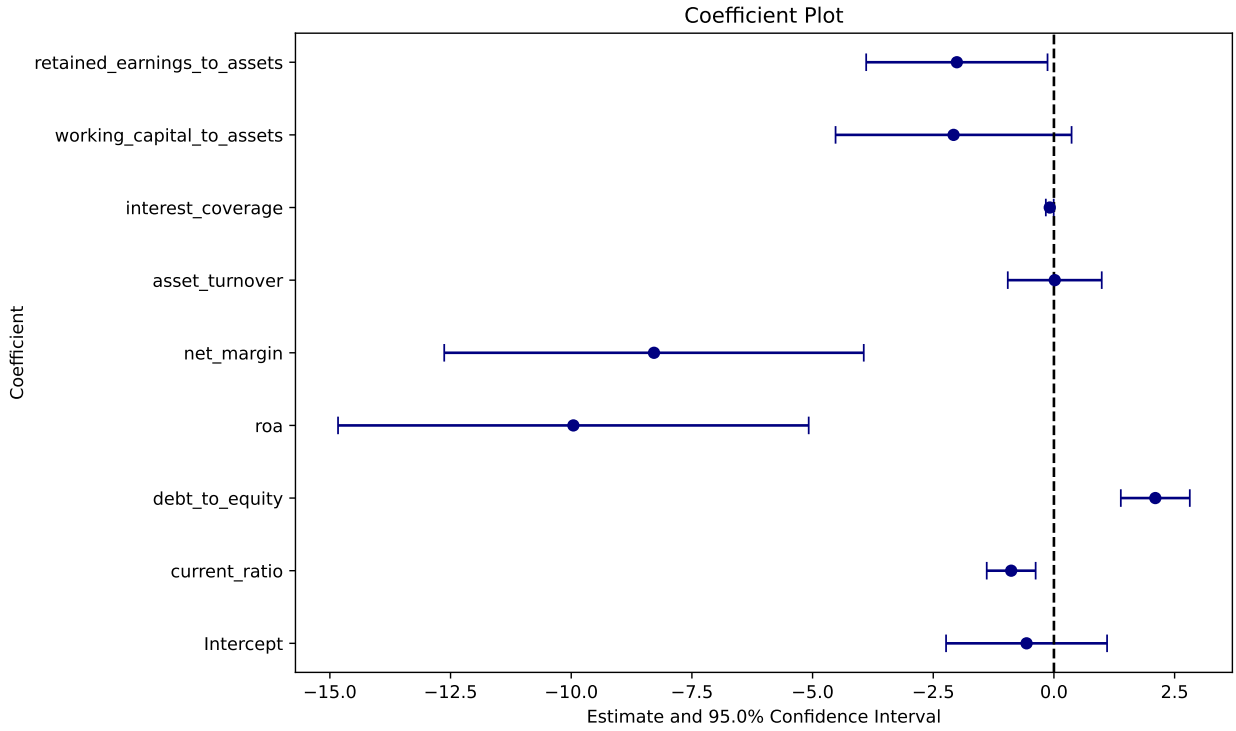
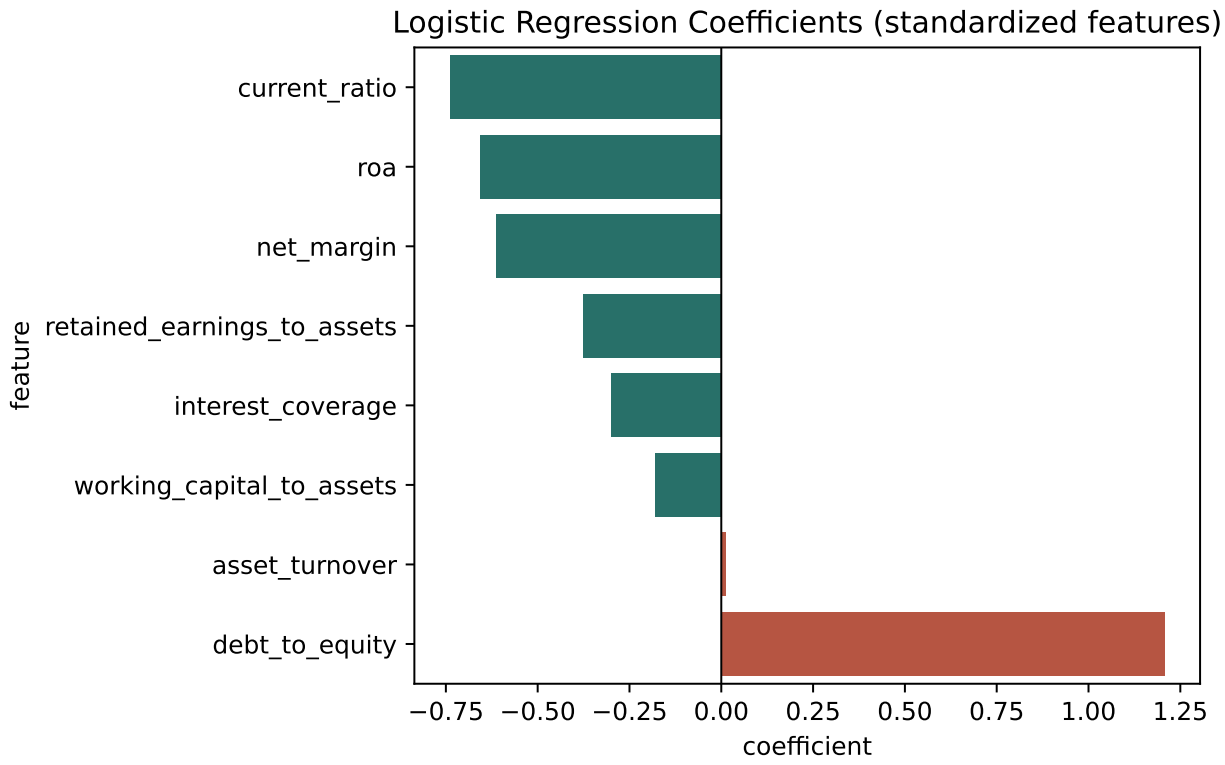


Figure 13.2.: Factors Affecting Financial Distress of a Company

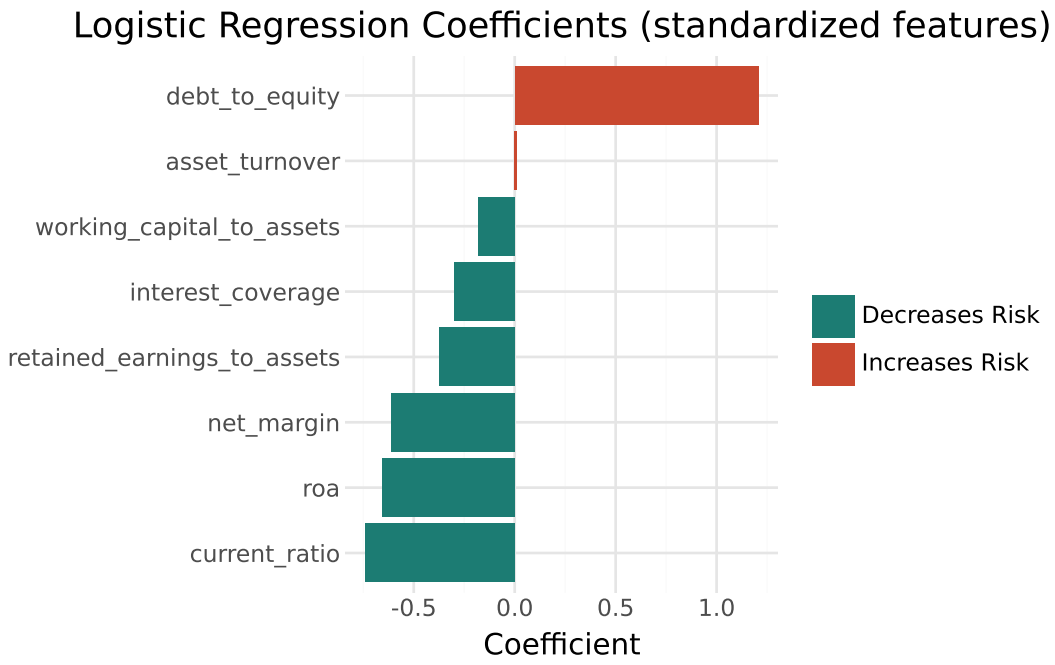
13.16. seaborn

```
plt.figure(figsize=(7, 4.5))
sns.barplot(
    data=coef_df, x="coefficient", y="feature", hue="direction",
    palette={"Increases Risk": "#c9482f", "Decreases Risk":
 ↪  "#1c7c73"}, legend=False
)
plt.title("Logistic Regression Coefficients (standardized features)")
plt.axvline(0, color="black", linewidth=0.8)
plt.tight_layout()
plt.show()
```



13.17. plotnine

```
(
    ggplot(coef_df, aes(x="reorder(feature, coefficient)",
 ↪  y="coefficient", fill="direction"))
    + geom_col()
    + coord_flip()
    + scale_fill_manual(values={"Increases Risk": "#c9482f",
     ↪  "Decreases Risk": "#1c7c73"})
    + labs(title="Logistic Regression Coefficients (standardized
     ↪  features)", x="", y="Coefficient", fill="")
    + theme_minimal()
)
```



`debt_to_equity` has the largest positive coefficient (higher leverage → higher distress risk), while `current_ratio` and `roa` show the largest negative coefficients (more liquidity and profitability → lower risk) — all consistent with the differences we saw in the group comparison above. Figure 13.2 presents the importance of coefficients.

13.17.1. Evaluating the Model

```
from sklearn.metrics import accuracy_score, precision_score,
 ↪  recall_score, f1_score, roc_auc_score, confusion_matrix

yc_pred = clf_model.predict(Xc_test_scaled)
yc_prob = clf_model.predict_proba(Xc_test_scaled)[:, 1]

print("Accuracy: ", round(accuracy_score(yc_test, yc_pred), 3))
print("Precision:", round(precision_score(yc_test, yc_pred), 3))
print("Recall:   ", round(recall_score(yc_test, yc_pred), 3))
print("F1:       ", round(f1_score(yc_test, yc_pred), 3))
print("ROC-AUC:  ", round(roc_auc_score(yc_test, yc_prob), 3))
```

```
Accuracy:  0.887
Precision: 0.75
Recall:    0.462
F1:        0.571
```

```
ROC-AUC:    0.836
```

> **!** The Accuracy Trap
>
> 88.8% accuracy sounds impressive — until you notice that simply predicting "no distress" for *every single company* would already achieve **83.75% accuracy**, since only about 16% of companies in the test set are actually distressed. Our model's real improvement over that naive baseline is much smaller than the headline accuracy number suggests.
> This is exactly why **precision, recall, and ROC-AUC matter more than accuracy alone** whenever the outcome you're predicting is rare — which describes most consequential accounting prediction problems (fraud, bankruptcy, default). A model can post a very high accuracy score while still being nearly useless at its actual job: identifying the rare cases that matter.

```
baseline_accuracy = 1 - yc_test.mean()
print(f"Baseline accuracy (always predict 'no distress'):
 ↪  {baseline_accuracy:.3f}")
```

```
Baseline accuracy (always predict 'no distress'): 0.838
```

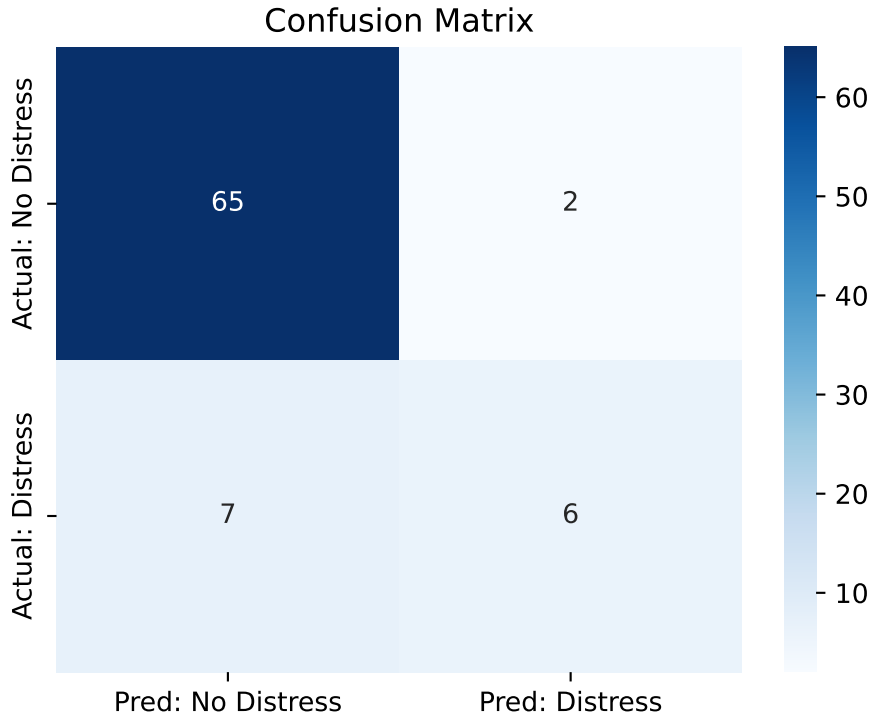
Our **recall** of roughly 0.46 means the model correctly identifies fewer than half of the companies that actually become distressed — a real limitation worth stating plainly rather than glossing over.

13.17.2. Confusion Matrix and ROC Curve

```
cm = confusion_matrix(yc_test, yc_pred)
cm_df = pd.DataFrame(
    cm,
    index=["Actual: No Distress", "Actual: Distress"],
    columns=["Pred: No Distress", "Pred: Distress"],
)
cm_df
```

	Pred: No Distress	Pred: Distress
Actual: No Distress	65	2
Actual: Distress	7	6

```
plt.figure(figsize=(5, 4))
sns.heatmap(cm_df, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()
```



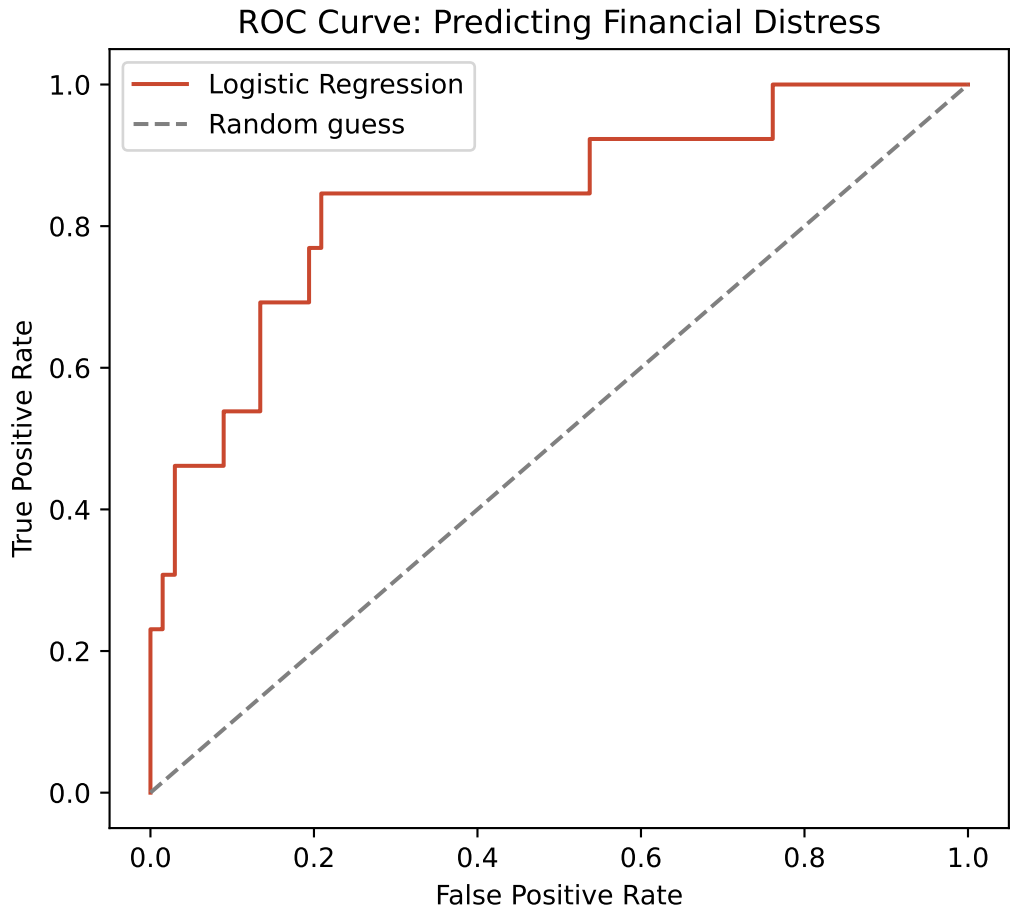

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(yc_test, yc_prob)
roc_df = pd.DataFrame({"fpr": fpr, "tpr": tpr})
```

13.18. seaborn

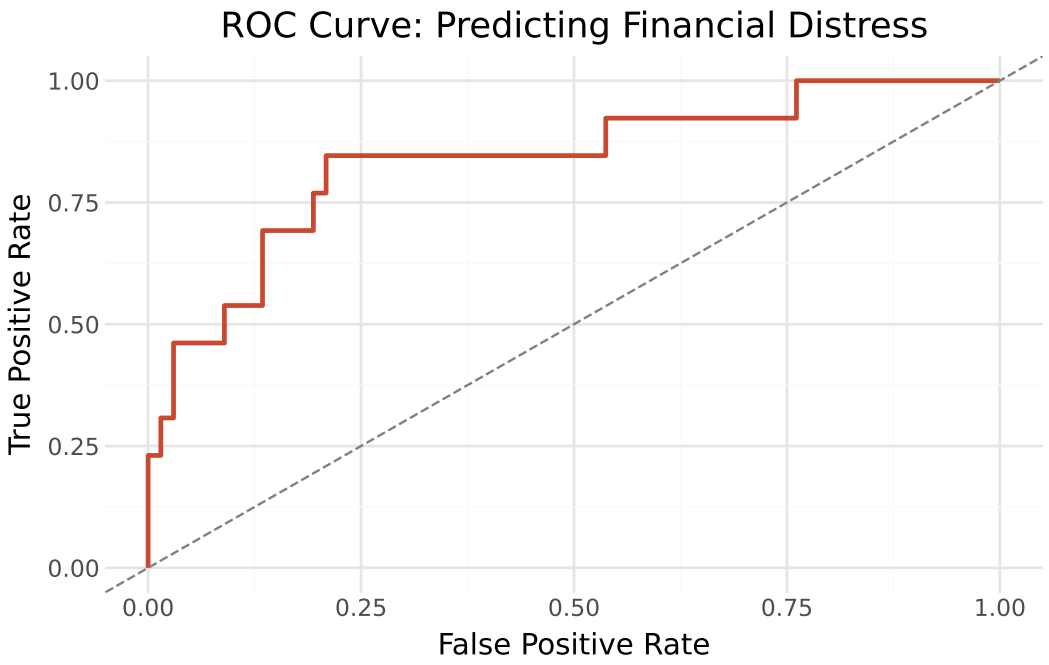
```
plt.figure(figsize=(5.5, 5))
plt.plot(fpr, tpr, color="#c9482f", label="Logistic Regression")
plt.plot([0, 1], [0, 1], "--", color="gray", label="Random guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve: Predicting Financial Distress")
```

```
plt.legend()
plt.tight_layout()
plt.show()
```



13.19. plotnine

```
(
    ggplot(roc_df, aes(x="fpr", y="tpr"))
    + geom_line(color="#c9482f", size=1)
    + geom_abline(slope=1, intercept=0, linetype="dashed",
     ↪  color="gray")
    + labs(title="ROC Curve: Predicting Financial Distress", x="False
     ↪  Positive Rate", y="True Positive Rate")
    + theme_minimal()
)
```



A ROC-AUC of 0.84 means the model does meaningfully better than random guessing at ranking companies by risk (an AUC of 0.5 would be no better than a coin flip), even though its **recall** at the default 50% probability threshold is modest. This is a useful reminder that a model's *ranking* ability and its performance at any *one specific threshold* are related but distinct questions — a point we return to in the exercises.

13.20. Chapter Summary

- Predictive analytics uses historical data to estimate an outcome that hasn't happened yet, building on the descriptive and diagnostic foundations from earlier chapters.
- Regression predicts a continuous outcome (days to collect); classification predicts a categorical outcome (financial distress) — the right choice depends entirely on the nature of the outcome you're trying to predict.
- Splitting data into training and test sets is essential for honestly evaluating whether a model generalizes, rather than just memorizing the data it was fit on.
- A weak regression result (R^2 of 0.14) is a legitimate and informative finding — not every prediction problem yields a strong model, and honestly reporting a weak fit is more valuable than manufacturing an artificially strong one.
- Standardizing features before logistic regression puts coefficients on a comparable scale, making it possible to compare which factors matter most.
- With imbalanced outcomes (here, ~17% distress), accuracy alone can be deeply misleading — a naive "always predict the majority class" baseline can already achieve high accuracy, so precision, recall, F1, and ROC-AUC are essential companions to any accuracy figure.

13.21. Discussion Questions

1. The regression model's R^2 was only 0.14. If you were presenting this result to a manager who wanted a "good" model, how would you explain why this result is still useful, rather than a failure?
2. Our classification model's recall was about 0.46 — it misses more than half of truly distressed companies. If you were designing a real early-warning system, would you rather have higher recall (catch more distressed companies, at the cost of more false alarms) or higher precision (fewer false alarms, but miss more real cases)? What accounting or business context would change your answer?
3. Explain, in your own words, why a model can have high accuracy but still be practically useless, using the baseline accuracy comparison from this chapter as your example.

13.22. Exercises

1. Add `quarter` as a categorical (one-hot encoded) feature instead of a numeric one in the regression model. Does R^2 improve, and does the interpretation of the `quarter` effect change?
2. Refit the logistic regression model using only four features of your choice (rather than all eight). How much do accuracy, recall, and ROC-AUC change compared to the full model?
3. The default classification threshold is 50% predicted probability. Using `clf_model.predict_proba()` manually apply a lower threshold (e.g., 30%) to classify companies as distressed, and recompute precision and recall. How do they change, and why?
4. Using `paid_pd`, engineer one new feature not used in this chapter (for example, a flag for whether the customer is one of the four high-risk customers identified in Chapter 10) and add it to the regression model. Does it improve R^2 meaningfully?

14. Text Analytics in Accounting

Learning Objectives

By the end of this chapter, you should be able to:

- Explain where unstructured text data shows up in accounting practice, and why it requires different techniques than the numeric, tabular data used throughout this book
- Preprocess raw text into an analyzable form: tokenization, stopword removal, and stemming
- Convert text into numeric features using bag-of-words and TF-IDF
- Apply sentiment analysis to financial narrative text, and explain why general-purpose sentiment tools often misread financial language
- Build a text classification model using the same train/test/evaluate workflow from Chapter 13, applied to text instead of numeric ratios
- Recognize the specific limitations of classical text analytics techniques in an accounting context

14.1. From Structured to Unstructured Data

Every dataset in this book so far has been **structured**: rows and columns, numbers and categories, ready to filter, group, and model. A huge and growing share of the data accountants and auditors actually encounter doesn't look like that at all — it's **unstructured text**: the paragraphs in a financial disclosure, the transcript of an earnings call, a free-text description in an audit workpaper, a customer complaint email. This chapter introduces **text analytics** (often called **Natural Language Processing**, or **NLP**) — a set of techniques for turning that unstructured text into something you can analyze quantitatively, using the same underlying workflow (train, test, evaluate) from Chapter 13, applied to text instead of numbers.

> **A Note on Tools for This Chapter**
>
> This chapter uses pandas (not polars) alongside scikit-learn, since text vectorization workflows in practice are built almost entirely on pandas + scikit-learn, and polars doesn't offer a meaningful advantage for the dataset sizes here. Visualizations continue to use both seaborn and plotnine, consistent with the rest of the book.

14.2. Where Text Shows Up in Accounting Practice

A few common sources of text data relevant to accounting and audit work:

- **MD&A (Management□s Discussion and Analysis)** and other financial statement narrative disclosures
- **Earnings call transcripts**, where tone and word choice are studied closely by analysts
- **Audit workpaper narratives** — free-text descriptions of findings, exceptions, and follow-up items
- **Whistleblower tips and complaint text**, often the first signal of a potential issue
- **Contract and lease language**, increasingly relevant for revenue recognition and lease accounting judgments
- **Vendor and customer correspondence**, sometimes reviewed during fraud investigations

14.3. Text Preprocessing Fundamentals

Raw text needs to be broken down and cleaned before it can be analyzed. We'll work with `mda_narratives.csv`, a set of short, original MD&A-style paragraphs written for this chapter, one per year (2021-2025), for the same fictional company from Chapter 7 — Ledger & Co. These are **not real filings**; they're written to reflect that chapter's actual computed ratio trends, so we can check whether text-based analysis lines up with what we already know from the numbers.

14.4. pandas

```
import pandas as pd
import re

mda = pd.read_csv("data/mda_narratives.csv")
mda[["year", "narrative_text"]]
```

	year	narrative_text
0	2021	Ledger & Co. delivered a solid year of growth,...
1	2022	The company continued to grow revenue and impr...
2	2023	Ledger & Co. achieved another year of strong p...
3	2024	The company again reported improved profitabil...
4	2025	Ledger & Co. delivered its strongest profitabi...

14.5. polars

```
import polars as pl
import re

mda_pl = pl.read_csv("data/mda_narratives.csv")
mda_pl.select(["year", "narrative_text"])
```

year	narrative_text
i64	str
2021	"Ledger & Co. delivered a solid...
2022	"The company continued to grow ...
2023	"Ledger & Co. achieved another ...
2024	"The company again reported imp...
2025	"Ledger & Co. delivered its str...

14.5.1. Tokenization and Cleaning

Tokenization splits text into individual words (tokens). Before that, we typically lowercase everything and remove punctuation, so "Revenue" and "revenue." are treated as the same word.

14.6. Not pandas

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", "", text)  # keep only letters and
 ↪  spaces
```

```
    return text.split()

tokens = preprocess(mda.loc[0, "narrative_text"])
tokens[:10]
```

```
['ledger',
 'co',
 'delivered',
 'a',
 'solid',
 'year',
 'of',
 'growth',
 'with',
 'revenue']
```

14.7. pandas

```
tokens_pd = mda.loc[0,'narrative_text'].lower().split()
tokens_pd[:10]
```

```
['ledger',
 '&',
 'co.',
 'delivered',
 'a',
 'solid',
 'year',
 'of',
 'growth,',
 'with']
```

14.8. polars

```
tokens_pl = (
    mda_pl
    .select(
```

```
        pl.col("narrative_text")
        .str.to_lowercase()
        .str.split(" ")
    )
    .row(0)[0]
)
tokens_pl[:10]
```

```
['ledger',
 '&',
 'co.',
 'delivered',
 'a',
 'solid',
 'year',
 'of',
 'growth,',
 'with']
```

Or you can use the simpler code like this -

```
tokens_pl = (
    mda_pl["narrative_text"][0]
    .lower()
    .split()
)
tokens_pl[:10]
```

```
['ledger',
 '&',
 'co.',
 'delivered',
 'a',
 'solid',
 'year',
 'of',
 'growth,',
 'with']
```

14.8.1. Removing Stopwords

Stopwords are common words ("the," "and," "is") that carry little analytical meaning on their own and are usually removed before further analysis.

14.9. Not pandas

```
stopwords =
 ↪  {"the","a","an","and","or","but","of","in","on","at","to","for","with","as",

                ↪  "is","was","were","be","been","this","that","its","it","company"

filtered = [t for t in tokens if t not in stopwords]
filtered[:10]
```

```
['ledger',
 'co',
 'delivered',
 'solid',
 'year',
 'growth',
 'revenue',
 'increasing',
 'expanded',
 'customer']
```

14.10. pandas

```
import nltk
from nltk.corpus import stopwords
nltk.download("stopwords")
nltk.download('punkt')
stop_words = stopwords.words("english")
custom_words = ["&", "co."] # adding some words to stop words list
stop_words.extend(custom_words)
filtered_pd = [word for word in tokens_pd if word.lower() not in
 ↪  stop_words]
filtered_pd[:10]
```

```
['ledger',
 'delivered',
 'solid',
 'year',
 'growth,',
 'revenue',
 'increasing',
 'company',
 'expanded',
 'customer']
```

14.11. polars

```
filtered_pl = [word for word in tokens_pl if word.lower() not in
 ↪  stop_words]
filtered_pl[:10]
```

```
['ledger',
 'delivered',
 'solid',
 'year',
 'growth,',
 'revenue',
 'increasing',
 'company',
 'expanded',
 'customer']
```

14.11.1. Stemming

Stemming reduces words to a common root, so “improve,” “improved,” and “improving” are all treated as the same underlying concept.

14.12. Not pandas

```
def simple_stem(word):
    for suffix in ["ing", "ed", "ly", "es", "s"]:
        if word.endswith(suffix) and len(word) - len(suffix) > 3:
            return word[: -len(suffix)]
    return word

stemmed = [simple_stem(t) for t in filtered]
stemmed[:10]
```

```
['ledger',
 'co',
 'deliver',
 'solid',
 'year',
 'growth',
 'revenue',
 'increas',
 'expand',
 'customer']
```

14.13. pandas

```
from nltk.stem import PorterStemmer
nltk.download("punkt")
stemmer_porter = PorterStemmer()
stemmed_pd = [stemmer_porter.stem(word) for word in filtered_pd]
stemmed_pd[:10]
```

```
['ledger',
 'deliv',
 'solid',
 'year',
 'growth,',
 'revenu',
 'increas',
 'compani',
 'expand',
```

```
 'custom']
```

14.14. polars

```
from nltk.stem import PorterStemmer
nltk.download("punkt")
stemmer_porter = PorterStemmer()
stemmed_pl = [stemmer_porter.stem(word) for word in filtered_pl]
stemmed_pl[:10]
```

```
['ledger',
 'deliv',
 'solid',
 'year',
 'growth,',
 'revenu',
 'increas',
 'compani',
 'expand',
 'custom']
```

> ⚠ A Deliberately Simple Stemmer, and Its Limitation
>
> Look closely: □**gross**□ **became** □**gros**□ — our simple rule stripped a trailing "s" without understanding it wasn't actually a plural or verb ending. This is a genuine, common failure mode of naive suffix-stripping stemmers, not a bug in this example specifically. Production NLP work typically uses more sophisticated **lemmatization** instead, which uses a dictionary and grammatical rules to find a word's true root ("gross" would correctly stay "gross"). We're using this simplified version purely to show the *concept* of reducing words to a common form without requiring an external dictionary download — know that real tools handle this more carefully.

14.14.1. Lemmatization

Lemmatization is an advanced text normalization technique in natural language processing (NLP) that reduces words to their meaningful, dictionary-based base form. To achieve high accuracy, a lemmatizer contextualizes the word by determining its part of speech (POS)—such as identifying whether "coding" functions as a noun or a verb in a sentence. This ensures that

complex irregular transformations, like mapping the word "better" back to "good" or "was" back to "be," are resolved correctly based on grammar rules rather than guesswork.

When evaluated side-by-side, **lemmatization** offers grammatical precision while **stemming** prioritizes processing speed. Stemming operates by applying crude, heuristic rules to slice off the ends of words, which frequently results in non-existent words or chopped roots like reducing "studies" and "studying" to just "studi." Conversely, lemmatization relies on deep semantic databases like WordNet to consistently output valid, real-world words like "study." While stemming is computationally lightweight and ideal for rapid search indexing on massive datasets, lemmatization is far superior for tasks demanding absolute contextual understanding, such as sentiment analysis, chatbots, and machine translation.

Feature	Stemming	Lemmatization
Approach	Rule-based suffix cutting (heuristic)	Dictionary-based morphological analysis
Output Word	Often an invalid word root (e.g., `chang`)	Always a valid dictionary word (e.g., `change`)
Context Aware	No (ignores surrounding sentence structure)	Yes (uses Part of Speech tagging)
Speed	Extremely fast and lightweight	Slower due to database lookups
Example (□better□)	`better`	`good`

14.15. pandas

```
import nltk
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lemmatizer_pd = [lemmatizer.lemmatize(word) for word in filtered_pd]
lemmatizer_pd[:10]
```

```
['ledger',
 'delivered',
 'solid',
 'year',
 'growth,',
 'revenue',
 'increasing',
 'company',
```

```
 'expanded',
 'customer']
```

14.16. polars

```
import nltk
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lemmatizer_pl = [lemmatizer.lemmatize(word) for word in filtered_pl]
lemmatizer_pl[:10]
```

```
['ledger',
 'delivered',
 'solid',
 'year',
 'growth,',
 'revenue',
 'increasing',
 'company',
 'expanded',
 'customer']
```

14.17. From Text to Numbers: Bag-of-Words and TF-IDF

Machine learning models need numbers, not words. The **bag-of-words** approach counts how often each word appears in each document, ignoring word order entirely.

```
from sklearn.feature_extraction.text import CountVectorizer

count_vec = CountVectorizer(stop_words="english")
count_matrix = count_vec.fit_transform(mda["narrative_text"])

count_df = pd.DataFrame(
    count_matrix.toarray(), columns=count_vec.get_feature_names_out(),
 ↪  index=mda["year"]
)
count_df[["revenue", "margin", "debt", "cash", "liquidity"]]
```

year	revenue	margin	debt	cash	liquidity
2021	1	1	1	0	1
2022	1	1	1	0	1
2023	0	1	1	1	0
2024	0	1	1	1	1
2025	0	3	2	2	1

Notice "cash" appears zero times in 2021-2022 but grows to 2 mentions by 2025 — consistent with Chapter 7's finding that the company's cash position grew substantially over this period.

TF-IDF (Term Frequency-Inverse Document Frequency) improves on simple counts by *down-weighting* words that appear in every document (and are therefore less distinctive) and *upweighting* words that are frequent in one document but rare across the rest.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vec = TfidfVectorizer(stop_words="english")
tfidf_matrix = tfidf_vec.fit_transform(mda["narrative_text"])

tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(), columns=tfidf_vec.get_feature_names_out(),
 ↪  index=mda["year"]
)
tfidf_df[["revenue", "margin", "debt", "cash", "liquidity"]].round(3)
```

year	revenue	margin	debt	cash	liquidity
2021	0.161	0.095	0.095	0.000	0.113
2022	0.158	0.093	0.093	0.000	0.110
2023	0.000	0.098	0.098	0.138	0.000
2024	0.000	0.095	0.095	0.133	0.112
2025	0.000	0.245	0.164	0.230	0.097

"Margin" and "debt" both carry noticeably higher TF-IDF weight in 2025 than in earlier years — reflecting that year's narrative repeatedly emphasizing margin improvement and debt reduction, exactly the two trends Chapter 7 identified as most significant.

14.18. Visualizing Word Frequency

Visualizing word frequency helps turn raw text into a quick, intuitive summary of the most common terms. It makes it easier to spot key themes, repeated concepts, and dominant topics in documents, survey responses, or research text.

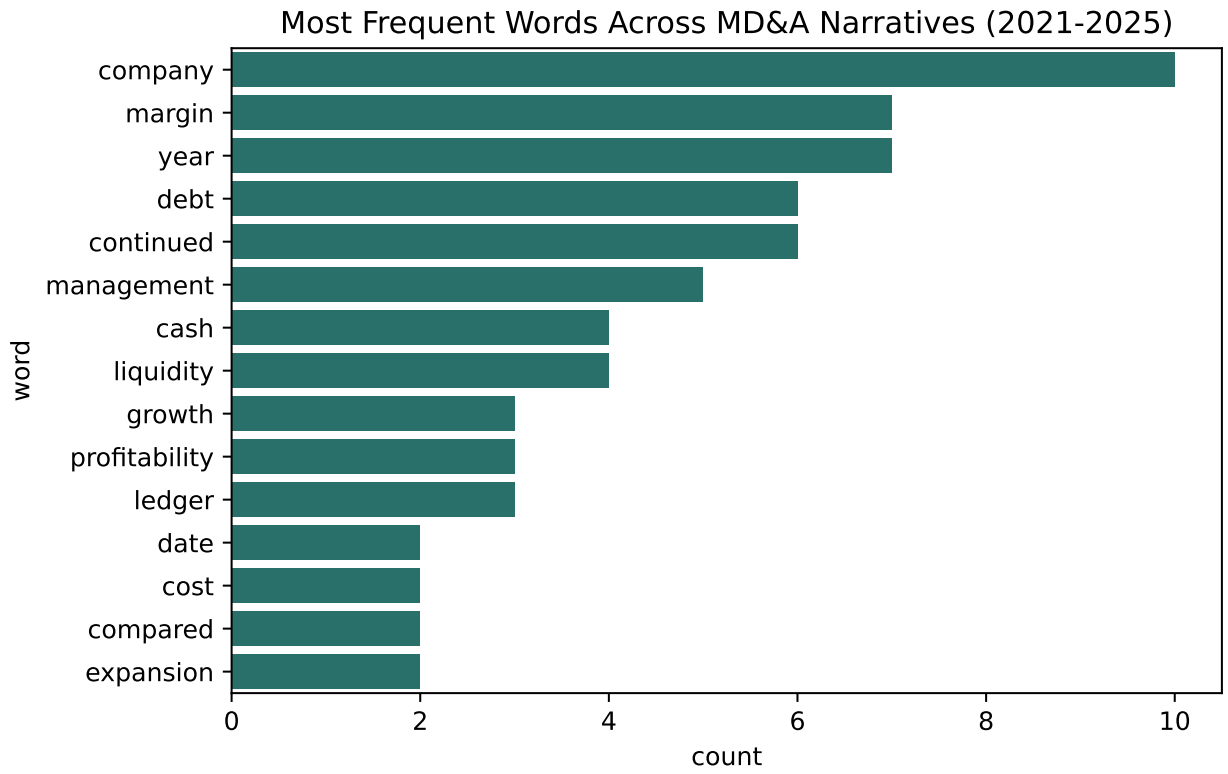
Common ways to visualize word frequency include bar charts, word clouds, and frequency tables. A bar chart is usually better for precise comparison, while a word cloud is more visually appealing for highlighting prominent words at a glance.

14.19. seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

count_vec_top = CountVectorizer(stop_words="english", max_features=15)
count_matrix_top = count_vec_top.fit_transform(mda["narrative_text"])
word_freq = pd.DataFrame({
    "word": count_vec_top.get_feature_names_out(),
    "count": count_matrix_top.toarray().sum(axis=0),
}).sort_values("count", ascending=False)

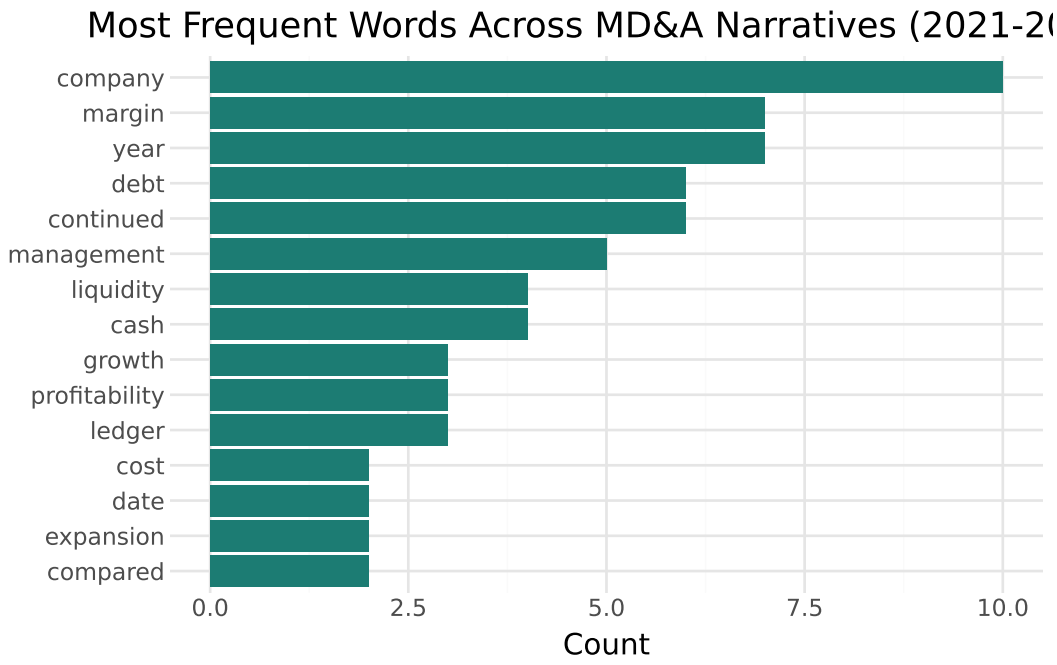
plt.figure(figsize=(7, 4.5))
sns.barplot(data=word_freq, x="count", y="word", color="#1c7c73")
plt.title("Most Frequent Words Across MD&A Narratives (2021-2025)")
plt.tight_layout()
plt.show()
```



14.20. plotnine

```
from plotnine import *

(
    ggplot(word_freq, aes(x="reorder(word, count)", y="count"))
    + geom_col(fill="#1c7c73")
    + coord_flip()
    + labs(title="Most Frequent Words Across MD&A Narratives
     ↪  (2021-2025)", x="", y="Count")
    + theme_minimal()
)
```



A word cloud is a text visualization where more frequent words appear larger. It is often used to quickly spot the dominant terms in a document, survey responses, or a corpus.

```
import re
from collections import Counter
from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

text = """
Accounting is the language of business, and it helps organizations
 ↪  record, classify, and summarize financial activity. In a typical
 ↪  accounting cycle, transactions are identified, journal entries are
 ↪  prepared, and reports  are produced for managers and outside
 ↪  users. Accurate accounting supports informed decisions and
 ↪  improves  transparency across the firm.

Financial accounting focuses on preparing financial statements for
 ↪  external stakeholders such as investors, creditors, regulators,
 ↪  and analysts. These statements include the balance sheet, income
 ↪  statement, cash flow statement, and statement of equity. Together,
 ↪  they show how a company performs, what it owns, what it owes, and
 ↪  how it generates cash.

Managerial accounting is used inside the organization for planning,
 ↪  control, and decision-making. It often includes budgeting, cost
 ↪  analysis, variance analysis, and performance measurement. Unlike
 ↪  financial accounting, managerial accounting is not governed by the
 ↪  same external reporting standards and can be tailored to internal
 ↪  needs.
```

```
Auditing plays an important role in ensuring that accounting
 ↪ information is reliable and fairly presented. External auditors
 ↪ examine financial statements and test internal controls to provide
 ↪ assurance to users. Internal auditors, by contrast, evaluate risk
 ↪ management, governance, and operational efficiency.

Corporate governance is closely connected to accounting because strong
 ↪ oversight reduces the risk of fraud and misstatement. Boards of
 ↪ directors, audit committees, and executives all influence the
 ↪ quality of reporting. Good governance helps ensure that financial
 ↪ reporting reflects economic reality rather than managerial
 ↪ manipulation.

The use of technology in accounting has grown rapidly through
 ↪ enterprise systems, data analytics, and artificial intelligence.
 ↪ Automated tools can process invoices, detect anomalies, and flag
 ↪ unusual transactions. At the same time, professionals must
 ↪ understand the limitations of algorithms and maintain ethical
 ↪ oversight.

Accounting standards provide consistency and comparability across
 ↪ firms and industries. In the United States, many public companies
 ↪ follow GAAP, while firms in other jurisdictions may use IFRS.
 ↪ Standards guide recognition, measurement, presentation, and
 ↪ disclosure decisions in financial reporting.

Tax accounting differs from financial accounting because it is
 ↪ concerned with compliance and tax law rather than external
 ↪ reporting alone. Firms must track deductible expenses, taxable
 ↪ income, credits, and timing differences. Effective tax planning
 ↪ can reduce costs while remaining within legal requirements.

Cost accounting helps organizations understand how resources are
 ↪ consumed in producing goods and services. It assigns direct
 ↪ materials, direct labor, and overhead to products, departments, or
 ↪ projects. This information supports pricing, profitability
 ↪ analysis, and operational improvement.

Modern accounting research examines topics such as earnings quality,
 ↪ audit quality, disclosure, ESG reporting, and the effects of AI on
 ↪ decision-making. Researchers use archival data, experiments, and
 ↪ advanced statistical methods to study these issues. As business
 ↪ environments evolve, accounting continues to adapt to new risks,
 ↪ technologies, and stakeholder expectations.
```

```
"""

# Clean text
text = text.lower()
text = re.sub(r"[^a-z\s]", " ", text)

# Remove stopwords
stopwords = set(STOPWORDS)
stopwords.update({
    "accounting", "financial", "company", "companies",
    "organization", "organizations" #, "reporting",
     ↪  "statement","statements"
    , "users"
})

clean_text = " ".join(word for word in text.split() if word not in
 ↪  stopwords)

# Generate word cloud
wc = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    stopwords=stopwords,
    collocations=False,
    max_words=100
).generate(clean_text)

# Display
plt.figure(figsize=(8, 4))
plt.imshow(wc, interpolation="bilinear")
plt.axis("off")
plt.tight_layout()
plt.show()
```



You can also provide different shapes to your word cloud. We generate a word cloud of the state of Illinois shape.

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud, STOPWORDS
import numpy as np
from PIL import Image
import re
# Load your Illinois shape image
img = Image.open("images/illinois.png").convert("L")  # grayscale
mask = np.array(img)

# Generate word cloud in Illinois shape
wc = WordCloud(
    background_color="white",
    mask=mask,
    stopwords=stopwords,
    contour_width=2,
    contour_color="darkblue",
    collocations=False
).generate(text)

# Show it
plt.figure(figsize=(10, 5))
plt.imshow(wc, interpolation="bilinear")
```

```
plt.axis("off")
plt.tight_layout()
plt.show()
```



14.21. Financial Sentiment Analysis

Sentiment analysis scores text as positive, negative, or neutral, typically using a **lexicon**: a list of words tagged with a sentiment polarity. This is where financial text creates a genuine, well-documented problem: general-purpose sentiment lexicons are usually built from product reviews or social media, where words like "liability," "tax," "cost," and "debt" are treated as negative — but in ordinary financial narrative, these are often just **neutral accounting terms**, not expressions of bad news.

```
# An illustrative generic-style lexicon (not a real published tool)
generic_positive =
 ↪  {"strong","solid","growth","improved","improve","optimistic","pleased",
```

```
                        ↪  "positive","opportunity","gains","growing","grow","expan
generic_negative =
 ↪  {"debt","decline","declined","liability","liabilities","risk","cost","costs",

                        ↪  "cautious","pressure","pressures","tax","expense","expen

# A finance-adjusted version: the same list, with neutral
 ↪  accounting/finance
# terms removed from the negative set, since they aren't inherently
 ↪  negative in this context
finance_adjusted_negative = generic_negative - {
    "debt", "liability", "liabilities", "tax", "expense", "expenses",
     ↪  "cost", "costs"
}

def score_sentiment(text, positive_words, negative_words):
    text = text.lower()
    tokens = re.sub(r"[^a-z\s]", "", text).split()
    pos_count = sum(1 for t in tokens if t in positive_words)
    neg_count = sum(1 for t in tokens if t in negative_words)
    total = pos_count + neg_count
    return 0.0 if total == 0 else (pos_count - neg_count) / total

mda["Generic Lexicon"] = mda["narrative_text"].apply(
    lambda t: score_sentiment(t, generic_positive, generic_negative)
)
mda["Finance-Adjusted Lexicon"] = mda["narrative_text"].apply(
    lambda t: score_sentiment(t, generic_positive,
     ↪  finance_adjusted_negative)
)

mda[["year", "Generic Lexicon", "Finance-Adjusted Lexicon"]].round(3)
```

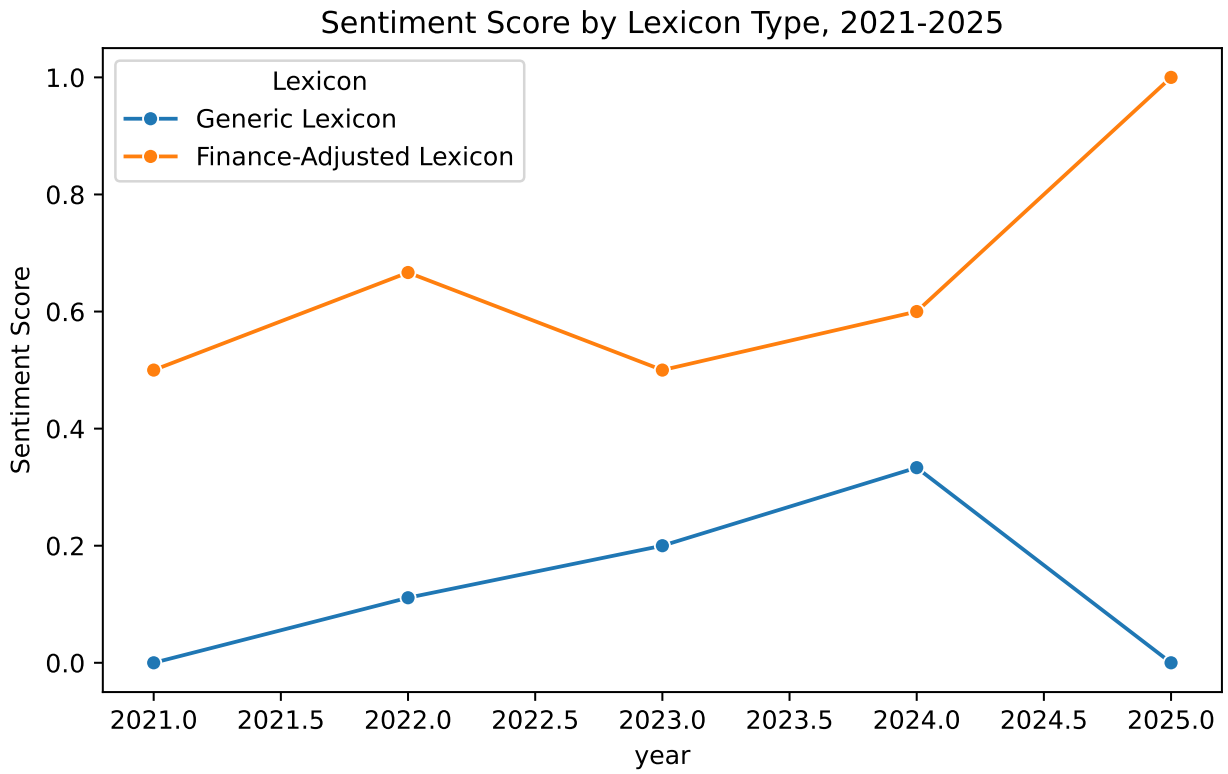
	year	Generic Lexicon	Finance-Adjusted Lexicon
0	2021	0.000	0.500
1	2022	0.111	0.667
2	2023	0.200	0.500
3	2024	0.333	0.600
4	2025	0.000	1.000

> **!** A Striking, Real Result From This Exact Data
>
> Look at **2025** — the year Chapter 7 identified as the company's *strongest* year across every profitability and leverage metric. The generic lexicon scores it as perfectly **neutral (0.000)**, entirely because the word "debt" appears twice (in the context of paying it down, a genuinely positive development). The finance-adjusted lexicon correctly scores 2025 as the **most positive** year in the entire series (1.000). This is not a contrived example — it's exactly what happens when a sentiment tool built for product reviews is applied unmodified to financial narrative. This is precisely why specialized financial sentiment dictionaries (the best-known

```
sentiment_long = mda.melt(
    id_vars="year", value_vars=["Generic Lexicon", "Finance-Adjusted
 ↪  Lexicon"],
    var_name="Lexicon", value_name="Sentiment Score"
)
```

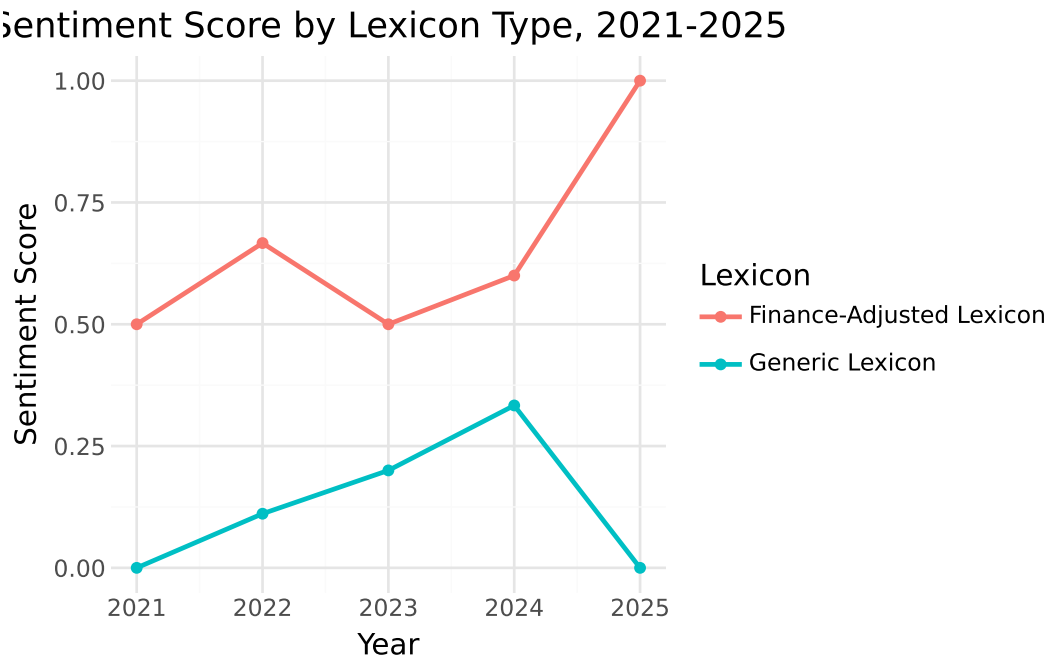
14.22. seaborn

```
plt.figure(figsize=(7, 4.5))
sns.lineplot(data=sentiment_long, x="year", y="Sentiment Score",
 ↪  hue="Lexicon", marker="o")
plt.title("Sentiment Score by Lexicon Type, 2021-2025")
plt.tight_layout()
plt.show()
```



14.23. plotnine

```
(
    ggplot(sentiment_long, aes(x="year", y="Sentiment Score",
    color="Lexicon"))
    + geom_line(size=1)
    + geom_point()
    + labs(title="Sentiment Score by Lexicon Type, 2021-2025",
      x="Year", y="Sentiment Score")
    + theme_minimal()
)
```



The finance-adjusted line trends upward, consistent with the company's actual improving fundamentals; the generic line is noisy and even dips in the company's best year — a clear illustration of why lexicon choice matters enormously in financial text analysis.

14.24. Text Classification

Beyond sentiment, a common accounting task is **classifying** text into categories — for example, sorting audit finding descriptions into issue types to prioritize review. This uses exactly the same train/test/evaluate workflow as Chapter 13's financial distress classifier, with TF-IDF features standing in for financial ratios.

```
findings = pd.read_csv("data/audit_findings_labeled.csv")
findings["category"].value_counts()
```

```
category
Documentation/Support Missing    43
Cutoff/Timing                    41
Authorization/Access             38
Duplicate/Data Entry             37
Name: count, dtype: int64
```

```
findings[["category", "description"]].head(5)
```

	category	description
0	Cutoff/Timing	Revenue recognized 16 days before shipment dat...
1	Duplicate/Data Entry	Identical debit and credit amounts posted on t...
2	Cutoff/Timing	Journal entry dated last day of the quarter re...
3	Authorization/Access	No segregation of duties was evident between p...
4	Authorization/Access	No segregation of duties was evident between p...

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

X_train_text, X_test_text, y_train, y_test = train_test_split(
    findings["description"], findings["category"],
    test_size=0.25, random_state=42, stratify=findings["category"]
)

vectorizer = TfidfVectorizer(stop_words="english", max_features=200)
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)

clf = LogisticRegression(max_iter=1000, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
                            precision    recall  f1-score   support

         Authorization/Access       1.00      1.00      1.00        10
               Cutoff/Timing       1.00      1.00      1.00        10
Documentation/Support Missing       1.00      1.00      1.00        11
         Duplicate/Data Entry       1.00      1.00      1.00         9

                     accuracy                           1.00        40
                    macro avg       1.00      1.00      1.00        40
                 weighted avg       1.00      1.00      1.00        40
```

```
# classify a brand-new, unseen finding description
new_finding = ["The journal entry was approved by someone without
 ↪  proper sign-off authority."]
new_vec = vectorizer.transform(new_finding)
clf.predict(new_vec)
```

```
array(['Authorization/Access'], dtype=object)
```

⚠ Why Is Accuracy a Perfect 100%? (And Why That's a Red Flag Here, Not a Success)

Unlike Chapter 13's financial distress model (which topped out around 88% accuracy on real-feeling data), this model classifies with perfect accuracy. That's **not** because text classification is inherently easier than numeric classification — it's because this chapter's finding descriptions were generated from a small number of category-specific templates, giving each category cleanly distinctive vocabulary ("duplicate," "authorization," "shipment," "supporting documentation") that a model can separate perfectly. **Real audit findings are almost never this clean**: the same finding might involve both a timing issue *and* a documentation gap, phrasing varies enormously between different auditors, and vocabulary overlaps heavily between categories. Treat this chapter's 100% accuracy as a property of intentionally clean training data, not as a realistic expectation for a production text classifier — a good habit is to be *more* suspicious of a perfect score, not more confident, since it often signals a data leakage or over-simplification problem rather than genuine model quality.

14.25. Limits and Risks of Text Analytics in Accounting

The classical techniques in this chapter — bag-of-words, TF-IDF, lexicon-based sentiment, and TF-IDF-based classification — are useful but genuinely limited in ways worth naming directly:

- **No understanding of context.** "The company **is not** at risk of default" and "the company **is** at risk of default" share almost identical bag-of-words representations despite opposite meanings — negation is invisible to these techniques.
- **No handling of sarcasm or subtlety.** A skeptical or hedging tone ("management remains *cautiously* optimistic, though several risks remain unresolved") is easily miscounted as simply positive by a lexicon that only tallies individual words.
- **Boilerplate dilutes signal.** Real financial disclosures contain large amounts of standardized legal and risk-factor boilerplate repeated across companies and years, which can drown out the specific, meaningful language analysts actually care about.
- **Category and vocabulary overlap.** As the audit finding classifier's suspiciously perfect accuracy illustrates, real-world text rarely separates as cleanly as engineered examples do.

Looking Ahead

Every one of these limitations is exactly where large language models — the subject of the next chapter — represent a genuine leap forward: they capture context, negation, and tone in ways bag-of-words and lexicon counting fundamentally cannot. Understanding *why* these classical techniques struggle is what makes it possible to appreciate specifically what's new and different about the GenAI-based approaches in Chapter 15, rather than treating "AI" as one undifferentiated technology.

14.26. Chapter Summary

- Text analytics (NLP) extends this book's analytical toolkit from structured, numeric data to unstructured text — MD&A disclosures, earnings call transcripts, audit narratives, and more.
- Preprocessing (tokenization, stopword removal, stemming) turns raw text into a cleaned, analyzable form, though simple stemming rules can produce their own errors, as this chapter's "gross" → "gros" example showed directly.
- Bag-of-words and TF-IDF convert text into numeric features usable by any machine learning model, with TF-IDF specifically upweighting distinctive, less common terms.
- General-purpose sentiment lexicons systematically misread financial text, since ordinary accounting terms like "debt," "tax," and "liability" get miscounted as negative — this chapter's own data showed a generic lexicon scoring the company's *best* year as perfectly neutral.
- Text classification follows the identical train/test/evaluate workflow from Chapter 13, just with TF-IDF features instead of financial ratios — and a suspiciously perfect accuracy score is a reason for more scrutiny, not less.
- Classical NLP techniques can't handle negation, sarcasm, tone, or boilerplate dilution — exactly the gap that large language models, covered next, are designed to close.

14.27. Discussion Questions

1. Why did the generic sentiment lexicon score 2025 — the company's strongest year by every financial metric in Chapter 7 — as perfectly neutral? What does this suggest about applying general-purpose NLP tools to financial text without adaptation?
2. The audit finding classifier achieved 100% accuracy. Why should that result make you *more* skeptical of the model, not more confident, and what would you want to test before trusting it on new, real findings?
3. Give an example sentence (financial or otherwise) where bag-of-words would completely miss the actual meaning due to negation or sarcasm.

14.28. Exercises

1. Add three new words to both the generic and finance-adjusted negative lexicons (e.g., "depreciation," "obligation," "amortization") and recompute the sentiment scores. Does the gap between the two lexicons widen or narrow?
2. Using `audit_findings_labeled.csv`, write five new finding descriptions in your own words for any one category, deliberately making the vocabulary overlap more with a *different* category than templates in this chapter did. Add them to the dataset and see whether the classifier's accuracy drops.
3. Rebuild the word frequency chart using only 2021 and 2025's narratives separately (rather than combined). How does the most frequent vocabulary differ between the company's weakest and strongest years?
4. Using `simple_stem()` from this chapter, find at least two other words from the MD&A narratives (besides "gross") where the naive stemming rule produces an incorrect or confusing result, and explain why.

15. Generative AI in Accounting

Learning Objectives

By the end of this chapter, you should be able to:

- Explain how generative AI differs from the predictive models built in Chapter 12
- Describe current, real-world use cases for generative AI in accounting and audit practice
- Apply basic prompt engineering principles to accounting-specific tasks
- Use an LLM API from Python to draft narrative content from computed financial data
- Systematically verify AI-generated content against source data, rather than trusting it by default
- Identify the key risks of using generative AI in accounting work: hallucination, confidentiality, bias, and overreliance
- Describe the current regulatory and professional landscape around AI use in audit and financial reporting

15.1. Generative AI vs. the Predictive Models in Chapter 12

Chapter 12 built models that predict a specific, narrow output from structured, numeric data: a number of days, a probability of distress. **Generative AI** — large language models (LLMs) like Claude, GPT, or Gemini — work differently: they're trained to produce open-ended, human-like text (or images, code, and other content) in response to a prompt, rather than a single predicted value.

This distinction matters practically. A logistic regression model can tell you *a company's probability of financial distress*, computed from specific numeric inputs, and that number is fully determined by the data and the model's fitted coefficients. An LLM asked to *write a paragraph about a company's financial health* will produce fluent, plausible-sounding prose — but that prose is generated by predicting likely word sequences, not by performing the underlying arithmetic itself, even if the numbers happen to appear correct. This is the source of nearly every risk discussed in this chapter, and it's worth keeping in mind throughout.

15.2. Current Use Cases in Accounting and Audit Practice

Generative AI is already in active, if still early-stage, use across the profession:

- **Audit documentation and workpaper drafting** — summarizing findings, drafting standard sections of workpapers from structured inputs
- **Anomaly explanation** — after a statistical or rule-based test (like the ones in Chapters 8-9) flags a transaction, an LLM can help draft a plain-language summary of *why* it was flagged, to speed up reviewer triage
- **Disclosure and financial narrative drafting** — drafting first-pass MD&A commentary, footnote language, or management representation letters, always subject to human review
- **Tax research assistance** — summarizing tax code sections or prior guidance as a starting point for a human researcher, not a final answer
- **Client communication drafting** — first drafts of client memos, follow-up questions, or engagement letters

> Connect to Practice
>
> Every one of these use cases follows the same pattern: **AI drafts, a human verifies.** None of them involve an LLM independently reaching a final conclusion that goes into a financial statement or audit opinion without review. That pattern isn't a limitation to work around — it's the appropriate way to use this technology given the risks covered later in this chapter.

15.3. Prompt Engineering Basics for Accounting Tasks

The quality of an LLM's output depends heavily on how you ask. A few principles that matter especially for accounting tasks:

- **Be specific about format and audience.** "Summarize this" produces something different from "Summarize this in three bullet points for a non-technical audit committee member."
- **Provide the actual data, don□t just describe it.** An LLM asked to comment on "a company with declining margins" will invent plausible-sounding but fictional specifics. An LLM given the actual computed ratios can reference them directly — though as we'll see, "given the numbers" doesn't guarantee it will use them correctly.
- **Ask for a specific structure.** Requesting a fixed structure (e.g., "one paragraph on profitability, one on leverage, one flagged concern") produces more consistent, checkable output than an open-ended request.

- **State what *not* to do.** Explicitly asking a model not to invent figures it wasn't given, and to flag uncertainty rather than guessing, measurably reduces (though does not eliminate) fabricated details.

15.4. Hands-On: Drafting a Financial Narrative with an LLM API

Let's connect this directly to Chapter 7's ratio analysis. We'll recompute this company's 2025 ratios, then use them to prompt an LLM to draft a short narrative — exactly the kind of first-draft assistance described above.

15.5. pandas

```
import pandas as pd

inc = pd.read_csv("data/income_statement.csv")
bs = pd.read_csv("data/balance_sheet.csv")
merged = inc.merge(bs, on="year")

ratios = pd.DataFrame({"year": merged["year"]})
ratios["net_margin"] = merged["Net Income"] / merged["Revenue"]
ratios["current_ratio"] = merged["Total Current Assets"] /
 ↪  merged["Total Current Liabilities"]
ratios["debt_to_equity"] = (merged["Short-term Debt"] +
 ↪  merged["Long-term Debt"]) / merged["Total Equity"]

latest = ratios.iloc[-1]
latest.round(3)
```

```
year              2025.000
net_margin           0.158
current_ratio        7.369
debt_to_equity       0.068
Name: 4, dtype: float64
```

15.6. polars

```
import polars as pl

# Read the data
inc_pl = pl.read_csv("data/income_statement.csv")
bs_pl = pl.read_csv("data/balance_sheet.csv")

# Merge the two DataFrames
merged_pl = inc_pl.join(
    bs_pl,
    on="year",
    how="inner"
)

# Compute financial ratios
ratios_pl = (
    merged_pl
    .select(
        "year",

        (
            pl.col("Net Income")
            / pl.col("Revenue")
        ).alias("net_margin"),

        (
            pl.col("Total Current Assets")
            / pl.col("Total Current Liabilities")
        ).alias("current_ratio"),

        (
            (pl.col("Short-term Debt") + pl.col("Long-term Debt"))
            / pl.col("Total Equity")
        ).alias("debt_to_equity"),
    )
)

# Get the latest year
latest_pl = (
    ratios_pl
```

```
    .sort("year")
    .tail(1)
    .with_columns(
        pl.exclude("year").round(3)
    )
)

latest_pl
```

year	net_margin	current_ratio	debt_to_equity
i64	f64	f64	f64
2025	0.158	7.369	0.068

```
# This chunk requires your own Anthropic API key and will not run
 ↪  automatically
# when this book is rendered. Install the SDK first: pip install
 ↪  anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY_HERE")

prompt = f"""
Using ONLY the figures provided below, write a two-sentence summary of
 ↪  this
company's 2025 financial position for a non-technical reader. Do not
 ↪  invent
any numbers not given here. If you are uncertain about an
 ↪  interpretation,
say so explicitly rather than guessing.

Net margin: {latest['net_margin']:.1%}
Current ratio: {latest['current_ratio']:.2f}
Debt-to-equity: {latest['debt_to_equity']:.2f}
"""

# NOTE: check https://docs.claude.com for the current model name
 ↪  before running
response = client.messages.create(
    model="claude-sonnet-4-5",
```

```
    max_tokens=300,
    messages=[{"role": "user", "content": prompt}],
)

print(response.content[0].text)
```

> **!** Why This Code Isn't Executed in This Book
>
> The chunk above is marked `eval: false`, meaning it's shown for reference but doesn't run when this book renders — it requires your own API key, which shouldn't be hardcoded into a shared document. When you run this yourself, store your key as an environment variable (`os.environ["ANTHROPIC_API_KEY"]`) rather than typing it directly into a notebook or `.qmd` file, especially one that might end up in a shared Git repository, as yours already is.

15.7. The Hallucination Check: Verifying AI Output Against Source Data

Suppose the model above returned the following (this is a realistic example of the kind of output an LLM might produce, written here for teaching purposes):

> *"This company shows strong and improving profitability, with net margin reaching 18.2% in 2025. Liquidity is very strong, with a current ratio of 7.37, and the company carries minimal financial risk with a debt-to-equity ratio of 0.07."*

This paragraph reads fluently and looks well-supported by numbers. It's also **subtly wrong** — one figure doesn't match what we actually computed. Let's check it the way you should check any AI-generated content containing numbers: line by line, against the source data.

```
ai_claims = {
    "net_margin": 0.182,
    "current_ratio": 7.37,
    "debt_to_equity": 0.07,
}

actual_values = {
    "net_margin": round(latest["net_margin"], 4),
    "current_ratio": round(latest["current_ratio"], 2),
    "debt_to_equity": round(latest["debt_to_equity"], 2),
```

```
}

for metric, claimed in ai_claims.items():
    actual = actual_values[metric]
    match = abs(claimed - actual) < 0.005
    print(f"{metric}: claimed={claimed}, actual={actual},
     ↪  matches={match}")
```

```
net_margin: claimed=0.182, actual=0.1576, matches=False
current_ratio: claimed=7.37, actual=7.37, matches=True
debt_to_equity: claimed=0.07, actual=0.07, matches=True
```

The `net_margin` claim of 18.2% doesn't match our actual computed value of 15.8% — a **hallucinated figure** that looks entirely plausible sitting next to two genuinely correct numbers. This is exactly what makes numeric hallucination dangerous in practice: a reader has no visual cue that one number in a paragraph is fabricated while the others are accurate. The only reliable defense is checking every figure against its source, every time — not spot-checking the ones that "feel" wrong.

> **!** Why LLMs Make This Kind of Mistake
>
> An LLM generates text by predicting plausible word (and number) sequences based on patterns in its training data, not by running a calculator. Even when given the correct inputs directly in a prompt, it can still produce a number that *sounds* right — close to the correct order of magnitude, formatted correctly — without it actually being the number it was given. This is fundamentally different from a calculation error in a spreadsheet formula, which is at least deterministic and reproducible; an LLM's numeric error can vary from one run to the next even with an identical prompt.

15.8. Using GenAI for Synthetic Text Generation

It's worth distinguishing two different things people mean by "synthetic data":

- **Statistical/numeric simulation** — generating realistic numeric datasets by sampling from probability distributions (this is exactly how every dataset in this book was built: lognormal transaction amounts, logistic financial distress probabilities, and so on). This is a traditional statistical technique, not generative AI.
- **AI-generated text content** — using an LLM to generate realistic *unstructured* text: transaction descriptions, vendor names, customer complaint narratives, or email bodies for a

training exercise. This is where generative AI genuinely adds something numeric simulation can't easily produce.

For example, an LLM is well-suited to generating a batch of realistic-sounding (but fictional) journal entry descriptions for a classroom exercise — "Q3 marketing consulting services - Contract #4471" reads far more like a real transaction memo than anything a random word generator would produce. But note that an LLM is a poor tool for generating the *dollar amounts* in that same dataset — that's exactly the kind of numeric task better suited to the statistical simulation techniques used throughout this book.

15.9. Risks and Limitations

- **Hallucination on numeric data.** As demonstrated above, LLMs can generate plausible-looking but incorrect figures, even when given correct source data. Never treat an AI-generated number as verified until you've checked it yourself.
- **Confidentiality and data privacy.** Pasting real client financial data, employee information, or unpublished results into a public AI tool can violate client confidentiality agreements or data privacy regulations. Firms increasingly require using only approved, firm-licensed AI tools with appropriate data handling agreements, not general consumer AI products, for any real client data.
- **Bias.** LLMs are trained on large text corpora that can encode historical biases; this is a concern for any application involving people (e.g., drafting language about employee performance or credit decisions), not just financial figures.
- **Overreliance and automation complacency.** Repeated correct AI output can lead reviewers to reduce their scrutiny over time — the exact failure mode that led to the "AI drafts, a human verifies" principle in the first place. A review process that exists on paper but isn't actually performed provides no real protection.

15.10. The Current Regulatory and Professional Landscape

This is a fast-moving area, and any specific description risks going stale — but as of mid-2026, a few themes are consistent across regulators:

- The **PCAOB** issued staff guidance addressing AI use from two angles: when audit firms use AI in their own procedures (requiring documentation and supervision sufficient for a reviewer to understand what the tool did and how its output was evaluated), and when audit clients themselves use AI in their financial reporting processes (requiring auditors to understand and evaluate those systems, similar to existing IT general control expectations) (PCAOB 2024b).

- The **SEC** launched an internal AI Task Force in 2025 focused on the agency's own use of AI, and its Division of Corporation Finance has reportedly begun including AI-related questions in comment letters to registrants about AI governance and risk disclosures — without yet finalizing a specific AI disclosure rule for financial reporting (US SEC 2025).
- Internationally, the **IAASB** has been developing updated technology-specific guidance, building on existing standards for understanding IT systems and automated controls (IAASB 2024).

> **Connect to Practice**
>
> Because this area changes quickly, treat any specific claim here as a starting point for your own research, not a final answer — exactly the same "AI drafts, verify independently" discipline this chapter has emphasized throughout. Check the PCAOB's and SEC's own websites for the most current guidance before relying on this for real engagement decisions.

15.11. Designing an □AI vs. Analyst□ Exercise

A useful way to build critical judgment about AI output is a structured comparison exercise:

1. Give the same financial dataset to a human analyst (or yourself) and to an LLM, with the same prompt/task (e.g., "identify the three most significant financial statement changes and their likely causes").
2. Have each work independently — the analyst without seeing the AI's draft first.
3. Compare: What did each get right? Did the AI fabricate or misstate any figures? Did the human miss a pattern the AI caught (or vice versa)?
4. Discuss: where would you trust the AI's output directly, where would you want independent verification, and why?

This exercise works well applied to any dataset from this book — for example, Chapter 7's five-year ratio trends, or Chapter 8's flagged transactions — precisely because you already know the "correct" answer independently and can grade both outputs against it.

15.12. Chapter Summary

- Generative AI produces open-ended text by predicting plausible sequences, unlike the predictive models in Chapter 12, which compute a specific output from a fitted statistical relationship — this distinction explains most of the risks in this chapter.

- Current accounting and audit use cases follow a consistent pattern: AI drafts, a human verifies — anomaly explanations, disclosure drafting, and research assistance, not final unsupervised conclusions.
- Effective prompts for accounting tasks are specific about format and audience, provide actual data rather than descriptions of it, request a defined structure, and explicitly instruct the model not to invent figures.
- AI-generated numeric claims must be verified against source data individually — a hallucinated figure can sit undetected next to entirely correct ones, with no stylistic cue distinguishing them.
- Generative AI is well-suited to producing realistic unstructured text (descriptions, narratives) but poorly suited to generating numeric data, which is better handled by the statistical simulation techniques used throughout this book.
- Confidentiality, bias, and automation complacency are risks independent of hallucination, and each requires its own safeguard rather than being solved by "just checking the numbers."
- Regulatory guidance (PCAOB, SEC, IAASB) is actively developing but not yet fully settled — current as of this writing, but worth verifying directly before relying on it professionally.

15.13. Discussion Questions

1. Why might an LLM produce a hallucinated number even when given the correct figures directly in the prompt, rather than only when working from memory?
2. The chapter distinguishes AI-generated text from statistical numeric simulation. Can you think of an accounting task where you'd want to combine both — using an LLM for some part of a dataset and statistical simulation for another?
3. If your firm required all AI-assisted workpapers to document "what the tool did and how its output was evaluated," what would that documentation look like for the hallucination-check exercise performed in this chapter?

15.14. Exercises

1. Using this chapter's `latest` ratios (or any other chapter's computed figures), write your own deliberately flawed AI-style paragraph containing exactly one incorrect figure, then write the verification code to catch it — the way the hallucination-check example did in this chapter.
2. Draft two versions of a prompt asking an LLM to summarize Chapter 9's fraud detection results: one vague ("tell me about this data") and one following this chapter's prompt engineering principles (specific format, actual data included, explicit instruction not to invent

figures). If you have API access, run both and compare; if not, predict in writing how the outputs would likely differ.

3. Design your own "AI vs. Analyst" exercise (following this chapter's structure) using Chapter 11's tax jurisdiction data. What's the "correct answer" you'd grade both outputs against?
4. Research your firm's (or a firm you're familiar with, or a hypothetical one) policy on using AI tools with client data. Write a short (150-word) summary of what it permits, what it prohibits, and whether it matches the confidentiality concerns raised in this chapter.

Part V.

Part V: Communication and Capstone

16. Automated Reporting and Dashboards with Quarto

Learning Objectives

By the end of this chapter, you should be able to:

- Explain what a parameterized report is and why it matters for recurring accounting deliverables
- Build a parameterized Quarto document using Python's parameters cell
- Render the same template multiple times with different parameter values, both manually and in a batch script
- Explain the difference between a Quarto Dashboard and the book/document format used throughout this course
- Build a simple interactive dashboard using value boxes, a Plotly chart, and a data table
- Understand where interactive tools like Plotly fit relative to the static charting tools (seaborn, plotnine) used earlier in this book

16.1. From One-Off Analysis to Repeatable Reporting

Every chapter so far has produced a single analysis, once. In practice, much of accounting reporting is **recurring**: the same audit risk summary needs to go out every department, every month; the same financial dashboard needs refreshing every quarter. Rewriting or manually re-running an analysis each time is slow and error-prone. This chapter covers two Quarto features built exactly for this problem: **parameterized reports** (one template, many rendered outputs) and **dashboards** (an interactive, at-a-glance layout rather than a linear document).

16.2. Parameterized Reports

A **parameterized report** is a single Quarto template that can be rendered multiple times with different input values, producing a different output each time — for example, one audit risk report

per department, generated from the exact same underlying code.

16.2.1. Declaring a Parameter

With Python (the Jupyter engine, which is what this book uses throughout), parameters are declared in a code chunk tagged `#| tags: [parameters]`. Any variable assigned in that chunk becomes overridable from the command line when rendering.

```
department = "Sales"
```

When this file is rendered normally (as we're doing here, without any override), `department` simply takes its default value, `"Sales"`. The rest of the document can now use `department` like any other variable:

16.3. pandas

```
import pandas as pd

audit_gl = pd.read_csv("data/audit_test_transactions.csv")
audit_gl["entry_date"] = pd.to_datetime(audit_gl["entry_date"])
audit_gl["amount"] = audit_gl[["debit", "credit"]].max(axis=1)

department_data = audit_gl[audit_gl["department"] == department]
print(f"Department: {department}")
print(f"Transaction lines: {len(department_data)}")
```

```
Department: Sales
Transaction lines: 58
```

16.4. polars

```
import polars as pl

# Read the data
audit_gl_pl = pl.read_csv(
    "data/audit_test_transactions.csv",
```

```
    try_parse_dates=True
)

# Create the amount column
audit_gl_pl = audit_gl_pl.with_columns(
    pl.max_horizontal("debit", "credit").alias("amount")
)

# Filter for the selected department
department_data_pl = (
    audit_gl_pl
    .filter(pl.col("department") == department)
)

print(f"Department: {department}")
print(f"Transaction lines: {department_data_pl.height}")
```

```
Department: Sales
Transaction lines: 58
```

16.4.1. Building the Rest of the Report Around the Parameter

The same risk-scoring logic from Chapter 8 works unchanged — it just now operates on whichever department the parameter specifies.

16.5. pandas

```
entry_level = department_data.groupby("entry_id").agg(
    entry_date=("entry_date", "first"),
    amount=("amount", "max"),
    posted_by=("posted_by", "first"),
).reset_index()

entry_level["is_weekend"] = entry_level["entry_date"].dt.weekday >= 5
entry_level["is_round_dollar"] = (entry_level["amount"] % 500 == 0) &
 ↪  (entry_level["amount"] > 0)
entry_level["is_year_end"] = entry_level["entry_date"] >=
 ↪  pd.Timestamp("2025-12-27")
```

```
full_poster_counts =
 ↪  audit_gl.groupby("posted_by")["entry_id"].nunique()
rare_threshold = full_poster_counts.quantile(0.1)
entry_level["is_rare_poster"] =
 ↪  entry_level["posted_by"].map(full_poster_counts) <= rare_threshold

flag_cols = ["is_weekend", "is_round_dollar", "is_year_end",
 ↪  "is_rare_poster"]
entry_level["risk_score"] = entry_level[flag_cols].sum(axis=1)

print(f"\n{department} department: {len(entry_level)} journal
 ↪  entries")
print(f"Average risk score: {entry_level['risk_score'].mean():.2f}")
entry_level.sort_values("risk_score",
 ↪  ascending=False).head(5)[["entry_id", "amount", "posted_by",
 ↪  "risk_score"]]
```

```
Sales department: 29 journal entries
Average risk score: 0.31
```

	entry_id	amount	posted_by	risk_score
28	JE5906	75000.00	ceverett	4
26	JE5900	50000.00	jsmith	2
27	JE5902	92500.00	rpatel	2
0	JE5105	1851.67	rpatel	1
1	JE5109	5812.50	agarcia	0

16.6. polars

```
import polars as pl

# Roll transaction lines up to journal-entry level
entry_level_pl = (
    department_data_pl
    .group_by("entry_id")
    .agg(
```

```
        pl.col("entry_date").first().alias("entry_date"),
        pl.col("amount").max().alias("amount"),
        pl.col("posted_by").first().alias("posted_by"),
    )
)

# Calculate poster counts from the FULL audit file
poster_counts_pl = (
    audit_gl_pl
    .group_by("posted_by")
    .agg(
        pl.col("entry_id")
        .n_unique()
        .alias("entry_count")
    )
)

# 10th percentile threshold
rare_threshold = (
    poster_counts_pl
    .select(pl.col("entry_count").quantile(0.1,
 ↪  interpolation="linear"))
    .item()
)

# Join poster counts and create risk flags
entry_level_pl = (
    entry_level_pl
    .join(
        poster_counts_pl,
        on="posted_by",
        how="left"
    )
    .with_columns(
        [
            # Saturday/Sunday
            (pl.col("entry_date").dt.weekday() >= 6)
                .alias("is_weekend"),

            (
                (pl.col("amount") % 500 == 0)
```

```
                & (pl.col("amount") > 0)
            ).alias("is_round_dollar"),

            (
                pl.col("entry_date") >= pl.date(2025, 12, 27)
            ).alias("is_year_end"),

            (
                pl.col("entry_count") <= rare_threshold
            ).alias("is_rare_poster"),
        ]
    )
    .with_columns(
        (
            pl.col("is_weekend").cast(pl.Int8)
            + pl.col("is_round_dollar").cast(pl.Int8)
            + pl.col("is_year_end").cast(pl.Int8)
            + pl.col("is_rare_poster").cast(pl.Int8)
        ).alias("risk_score")
    )
)

print(f"\n{department} department: {entry_level_pl.height} journal
 ↪  entries")

print(
    f"Average risk score: "
    f"{entry_level_pl['risk_score'].mean():.2f}"
)

(
    entry_level_pl
    .sort( ["risk_score", "entry_id"], descending=[True, False])
    .head(5)
    .select(
        "entry_id",
        "amount",
        "posted_by",
        "risk_score",
    )
)
```

```
Sales department: 29 journal entries
Average risk score: 0.31
```

entry_id str	amount f64	posted_by str	risk_score i8
”JE5906”	75000.0	”ceverett”	4
”JE5900”	50000.0	”jsmith”	2
”JE5902”	92500.0	”rpatel”	2
”JE5105”	1851.67	”rpatel”	1
”JE5109”	5812.5	”agarcia”	0

This is a complete, working department risk report — for Sales, since that's the parameter's default value.

16.6.1. Rendering the Same Template for a Different Department

> **!** Turning This Into a Reusable Template
>
> To actually use this as a parameterized report, save the code above (the parameters cell plus the analysis that follows it) into its own file — for example, `department-risk-report.qmd` — separate from this book's chapters. Then, from the terminal, you can render it for any department without touching the code:
>
> ```
> quarto render department-risk-report.qmd -P
>  ↪  department:"Operations" --output operations-report.html
> quarto render department-risk-report.qmd -P department:"HR"
>  ↪  --output hr-report.html
> quarto render department-risk-report.qmd -P department:"Finance"
>  ↪  --output finance-report.html
> ```
>
> Each command produces a fully independent HTML report for that one department, using identical code — exactly the same relationship between a template and its output as calling a function with different arguments.

16.7. Batch Rendering Multiple Reports

Running four `quarto render` commands by hand is manageable. Running fifty is not. A short script automates the repetition.

```
#!/bin/bash
# render_all_departments.sh
departments=("Sales" "Operations" "HR" "Finance")

for dept in "${departments[@]}"; do
  output_name=$(echo "$dept" | tr '[:upper:]' '[:lower:]')
  quarto render department-risk-report.qmd \
    -P department:"$dept" \
    --output "${output_name}-risk-report.html"
  echo "Rendered report for $dept"
done
```

Running `bash render_all_departments.sh` once produces all four department reports automatically. This is the practical payoff of parameterization: what would be an evening of manual copy-pasting and re-running becomes a single command.

> **A Python-Native Alternative**
>
> If you'd rather stay inside Python entirely, the `quarto` PyPI package (`pip install quarto`) provides a `render()` function that does the same thing programmatically:
>
> ```
> from quarto import render
>
> departments = ["Sales", "Operations", "HR", "Finance"]
>
> for dept in departments:
>     render(
>         "department-risk-report.qmd",
>         execute_params={"department": dept},
>         output_file=f"{dept.lower()}-risk-report.html",
>     )
> ```
>
> Both approaches accomplish the same thing; use whichever fits more naturally into your existing workflow. A script like either of these is also exactly what you'd hand off to a scheduler (a cron job, a scheduled task, or a CI pipeline step) to regenerate reports automatically on a recurring basis, without any manual steps at all.

16.8. Quarto Dashboards

A **dashboard** is a different kind of output entirely: instead of a linear document meant to be read top to bottom (like every chapter in this book), it arranges content into cards, value boxes, and panels meant to be scanned at a glance. Quarto supports this natively via `format: dashboard`.

> **!** Why This Book Doesn't Use Dashboards Directly
>
> `format: dashboard` is a distinct project type from the `format: html/format: pdf` book you've built throughout this course — a dashboard can't be one chapter inside your book the way earlier chapters have been. To build one, you'd create it as its own separate `.qmd` file with `format: dashboard` in its YAML header, rendered independently of your book project.

16.8.1. Building the Dashboard□s Content

The data and chart that would populate a dashboard can be built and tested exactly like everything else in this book — it's only the surrounding layout markup that's dashboard-specific.

```
full_entry_level = audit_gl.groupby("entry_id").agg(
    entry_date=("entry_date", "first"),
    amount=("amount", "max"),
    department=("department", "first"),
    posted_by=("posted_by", "first"),
).reset_index()

full_entry_level["is_weekend"] =
 ↪  full_entry_level["entry_date"].dt.weekday >= 5
full_entry_level["is_round_dollar"] = (full_entry_level["amount"] %
 ↪  500 == 0) & (full_entry_level["amount"] > 0)
full_entry_level["is_year_end"] = full_entry_level["entry_date"] >=
 ↪  pd.Timestamp("2025-12-27")
full_entry_level["is_rare_poster"] =
 ↪  full_entry_level["posted_by"].map(full_poster_counts) <=
 ↪  rare_threshold

full_flag_cols = ["is_weekend", "is_round_dollar", "is_year_end",
 ↪  "is_rare_poster"]
full_entry_level["risk_score"] =
 ↪  full_entry_level[full_flag_cols].sum(axis=1)
```

```
total_entries = len(full_entry_level)
high_risk_count = (full_entry_level["risk_score"] >= 3).sum()
avg_risk_score = full_entry_level["risk_score"].mean()

print(f"Total entries: {total_entries}")
print(f"High-risk entries (score >= 3): {high_risk_count}")
print(f"Average risk score: {avg_risk_score:.2f}")
```

```
Total entries: 117
High-risk entries (score >= 3): 2
Average risk score: 0.16
```

16.8.2. An Interactive Chart with Plotly

Everywhere else in this book, we've used seaborn and plotnine — both produce **static images**, which is exactly what's needed for a document that renders to both HTML and PDF, since a PDF page can't respond to a mouse hover. A dashboard, by contrast, is HTML-only and interactive by nature, which makes **Plotly** a natural fit: its charts support hovering for details, zooming, and panning directly in the browser.

```
import plotly.express as px

fig = px.scatter(
    full_entry_level, x="entry_date", y="amount", color="risk_score",
    size="risk_score", hover_data=["entry_id", "posted_by",
 ↪  "department"],
    color_continuous_scale="OrRd",
    title="Journal Entries by Risk Score",
    labels={"entry_date": "Entry Date", "amount": "Amount ($)",
 ↪  "risk_score": "Risk Score"},
)
fig.update_layout(height=400)
fig.show()
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

Hovering over any point in this chart (when rendered to HTML) reveals the entry ID, poster, and department — detail that a static chart could only show by cluttering the plot with labels.

> **Plotly and PDF Output**
>
> If you ever do need a Plotly chart to appear in a PDF (outside of a dashboard, which is HTML-only anyway), Quarto converts it to a static image automatically — but this requires the `kaleido` package (`pip install kaleido`). Without it, rendering a Plotly chunk to PDF will fail. This is exactly the kind of cross-format consideration that shaped this book's own tooling choices back in Chapter 6: seaborn and plotnine were chosen specifically because they produce static images that work identically well in both this book's HTML and PDF outputs, with no extra dependency required.

16.8.3. Assembling the Full Dashboard

Here's what the complete standalone dashboard file would look like, combining value boxes, the chart above, and a data table into a proper dashboard layout:

````
---
title: "Audit Risk Dashboard"
format: dashboard
---


## Row

::: {.cell content='valuebox' title='Total Entries' execution_count=9}
``` {.python .cell-code}
dict(icon = "list-ul", color = "primary", value = total_entries)
```

::: {.cell-output .cell-output-display execution_count=9}
```
{'icon': 'list-ul', 'color': 'primary', 'value': 117}
```
:::
:::


::: {.cell content='valuebox' title='High-Risk Entries'
 ↪  execution_count=10}
````

```
``` {.python .cell-code}
dict(icon = "exclamation-triangle", color = "danger", value =
 ↪  int(high_risk_count))
```

::: {.cell-output .cell-output-display execution_count=10}
```
{'icon': 'exclamation-triangle', 'color': 'danger', 'value': 2}
```
:::
:::

::: {.cell content='valuebox' title='Average Risk Score'
 ↪ execution_count=11}
``` {.python .cell-code}
dict(icon = "speedometer2", color = "info", value =
 ↪  round(avg_risk_score, 2))
```

::: {.cell-output .cell-output-display execution_count=11}
```
{'icon': 'speedometer2', 'color': 'info', 'value': np.float64(0.16)}
```
:::
:::

Row

Column {width="65%"}

::: {.cell title='Journal Entries by Risk Score' execution_count=12}
``` {.python .cell-code}
fig.show()
```

::: {.cell-output .cell-output-display}
```
Unable to display output for mime type(s): text/html
```
```

```
:::
:::


### Column {width="35%"}

::: {.cell title='Highest-Risk Entries' execution_count=13}
``` {.python .cell-code}
full_entry_level.sort_values("risk_score", ascending=False).head(10)[
    ["entry_id", "department", "amount", "risk_score"]
]
```

::: {.cell-output .cell-output-display execution_count=13}
```{=html}
<div>
<style scoped>
    .dataframe tbody tr th:only-of-type {
        vertical-align: middle;
    }

    .dataframe tbody tr th {
        vertical-align: top;
    }

    .dataframe thead th {
        text-align: right;
    }
</style>
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>entry_id</th>
      <th>department</th>
      <th>amount</th>
      <th>risk_score</th>
    </tr>
  </thead>
  <tbody>
    <tr>
```
```

```
      <th>116</th>
      <td>JE5906</td>
      <td>Sales</td>
      <td>75000.00</td>
      <td>4</td>
    </tr>
    <tr>
      <th>115</th>
      <td>JE5905</td>
      <td>HR</td>
      <td>41000.00</td>
      <td>3</td>
    </tr>
    <tr>
      <th>112</th>
      <td>JE5902</td>
      <td>Sales</td>
      <td>92500.00</td>
      <td>2</td>
    </tr>
    <tr>
      <th>110</th>
      <td>JE5900</td>
      <td>Sales</td>
      <td>50000.00</td>
      <td>2</td>
    </tr>
    <tr>
      <th>5</th>
      <td>JE5105</td>
      <td>Sales</td>
      <td>1851.67</td>
      <td>1</td>
    </tr>
    <tr>
      <th>113</th>
      <td>JE5903</td>
      <td>Operations</td>
      <td>3000.00</td>
      <td>1</td>
    </tr>
```

```
    <tr>
      <th>114</th>
      <td>JE5904</td>
      <td>Operations</td>
      <td>3000.00</td>
      <td>1</td>
    </tr>
    <tr>
      <th>111</th>
      <td>JE5901</td>
      <td>Operations</td>
      <td>10000.00</td>
      <td>1</td>
    </tr>
    <tr>
      <th>79</th>
      <td>JE5179</td>
      <td>Operations</td>
      <td>503.11</td>
      <td>1</td>
    </tr>
    <tr>
      <th>16</th>
      <td>JE5116</td>
      <td>Operations</td>
      <td>2860.71</td>
      <td>1</td>
    </tr>
  </tbody>
</table>
</div>
```
:::
:::
```

A few things worth noting about this structure:

- `## Row` and `### Column` headings define the dashboard's layout grid — no CSS or HTML required.
```

- `#| content: valuebox` turns a code chunk's output into one of those large single-number summary cards, commonly seen atop real business dashboards.
- Everything below the value boxes is still just pandas and Plotly code you already know how to write — the dashboard format only changes how the *output* is arranged, not how you compute it.

Saved as `audit-risk-dashboard.qmd` and rendered with `quarto render audit-risk-dashboard.qm` this produces a single interactive HTML page: three value boxes across the top, an interactive scatter plot, and a sortable table of the highest-risk entries — a genuinely useful artifact you could share with an audit team lead for daily monitoring, built entirely from tools covered in this book.

16.9. Putting It Together: A Recurring Reporting Workflow

Combining everything in this chapter, a realistic recurring reporting setup looks like this:

1. **A parameterized template** (like `department-risk-report.qmd`) containing the actual analysis logic
2. **A batch script** (like `render_all_departments.sh`) that renders one output per department, or per period, automatically
3. **A dashboard** (like `audit-risk-dashboard.qmd`) giving a real-time, interactive summary view, refreshed by simply re-running its render command whenever the underlying data updates
4. **A scheduler** (cron, a CI pipeline, or your organization's task scheduler) that runs the batch script on a recurring basis, so reports regenerate automatically without anyone remembering to run them by hand

None of these pieces require new analytical skills beyond what earlier chapters already taught — this chapter is entirely about *packaging* that same analytical work so it can run repeatedly, for different inputs, with minimal ongoing manual effort.

16.10. Chapter Summary

- Parameterized reports let one Quarto template produce many customized outputs by declaring parameters in a `#| tags: [parameters]` cell (for Python) and overriding them at render time with `-P key:value`.
- A batch script (bash or Python) automates rendering a template across many parameter values in one run, turning what would be repetitive manual work into a single command.

- Quarto Dashboards (`format: dashboard`) are a distinct output type from the book/document format used throughout this course, built for at-a-glance scanning rather than linear reading, using value boxes, rows, and columns.
- Plotly is well-suited to dashboards specifically because dashboards are HTML-only and benefit from interactivity (hover, zoom); seaborn and plotnine remain the right choice for this book's chapters because they need to render identically well to both HTML and PDF.
- A complete recurring reporting workflow combines a parameterized template, a batch rendering script, an optional dashboard for real-time monitoring, and a scheduler — all built from skills already covered earlier in this book.

16.11. Discussion Questions

1. Why can't a Quarto Dashboard simply be added as another chapter in this book's `_quarto.yml`, the way Chapters 1-13 were?
2. If you were building a monthly reporting package for four regional offices, would you use a parameterized report, a dashboard, or both? Explain your reasoning.
3. Why does this book use seaborn and plotnine for its own charts rather than Plotly, given that Plotly charts are arguably more impressive to look at?

16.12. Exercises

1. Save this chapter's parameterized code (the parameters cell plus the analysis that follows it) into its own file, `department-risk-report.qmd`, and render it for each of the four departments in `audit_test_transactions.csv` using the `-P` flag from the terminal.
2. Extend `render_all_departments.sh` to also print each department's average risk score to the terminal as it renders (hint: this requires a small Python helper script called from the bash loop, since bash itself can't easily compute a pandas aggregation).
3. Add a fourth value box to the dashboard listing in this chapter showing the single highest-risk department (by average risk score) rather than a company-wide total.
4. Rebuild this chapter's Plotly scatter plot as a bar chart instead, showing average risk score by department, and add it as a second chart to the dashboard layout shown in this chapter.

17. Data Ethics, Governance, and Reproducibility

Learning Objectives

By the end of this chapter, you should be able to:

- Explain why data privacy, governance, and reproducibility are core professional responsibilities in accounting analytics, not optional extras
- Apply pseudonymization and generalization techniques to reduce the sensitivity of a dataset
- Build a simple transformation log that documents what was done to a dataset, when, and why
- Explain how version control (Git) functions as a form of audit trail for analytics work
- Capture the software environment behind an analysis so it can be reproduced later
- Apply a practical checklist covering ethics, governance, and reproducibility to your own analytics projects

17.1. Why This Chapter Matters

Every technique in this book — audit analytics, fraud detection, predictive modeling — assumes you're working with real, sensitive data: customer records, employee salaries, vendor relationships, financial results not yet public. Getting the *analysis* right isn't enough. You also need to handle that data responsibly, be able to explain what you did to it, and be able to reproduce your results later — whether for a regulator, an auditor reviewing your work, or just yourself six months from now trying to remember how you got a number.

This chapter covers three related responsibilities: **data privacy** (protecting sensitive information), **governance** (documenting and controlling how data is used), and **reproducibility** (ensuring your work can be verified and repeated).

17.2. Data Privacy and Confidentiality

Accounting datasets routinely contain information that must be protected: customer names and account numbers, employee salaries, vendor banking details, and unpublished financial results. Two general principles matter regardless of which specific regulation applies (GDPR, CCPA, or an internal firm policy):

- **Data minimization** — only collect, retain, and share the data actually needed for the task at hand. If an analysis doesn't require a customer's name, don't include it.
- **Least exposure** — even when sensitive data is needed for analysis, reduce how many people and systems see the *identifiable* version of it. This is where pseudonymization and generalization come in.

17.2.1. Pseudonymization

Pseudonymization replaces an identifying value with a consistent substitute, so the same real-world entity always maps to the same pseudonym — which preserves the ability to group and analyze by that entity — without exposing the original value to anyone who only sees the pseudonymized data.

```
import pandas as pd
import hashlib

sales = pd.read_csv("data/sales_transactions.csv")
print(sales["customer_id"].nunique(), "unique customers")
```

```
35 unique customers
```

```
SALT = "course-project-2026"  # kept separate from any shared or
 ↪  published data

def pseudonymize(value, salt=SALT):
    combined = f"{salt}{value}".encode("utf-8")
    return "CUST_" + hashlib.sha256(combined).hexdigest()[:8]

sales["customer_id_pseudonym"] =
 ↪  sales["customer_id"].apply(pseudonymize)
sales[["customer_id",
 ↪  "customer_id_pseudonym"]].drop_duplicates().head(6)
```

	customer_id	customer_id_pseudonym
0	CUST103	CUST_0d15e223
1	CUST117	CUST_9b50fbbd
2	CUST113	CUST_54f74765
3	CUST122	CUST_e2525ed5
4	CUST112	CUST_0f68e998
6	CUST125	CUST_41da5f98

```
# same input always produces the same pseudonym, so grouping/joining
 ↪  still works
check1 = pseudonymize("CUST100")
check2 = pseudonymize("CUST100")
print("Consistent mapping:", check1 == check2)
```

```
Consistent mapping: True
```

> **!** Pseudonymization Is Not Anonymization
>
> Anyone who knows the `SALT` value can recompute the same hash and re-identify a customer — which is exactly the point: the salt is a secret, kept separate from the pseudonymized dataset, so someone with only the pseudonymized data *cannot* reverse it, but someone with legitimate access to both the salt and the mapping can. This is why the salt itself needs its own protection (e.g., a secrets manager or restricted config file, never committed to a public Git repository) — pseudonymization only protects the data as well as the salt is protected.

17.2.2. Generalization

Generalization reduces precision rather than replacing identity — turning an exact value into a broader category, which makes any single record less identifiable while preserving overall patterns for analysis.

```
def generalize_amount(amount):
    lower = int(amount // 1000) * 1000
    return f"${lower:,}-${lower + 1000:,}"

sales["invoice_date"] = pd.to_datetime(sales["invoice_date"])
sales["amount_range"] = sales["amount"].apply(generalize_amount)
sales["invoice_month"] =
 ↪  sales["invoice_date"].dt.to_period("M").astype(str)
```

```
sales[["amount", "amount_range", "invoice_date",
 ↪  "invoice_month"]].head(5)
```

	amount	amount_range	invoice_date	invoice_month
0	5544.89	$5,000-$6,000	2025-01-01	2025-01
1	6579.43	$6,000-$7,000	2025-01-02	2025-01
2	2640.39	$2,000-$3,000	2025-01-03	2025-01
3	7605.61	$7,000-$8,000	2025-01-03	2025-01
4	4942.04	$4,000-$5,000	2025-01-04	2025-01

```
print("Unique exact amounts:", sales["amount"].nunique())
print("Unique amount ranges:", sales["amount_range"].nunique())
```

```
Unique exact amounts: 427
Unique amount ranges: 23
```

Generalizing amounts into $1,000 bands reduces 427 distinct exact values down to just 23 categories — most quarterly trend or department-level analysis in this book would look nearly identical using either version, but the generalized version is meaningfully harder to use to single out one specific transaction.

> **Connect to Practice**
>
> Notice the tradeoff: generalization makes data safer to share but also less useful for some analyses. Chapter 9's Isolation Forest model, for example, needed exact dollar amounts to detect the seven outlier transactions — a generalized `amount_range` column would have hidden exactly the signal that model relied on. Privacy protection and analytical usefulness are frequently in tension, and the right balance depends on who the data is being shared with and why.

17.3. Data Governance and Audit Trails

Governance is about being able to answer, at any point, three questions: who has access to this data, what has been done to it, and why. The second question — what's been done to it — is where analytics work generates its own audit trail.

17.3.1. Version Control as an Audit Trail

You've been using Git since Chapter 2 to track changes to this book's chapters. The same mechanism is a legitimate governance tool for analytics work: every commit records *what* changed, *when*, and (via the commit message) *why* — exactly the kind of trail a regulator or reviewer might ask for when questioning how an analysis was produced.

```
git log --oneline data/sales_transactions.csv
```

Running this shows every commit that touched this specific file — a complete history of when the dataset was added or modified, without needing any separate logging system.

17.3.2. Logging Data Transformations

Git tracks *file-level* history well, but for a single analytical session, it's often useful to log each transformation step directly, especially for privacy-sensitive operations like the pseudonymization above.

```
from datetime import datetime, timezone

class TransformLog:
    def __init__(self):
        self.entries = []

    def record(self, step_name, description, rows_before, rows_after):
        self.entries.append({
            "timestamp":
             ↪  datetime.now(timezone.utc).isoformat(timespec="seconds"),
            "step": step_name,
            "description": description,
            "rows_before": rows_before,
            "rows_after": rows_after,
        })

    def as_dataframe(self):
        return pd.DataFrame(self.entries)

log = TransformLog()
```

```
raw_sales = pd.read_csv("data/sales_transactions.csv")
log.record("load", "Loaded raw sales_transactions.csv", None,
 ↪  len(raw_sales))

before = len(raw_sales)
raw_sales["customer_id"] =
 ↪  raw_sales["customer_id"].apply(pseudonymize)
log.record("pseudonymize", "Replaced customer_id with salted-hash
 ↪  pseudonym", before, len(raw_sales))

before = len(raw_sales)
clean_sales = raw_sales.dropna(subset=["amount"])
log.record("drop_missing", "Dropped rows with missing amount", before,
 ↪  len(clean_sales))

log.as_dataframe()
```

	timestamp	step	description	rows
0	2026-08-03T19:27:14+00:00	load	Loaded raw sales_transactions.csv	NaN
1	2026-08-03T19:27:14+00:00	pseudonymize	Replaced customer_id with salted-hash pseudonym	427.0
2	2026-08-03T19:27:14+00:00	drop_missing	Dropped rows with missing amount	427.0

This log is itself a small dataset you could save alongside your analysis output — a concrete, reviewable record of exactly what happened to the data, in what order, and why each step was taken.

17.3.3. Access Control

Governance also means limiting *who* can see identifiable data in the first place, following the principle of **least privilege**: a person should have access to only the data their role genuinely requires. In practice this means the pseudonymized version of a dataset (like `customer_id_pseudonym` above) might be shared broadly with a team performing pattern analysis, while the mapping needed to reverse it stays restricted to a much smaller group with a specific, documented business need.

17.4. Reproducibility

Reproducibility means someone else — or you, months later — can rerun your analysis and get the same result. This has come up implicitly throughout this book (every synthetic dataset used

a fixed random seed), and it's worth making the practice explicit.

17.4.1. Fixed Random Seeds

Every dataset in this book was generated with an explicit seed, for example `rng = np.random.default_rng(` back in Chapter 5. This is why the specific numbers in every chapter — the 7 z-score outliers, the exact ETR figures, the exact model coefficients — are exactly reproducible if you rerun the generation code, rather than different every time.

```
import numpy as np

rng1 = np.random.default_rng(seed=42)
rng2 = np.random.default_rng(seed=42)

print(rng1.normal(0, 1, 3))
print(rng2.normal(0, 1, 3))
```

```
[ 0.30471708 -1.03998411  0.7504512 ]
[ 0.30471708 -1.03998411  0.7504512 ]
```

Both sequences are identical — the seed fully determines the "random" output, which is exactly what makes a simulation reproducible rather than a one-time, unrepeatable event.

17.4.2. Capturing the Software Environment

The same code can produce different results on different machines if package versions differ — a chart's default colors change, a statistical function's default parameter changes, a bug gets fixed (or introduced) between versions. Recording exactly which versions were used is essential for anyone trying to reproduce your work later.

```
import importlib.metadata as metadata

packages_of_interest = ["pandas", "polars", "numpy", "scikit-learn",
 ↪  "seaborn", "plotnine", "matplotlib"]

for pkg in packages_of_interest:
    try:
        print(f"{pkg}: {metadata.version(pkg)}")
    except metadata.PackageNotFoundError:
        print(f"{pkg}: not installed")
```

```
pandas: 3.0.5
polars: 1.43.2
numpy: 2.4.6
scikit-learn: 1.9.0
seaborn: 0.13.2
plotnine: 0.15.7
matplotlib: 3.11.1
```

In practice, this is usually captured once for an entire project rather than checked manually each time, using:

```
pip freeze > requirements.txt
```

Committing `requirements.txt` to your project's Git repository (alongside the `.qmd` files themselves) means anyone who clones the repository can recreate the exact same package environment with `pip install -r requirements.txt`, rather than guessing which versions you originally used.

17.4.3. Quarto□s Own Reproducibility Features

Recall from Chapter 2 that this book's `_quarto.yml` includes:

```
execute:
  freeze: auto
```

This caches each chapter's executed output and only re-runs code when that chapter's content actually changes — which serves reproducibility in a subtle but important way: it guarantees that a chapter's rendered output reflects a specific, known execution of its code, rather than silently drifting if, say, a shared dataset file changes without the chapter itself being edited.

17.5. A Practical Ethics, Governance, and Reproducibility Checklist

Before sharing or submitting any analytics project, it's worth checking:

- ☐ Does this dataset contain any identifying information that isn't strictly necessary for the analysis? If so, has it been removed, pseudonymized, or generalized?

- ☐ If pseudonymization was used, is the salt or key stored separately from the shared data, and never committed to a public repository?
- ☐ Is there a record (a transformation log, commit history, or both) of what was done to the data and why?
- ☐ Are random seeds fixed anywhere randomness is used, so results are reproducible?
- ☐ Is the software environment (package versions) documented, e.g., via `requirements.txt`?
- ☐ Would someone with appropriate access, given your code and your documented environment, be able to reproduce your exact results?

17.6. Chapter Summary

- Data privacy in accounting analytics rests on two general principles: collect and share only what's needed (data minimization), and reduce exposure of identifiable data even when it's needed (least exposure).
- Pseudonymization (consistent, salted replacement of identifiers) preserves the ability to analyze by entity while protecting identity — but only as well as the salt itself is protected, and it is not the same as true anonymization.
- Generalization reduces precision (exact amounts to ranges, exact dates to months) to make individual records harder to single out, at some cost to analytical precision — a real tradeoff, not a free improvement.
- Git commit history and an explicit transformation log both serve as governance audit trails, recording what was done to data, when, and why.
- Least privilege access control means limiting identifiable data access to those with a specific, documented need, separate from broader access to de-identified data for analysis.
- Reproducibility depends on fixed random seeds and a documented software environment (e.g., `requirements.txt`) — both practices already used implicitly throughout this book's own dataset generation.
- A short pre-sharing checklist covering privacy, governance, and reproducibility should become a standard last step before submitting or distributing any analytics project.

17.7. Discussion Questions

1. Why is pseudonymization described in this chapter as "not the same as anonymization"? What would need to be true for a dataset to be genuinely anonymized rather than merely pseudonymized?
2. Chapter 9's Isolation Forest model needed exact dollar amounts to detect outliers, which generalization would have obscured. Can you think of an accounting analysis from an earlier chapter where generalized (rather than exact) data would have worked just as well?

3. If you inherited a dataset and analysis from a colleague who left the firm, with no transformation log and no record of package versions used, what specific problems might you run into trying to extend their work?

17.8. Exercises

1. Apply the pseudonymization function from this chapter to the `posted_by` column in `audit_test_transactions.csv`, and confirm that Chapter 8's rare-poster test still identifies the same flagged individual after pseudonymization (by pseudonym rather than by name).
2. Extend the `TransformLog` class in this chapter to also record the specific columns affected by each transformation step, not just row counts.
3. Write a `requirements.txt`-style list (by hand or using the `importlib.metadata` approach shown in this chapter) documenting every package used across this book's chapters, and commit it to your project's Git repository.
4. Design a generalization scheme (amount ranges, date granularity, or both) for `sales_transactions.csv` that would still allow Chapter 10's quarter-end concentration analysis to reach the same conclusion. How coarse can the generalization be before that analysis's conclusion becomes unclear?

18. Capstone Project

> **What This Chapter Is**
>
> Unlike every other chapter in this book, this one doesn't teach a new technique. It's a project brief: a single integrated dataset, a choice of analytical tracks, and a structure for applying what you've learned across Chapters 1-16 to an open-ended, realistic engagement. Plan to spend the final 1-2 weeks of the course on this.

18.1. The Scenario

You've been engaged to analyze one fiscal year (2025) of data for **Alderwood Supply Co.**, a mid-sized distribution company. You have access to:

File	Description
`capstone_chart_of_accounts.csv`	The company's chart of accounts (21 accounts)
`capstone_general_ledger.csv`	A sample of FY2025 transaction-level journal entries (692 lines, 346 entries)
`capstone_income_statement.csv`	Income statements for FY2024 and FY2025
`capstone_balance_sheet.csv`	Balance sheets for FY2024 and FY2025
`capstone_sales_transactions.csv`	FY2025 invoice-level sales and accounts receivable detail (151 invoices)
`capstone_department_budget_actual.csv`	FY2025 budget-to-actual spending by department and category

> **A Naming Note**
>
> Every file in this capstone is prefixed with `capstone_`, even though a few cover the same *type* of data as earlier chapters (an income statement, a chart of accounts, sales transactions). This is deliberate: earlier chapters already have files named `income_statement.csv`, `balance_sheet.csv`, `chart_of_accounts.csv`, and `sales_transactions.csv` sitting in your project root, and this capstone uses a **different fictional company** with com-

pletely different numbers. Keeping the `capstone_` prefix means both sets of files can coexist in the same project folder without one silently overwriting the other.

! This Data Has Not Been Pre-Cleaned or Pre-Explained

Every dataset in Chapters 1-16 came with a narrative explaining exactly what to look for. This one doesn't. Some of it may be completely unremarkable. Some of it may not be. Part of this capstone is deciding, using the techniques from this book, what's actually worth investigating and building a defensible case — not being told in advance where to look. Treat the general ledger sample specifically the way Chapter 9 treated sampling: as a *sample* of the year's activity, not a complete population, and reason about your conclusions accordingly.

18.2. Choosing a Track

Pick **one** primary track to focus your capstone on. All three draw on the same underlying company, but emphasize different chapters. Regardless of track, you're expected to draw on the foundational skills from Chapters 3-6 (data fundamentals, wrangling, EDA, visualization) throughout.

18.2.1. Track A: Audit & Fraud Analytics

Primarily draws on Chapters 8-10 (management accounting variance concepts, audit analytics, fraud detection)

Using `capstone_general_ledger.csv`, apply journal entry testing, sampling, and anomaly detection techniques to identify transactions or patterns worth flagging for further review. Build a composite risk score. Decide which flagged items you'd escalate for follow-up and which you'd consider immaterial or explainable.

18.2.2. Track B: Financial Reporting & Revenue Analytics

Primarily draws on Chapters 7 and 11-12 (financial statement analysis, revenue recognition, tax)

Using `capstone_income_statement.csv`, `capstone_balance_sheet.csv`, and `capstone_sales_transactions.csv`, perform ratio and trend analysis across the two years provided, and evaluate the sales data for revenue recognition risk (timing concentration, invoice-to-shipment patterns, AR aging). Build a short financial health assessment.

18.2.3. Track C: Management Accounting & Predictive Analytics

Primarily draws on Chapter 8 (management accounting) and Chapter 13 (predictive modeling)

Using `capstone_department_budget_actual.csv` alongside the financial statements, analyze budget variances by department and category, and build a simple predictive model (for example, predicting which department/category combinations are most likely to show a large variance, or modeling a cost driver relationship) using techniques from Chapter 13.

> **Not Sure Which Track to Pick?**
>
> Track A tends to suit students who enjoyed Chapters 8-10 most and like investigative, detective-style work. Track B suits students more interested in external financial analysis and reporting. Track C suits students who want more hands-on time with the modeling techniques from Chapter 13. There's no wrong choice — pick based on genuine interest, since that will show in the quality of your final report.

18.3. Deliverables

1. **A rendered Quarto report** (HTML, PDF, or both) presenting your analysis, findings, and recommendations — written for a specific, named audience (e.g., "the audit committee," "the CFO," "the controller"), not a generic reader.
2. **Clean, reproducible code** supporting every claim in your report, following the practices from Chapter 16: a fixed random seed anywhere randomness is used, a documented package environment, and clear organization so someone else could rerun your analysis and get the same result.
3. **A short presentation** (5-10 minutes, in-class or recorded, per your instructor's preference) summarizing your findings for a non-technical audience — practice communicating results the way Chapter 6 and Chapter 7 emphasized, not just producing them.

> **Individual or Group?**
>
> This capstone works either individually or in small groups of 2-3, at your instructor's discretion. If working in a group, your report should briefly note who contributed to which sections.

18.4. Suggested Milestones

A capstone with no intermediate checkpoints tends to become a last-minute scramble. A reasonable pacing:

Week	Milestone
1	Explore all provided datasets, choose a track, and submit 2-3 specific questions you plan to investigate
1-2	Perform your core analysis; meet with your instructor or peers for a brief progress check
2	Draft your Quarto report; conduct a peer review exchange with another student or group
2	Finalize your report, prepare your presentation, and submit

18.5. Example Starter Questions

You don't have to use these exact questions, but they illustrate the kind of open-ended question this capstone expects, and roughly how much depth is expected per track.

Track A (Audit & Fraud): - Does the general ledger sample balance? If not, what does that tell you, and how would you investigate it? - Which journal entries score highest on a composite red-flag basis, and what specific follow-up would you request for each? - Are there any postings whose *description* field is worth a second look, independent of their dollar amount?

Track B (Financial Reporting & Revenue): - How did the company's profitability, liquidity, and leverage change from FY2024 to FY2025, and what's driving the biggest change? - Does the sales data show any quarter-end concentration or invoice-to-shipment timing patterns worth flagging? - Which customers, if any, warrant a closer look based on payment behavior or revenue timing?

Track C (Management Accounting & Predictive): - Which department/category combinations show the largest budget variances, and are they favorable or unfavorable? - Can you build a model that predicts which combinations are likely to see a large variance, using the patterns in this year's data? - What would you recommend to management based on this year's variance patterns going into next year's budget cycle?

18.6. Grading Rubric

Criterion	What It Assesses
Technical correctness	Are your calculations, code, and statistical methods correct and appropriately applied?
Technique selection	Did you choose analytical methods appropriate to your question, rather than applying a technique just because it was covered in the book?
Depth of investigation	Did you follow up on interesting findings, or stop at the first surface-level observation?
Clarity of communication	Would your named audience actually understand your findings and recommendations?
Reproducibility	Could someone else rerun your code and get your exact results?
Presentation	Did you communicate your findings clearly and appropriately for a non-technical audience?

> **!** A Note on "Finding Something"
>
> Not every dataset contains a dramatic discovery, and manufacturing a finding that isn't really there is a worse outcome than honestly reporting "I investigated X and found no significant concern." Chapter 12 made exactly this point about a weak regression result — an honest null finding, properly investigated and clearly communicated, is a legitimate and valuable outcome. Grading rewards rigorous investigation and clear communication, not a dramatic conclusion for its own sake.

18.7. Final Thoughts

Every technique in this capstone — reading messy data, computing the right summary statistic, building a red-flag score, checking a model's assumptions, communicating a result honestly — is something you've now done at least once in this book, on data built specifically to teach that one skill in isolation. The capstone's real challenge isn't learning anything new; it's deciding, on your own, which of those sixteen chapters' worth of tools actually fit the questions in front of you, and using them with the same judgment a professional would bring to an unfamiliar engagement. That judgment is the actual subject this course has been teaching all along.

References

Financial Accounting Standards Board (FASB). Revenue from Contracts with Customers, 2014. URL https://asc.fasb.org/1943274/2147480014.

Public Company Accounting Oversight Board (PCAOB). AS 2401: Consideration of Fraud in a Financial Statement Audit, 2024a. URL https://pcaobus.org/oversight/standards/auditing-standards/details/AS2401.

Public Company Accounting Oversight Board (PCAOB). Staff Update on Outreach Activities Related to the Integration of Generative Artificial Intelligence in Audits and Financial Reporting, 2024b. URL https://pcaobus.org/news-events/news-releases/news-release-detail/pcaob-staff-shares-observations-from-outreach-on-use-of-generative-artificial-intelligence-in-audits-and-financial-reporting.

Tim Stobierski. Data Wrangling: What It Is & Why It's Important, 2021. URL https://online.hbs.edu/blog/post/data-wrangling.

The International Auditing and Assurance Standards Board (IAASB). IAASB Unveils New Technology Position to Shape the Future of Audit and Assurance Standards, 2024. URL https://www.iaasb.org/news-events/2024-10/iaasb-unveils-new-technology-position-shape-future-audit-and-assurance-standards.

US Securities & Exchange Commission (US SEC). SEC Creates Task Force to Tap Artificial Intelligence for Enhanced Innovation and Efficiency Across the Agency, 2025. URL https://www.sec.gov/newsroom/press-releases/2025-103-sec-creates-task-force-tap-artificial-intelligence-enhanced-innovation-efficiency-across-agency.

A. A Primer on Python

> **Who This Appendix Is For**
>
> This appendix is a self-contained introduction to Python fundamentals — the language mechanics this book assumes you're comfortable with starting in Chapter 3. If you've never written Python before, work through this appendix before continuing. If you already know another programming language, this should read quickly as a syntax reference rather than a full course.
> This appendix doesn't cover pandas or polars directly — those get a full, dedicated treatment starting in Chapter 4. What's here is the foundation those chapters build on: values, data structures, functions, NumPy arrays, and just enough object-oriented programming to understand what's actually happening every time you call `.groupby()` or `.mean()` on a DataFrame later in this book.

A.1. What Is Python, and Why This Book Uses It

Python is a general-purpose programming language that has become the dominant choice for data analysis, thanks to a large ecosystem of libraries (pandas, NumPy, scikit-learn, and others used throughout this book) and a syntax designed to be readable. Code in Python is organized into **statements** (individual instructions) and relies on **indentation** — consistent spacing at the start of a line — to define which statements belong together, rather than curly braces or explicit block markers used by some other languages.

```
print("This is a Python statement.")
```

```
This is a Python statement.
```

A.2. Data Types

Every value in Python has a **type**, which determines what you can do with it. The `type()` function tells you what type any value is.

```
x = 42            # int -- a whole number
y = 3.14           # float -- a decimal number
name = "Alice"     # str -- text, in quotes
is_active = True   # bool -- True or False
nothing = None     # NoneType -- represents "no value"

print(type(x), type(y), type(name), type(is_active), type(nothing))
```

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'> <class
'NoneType'>
```

A few things worth knowing about these five types:

- **Integers and floats** behave as you'd expect arithmetically, but dividing two integers with `/` always produces a float in Python, even if the result is a whole number (`10 / 2` gives `5.0`, not `5`).
- **Strings** support many built-in operations: `name.upper()`, `name.lower()`, `len(name)`, and combining strings with `+`.
- **Booleans** are the result of any comparison (`5 > 3` evaluates to `True`) and are essential for the filtering operations used constantly in later chapters (e.g., `debit > 0` in Chapter 4).
- **`None`** represents the deliberate absence of a value — different from `0` or an empty string, and directly related to how pandas represents missing data (`NaN`) starting in Chapter 3.

```
print(10 / 2)       # true division always returns a float
print(10 // 3)      # floor division discards the remainder
print(10 % 3)       # modulo returns the remainder
print("abc" + "def") # string concatenation
```

```
5.0
3
1
abcdef
```

A.3. Data Structures

Beyond single values, Python has four built-in structures for holding *collections* of values. Choosing the right one matters — each has different rules about ordering, uniqueness, and whether it can be changed after creation.

Structure	Ordered?	Duplicates Allowed?	Mutable?	Syntax
List	Yes	Yes	Yes	`[1, 2, 3]`
Tuple	Yes	Yes	No	`(1, 2, 3)`
Dictionary	Yes (insertion order)	Keys must be unique	Yes	`{"key": "value"}`
Set	No	No (auto-removes duplicates)	Yes	`{1, 2, 3}`

```
my_list = [1, 2, 3, "four"]      # a list can hold mixed types
my_tuple = (1, 2, 3)              # tuples look like lists but can't
 ↪  be changed
my_dict = {"account": "Cash", "balance": 1000}
my_set = {1, 2, 2, 3}              # note: the duplicate 2 is
 ↪  automatically dropped

print(my_list, my_tuple, my_dict, my_set)
```

```
[1, 2, 3, 'four'] (1, 2, 3) {'account': 'Cash', 'balance': 1000} {1,
2, 3}
```

Indexing and accessing values:

```
print(my_list[0])          # first item in a list (indexing starts at
 ↪  0)
print(my_dict["account"])  # access a dictionary value by its key
```

```
1
Cash
```

Lists are mutable □ they can be changed after creation:

```
my_list.append(5)
print(my_list)
```

```
[1, 2, 3, 'four', 5]
```

> **Connect to This Book**
>
> Every one of these structures shows up constantly once you start using pandas. A DataFrame's columns behave much like dictionary keys. A single column of data, once extracted, behaves a lot like a list or a NumPy array (covered next). Understanding these four structures now makes almost every later chapter's syntax feel far more predictable.

A.4. Functions

A **function** packages a block of reusable code that can be called with different inputs. You've already seen one built-in function (`print()`); Python also lets you define your own with `def`.

```
def add_tax(amount, rate=0.07):
    """Return the amount plus tax, using a default 7% rate unless told
     ↪ otherwise."""
    return amount * (1 + rate)

print(add_tax(100))          # uses the default rate
print(add_tax(100, rate=0.10)) # overrides the default
```

```
107.0
110.00000000000001
```

A few pieces of that function definition worth naming explicitly:

- **`amount`** and **`rate`** are **parameters** — placeholders for values the function needs to run.
- **`rate=0.07`** gives `rate` a **default value**, so callers don't have to specify it every time.
- **`return`** sends a value back to whatever called the function — without it, a function produces `None`.
- The text in triple quotes right after `def` is a **docstring**, a convention for documenting what a function does (you'll see this same convention throughout well-written pandas/polars code, and it's good practice in your own functions too — recall the `compute_ratios()` function from Chapter 7).

Lambda functions are a compact way to write small, throwaway functions in a single line — you'll see these frequently as arguments to pandas methods like `.apply()`:

```
square = lambda x: x ** 2
print(square(5))
```

A.5. NumPy and Arrays

NumPy (Numerical Python) is the foundational library for numeric computing in Python — pandas itself is built on top of it, and you'll use it directly throughout this book, especially for generating and transforming numeric data.

```
import numpy as np
```

A.5.1. Why Not Just Use a List?

A Python list can hold numbers, but it isn't built for fast numeric computation — operations on a list happen one element at a time in a slow loop under the hood. A NumPy **array** stores numbers far more efficiently and supports **vectorized operations** — applying an operation to every element at once, without writing an explicit loop.

```
arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(arr.shape, arr.dtype, arr.ndim)
```

```
[1 2 3 4 5]
(5,) int64 1
```

- **`.shape`** — the array's dimensions (a 1-D array of 5 elements here)
- **`.dtype`** — the data type stored in the array (all elements in a NumPy array share one type)
- **`.ndim`** — the number of dimensions (1 here; a table of data would have `ndim = 2`)

A.5.2. Creating Arrays

```
arr2 = np.arange(0, 10, 2)   # like Python's range(), but returns an
 ↪  array
zeros = np.zeros((2, 3))      # a 2x3 array of zeros
ones = np.ones(4)             # a 1-D array of four ones
lin = np.linspace(0, 1, 5)    # 5 evenly spaced values between 0 and 1

print(arr2)
print(zeros)
print(ones)
print(lin)
```

```
[0 2 4 6 8]
[[0. 0. 0.]
 [0. 0. 0.]]
[1. 1. 1. 1.]
[0.   0.25 0.5  0.75 1.  ]
```

A.5.3. Vectorized Operations and Filtering

```
print(arr * 2)          # every element multiplied by 2, no loop
 ↪  needed
print(arr[arr > 2])     # boolean filtering: keep only elements
 ↪  greater than 2
```

```
[ 2  4  6  8 10]
[3 4 5]
```

That second line — indexing an array with a condition — is exactly the same logic behind filtering a pandas DataFrame in Chapter 4 (e.g., `gl_pd[gl_pd["debit"] > 5000]`). It's not a coincidence; pandas' filtering syntax is built directly on this NumPy pattern.

A.5.4. Common Aggregate Functions

```
print(arr.sum(), arr.mean(), arr.std(), arr.min(), arr.max())
```

```
15 3.0 1.4142135623730951 1 5
```

These same five operations — sum, mean, standard deviation, min, max — are exactly the summary statistics computed constantly starting in Chapter 5's exploratory data analysis, just called on a pandas Series instead of a raw NumPy array.

A.5.5. Reshaping Arrays

```
mat = np.array([[1, 2, 3], [4, 5, 6]])
print(mat.reshape(3, 2))
```

```
[[1 2]
 [3 4]
 [5 6]]
```

`.reshape()` rearranges the same data into a different shape, as long as the total number of elements stays the same (here, 6 elements stay 6 elements, just arranged as 3 rows × 2 columns instead of 2 × 3).

A.5.6. Conditional Logic with `np.where()`

```
print(np.where(arr > 2, "high", "low"))
```

```
['low' 'low' 'high' 'high' 'high']
```

`np.where(condition, value_if_true, value_if_false)` builds a new array by checking a condition element-by-element — the same logic behind flagging outliers or building categorical labels throughout the audit and fraud detection chapters (e.g., Chapter 9's `is_outlier` flag).

A.5.7. Random Number Generation

Every synthetic dataset in this book was built using NumPy's random number generator, seeded for reproducibility (see Appendix on reproducibility practices in Chapter 16):

```
rng = np.random.default_rng(seed=42)
print(rng.normal(0, 1, 3))   # 3 random values from a normal
 ↪ distribution
```

```
[ 0.30471708 -1.03998411  0.7504512 ]
```

Using a fixed `seed` means this exact code produces this exact output every time it runs — the same principle behind every dataset generated throughout this book being fully reproducible.

A.6. Classes, Objects, Attributes, and Methods

Everything in Python — including a NumPy array and, later, every pandas DataFrame you'll create — is an **object**: a bundle of data (**attributes**) and the functions that operate on that data (**methods**), defined by a **class**.

A.6.1. Defining a Class

```
class Account:
    """A simple representation of a financial account."""

    def __init__(self, name, balance=0):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount
```

A few pieces of this definition worth naming explicitly:

- **`class Account:`** defines a new type, much like `int` or `str` are types, but one you've designed yourself.
- **`__init__`** is a special method that runs automatically whenever a new `Account` is created, setting up its initial attributes.
- **`self`** refers to the specific object being worked with — it's how a method knows *which* account's balance to change, when many `Account` objects might exist at once.
- **`self.name`** and **`self.balance`** are **attributes** — data that belongs to each individual object.
- **`deposit()`** and **`withdraw()`** are **methods** — functions that belong to the class and operate on its attributes.

A.6.2. Creating and Using an Object

```
acct = Account("Cash", 1000)
acct.deposit(500)
acct.withdraw(200)

print(acct.name, acct.balance)
```

```
Cash 1300
```

`acct` is an **instance** of the `Account` class — a specific object with its own `name` and `balance`, created from the general blueprint the class defines. You could create as many separate `Account` objects as you like, each with independent attribute values.

> **!** Why This Matters for the Rest of This Book
>
> Every time you write `gl_pd.groupby("account")` or `df.shape` starting in Chapter 4, you're calling a **method** (`.groupby()`) or reading an **attribute** (`.shape`) on a pandas DataFrame **object** — `gl_pd` is an instance of pandas' `DataFrame` class, in exactly the same sense that `acct` above is an instance of `Account`. Understanding this pattern demystifies a lot of pandas/polars syntax: `.mean()`, `.describe()`, `.sort_values()`, and every other method you'll use throughout this book are simply methods defined on the DataFrame class, working on the attributes (the actual data) that a specific DataFrame object holds. You'll never need to *write* a class as complex as pandas' `DataFrame` yourself, but recognizing this object → attribute / method relationship makes the rest of this book's syntax far more predictable rather than something to memorize line by line.

A.7. Summary

- Every Python value has a type (`int`, `float`, `str`, `bool`, `NoneType`), and `type()` tells you which one you're working with.
- Lists, tuples, dictionaries, and sets each store collections of values with different rules around ordering, uniqueness, and mutability — choosing the right one matters.
- Functions (`def`) package reusable code, accept parameters with optional default values, and `return` a result; `lambda` provides a compact syntax for small one-line functions.
- NumPy arrays support fast, vectorized numeric operations without explicit loops — the foundation pandas itself is built on, and the source of syntax patterns (boolean filtering, aggregate functions, `np.where()`) that reappear constantly starting in Chapter 4.
- A fixed random seed (`np.random.default_rng(seed=...)`) makes randomly generated data fully reproducible — the technique behind every synthetic dataset in this book.
- Python objects bundle data (attributes) and behavior (methods), defined by a class — understanding this pattern is what makes pandas and polars syntax feel logical rather than arbitrary, since a DataFrame is simply an object like any other, with its own attributes and methods.

B. The Accounting Cycle: From Journal to Financial Statements

> **Who This Appendix Is For**
>
> This appendix is a self-contained primer on the mechanics of the accounting cycle — how a single transaction turns into a full set of financial statements. If you've already taken introductory financial accounting, most of this will be a refresher. If accounting analytics is your first real exposure to accounting mechanics, work through this appendix *before* Chapter 3 — everything in this book assumes you're comfortable with the concepts covered here.
> This appendix uses one small, complete worked example throughout — a single company, a single month, from its very first transaction all the way to its financial statements and post-closing trial balance — rather than isolated examples for each step. Every number in every table below was computed and verified to tie out exactly.

B.1. The Accounting Cycle - at a Glance

The **accounting cycle** is the sequence of steps a business follows, every period, to turn raw transactions into financial statements:

1. **Journalize transactions** — record each transaction in the general journal as it occurs
2. **Post to the ledger** — transfer each journal entry into the relevant account
3. **Prepare an unadjusted trial balance** — list every account and its balance, to confirm debits equal credits
4. **Record adjusting entries** — update accounts for items not yet captured by day-to-day transactions (accruals, deferrals, depreciation)
5. **Prepare an adjusted trial balance** — confirm the books still balance after adjustments
6. **Prepare financial statements** — the income statement, statement of retained earnings, and balance sheet, built directly from the adjusted trial balance
7. **Record closing entries** — zero out revenue, expense, and dividend accounts, transferring their net effect into retained earnings
8. **Prepare a post-closing trial balance** — confirm only permanent (balance sheet) accounts carry a balance into the next period

We'll now work through all eight steps for one company.

B.2. Our Worked Example: Bright Consulting LLC

Bright Consulting LLC is a new consulting business. We'll record its very first month of operations, January 2026, from its first transaction through to its financial statements.

> **A Quick Refresher on Debits and Credits**
>
> Every transaction affects at least two accounts, and total debits must always equal total credits. The normal balance of each account type:
>
> | Account Type | Normal Balance | Increases With | Decreases With |
> |---|---|---|---|
> | Assets | Debit | Debit | Credit |
> | Liabilities | Credit | Credit | Debit |
> | Equity | Credit | Credit | Debit |
> | Revenue | Credit | Credit | Debit |
> | Expenses | Debit | Debit | Credit |
>
> If this table doesn't feel automatic yet, keep it nearby while working through the rest of this appendix.

B.3. Step 1: Journalize Transactions

Here are Bright Consulting's transactions for January 2026, recorded in the general journal:

Date	Description	Account	Debit	Credit
Jan 1	Owner invests cash for common stock	Cash	20,000	
		Common Stock		20,000
Jan 3	Purchase equipment for cash	Equipment	6,000	
		Cash		6,000
Jan 5	Pay 12-month insurance policy in advance	Prepaid Insurance	2,400	

Date	Description	Account	Debit	Credit
		Cash		2,400
Jan 8	Purchase office supplies on account	Supplies	800	
		Accounts Payable		800
Jan 10	Provide consulting services for cash	Cash	5,000	
		Service Revenue		5,000
Jan 15	Provide consulting services on account	Accounts Receivable	3,500	
		Service Revenue		3,500
Jan 18	Pay employee salaries	Salaries Expense	2,200	
		Cash		2,200
Jan 20	Receive cash in advance for February services	Cash	1,500	
		Unearned Revenue		1,500
Jan 22	Pay accounts payable	Accounts Payable	800	
		Cash		800
Jan 28	Collect cash from customer on account	Cash	2,000	
		Accounts Receivable		2,000

! Notice Two Transactions That Are Easy to Misread

The January 5 entry (prepaid insurance) and January 20 entry (cash received in advance) are **not** revenue or expense at the time they're recorded — Bright Consulting hasn't yet used the insurance or performed the February services. Both will need an adjusting entry later, in Step 4, once some of that time or service has actually passed. This is one of the most common sources of confusion for students new to accounting: cash changing hands does

not necessarily mean revenue or expense has been earned or incurred yet.

Check: total debits across all ten entries = $44,200; total credits = $44,200. They match, which is the first sign nothing was recorded incorrectly.

B.4. Step 2: Post to the Ledger

Posting means transferring each journal entry into its own running account (traditionally drawn as a "T-account," named for its shape). Here's Bright Consulting's Cash T-account after posting every January transaction involving cash:

```
                              Cash
-------------------------------------------------------------
Jan 1    20,000  |  Jan 3     6,000
Jan 10    5,000  |  Jan 5     2,400
Jan 20    1,500  |  Jan 18    2,200
Jan 28    2,000  |  Jan 22      800
-------------------------------------------------------------
Total    28,500  |  Total    11,400

Ending balance: 28,500 - 11,400 = 17,100 (debit balance, as expected
for an asset)
```

Every other account gets its own T-account the same way. Once every journal entry has been posted, you're ready for Step 3.

B.5. Step 3: Unadjusted Trial Balance

A **trial balance** simply lists every account and its ending balance, to confirm total debits still equal total credits after posting.

Account	Debit	Credit
Cash	17,100	
Accounts Receivable	1,500	
Supplies	800	
Prepaid Insurance	2,400	
Equipment	6,000	
Accounts Payable		0
Unearned Revenue		1,500

Account	Debit	Credit
Common Stock		20,000
Service Revenue		8,500
Salaries Expense	2,200	
Total	**29,000**	**29,000**

> **Why Accounts Payable Shows Zero**
>
> Accounts Payable was debited $800 (the January 22 payment) and credited $800 (the January 8 purchase) — the account was opened and fully paid off within the same month, so its net balance happens to be zero. It's still listed, since a real trial balance includes every account that had *any* activity, even if the ending balance nets to nothing.

Debits equal credits ($29,000 = $29,000), so we're ready to move on. But notice this trial balance is *incomplete* — it doesn't yet reflect the insurance that's expired, the supplies that have been used, or several other things that happened silently over the course of the month. That's exactly what Step 4 fixes.

B.6. Step 4: Adjusting Entries

Adjusting entries update the books at the end of a period for economic events that occurred but weren't triggered by an obvious external transaction — no invoice arrived, no cash changed hands, but something real happened that the financial statements need to reflect. There are four common types, all illustrated below.

B.6.1. (a) Deferred Expense: Supplies Used

Bright Consulting purchased $800 of supplies on January 8. A physical count on January 31 shows $300 worth remaining — meaning $500 was used during the month.

Account	Debit	Credit
Supplies Expense	500	
Supplies		500

B.6.2. (b) Deferred Expense: Insurance Expired

The $2,400 insurance policy purchased January 5 covers 12 months. One month (1/12) has now passed: $2,400 / 12 = $200.

Account	Debit	Credit
Insurance Expense	200	
Prepaid Insurance		200

B.6.3. (c) Depreciation

The $6,000 equipment purchased January 3 is expected to last 5 years (60 months) with no salvage value, depreciated evenly: $6,000 / 60 = $100 per month.

Account	Debit	Credit
Depreciation Expense	100	
Accumulated Depreciation — Equipment		100

! Why a Separate "Accumulated Depreciation" Account?

Depreciation is never credited directly to the Equipment account. Instead, **Accumulated Depreciation** is a *contra-asset* account — it's reported as a deduction from Equipment on the balance sheet, but keeps the original cost ($6,000) and the total depreciation taken visible separately, rather than silently shrinking the Equipment balance itself.

B.6.4. (d) Accrued Expense: Unrecorded Salaries

By January 31, employees have earned an additional $400 in wages that won't actually be paid until February — but the expense belongs to January, since that's when the work was performed.

Account	Debit	Credit
Salaries Expense	400	
Salaries Payable		400

B.6.5. (e) Deferred Revenue: Unearned Revenue Now Earned

Recall the $1,500 received January 20 for February services. By January 31, Bright Consulting has actually performed $500 worth of that work already.

Account	Debit	Credit
Unearned Revenue	500	
Service Revenue		500

The Pattern Behind All Five Adjustments

Every adjusting entry above does the same job: it moves something out of a balance sheet "holding" account (Supplies, Prepaid Insurance, Unearned Revenue) or recognizes something not yet recorded at all (Depreciation, accrued Salaries) and puts it in the right period's income statement. None of these five entries involve cash — that's precisely why they'd be missed without a deliberate adjusting step at period-end.

B.7. Step 5: Adjusted Trial Balance

After posting all five adjusting entries, here's the complete adjusted trial balance:

Account	Debit	Credit
Cash	17,100	
Accounts Receivable	1,500	
Supplies	300	
Prepaid Insurance	2,200	
Equipment	6,000	
Accumulated Depreciation — Equipment		100
Accounts Payable		0
Salaries Payable		400
Unearned Revenue		1,000
Common Stock		20,000
Service Revenue		9,000
Salaries Expense	2,600	
Supplies Expense	500	
Insurance Expense	200	
Depreciation Expense	100	
Total	**30,500**	**30,500**

Debits still equal credits ($30,500 = $30,500) — the adjustments were posted correctly.

B.8. Step 6: Prepare the Financial Statements

The adjusted trial balance contains everything needed to build all three core financial statements — in a specific order, since each one feeds the next.

B.8.1. Income Statement

Bright Consulting LLC — Income Statement		
For the Month Ended January 31, 2026		
Service Revenue		9,000
Expenses:		
Salaries Expense	2,600	
Supplies Expense	500	
Insurance Expense	200	
Depreciation Expense	100	
Total Expenses		3,400
Net Income		**5,600**

B.8.2. Statement of Retained Earnings

Bright Consulting LLC — Statement of Retained Earnings	
For the Month Ended January 31, 2026	
Beginning Retained Earnings (new company)	0
Add: Net Income	5,600
Less: Dividends	0
Ending Retained Earnings	**5,600**

B.8.3. Balance Sheet

Bright Consulting LLC — Balance Sheet
As of January 31, 2026
Assets

Bright Consulting LLC — Balance Sheet		
Cash		17,100
Accounts Receivable		1,500
Supplies		300
Prepaid Insurance		2,200
Equipment	6,000	
Less: Accumulated Depreciation	(100)	5,900
Total Assets		**27,000**
Liabilities		
Accounts Payable		0
Salaries Payable		400
Unearned Revenue		1,000
Total Liabilities		**1,400**
Equity		
Common Stock		20,000
Retained Earnings		5,600
Total Equity		**25,600**
Total Liabilities and Equity		**27,000**

Total Assets ($27,000) = Total Liabilities and Equity ($27,000). The balance sheet balances, confirming everything up to this point was recorded and adjusted correctly.

B.9. Step 7: Closing Entries

Closing entries reset every *temporary* account (revenue, expenses, and dividends) to zero at the end of the period, transferring their combined effect into Retained Earnings — a *permanent* account that carries forward into the next period. This is what allows revenue and expense accounts to start fresh at zero every new period, rather than accumulating forever.

Closing entry 1 □ close revenue:

Account	Debit	Credit
Service Revenue	9,000	
Income Summary		9,000

Closing entry 2 □ close expenses:

Account	Debit	Credit
Income Summary	3,400	
Salaries Expense		2,600
Supplies Expense		500
Insurance Expense		200
Depreciation Expense		100

Closing entry 3 □ close Income Summary to Retained Earnings:

Account	Debit	Credit
Income Summary	5,600	
Retained Earnings		5,600

What Is "Income Summary"?

Income Summary is a temporary clearing account used only during the closing process — it exists purely to collect the total of all revenue and expense accounts in one place, before that net figure (here, $5,600 — exactly this period's net income) moves into Retained Earnings. It never appears on any financial statement and always ends the closing process with a zero balance.

A Fourth Closing Entry, If Dividends Were Paid

Bright Consulting paid no dividends this month, so there's no fourth entry here. If it had, a closing entry would debit Retained Earnings and credit Dividends directly (Dividends does not flow through Income Summary, since dividends are a distribution of profit, not a component of it).

B.10. Step 8: Post-Closing Trial Balance

After posting all closing entries, only **permanent** (balance sheet) accounts carry a balance — every revenue, expense, and Income Summary account is now exactly zero.

Account	Debit	Credit
Cash	17,100	
Accounts Receivable	1,500	
Supplies	300	

Account	Debit	Credit
Prepaid Insurance	2,200	
Equipment	6,000	
Accumulated Depreciation — Equipment		100
Accounts Payable		0
Salaries Payable		400
Unearned Revenue		1,000
Common Stock		20,000
Retained Earnings		5,600
Total	**27,100**	**27,100**

Debits equal credits one final time ($27,100 = $27,100), and this trial balance's ending balances become the **beginning** balances for February — the accounting cycle then repeats.

B.11. Seeing the Whole Cycle in Python

Since this book uses Python throughout, it's worth briefly seeing how naturally this entire cycle maps onto a pandas DataFrame — exactly the structure used for the general ledger datasets in Chapters 3-4.

```
import pandas as pd

# Every journal entry (including adjusting entries) as one tidy table
# --
# this is exactly the shape of general_ledger_sample.csv from Chapter 3
entries = [
    ("Jan 1", "Cash", 20000, 0), ("Jan 1", "Common Stock", 0, 20000),
    ("Jan 3", "Equipment", 6000, 0), ("Jan 3", "Cash", 0, 6000),
    ("Jan 31 (a)", "Supplies Expense", 500, 0), ("Jan 31 (a)",
    "Supplies", 0, 500),
    ("Jan 31 (e)", "Unearned Revenue", 500, 0), ("Jan 31 (e)",
    "Service Revenue", 0, 500),
    # ... the remaining entries from this appendix would continue here
]

journal = pd.DataFrame(entries, columns=["date", "account", "debit",
    "credit"])
```

```
# A trial balance is just a groupby -- the same technique from Chapter
 ↪ 4
trial_balance = journal.groupby("account")[["debit", "credit"]].sum()
trial_balance["balance"] = trial_balance["debit"] -
 ↪ trial_balance["credit"]
trial_balance
```

account	debit	credit	balance
Cash	20000	6000	14000
Common Stock	0	20000	-20000
Equipment	6000	0	6000
Service Revenue	0	500	-500
Supplies	0	500	-500
Supplies Expense	500	0	500
Unearned Revenue	500	0	500

The Connection to This Book

Every general ledger dataset used in Chapters 3 through 11 *is* the journal from this appendix, just at a larger scale — hundreds or thousands of entries instead of ten. A trial balance is a `groupby`. A balance sheet is a filtered, reshaped version of that same trial balance. The audit tests in Chapter 9 look for unusual patterns in exactly the kind of journal entries you just recorded by hand in this appendix. If the mechanics in this appendix make sense, you already understand *what* the data in the rest of this book represents — the remaining chapters are about doing this same work at a scale no one could manage by hand.

B.12. Summary

- The accounting cycle has eight steps: journalize, post, unadjusted trial balance, adjusting entries, adjusted trial balance, financial statements, closing entries, post-closing trial balance.
- Adjusting entries fall into four patterns: deferred expenses (supplies, prepaid insurance), depreciation, accrued expenses (unpaid salaries), and deferred revenue (unearned revenue now earned) — none involve cash changing hands at the moment of adjustment.
- Financial statements are prepared in a specific order because each one depends on the last: the income statement's net income feeds the statement of retained earnings, whose ending balance feeds the balance sheet.

- Closing entries reset temporary accounts (revenue, expenses, dividends) to zero via the Income Summary account, transferring the period's net income into Retained Earnings, a permanent account.
- A post-closing trial balance should contain only balance sheet accounts, and its ending balances become the next period's beginning balances — the cycle then repeats.
- Every technique in this appendix — the journal, the trial balance, the adjustments — is the same structure this entire book works with in pandas and polars, just at a much larger scale.

C. An Introduction to EDGAR Database

> **Who This Appendix Is For**
>
> This appendix provides idea about EDGAR database, which is maintained by US SEC as a central repository of corporate disclosures. Therefore, EDGAR contains real companies' data. If you need to use these data, you should know how to extract data from EDGAR database.
> Since accounting numbers have implications for capital market, you need to combine accounting data with share price. Therefore, you should know how to extract capital market data. This appendix provides you with tools and knowledge about how to extract data from both sources. You can combine them and test the significance or implications of accounting numbers.

C.1. EDGAR Database and Its Implications

The Electronic Data Gathering, Analysis, and Retrieval (EDGAR) database is an online filing system maintained by the U.S. Securities and Exchange Commission (SEC). Established in 1992, EDGAR serves as a centralized repository for corporate disclosures required under U.S. federal securities laws. The system provides investors, researchers, analysts, and the general public with free access to millions of regulatory filings submitted by publicly traded companies and other reporting entities. The url of EDGAR database is - https://www.sec.gov/edgar.

Public companies are required to submit various financial and non-financial reports through EDGAR, including annual reports (Form 10-K), quarterly reports (Form 10-Q), current reports (Form 8-K), proxy statements (Form DEF 14A), and registration statements. These filings contain important information about a company's financial performance, operating activities, risk factors, management discussion and analysis (MD&A), executive compensation, and corporate governance practices.

For financial analysis and academic research, EDGAR is an important source of primary corporate information because the data comes directly from company filings submitted to regulators.

Researchers can use EDGAR filings to analyze financial statements, calculate financial ratios, evaluate business performance, study corporate decisions, and conduct empirical research in accounting, finance, and economics. The database also supports automated data collection through SEC-provided APIs and structured filing formats, making it useful for large-scale financial data analysis.

Overall, EDGAR plays a critical role in promoting transparency and efficiency in U.S. capital markets by ensuring that investors have timely and reliable access to corporate disclosure information.

C.2. XBRL

XBRL, or eXtensible Business Reporting Language, is an XML-based language for the electronic transmission of business and financial data. It is designed to improve the way financial data is communicated and shared, making it easier to prepare, analyze, and exchange financial information. XBRL uses tags to identify each piece of financial data, which then allows it to be used programmatically by an XBRL-compatible program.

According to XBRL, XBRL plays a huge role in increasing *corporate transparency*, *reducing information asymmetries*, and *improving the efficiency of capital markets*. When digital reporting – supplying clear, consistent digital meanings – is combined with rigorous management oversight and assurance processes, users can have a high degree of confidence in business data and in the insights they draw from it.

- For **investors**, finding information, comparing corporate performance and uncovering new opportunities all becomes easier, cheaper, faster and more accurate. XBRL data can open up wider possibilities for investment, less limited by language and location.
- **Regulators** too benefit from streamlined data collection, improved understanding of market, sustainability or other risks, and ultimately more efficient and effective oversight.
- **Corporates**, meanwhile, are able to tell their own stories. Digital reporting can open up increased access to capital, potentially at a reduced cost. For companies with an eye on the future, getting to grips with XBRL offers the opportunity to go beyond compliance, for example in monitoring and benchmarking performance.

XBRL brings reporting into the digital age, and offers powerful capabilities across the reporting chain. Market demand for reliable digital data is high: for users from investors to regulators, XBRL is the gold standard for decision-useful analysis.

C.3. edgartools - Python Package for Parsing EDGAR Database

`edgartools` is a Python library for accessing SEC EDGAR filings as structured data. SEC EDGAR has every filing back to 1994, free — and almost none of it is ready to use. `edgartools` turns any filing into a tidy data object. To learn more about the `edgartools`, you can go through documentation.

After installing the `edgartools`, you need to identify yourself to the SEC because EDGAR requires an email with every request.

```
from edgar import *
set_identity("Sharif Islam mdshariful.islam@siu.edu")  # required by
 ↪  SEC
```

```
from edgar import Company, set_identity
from edgar.xbrl import XBRLS
```

```
# Getting data for one company
company = Company("AAPL")

# Get all 10-K filings, then filter to the date range you want
filings = company.get_filings(form="10-K",
 ↪  date="2010-01-01:2015-12-31")
# Stitch multiple filings' financials into one multi-period view
xbrls = XBRLS.from_filings(filings)
```

```
# Income Statement
income_statement = xbrls.statements.income_statement()
# Convert to a DataFrame for easier analysis/plotting
income_df = income_statement.to_dataframe()
```

```
# Balance Sheet
balance_sheet = xbrls.statements.balance_sheet()
balance_df = balance_sheet.to_dataframe()
```

```
# Cash Flow Statement
cash_flow_statement = xbrls.statements.cash_flow_statement()
cash_flow_statement_df = cash_flow_statement.to_dataframe()
cash_flow_statement2 = xbrls.statements.cashflow_statement()
cash_flow_statement_df2 = cash_flow_statement2.to_dataframe()
```

```
# Statement of Equity
statement_of_equity = xbrls.statements.statement_of_equity()
statement_of_equity_df = statement_of_equity.to_dataframe()
```

```
# Comprehensive Income Statement
comp_income_statement = xbrls.statements.comprehensive_income()
comp_income_statement_df = comp_income_statement.to_dataframe()
```

```
income_statement_detailed =
 ↪  xbrls.statements.income_statement(view="detailed")
income_df_detailed = income_statement_detailed.to_dataframe()
```

Alternatively, you can get some specific values from the financial statements instead of the whole financial statements to calculate different ratios using `edgartools`.

```
financials = company.get_financials()
revenue = financials.get_revenue()
net_income = financials.get_net_income()
total_assets = financials.get_total_assets()
total_liabilities = financials.get_total_liabilities()
stockholders_equity = financials.get_stockholders_equity()
current_assets = financials.get_current_assets()
current_liabilities = financials.get_current_liabilities()
operating_cf = financials.get_operating_cash_flow()
capex = financials.get_capital_expenditures()
free_cash_flow = financials.get_free_cash_flow()  # calculated
 ↪  automatically

# Roll your own ratio
debt_to_equity = total_liabilities / stockholders_equity
net_margin = net_income / revenue
```

If you want to extract financial statement information from more than one company, you can run the following code.

```
import pandas as pd
from edgar import Company, set_identity
from edgar.xbrl import XBRLS
```

```
set_identity("Sharif Islam mdshariful.islam@siu.edu")

tickers = ["AAPL", "MSFT", "WMT"]
date_range = "2010-01-01:2025-12-31"

# Step 1: Fetch and stitch XBRL data for each company
xbrls_by_ticker = {}

for ticker in tickers:
    company = Company(ticker)
    filings = company.get_filings(form="10-K", date=date_range)
    xbrls = XBRLS.from_filings(filings)
    xbrls_by_ticker[ticker] = xbrls
    print(f"{ticker}: {len(filings)} filings retrieved")

# Step 2: Pull each statement type per company, tag with ticker, and
 ↪  combine
def build_combined_df(statement_method_name):
    """Fetch a given statement type for all companies and combine into
     ↪  one DataFrame."""
    frames = []
    for ticker, xbrls in xbrls_by_ticker.items():
        try:
            statement = getattr(xbrls.statements,
 ↪  statement_method_name)()
            df = statement.to_dataframe()
            df["ticker"] = ticker
            frames.append(df)
        except Exception as e:
            print(f"Could not get {statement_method_name} for
             ↪  {ticker}: {e}")
    return pd.concat(frames, ignore_index=True) if frames else
     ↪  pd.DataFrame()

income_statement_df = build_combined_df("income_statement")
balance_sheet_df = build_combined_df("balance_sheet")
cashflow_statement_df = build_combined_df("cashflow_statement")
equity_statement_df = build_combined_df("statement_of_equity")  #
 ↪  method name unverified - see note below
```

```
# Quick check
#print(income_statement_df.head())
#print(balance_sheet_df.head())
#print(cashflow_statement_df.head())
#print(equity_statement_df.head())
```

```
AAPL: 17 filings retrieved
MSFT: 16 filings retrieved
WMT: 16 filings retrieved
```

C.4. Unstructured Data in Company Filings

Companay filings (e.g., 10-K) contain a large amount of unstructured text data, especially in sections such as the business description, risk factors, legal proceedings, and management's discussion and analysis. Unlike structured financial statement data, this narrative information is not arranged in fixed rows and columns, which makes it more difficult to analyze using traditional spreadsheet methods. However, it is often rich in useful signals about firm strategy, uncertainty, governance, litigation exposure, and future expectations. `edgartools` can be used to extract unstructured data as well.

```
from edgar import *
set_identity("Sharif Islam mdshariful.islam@siu.edu")
filings = get_filings(form='10-K', amendments=False,
 ↪  filing_date="2026-01-05:2026-01-12")
filings_df = filings.to_pandas()

# Create a list to store the Item 1a and Item 7 text
items_texts = []

for n, filing in enumerate(filings):
    url = filing.url
    cik = filing.cik
    filing_date = filing.header.filing_date,
    reporting_date = filing.header.period_of_report,
    comn = filing.company

    # Extract the text for Item 1c
    TenK = filing.obj()
```

```
    # Bypass None values
    try:
        item7_text = TenK['Item 7'] # Management Discussion & Analysis
 ↪  (MDA)
        item1a_text = TenK['Item 1A'] # Risk Factors
    except:
        item7_text = None
        item1a_text = None

    # Append the data to the list
    # item1c_text = TenK['Item 1C']
    items_texts.append({
        'CIK': cik,
        'Filing Date': str(filing_date),
        'Item 7 Text': item7_text,
        'Item 1a Text': item1a_text,
        'url': url,
        'reporting_date': str(reporting_date),
        'comn': comn
    })

# Create a DataFrame from the Item 1c text data
items_df = pd.DataFrame(items_texts)
```

If you want to extract some specific companies' unstructured data, you can do so as well.

```
tickers = ["AAPL", "MSFT", "WMT"]
date_range = "2018-01-01:2022-12-31"

# Step 1: Fetch and stitch XBRL data for each company
# Create a list to store the Item 1c text
items_texts = []

for ticker in tickers:
    filings = company.get_filings(form="10-K",
 ↪  date=date_range,amendments=False)
    for n, filing in enumerate(filings):
        url = filing.url
        cik = filing.cik
        filing_date = filing.header.filing_date,
        reporting_date = filing.header.period_of_report,
```

```
        comn = filing.company,
        tic = Company(filing.cik).tickers

        # Extract the text for Item 1c
        TenK = filing.obj()

        # Bypass None values
        try:
            item7_text = TenK['Item 7'] # Management Discussion &
↪  Analysis (MDA)
            item1a_text = TenK['Item 1A'] # Risk Factors
        except:
            item7_text = None
            item1a_text = None

        # Append the data to the list
        # item1c_text = TenK['Item 1C']
        items_texts.append({
        'Ticker': tic,
        'CIK': cik,
        'Filing Date': str(filing_date),
        'Item 7 Text': item7_text,
        'Item 1a Text': item1a_text,
        'url': url,
        'reporting_date': str(reporting_date),
        'comn': comn
    })
# Create a DataFrame from the Item 1c text data
company_items_df = pd.DataFrame(items_texts)
```

If you want to extract some accounting policies of some specific companies, you can do so using `edgartools` python package.

```
from edgar import Company
import pandas as pd

tickers = ["AAPL", "MSFT", "WMT"]
date_range = "2018-01-01:2022-12-31"

records = []
```

```
for ticker in tickers:
    company = Company(ticker)
    filings = company.get_filings(
        form="10-K",
        date=date_range,
        amendments=False,
    )

    for filing in filings:

        try:
            xbrl = filing.xbrl()

            # Accounting Policies statement
            acct_policy = xbrl.get_statement("accounting_policies")

            records.append({
                "ticker": ticker,
                "cik": filing.cik,
                "filing_date": filing.filing_date,
                "report_date": filing.report_date,
                #"fiscal_year": filing.report_date.year,
                "accounting_policies": acct_policy,
            })

        except Exception as e:

            records.append({
                "ticker": ticker,
                "cik": filing.cik,
                "filing_date": filing.filing_date,
                "report_date": filing.report_date,
                #"fiscal_year": filing.report_date.year,
                "accounting_policies": None,
                "error": str(e),
            })

accounting_policy_df = pd.DataFrame(records)
```

```
# How to convert the accounting policies into a dataframe
import ast
```

```
import pandas as pd
from bs4 import BeautifulSoup

def clean_html(x):
    """Strip HTML tags and return plain text."""
    if not isinstance(x, str):
        return x
    return BeautifulSoup(x, "html.parser").get_text(" ", strip=True)

def policies_to_df(items):
    """Convert one cell (a list of concept dicts) into a tidy
    ↪ DataFrame."""
    if isinstance(items, str):          # if the cell was saved to CSV
    ↪ as text
        items = ast.literal_eval(items)
    rows = []
    for d in items or []:
        if d.get("is_abstract") or not d.get("values"):
            continue                    # skip header rows with no
            ↪ text
        for period, text in d["values"].items():
            parts = period.split("_")   # e.g.
↪ duration_2021-09-26_2022-09-24
            rows.append({
                "concept": d["concept"],
                "label": d["label"],
                "period_start": parts[1] if len(parts) > 2 else None,
                "period_end": parts[-1],
                "text": clean_html(text),
            })
    return pd.DataFrame(rows)



id_cols = ["ticker", "cik", "filing_date", "report_date"]

frames = []
for idx, row in accounting_policy_df.iterrows():
    out = policies_to_df(row["accounting_policies"])      # use your
↪ column name
    for i, col in enumerate(id_cols):
```

```
        out.insert(i, col, row[col])
    frames.append(out)

policies_df = pd.concat(frames, ignore_index=True)
policies_df
```

	ticker	cik	filing_date	report_date	concept
0	AAPL	320193	2022-10-28	2022-09-24	us-gaap_BasisOfAccountingPolicyPolicyTextBlock
1	AAPL	320193	2022-10-28	2022-09-24	us-gaap_FiscalPeriod
2	AAPL	320193	2022-10-28	2022-09-24	us-gaap_RevenueRecognitionPolicyTextBlock
3	AAPL	320193	2022-10-28	2022-09-24	us-gaap_CompensationRelatedCostsPolicyTextBlock
4	AAPL	320193	2022-10-28	2022-09-24	us-gaap_EarningsPerSharePolicyTextBlock
...	...	...	...	...	...
293	WMT	104169	2018-03-30	2018-01-31	us-gaap_SellingGeneralAndAdministrativeExpense...
294	WMT	104169	2018-03-30	2018-01-31	us-gaap_AdvertisingCostsPolicyTextBlock
295	WMT	104169	2018-03-30	2018-01-31	us-gaap_StartUpActivitiesCostPolicy
296	WMT	104169	2018-03-30	2018-01-31	us-gaap_ForeignCurrencyTransactionsAndTranslat...
297	WMT	104169	2018-03-30	2018-01-31	us-gaap_NewAccountingPronouncementsPolicyPolic...

C.5. tidyfinance - Python package for Share Price Data

In Chapter 7, we talked about the significance of financial statement information for capital market efficiency. Therefore, just getting data or information from financial statements is not enough, we also need capital market data, particulary share price data of a given company. We can get these data using `tidyfinance` python package. There are two widely used models that examine whether stock prices are associated with accounting numbers. One model is called price model, and the other one is called Return model.

The **price model** examines the relationship between a firm's stock price and its accounting information, such as earnings per share (EPS) and book value per share (BVPS). A commonly used specification based on the Ohlson (1995) valuation model is:

$$\text{Price}_i = \beta_0 + \beta_1 \text{EPS}_i + \beta_2 \text{BVPS}_i + \varepsilon_i$$

where:

- Price_i = Stock price of firm i
- EPS_i = Earnings per share of firm i

- BVPS_i = Book value per share of firm i
- β_0 = Intercept
- β_1, β_2 = Regression coefficients
- ε_i = Error term

The model tests whether accounting information explains cross-sectional differences in stock prices.

The **return model** examines whether accounting earnings explain stock returns. A simple specification is:

$$\text{Return}_i = \beta_0 + \beta_1 \text{Earnings}_i + \varepsilon_i$$

A more common empirical specification uses changes in earnings:

$$\text{Return}_i = \beta_0 + \beta_1 \Delta\text{EPS}_i + \varepsilon_i$$

where:

- Return_i = Stock return for firm i
- Earnings_i = Earnings of firm i
- ΔEPS_i = Change in earnings per share
- β_0 = Intercept
- β_1 = Earnings response coefficient (ERC)
- ε_i = Error term

The return model evaluates whether new accounting information is reflected in stock returns.

```
import polars as pl
import tidyfinance as tf
tf.set_backend("polars")
```

```
prices = tf.download_data(
    domain="Stock Prices",
    symbols= ["AAPL", "GOOGL", "WMT"],
    start_date="2000-01-01",
    end_date="2025-12-31"
)
prices.head()
```

symbol str	date date	volume i64	open f64	low f64	high f64	close f64	adjusted_close f64
"AAPL"	2000-01-03	535796800	0.936384	0.907924	1.004464	0.999442	0.837002
"AAPL"	2000-01-04	512377600	0.966518	0.90346	0.987723	0.915179	0.766434
"AAPL"	2000-01-05	778321600	0.926339	0.919643	0.987165	0.928571	0.77765
"AAPL"	2000-01-06	767972800	0.947545	0.848214	0.955357	0.848214	0.710353
"AAPL"	2000-01-07	460734400	0.861607	0.852679	0.901786	0.888393	0.744002

The variable `volume` indicates trading volume; the variables `open`, `low`, `high`, and `close` indicate opening, low, high, and closing price of the stock in a given day. The `adjusted_close` takes the closing price and adjusts retroactively for dividend payouts or stock splits or other corporate actions. We can merge the data extracted using `edgartools` and `tidyfinance` and run either price or return model to test the economic significance of accounting numbers.

If we want to use **return model**, we need returns for each stock. The return are calculated:

$$r_t = P_t / P_{t-1} - 1$$

where:

- P_t = Adjusted Price at the End of Day t

```
returns = (
     prices
     .sort(["symbol", 'date'])

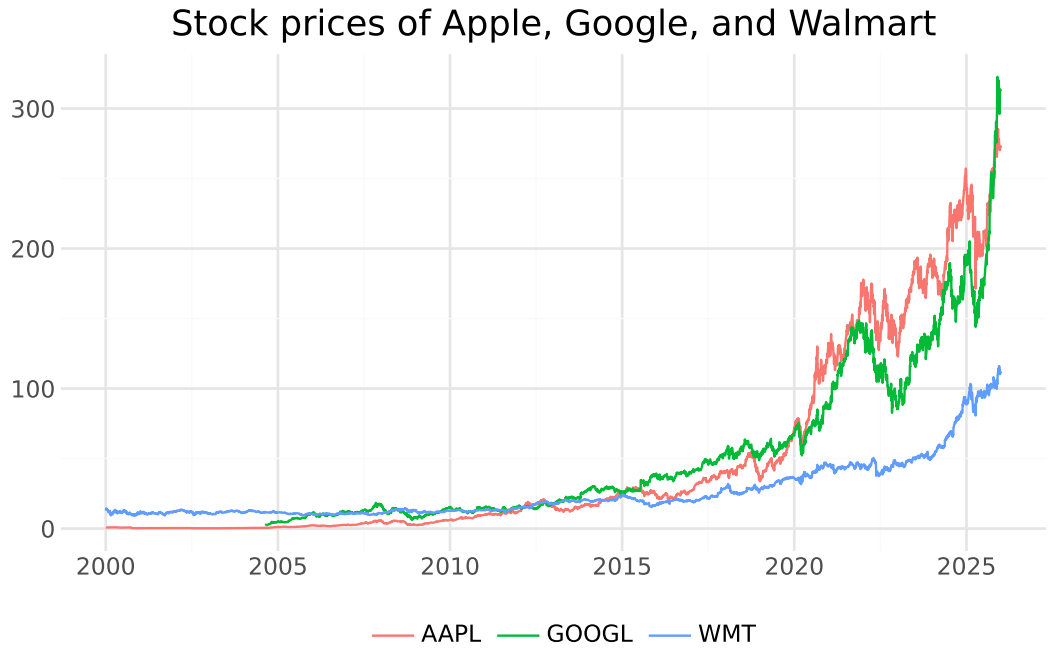
 ↪  .with_columns(pl.col("adjusted_close").pct_change().over("symbol").alias("ret
     .select("symbol", "date", "adjusted_close", "ret")
)
returns
```

symbol str	date date	adjusted_close f64	ret f64
"AAPL"	2000-01-03	0.837002	null
"AAPL"	2000-01-04	0.766434	-0.08431
"AAPL"	2000-01-05	0.77765	0.014634
"AAPL"	2000-01-06	0.710353	-0.086539
"AAPL"	2000-01-07	0.744002	0.047369
…	…	…	…

symbol str	date date	adjusted_close f64	ret f64
"WMT"	2025-12-23	110.198242	-0.015098
"WMT"	2025-12-24	110.903748	0.006402
"WMT"	2025-12-26	111.032936	0.001165
"WMT"	2025-12-29	111.817924	0.00707
"WMT"	2025-12-30	111.211784	-0.005421

We can also visualize the prices or returns of companies to identify patterns, if any.

```
from plotnine import *
theme_set(theme_minimal())
price_viz = (
    ggplot(prices, aes(y = "adjusted_close", x = "date", color =
 ↪  "symbol"))
    + geom_line()
    + scale_x_date(date_breaks="5 years", date_labels="%Y")
    + labs(x="", y="", color="", title="Stock prices of Apple, Google,
     ↪  and Walmart")
    + theme(legend_position="bottom")
)
price_viz.show()
```



C.6. Summary

- EDGAR database is a central repository of corporate disclosures in the US. All US public companies issue their financial statement using Form 10-K in EDGAR. Therefore, understanding the EDGAR database is critical for hands-on application of accounting knowledge.
- `edgartools` is a python package that is very efficient in parsing data from EDGAR databases. All kinds of forms 10-K, 10-Q, 8-K can be easily parsed by using edgartools
- `tidyfinance` python package is a python package that helps to get data about stock price of companies.